

FR-4

Automatic Slip Detection

สแกนสลิปในแคสเลอริวต์โนมิตด้วย AI
เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ
คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-4 ละเอียดทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-4 Automatic Slip Detection เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

i ภาค ก • พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข • FR-4 Automatic Slip Detection

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผล การออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปปต์นำเสนอ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบล่างแบบนี้:

src/domain/runway.ts

ตัวอย่าง • บรรทัด 17-23

```
17 * so the status is "no_spending" and no date is shown.
18 */
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด **Ctrl+P** (Mac: **Cmd+P**) แล้วพิมพ์ข้อความตัวหน้าที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น `src/domain/runway.ts` ก็แตะเปิดได้เหมือนกัน

i ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดยังไ

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

!ระวัง

จุดที่คนมักเข้าใจผิด หรือนึกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดทีละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างขีดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอต่่ายจากเว็บเวอร์ชันจริงในโหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไปเล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดอย่างไร	2
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	12
MindPay คืออะไร และทีมเราสร้างอะไร	13
ปัญหาที่ทีมเราอยากแก้	13
6 ฟังก์ชันหลัก (Functional Requirements)	13
หน้าต่างแอปโดยรวม	14
ใช้แอปได้ 3 แบบ	15
ตัวเลขของโปรเจกต์	15
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	16
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันอย่างไร	17
ระบบมี 4 ส่วนใหญ่	17
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	17
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนถึงเลขบนหน้าหลักเปลี่ยน	18
ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ	22
เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	24
กล่องเครื่องมือของทีมเรา	24
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	24
เอาโค้ด MindPay มาไว้ในเครื่อง	24
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	25
คำสั่งที่ใช้บ่อย	28
รู้จักหน้าจอส VS Code	28
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	29
ส่วนเสริม (Extensions) ที่แนะนำ	30
ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	30
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	30
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย	31
โปรแกรมคืออะไร	31
1. ค่า (value) และชนิดของค่า	31
2. ตัวแปร: กล่องที่มีชื่อ	31
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	32
4. เชื้อไข if: ถ้า... ให้ทำ...	32
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	33
6. Array: รายการของหลายค่า	33
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	35
8. วงซ้ำ (loop)	36
9. ข้อความ (string) ที่ใช้บ่อย	36
10. คณิตศาสตร์ที่ใช้	36
แบบฝึกหัด A4	36

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay	38
ทำไมต้อง TypeScript	38
การบอชนิดข้อมูล (type annotation)	38
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	38
ชนิดข้อมูลแบบพิเศษที่เจอบ่อย	40
import และ export: แคร่โค้ดระหว่างไฟล์	41
งานที่ต้องรอ: async, await และ Promise	42
จัดการความผิดพลาด: try, catch, throw	42
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	43
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	43
Regular expression (regex) อ่านยังงี้	45
แปลงสตางค์กลับเป็นข้อความบาท	46
React: วาดหน้าจอจากข้อมูล	48
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	48
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	48
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	50
Hook ที่ใช้ในโปรเจกต์	51
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	53
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังงี้	54
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	56
React Native คืออะไร	56
Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น	57
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	57
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	59
แถบเมนูล่าง: (tabs)/_layout.tsx	63
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	64
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	66
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	68
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	68
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	68
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	69
แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน	70
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	72
ฐานข้อมูลคืออะไร	72
Supabase ให้อะไรทีมเราบ้าง	72
Migration: ไฟล์สร้างฐานข้อมูล	73
Row Level Security (RLS): หัวใจความปลอดภัย	76
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	78
แอปต่อ Supabase ยังงี้: src/data/supabase.ts	78
การสมัครและเข้าสู่ระบบ	80
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	81
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	86
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังงี้	89
AI ในแอปคืออะไร	89
ทำไมแอปไม่เรียก AI เอง	89
เส้นทางของคำถาม 1 ข้อถึงน้องกล่า	89

ฝั่งแอป: src/services/coach.ts	90
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบัสรถ	91
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	93
เลือก AI ตัวไหน: Claude หรือ Gemini	96
ส่งคำขอไปที่ Claude: callClaude	98
ตั้งค่า AI (ผู้ดูแลเครื่องเดียว)	101
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	101
UX/UI: การออกแบบหน้าจอและ Design System	102
หลักการออกแบบของ MindPay	102
Design System คืออะไร	102
สี: Green + Gold	102
ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ	104
ตัวอักษร	106
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	106
Accessibility: ทุกคนใช้ได้	109
แอนิเมชัน: สวยแต่ไม่กินแบต	109
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	109
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	111
ทำไมต้องเขียนเทสต์	111
Unit test: ตรวจสอบฟังก์ชันทีละตัว	111
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	113
Manual test: ทดสอบบนมือถือจริง	116
Traceability: ผูก Requirement → โค้ด → เทสต์	117
ตรวจสอบคุณภาพอื่น ๆ	117
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	119
Git: ระบบบันทึกประวัติ	119
ใช้ Git ประจำวัน	119
ประวัติของ MindPay UU GitHub	120
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push	120
Secrets UU GitHub	125
ไฟล์เอกสารใน repo	125
6 FR ทำงานร่วมกันอย่างไร และคำถามระดับระบบ	127
สายข้อมูลของทั้งระบบ	127
ใครใช้ของใคร	127
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	127
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	128
คำศัพท์รวม	129
FR-4 AUTOMATIC SLIP DETECTION	131
FR-4 ในหน้าเดียว	132
อยู่ตรงไหนในแอป	132
4 ข้อที่ต้องจำ	132
ปัญหาที่ฉันทายากแก้	134
บทเรียนจาก prototype รุ่นแรก	134
เป้าหมายที่ฉันทั้งไว้ตอนออกแบบ	134
Requirement และเงื่อนไขผ่าน	135

เส้นทางของสลิป 1 ใน	137
แผนที่โค้ดของ FR-4	137
ทัวร์หน้าจ่อ: เปิดแอปแล้วกดล้าวดให้เอง	139
6 รูปในแคลเลอริ์ เกิดอะไรขึ้นกับแต่ละรูป	140
การ์ดนี้มีได้ 5 หน้าตา	140
ทัวร์หน้าจ่อ: หน้าสแกนสลิป	142
ระหว่างอ่าน: คิว AI เติม แล้วอ่านต่อเอง	142
4 รูปที่เลือก กลายเป็นอะไร	143
ทัวร์หน้าจ่อ: ผลการสแกนตามช่วงเวลาและตามวัน	145
บรรทัดสรุปตอนอ่านจบ บอกครบทุกรูป	145
ทัวร์หน้าจ่อ: ไม่มีรูปใหม่ และเน็ตหลุด	147
ไม่มีรูปใหม่ใน 30 วันล่าสุด	147
เน็ตหลุดระหว่างอ่าน	147
ข้อความทั้งหมดที่ผู้ใช้อาจเห็นเมื่อมีปัญหา	148
ทัวร์หน้าจ่อ: สลิปรอยืนยัน และหน้าตรวจสอบสลิป	149
หน้าสลิปรอยืนยัน แบ่ง 2 กลุ่ม	149
หน้าตรวจสอบสลิป = พอร์มของ FR-1 ที่มีกล้องช่วยตรวจ	149
ทัวร์หน้าจ่อ: ตั้งค่าการสแกน และรูปทดสอบ	151
รูปทดสอบที่ใช้ในเล่มนี้และในเกสท์	151
หลักการที่ 1: QR ตรวจสอบสลิป ด้านแรกทีฟรีและแม่นที่สุด	153
ข้างใน QR มีอะไร: แกะทีละตัวอักษร	153
CRC: ตัวเลขตรวจสอบท้าย QR	154
QR ของรูปทดสอบทั้ง 3 ใบ (ผลจากการรันโค้ดจริง)	154
รหัสธนาคาร 21 แห่งที่แอปรู้จัก	155
ปัญหา: QR บนสลิปเล็กมาก	155
ข้อยกเว้น: สลิป e-wallet	156
หลักการที่ 2: เกณฑ์ความมั่นใจ 80%	157
ธง 4 ชนิด และเงื่อนไขที่ทำให้ติดธง	157
จากธง ไปเป็นสถานะของรายการ	157
ตัวอย่างจริง 8 แบบ (รันโค้ดจริง วันที่ 9 ต.ค. 2569)	157
ทำไมเลือก 80%	158
หลักการที่ 3: อ่าน 2 ครั้งด้วย AI 2 รุ่น	159
2 รุ่นที่ใช้ (เมื่อใช้ Gemini)	159
กติการวม 2 คำตอบ (mergeReadings)	159
ชื่อเดียวกันใหม่ (similarName)	160
จากเซิร์ฟเวอร์กลับมาทีแอป	160
หลักการที่ 4: ยอดเงินต้องตรงกับตัวเอง	162
ฟังก์ชัน normalizeReading ผังเซิร์ฟเวอร์ทำอะไรบ้าง	162
หลักการที่ 5: รายรับหรือรายจ่าย? ดูจากชื่อบนสลิป	163
เรียนรู้ชื่อของฉันทจากสลิปในเครื่อง	163
ไม่เก็บชื่อจริง เก็บแค่ "ลายนิ้วมือ"	163
ตัวอย่าง: เครื่องที่มีสลิป 6 ใบ	163
ตัดสินใจรายรับ/รายจ่าย (decideDirection)	164

หลักการที่ 6: กันรายการซ้ำ 4 ชั้น	165
ตัวอย่างจริง (รับโค้ดจริง)	165
หลักการที่ 7: วันที่บนสลิปแบบไทย	167
รูปแบบที่อ่านได้ (ผลจริง สมมติวันนี้ 9 ต.ค. 2569)	167
ปี 2 หลัก: พ.ศ. หรือ ค.ศ.?	167
เวลา และสิ่งที่เกิดเมื่ออ่านวันที่ไม่ได้	168
หลักการที่ 8: state machine ของการสแกน	169
บ๊ักใน prototype ที่ออกแบบให้เกิดขึ้นไม่ได้	169
ไล่ตามจริงทีละคำสั่ง (รับโค้ดจริง)	169
หลักการที่ 9: สแกนอัตโนมัติ มองย้อนหลังแค่ไหน	171
ตัวอย่างจริง: เริ่มมองจากเวลาไหน (ตอนนี้ 9 ต.ค. 12:00 น.)	171
เลื่อนเวลา "สแกนล่าสุด" เมื่อไร	171
เมื่อมีปัญหาระหว่างสแกนอัตโนมัติ	172
หลักการที่ 10: ต้นทุน ความเป็นส่วนตัว และความปลอดภัย	173
ด่านป้องกันบนเชิร์ฟเวอร์ (ฟังก์ชัน parse-slip)	173
ต้นทุน	173
อ่านโค้ด: slip.ts ตอนที่ 1 – ชนิดข้อมูลและ QR	175
ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import (บรรทัด 1–20)	175
ส่วนที่ 2: เกล็ด 80% และหน้าต่างของผลอ่าน (บรรทัด 22–57)	176
ส่วนที่ 3: ผู้สมัคร (บรรทัด 59–74)	177
ส่วนที่ 4: parseTlv – แกะรูปแบบ Tag-Length-Value (บรรทัด 80–95)	178
ส่วนที่ 5: crc16 – ตัวเลขตรวจสอบ (บรรทัด 97–107)	179
ส่วนที่ 6: parseSlipQr – QR นี้เป็น QR ตรวจสอบสลิปไหม (บรรทัด 109–134)	180
ส่วนที่ 7: รหัสธนาคาร (บรรทัด 136–168)	181
ส่วนที่ 8: firstSlipQr (บรรทัด 170–177)	182
ส่วนที่ 9: isScreenSized (บรรทัด 179–194)	182
ส่วนที่ 10: ส่วนของรูปที่ตัดมาสแกนใหม่ (บรรทัด 196–217)	183
อ่านโค้ด: slip.ts ตอนที่ 2 – จากผลอ่านสู่การตัดสินใจ	185
ส่วนที่ 1: normalizeRef และ clamp01 (บรรทัด 219–229)	185
ส่วนที่ 2: normalizeReading – หัวใจของ FR-4 (บรรทัด 231–277)	185
ส่วนที่ 3: slipNote – บันทึกของรายการ (บรรทัด 279–303)	187
ส่วนที่ 4: amountChoices – ดึงยอด 2 แบบกลับจากบันทึก (บรรทัด 305–315)	188
ส่วนที่ 5: ผลการตัดสินใจและดัชนีกันซ้ำ (บรรทัด 317–342)	189
ส่วนที่ 6: สร้างและเติมสมุดจำ (บรรทัด 344–367)	190
ส่วนที่ 7: isDuplicate (บรรทัด 369–386)	191
ส่วนที่ 8: classifyCandidate และ initialStatus (บรรทัด 388–405)	191
อ่านโค้ด: slipNames.ts และ myNames.ts – ใครคือ "ฉัน"	193
ส่วนที่ 1: หัวไฟล์ ชนิดข้อมูล และค่าคงที่ (บรรทัด 1–28)	193
ส่วนที่ 2: ลายนิ้วมือและคีย์ของชื่อ (บรรทัด 30–58)	194
ส่วนที่ 3: นับและตัดสินใจ (บรรทัด 60–79)	195
ส่วนที่ 4: decideDirection (บรรทัด 81–119)	196
ส่วนที่ 5: myNames.ts – เก็บสถิติบนมือถือ (บรรทัด 1–42)	197
อ่านโค้ด: dates.ts ส่วนอ่านวันที่บนสลิป	199
ส่วนที่ 1: ชื่อเดือนแบบไม่มีจุด (บรรทัด 159–185)	199
ส่วนที่ 2: normalizeYear และ validDay (บรรทัด 187–214)	199

ส่วนที่ 3: parseSlipDate และ parseSlipTime (บรรทัด 216–261)	200
อ่านโค้ด: scanQueue.ts — state machine ของการสแกน	203
ส่วนที่ 1: คอมเมนต์และชนิดข้อมูล (บรรทัด 1–48)	203
ส่วนที่ 2: คำสั่ง (actions) และสถานะเริ่มต้น (บรรทัด 50–77)	204
ส่วนที่ 3: scanReducer (บรรทัด 79–142)	205
ส่วนที่ 4: ตัวช่วยอ่านสถานะ (บรรทัด 144–173)	207
อ่านโค้ด: autoScan.ts และ prefs.ts	209
ส่วนที่ 1: autoScan.ts ทั้งไฟล์ (บรรทัด 1–45)	209
ส่วนที่ 2: prefs.ts — ตั้งค่าการสแกน (บรรทัด 1–48)	210
อ่านโค้ด: gallery.ts และ gallery.web.ts — ค้นรูปในเครื่อง	212
ส่วนที่ 1: สิทธิ์เข้าถึงรูปภาพบนมือถือ (บรรทัด 1–23)	212
ส่วนที่ 2: ค้นรูปตามเวลา (บรรทัด 25–60)	213
ส่วนที่ 3: เวอร์ชันเว็บ และ "เกลเลอรีทดสอบ" (บรรทัด 1–49)	214
อ่านโค้ด: slips.ts — หา QR ย่อรูป และเรียกตัวอ่าน	216
ส่วนที่ 1: หัวไฟล์และ import (บรรทัด 1–27)	216
ส่วนที่ 2: จำรูปที่เคยตรวจ และจำนวนครั้งที่ล้มเหลว (บรรทัด 29–83)	217
ส่วนที่ 3: detectSlipQr — หา QR บนมือถือ (บรรทัด 85–148)	218
ส่วนที่ 4: prepareImage — ย่อรูปและทำลายนิ้วมือ (บรรทัด 150–160)	220
ส่วนที่ 5: ชนิดของปัญหาและข้อความ (บรรทัด 162–204)	221
ส่วนที่ 6: readSlip และ readSlipOnce — เรียกเซิร์ฟเวอร์ (บรรทัด 206–253)	222
อ่านโค้ด: processSlip.ts — รูป 1 รูป ตั้งแต่ต้นจนบันทึก	224
ส่วนที่ 1: หัวไฟล์และผลลัพธ์ (บรรทัด 1–31)	224
ส่วนที่ 2: ข้อมูลเข้าและอ่าน QR (บรรทัด 33–60)	224
ส่วนที่ 3: ย่อ อ่าน และตัดสินใจ (บรรทัด 62–78)	225
ส่วนที่ 4: บันทึก (บรรทัด 79–108)	226
อ่านโค้ด: useSlipScanner.ts — ตัวควบคุมของหน้าสแกน	228
ส่วนที่ 1: หัวไฟล์ import และค่าคงที่ (บรรทัด 1–38)	228
ส่วนที่ 2: state, ref และ apply (บรรทัด 40–80)	229
ส่วนที่ 3: ใส่รูปเข้าคิว 2 ทาง (บรรทัด 82–128)	230
ส่วนที่ 4: processOne — ทำ 1 รูป (บรรทัด 130–163)	232
ส่วนที่ 5: runLoop — ลูปที่ทำทีละรูปจนหมด (บรรทัด 165–211)	233
ส่วนที่ 6: สิ่งที่ hook ส่งให้หน้าจ่อ (บรรทัด 213–240)	235
อ่านโค้ด: AutoScanProvider.tsx — สแกนอัตโนมัติตอนเปิดแอป	236
ส่วนที่ 1: คอมเมนต์ต่อออกแบบ และค่าคงที่ (บรรทัด 1–51)	236
ส่วนที่ 2: ชนิดข้อมูล และ Context (บรรทัด 53–87)	237
ส่วนที่ 3: Provider เริ่มต้น และการสลับบัญชี (บรรทัด 89–121)	238
ส่วนที่ 4: run — เริ่มรอบ และเลือกรูป (บรรทัด 123–158)	239
ส่วนที่ 5: ลูปทีละรูป และการจัดการปัญหา (บรรทัด 154–225)	240
ส่วนที่ 6: จบรอบ (บรรทัด 227–250)	243
ส่วนที่ 7: maybeRun — ควรสแกนตอนนี้ไหม (บรรทัด 252–287)	244
ส่วนที่ 8: คำสั่งที่ให้หน้าจ่อ (บรรทัด 289–321)	245
อ่านโค้ด: scan.tsx — หน้าสแกนสลิป	247
ส่วนที่ 1: คอมเมนต์และ import (บรรทัด 1–37)	247
ส่วนที่ 2: state และการเริ่มค้นอัตโนมัติ (บรรทัด 38–61)	248
ส่วนที่ 3: ข้อมูลที่คำนวณจากผลของรอบนี้ (บรรทัด 63–90)	249

ส่วนที่ 4: หัวหน้าจอ โหมดทดลอง และการ์ดของเว็บ (บรรทัด 92–118)	250
ส่วนที่ 5: สถานะของมือถือ และข้อความปัญหา (บรรทัด 120–166)	251
ส่วนที่ 6: การ์ดความคืบหน้า (บรรทัด 168–218)	252
ส่วนที่ 7: ผลตามช่วงและตามวัน และกล่องความเป็นส่วนตัว (บรรทัด 220–259)	254
อ่านโค้ด: drafts.tsx และส่วนตรวจสอบใน transaction.tsx	256
ส่วนที่ 1: drafts.tsx ตรรกะ (บรรทัด 1–42)	256
ส่วนที่ 2: drafts.tsx หน้าจอ (บรรทัด 44–96)	257
ส่วนที่ 3: transaction.tsx เตรียมข้อมูลของแบบร่าง (บรรทัด 56–60 และ 157)	258
ส่วนที่ 4: transaction.tsx กล่อง "ช่วยตรวจสอบ" (บรรทัด 194–230)	259
ส่วนที่ 5: transaction.tsx ปุ่มยืนยัน (บรรทัด 341–350 และ 106–111)	260
ส่วนที่ 6: TxRow แสดงธงเป็นป้าย (บรรทัด 57–72)	261
อ่านโค้ด: AutoScanBanner, ScannerStage และ WaitNotice	262
ส่วนที่ 1: AutoScanBanner ระหว่างสแกน (บรรทัด 1–48)	262
ส่วนที่ 2: ขอสถานะ และเมื่อมีปัญหา (บรรทัด 50–87)	263
ส่วนที่ 3: เมื่อเสร็จ (บรรทัด 89–138)	264
ส่วนที่ 4: ScannerStage เวทัสแกน (บรรทัด 1–71)	265
ส่วนที่ 5: WaitNotice นับถอยหลัง (บรรทัด 1–21)	267
อ่านโค้ด: parse-slip/index.ts — ฟังก์ชันอ่านสลิปบนเชิร์ฟเวอร์	269
ส่วนที่ 1: คอมเมนต์ออกแบบ import และขีดจำกัด (บรรทัด 1–23)	269
ส่วนที่ 2: SCHEMA — รูปแบบคำตอบที่บังคับ AI (บรรทัด 25–57)	270
ส่วนที่ 3: SYSTEM — คำสั่งให้ AI (บรรทัด 59–79)	271
ส่วนที่ 4: readOnce — อ่าน 1 รอบ (บรรทัด 81–94)	272
ส่วนที่ 5: ตัวจัดการคำขอ (บรรทัด 96–154)	273
อ่านโค้ด: helpers.ts — เลือกรุ่น AI ทำความสะอาด และรวม 2 ผล	276
ส่วนที่ 1: รุ่น AI และการตัดสินใจเมื่อเรียกไม่สำเร็จ (บรรทัด 47–99)	276
ส่วนที่ 2: อ่านข้อความตอบของ AI (บรรทัด 101–122)	277
ส่วนที่ 3: SlipReadingOut และตัวช่วยทำความสะอาด (บรรทัด 136–194)	278
ส่วนที่ 4: normalizeReading ฝั่งเชิร์ฟเวอร์ (บรรทัด 196–253)	279
ส่วนที่ 5: ค่าคงที่และตัวช่วยเทียบ (บรรทัด 255–341)	281
ส่วนที่ 6: mergeReadings และ mergeTwo (บรรทัด 343–441)	283
อ่านโค้ด: callGemini ใน common.ts — เรียก Gemini อย่างทันท่วงที	286
การทดสอบ FR-4	289
Unit test ทั้งหมดของ FR-4	289
ตัวอย่างเทสต์ 1: TC-23 กันซ้ำ	289
ตัวอย่างเทสต์ 2: TC-63 รวม 2 ผลอ่าน	290
E2E: เบราวเซอร์จริง เชิร์ฟเวอร์จำลอง (e2e/web-auth.mjs)	291
MT: ทดสอบด้วยมือบนมือถือจริง (docs/TRACEABILITY.md)	292
FR-4 เชื่อมกับ FR อื่นยัง	293
ทำไมฉันออกแบบแบบนี้	294
ข้อจำกัด และสิ่งที่จะทำต่อ	295
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	296
แบบฝึกหัดเช็คความเข้าใจ	298
เฉลย	298

สคริปต์นำเสนอ 1 นาที และคำศัพท์	299
คำศัพท์ของ FR-4	299

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คอยกันยังไง

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไง
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

- นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ
- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
 - **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
 - **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

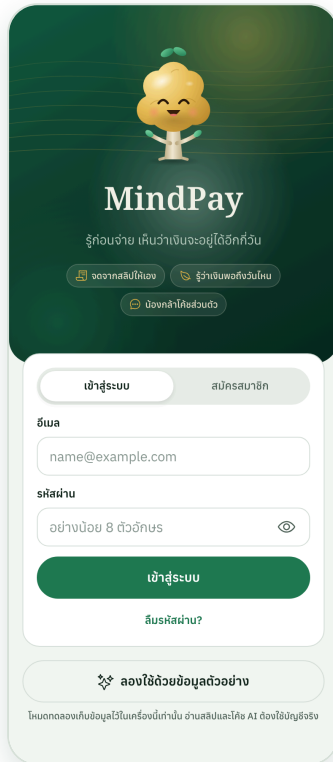
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ทองบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขียบบนหน้าหลัก

★ จำให้ได้เลย: ทุก FR ต่อกันเป็นสายเดี่ยว

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จุดเอง พุดจกด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนพัง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดตามไปด้วย

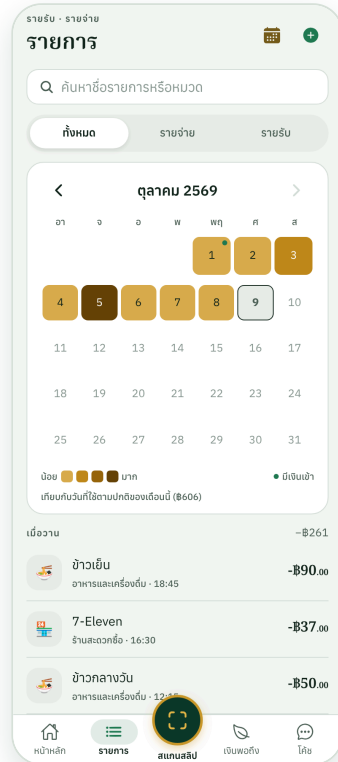
หน้าตาแอปโดยรวม



หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"



หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), น้องกล้วย พุดสรุป



แท็บรายการ (FR-1): รายการทั้งหมด ค้นหา กรองรายรับ/รายจ่าย



สแกนสลิป (FR-4): อ่านสลิปแล้ว
สรุปตามช่วงเวลา

พุดจดยอด (FR-3): พุดประโยคเดียว ได้
หลายรายการ

เงินพอถึง (FR-6): 16 วัน ถึง
ประมาณ 25 ค.ศ.

ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน `src/app/(tabs)/_layout.tsx` ซึ่งจะอ่านทีละบรรทัดใน un A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไ้	ต่างกันยังไ้
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแคเลเลอร์ ทั้งหมดเอง (บนเว็บต้องกดเลือกรูป สลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest -apk	สแกนแคเลเลอร์อัตโนมัติ พุดจดยอดด้วยไมค์ น้องกล้าพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและไอซ์ได้
- **โหมดทดลอง** แตะ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียวไม่ต้องสมัคร ใช้ตอนฟรีเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700

บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับ
เทสต์)

157

unit test ผ่านทั้งหมด

145

ข้อตรวจในเบราร์เซอร์ (E2E) ผ่านทั้งหมด

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไมค์ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสต์ในเบราร์เซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอ)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสต์ (ไฟล์เทสต์อีกประมาณ 2,000 บรรทัด)

≈ เปรียบแอปเป็นร้านอาหาร

หน้าจอ (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโถงกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บทต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ใช้ตรวจ ยังไม่นับในยอดเงิน
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล้า	มาสคอตของแอป เป็นต้นกล้าจากต้นไม้เงิน และเป็นโค้ช AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงผลหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ต (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขได้เฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วคือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)

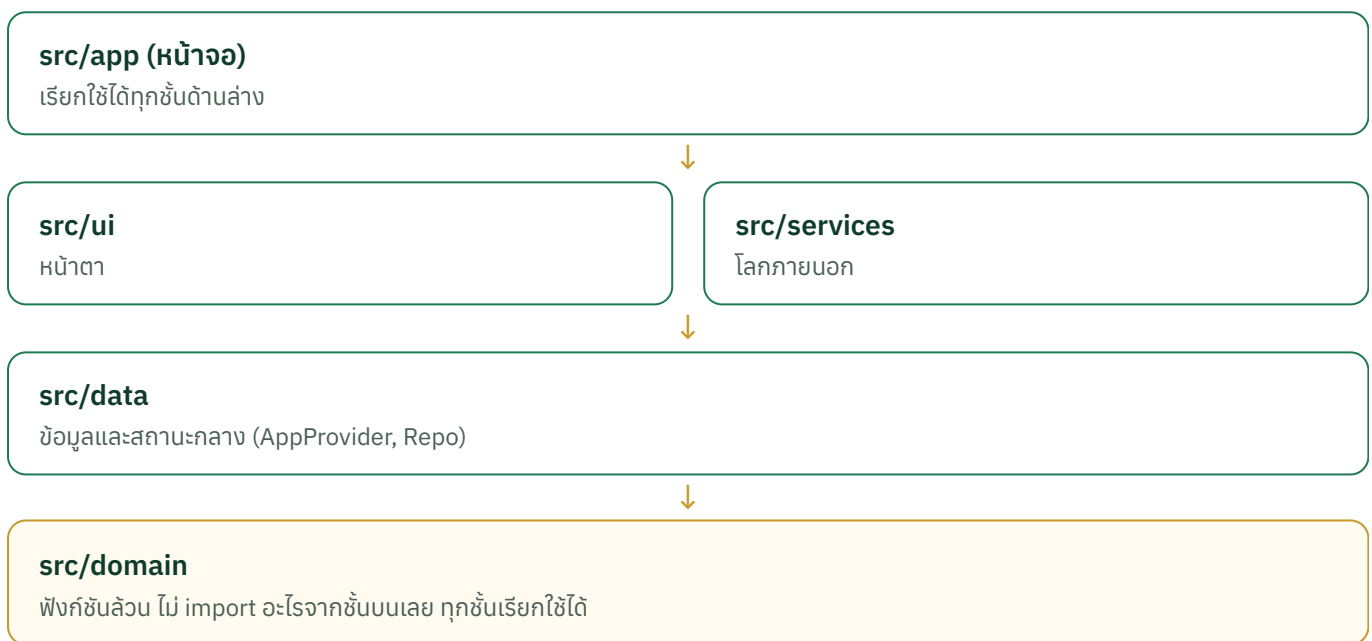
1. เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
2. เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
3. สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
4. กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	src/app/transaction.tsx หน้าจอดรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	src/ui/components.tsx , src/ui/theme.ts

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
กฎและการคำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แกลเลอรี่รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แค่นี้เดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน src/domain ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามมันไ้฿85" แล้วกด "บันทึกรายการ"

1. หน้าจอดรายการ [src/app/transaction.tsx](#)

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง [src/data/AppProvider.tsx](#)

`addTx` ส่งต่อให้ Repo บันทึก รวบรวมกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTx`)



3. Repo [src/data/repo.ts](#)

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase [supabase/migrations/20260927000000_init.sql](#)

ตรวจกฎ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน [src/data/AppProvider.tsx](#)

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละชั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนีดูแค่ว่ามันต่อกันยังไง)

ชั้น 1: หน้าจอดรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
```



```

113     await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ต)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนให้ช่องและหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหนวนรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการจดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385     async addTx(input) {
386         const r = need();
387         const tx = await r.insert(input);
388         if (repoRef.current !== r) return tx;
389         noteLocal(tx.id, tx);
390         setTxs((list) => [tx, ...list]);
391         return tx;
392     },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยาบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need () จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้ลบบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีการเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้ากำลังโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxS ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts	บรรทัด 217–222
<pre> 217 async insert(input) { 218 const { data, error } = await db.from('transactions').insert(toRow(input)).select().single(); 219 if (isUniqueViolation(error)) throw new DuplicateSlipError(); 220 if (error) throw error; 221 return fromRow(data as TxRow); 222 }, </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217	

บรรทัด	ความหมาย
218	อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
219	ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
220	error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
221	แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx	บรรทัด 442–459
<pre> 442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */ 443 export function useMoney() { 444 const { profile, txs, carrySatang, goals, today } = useApp(); 445 return useMemo(() => { 446 const opening = profile?.openingBalanceSatang ?? 0; 447 const floor = profile?.runwayFloorSatang ?? 50_000; 448 // Money from before the loaded window (over ~13 months ago) still counts. 449 const balance = computeBalance(opening + carrySatang, txs); 450 const average = averageDailyExpense(txs); 451 // Money in savings goals is set aside: it does not count as money to spend. 452 const reserved = reservedSatang(goals); 453 const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang: average.averageSatang }); 454 const drafts = txs.filter((t) => t.status === 'draft'); 455 return { balance, average, runway, drafts, reserved, floor }; </pre>	

```

456 // `today` makes the runway and averages start over at midnight.
457 // eslint-disable-next-line react-hooks/exhaustive-deps
458 }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
443	useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
444	ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกลูก และวันนี้
445	useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
446–447	ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
449	คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
450–453	คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุก และ Money Runway (FR-6)
454	หารายการแบบร่างที่รอตรวจ (FR-4)
455	ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
458	รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่งเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จากรายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลบอกอย่างหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ [src/data/AppProvider.tsx](#) เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มือถือหารูปใหม่ในแคเลเลอรี่ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปไหนไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจว่าเข้าสู่ระบบจริง ตรวจโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ตรวจ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ตรวจ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขึ้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รันแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรับภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รัน SQL ใส่กุญแจ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

- VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows คลิก "Add to PATH" และ "Open with Code" ไว้ด้วย)
- Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ `node -v` ต้องขึ้น v22 หรือสูงกว่า
- Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย `git --version`
- บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไข (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
- Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminalคำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน package.json แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ node_modules (ประมาณ 1-3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น .env ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ copy .env.example .env)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

.env.exampleบรรทัด 1-6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → .env.example:1

บรรทัด	ความหมาย
1-4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บท A9) ชื่อที่ขึ้นต้นด้วย EXPO_PUBLIC_ คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งทำอะไรบ้าง

package.jsonบรรทัด 1-59

```

1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
17    "expo-haptics": "~57.0.3",
18    "expo-image-manipulator": "~57.0.20",
19    "expo-image-picker": "~57.0.20",
20    "expo-linear-gradient": "~57.0.2",
21    "expo-linking": "~57.0.11",
22    "expo-media-library": "~57.0.5",
23    "expo-router": "~57.0.23",
24    "expo-speech": "~57.0.3",
25    "expo-speech-recognition": "^57.1.0",
26    "expo-splash-screen": "~57.0.9",
27    "expo-sqlite": "~57.0.3",
28    "expo-status-bar": "~57.0.1",
29    "expo-system-ui": "~57.0.4",
30    "expo-updates": "~57.0.23",
31    "react": "19.2.3",
32    "react-dom": "19.2.3",
33    "react-native": "0.86.3",
34    "react-native-safe-area-context": "~5.7.0",
35    "react-native-screens": "~4.26.0",
36    "react-native-svg": "15.15.4",
37    "react-native-web": "~0.21.0"
38  },
39  "devDependencies": {
40    "@types/react": "~19.2.2",
41    "eslint": "^9.0.0",
42    "eslint-config-expo": "~57.0.2",
43    "playwright": "^1.56.0",
44    "typescript": "~6.0.3",
45    "vitest": "^5.0.2"
46  },
47  "scripts": {
48    "start": "expo start",
49    "android": "expo start --android",
50    "ios": "expo start --ios",
51    "web": "expo start --web",
52    "test": "vitest run",
53    "typecheck": "tsc --noEmit",
54    "lint": "expo lint",
55    "e2e:web": "EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist && EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56    "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-video dark android && node e2e/demo-video.mjs dist demo-video light web"
57  },

```

```
58 "private": true
59 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → package.json:1

บรรทัด	ความหมาย
2–3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5–38	dependencies แพ็กเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6–7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลอรี่ทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (บท A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขอมจ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าวกราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้ลง: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)

บรรทัด	ความหมาย
47-57	scripts คำสั่งลัด พิมพ์ <code>npm run</code> ชื่อ (หรือ <code>npm test</code> , <code>npm start</code>) แทนคำสั่งยาว ๆ ทางขวา
58	<code>private</code> = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
<code>npm run web</code>	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก้โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
<code>npx expo start</code>	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
<code>npm test</code>	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
<code>npm run typecheck</code>	ให้ TypeScript ตรวจสอบข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
<code>npx expo lint</code>	ตรวจสอบรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
<code>npm run e2e:web</code>	build เว็บแล้วรันเทสต์ในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำ หรือค้าง = รอได้ หรือลบโฟลเดอร์ `node_modules` แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ `.env` (บรรทัด 4) · ห้ามใช้ `npm install` ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ `npx expo install` ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์ คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น <code>computeRunway</code> จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Terminal	ด้านล่าง (Ctrl+` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แถบข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบล่างสุด	บอก branch ของ Git, บรรทัด/คอมมิตที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อท้ายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หากทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเมนต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าจอหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด Ctrl+Shift+X แล้วค้นชื่อ กด Install

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ <code>npm run lint</code>
Expo Tools	ช่วยเติมคำและตรวจไฟล์ <code>app.json</code> / <code>eas.json</code>
Vitest	มีปุ่ม ▶ ข้างเกสต์แต่ละข้อ กดรันทีละข้อได้
GitLens (ไม่บังคับ)	ขีดบรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน commit อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ท้ายบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (`node_modules/typescript`) ตามไฟล์ `tsconfig.json`

ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แตะชื่อไฟล์บนหัวโค้ดก้อนไหนก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (Ctrl+Shift+F) กด F12 ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน `npm test` หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. Ctrl+P พิมพ์ `index.tsx` เลือกไฟล์ใน `src/app/` (tabs) (หน้าจอหลัก)
2. Ctrl+F ค้น `useMoney` แล้วกด F12 ที่คำนั้น จะแดงไป `src/data/AppProvider.tsx` บรรทัด 443
3. ที่บรรทัด 453 คลิก `computeRunway` แล้วกด F12 จะไปที่ `src/domain/runway.ts` บรรทัด 72 ซึ่งเป็นสูตรของ FR-6
4. กด Shift+F12 ที่ชื่อฟังก์ชัน จะเห็นว่ามีเกสต์เรียกใช้ใน `src/domain/__tests__/domain.test.ts` ด้วย
5. กด Alt+← สองครั้ง ย้อนกลับไปหน้าจอหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (บท A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React อยู่นะ (บท A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000;           // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0;                  // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500;           // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- `const` ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ `let` เฉพาะตอนต้องเปลี่ยนจริง ๆ
- `50_000` คือ 50000 เครื่องหมาย `_` ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง `//` คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน `/* ... */` และ `/** ... */` เป็นคอมเมนต์หลายบรรทัด (แบบ `/**` VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ **camelCase**: คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น `balanceSatang`, `averageDailyExpense` คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น `RUNWAY_CAP_DAYS`

★ กฎการตั้งชื่อของทิมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากับ / ไม่เท่ากับ (แบบเคร่ง ชนิดต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'ลดรายการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่ขึ้นใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก `src/data/AppProvider.tsx:447` ซึ่งใช้ทั้ง ?. และ ??

`src/data/AppProvider.tsx`

บรรทัด 447-447

```
447 const floor = profile?.runwayFloorSatang ?? 50_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:447`

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เงื่อนไข if: ถ้า... ให้ทำ...

`if / else`

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 if (spendable <= 0) {
2   // ทำส่วนนี้เมื่อเงื่อนไขจริง
3 } else if (averageSatang <= 0) {
4   // ไม่จริง ถ้าเงื่อนไขนี้จริง ทำส่วนนี้
5 } else {
6   // ไม่ตรงอะไรเลย ทำส่วนนี้
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

src/domain/runway.ts

บรรทัด 83-88

```
83  if (spendable <= 0) {
84    return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85  }
86  if (averageSatang <= 0) {
87    return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88  }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:83

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด่านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ พารามิเตอร์ (ของที่ส่งเข้า) และ return ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1  // แบบที่ 1: function
2  function toBaht(satang: number): number {
3    return satang / 100;
4  }
5
6  // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกัน เขียนสั้นกว่า
7  const toBaht2 = (satang: number) => satang / 100;
8
9  toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไปใช้ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปแคะอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] ค้นด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0];           // 12000 (ตัวแรก)
3 amounts.length;       // 4 (มีกี่ตัว)
4 amounts.includes(0);   // true

```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกแต่ละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยัง (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```

9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คัดเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คำนวณเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว

บรรทัด	ความหมาย
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชั้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
<code>const { a, b } = obj;</code>	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น <code>const { balance, runway } = useMoney();</code>
<code>{ ...obj, status: 'confirmed' }</code>	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
<code>[tx, ...list]</code>	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ในบท A2)
<code>{ kind, title }</code>	shorthand	ย่อจาก <code>{ kind: kind, title: title }</code>

! ทำไมโค้ดชอบสร้าง object ใหม่แทนการแก้ไขเดิม

React (un A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ไขใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอก็ไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วงซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;       // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++)
ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดู un A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
Math.floor(20.69)	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
Math.round(680.2857)	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
Math.max(0, -500)	0 (ตัวที่มากกว่า)	กันเงินที่ใช้ได้ติดลบ
Math.min(7, 3)	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
Math.abs(-85)	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า `profile` เป็น null ค่าของ `profile?.runwayFloorSatang ?? 50_000` คืออะไร
2. `[100, 200, 300].filter((x) => x > 150).map((x) => x / 100)` ได้อะไร
3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 `computeBalance` คืนค่าสตางค์
4. `const t2 = { ...t, title: 'ข้าวผัด' }` แล้ว `t.title` เปลี่ยนไหม

i เฉลย

- 1) 50000 · 2) `[2, 3]` (กรองเหลือ 200, 300 แล้วหาร 100) · 3) $50,000 + 100,000 - 30,000 = 120,000$ สตางค์ (แบบร่างไม่นับ) · 4) ไม่เปลี่ยน `t2` เป็น object ใหม่ `t` ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatnag` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเพี้ยนตอนผู้ใช้งานใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด ("`strict`": `true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ตักกลับทันทีที่โต๊ะ ตักว่าไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกลบออก เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น number เอาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์นี้สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
```

```

11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
22   note: string | null;
23   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24   occurredAt: string;
25   source: TxSource;
26   status: TxStatus;
27   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28   slipRef: string | null;
29   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30   slipImageHash: string | null;
31   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32   ocrConfidence: number | null;
33   /** Fields the slip reader was unsure about, shown to the user during review. */
34   reviewFlags: string[];
35   createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3
9	union type: ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที

บรรทัด	ความหมาย
11-12	สถานะของรายการ: 'draft' (แบบร่าง ยังไม่นับ) หรือ 'confirmed' (ยืนยันแล้ว นับในยอด) คอมเมนต์อ้างถึง Charter ของทีมว่าผลจากสลิปเป็นแบบร่างก่อน
14	รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) นับเป็น manual
16	interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร"
17	id รหัสไม่ซ้ำของรายการ (ฐานข้อมูลสร้างให้)
18	kind ใช้ชนิด TxKind จากบรรทัด 9
19	amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind)
20	categoryKey รหัสหมวด เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts)
21-22	ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้)
23-24	เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย)
25-26	ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12
27-30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31-32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จดเองเป็น null
33-34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้นำมาตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38-39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทนโคช 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล่าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43-53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เงินสำรองของ FR-6 (บรรทัด 49), จบต่อเดือน (ไม่ตั้งก็ได้ = null), โทนโคช และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใส่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "ชื่อฉัน x เป็นชนิด Y" (ใช้น้อย ๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โค้ดระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';
10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด	ความหมาย
6	นำ Storage มาจากไฟล์ ./storage (จุด = โฟลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์)
7	นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules
9-13	../domain/... = ถอยออกไป 1 โฟลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้
14	import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป)

11 type GoalInput ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว

ฝั่งไฟล์ต้นทางต้องเขียน export หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน export default ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจออกจาก default export)

งานที่ต้องรอ: async, await และ Promise

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครั้งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 async function save() {           // async = ฟังก์ชันนี้มีงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอตรงนี้ได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }
```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ `Promise.all` ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออม พร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131-133

```
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
      r.listGoals().catch(() => [] as SavingsGoal[])]);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:131

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แะใส่ตัวแปร p, list, g ส่วน `.catch(() => [])` ถ้างานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ ใน Edge Function ก็ใช้ `Promise.allSettled` อ่านสลิป 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: try, catch, throw

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 try {
2   await addTx(input);           // ลองทำ
3   toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) {                   // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5   toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally {                     // ทำเสมอ ไม่ว่าสำเร็จหรือพลาด
7   setBusy(false);
8 }
```

`throw` คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น `DuplicateSlipError` (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

class ... extends Error = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา super(...) ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอลูกได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2      → 0.30000000000000004
19.99 * 100    → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกหลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผลบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ amount_satang bigint)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน parseBahtToSatang ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กรหัสจ่ายของ FR-6, ระบุยอด) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value).
6  */
7 export const MAX_SATANG = 100_000_000_000;
8
9 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
10 export function toAsciiDigits(s: string): string {
11   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
12 }
13
14 /**
15  * Parse what a user (or a slip) wrote into satang.
16  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
17  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
18  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
19  * Returns null for anything that is not a positive amount with at most 2 decimals, or above
20  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).

```



```

21 export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?:
    boolean } = {}): number | null {
22   if (input === null || input === undefined) return null;
23   const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero &&
    satang === 0) ? satang : null);
24   if (typeof input === 'number') {
25     if (!Number.isFinite(input) || input < 0) return null;
26     return ok(Math.round(input * 100));
27   }
28   let cleaned = toAsciiDigits(input)
29     .replace(/฿|บาท|THB|baht/gi, '')
30     .replace(/\s/g, '');
31   if (/^\d+,\d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32   else if (cleaned.includes(',')) {
33     if (!/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/.test(cleaned)) return null;
34     cleaned = cleaned.replace(/,/g, '.');
35   }
36   if (!/^\d+(\.\d{0,2})?$/.test(cleaned)) return null;
37   const [whole, frac = ''] = cleaned.split('.');
38   if (whole.length > 12) return null;
39   return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:1`

บรรทัด	ความหมาย
5–6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกัน กับค่าที่อ่านผิดใหญ่เกินจริง)
8–11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0E50) จะได้ 0–9
13–20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีมือถือบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ถูกต้อง
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24–27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา $19.99 \times 100 = 1998.999\dots$)
28–30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1–2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32–35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่ขึ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่ขึ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง

บรรทัด	ความหมาย
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท × 100 + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์ชันจริง

ผู้ใช้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" → 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรรทัด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แค่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่มีก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙

สัญลักษณ์	แปลว่า
g, i (ท้าย /)	g = หากจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1-3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1-2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตริงกลับเป็นข้อความบาท

src/domain/money.ts

บรรทัด 42-63

```

42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
45   const negative = satang < 0;
46   const abs = Math.abs(Math.round(satang));
47   // Whole baht only: round to the nearest baht (#99.99 reads #100, not #99).
48   const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49   const frac = abs % 100;
50   const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51   const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52   return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "#1,234.50"; with sign: "+#1,234.50" / "-#1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}฿${body}`;
62   return `${satang < 0 ? '-' : ''}฿${body}`;
63 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:42

บรรทัด	ความหมาย
42-43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45-46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47-48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ #99.99 จึงแสดงเป็น #100 ไม่ใช่ #99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมาคั่นหลักพันด้วย regex (หาตำแหน่งที่ด้านหลังมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51-52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55-63	formatBaht เพิ่ม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i **ผลลัพธ์**

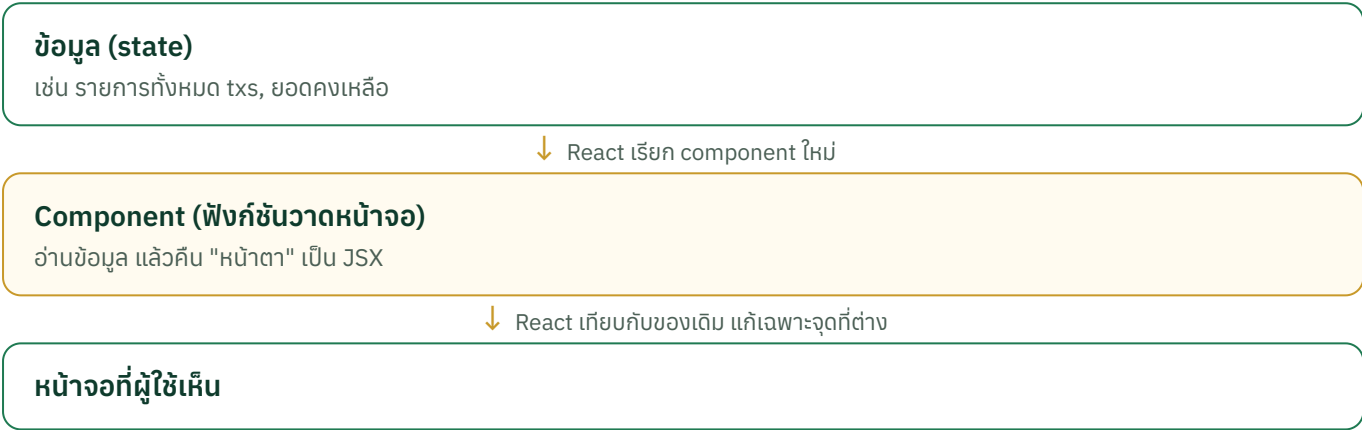
```
formatBaht(123450) → "฿1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·  
formatBaht(9999, { decimals: false }) → "฿100"
```

React: วาดหน้าจจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจก็ขัดกันเอง React เปลี่ยนวิธีคิด: เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจหน้าตาแบบนี้" แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน



Component: ชิ้นส่วนหน้าจที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {
2   return <Text>สวัสดี {name}</Text>;
3 }
4
5 // เรียกใช้ที่อื่น:
6 <Hello name="มินท์" />    // แสดง "สวัสดี มินท์"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แท็กเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แกะเอาช่อง name ออกมา
<Hello name="มินท์" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

```

36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
40   h1: fonts.serif,
41   h2: fonts.sansBold,
42   h3: fonts.sansSemi,
43   body: fonts.sans,
44   small: fonts.sans,
45   micro: fonts.sansMedium,
46   label: fonts.sansSemi,
47 };
48
49 export function T({
50   v = 'body',
51   color,
52   style,
53   children,
54   numberOfLines,
55   center,
56   selectable,
57   accessibilityRole,
58 }: {
59   v?: Variant;
60   color?: string;
61   style?: StyleProp<TextStyle>;
62   children: ReactNode;
63   numberOfLines?: number;
64   center?: boolean;
65   selectable?: boolean;
66   accessibilityRole?: 'header' | 'text' | 'alert';
67 }) {
68   const theme = useTheme();
69   const size = v === 'label' ? type.micro : type[v];
70   return (
71     <Text
72       accessibilityRole={accessibilityRole}
73       numberOfLines={numberOfLines}
74       selectable={selectable}
75       style={[
76         size,
77         {
78           fontFamily: VARIANT_FONT[v],
79           color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80           textAlign: center ? 'center' : undefined,
81         },
82         // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below
4.5:1).
83         v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ??
theme.inkSoft },
84         style,
85       ]}
86     >
87       {children}
88     </Text>
89   );

```

บรรทัด	ความหมาย
36	ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
38–47	ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
49–57	ประกาศ component ชื่อ T พร้อมแกะ props ออกมา v = 'body' แปลว่าถ้าไม่ระบุระดับ ใช้ body
58–67	ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole บอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน
68	เรียก useTheme () เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ป่าทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จันข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: <T v="h3">จดรายการ</T> หรือ <T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอลoadใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ useState

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount    = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
```

```

36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก้ไขรายการเดิม ใช้ประเภทเดิม ไม่จําเป็นดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จําเป็นเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) useState<TxKind> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจํานวนเงิน เก็บเป็น ข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก้ไขรายการเดิม เติมค่าเดิมให้
38–39	ชื่อรายการ และหมวด
40	จํานวนที่ผู้ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเดาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จํา state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จําค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจําผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney () คำนวณยอดเงินและ Runway

Hook	ทำอะไร	ตัวอย่างใน MindPay
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use เอง	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })
204     .catch(() => {});
205   const sub = AppState.addEventListener('change', (s) => {
206     if (s === 'active') check();
207   });
208   return () => {
209     clearTimeout(timer);
210     sub.remove();
211   };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array วางท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา

บรรทัด	ความหมาย
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกล่าวกักตายนและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกนาฬิกาเดิม แล้วตั้งใหม่ให้ปลุกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ) msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งล่าสุดจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่หน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แสร้งข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบคลุมทั้งแอปไว้ใน `src/app/_layout.tsx` (un A7) ตัวสำคัญที่สุดคือ **AppProvider**

`src/data/AppProvider.tsx`

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the
96     companion). */
97 export function useAppMaybe() {
98   return useContext(AppContext);
99 }
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);

```

```

113 const todayRef = useRef(today);
114 const repoRef = useRef<Repo | null>(null);
115 /** The account whose data is loaded (or being loaded): prevents loading the same account
    twice. */
116 const activeUserRef = useRef<string | null>(null);
117 const handledLinks = useRef(new Set<string>());
118 /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119 const loadGen = useRef(0);
120 /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121 const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หับข้อมูลกลาง ถ้าเรียกนอก Provider จะ error กันที (กันลืมหวน)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คลาวด์หรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โปรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกลูก
108–110	ข้อความ error ตอนโหลด, คำสั่งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	รวันใหม่
113–121	ค่าที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme();           // สีของธีม
3   const { txs, addTx } = useApp();     // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6
7   return (
8     <Screen>

```

```

9      <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้อีก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุปบท A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าจอ (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (un A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสั้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (bussกั 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนกับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)

style	ความหมาย
gap: 8	ระยะห่างระหว่างลูกแต่ละตัว
padding / margin	ระยะห่างด้านใน / ด้านนอก
borderRadius	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

src/ui/components.tsx

บรรทัด 189–203

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }: {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap },
203     style] >{children}</View>;
204 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

บรรทัด 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวตั้ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่ามากมาย **Expo** ห่อทุกอย่างไว้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที่ Expo ให้กับเรา:

- **Expo Go + dev server:** แก้โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หัวข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ที่เราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน src/app/ ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
_layout.tsx	-	โครงทั้งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
welcome.tsx	/welcome	สมัคร เข้าสู่ระบบ ลืมรหัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
onboarding.tsx	/onboarding	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, งบ)	FR-2, FR-6
(tabs)/_layout.tsx	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
(tabs)/index.tsx	/	หน้าหลัก	FR-2 (+FR-6)
(tabs)/transactions.tsx	/transactions	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
(tabs)/runway.tsx	/runway	แท็บเงินพอถึง	FR-6
(tabs)/coach.tsx	/coach	แท็บโค้ชน้องกล้า	FR-5
transaction.tsx	/transaction	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
voice.tsx	/voice	พูดจด	FR-3
scan.tsx	/scan	สแกนสลิป	FR-4
drafts.tsx	/drafts	สลิปรอยืนยัน	FR-4
settings.tsx	/settings	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
goals.tsx, goal.tsx	/goals, /goal	กระปุกออม	FR-6
recap.tsx	/recap	สรุปเดือนแบบเรื่องเล่า	FR-2
achievements.tsx, skins.tsx	/achievements, /skins	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
whatsnew.tsx	/whatsnew	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น (tabs) เป็น "กลุ่ม" ไม่โฟลในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- index.tsx คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ /)
- ไฟล์ _layout.tsx ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
<code>router.push('/scan')</code>	เปิดหน้าจอสแกนซ้อนขึ้นมา (กดกลับได้)
<code>router.push({ pathname: '/transaction', params: { kind: 'income' } })</code>	เปิดหน้าจอรายการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
<code>useLocalSearchParams()</code>	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
<code>router.navigate('/runway')</code>	ไปที่เก็บเงินพอถึง
<code>goBack()</code>	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: `_layout.tsx` โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

`src/app/_layout.tsx`

บรรทัด 1-51

```
1 import { useFonts } from 'expo-font';
2 import { Stack } from 'expo-router';
3 import * as SplashScreen from 'expo-splash-screen';
4 import { StatusBar } from 'expo-status-bar';
5 import { useCallback, useState, type ReactNode } from 'react';
6 import { Platform, View } from 'react-native';
7 import { SafeAreaProvider } from 'react-native-safe-area-context';
8 import {
9   Anuphan_400Regular,
10  Anuphan_500Medium,
11  Anuphan_600SemiBold,
12  Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
```



```

35   });
36   return (
37     <SafeAreaProvider>
38       <WebFrame>
39         <CelebrationProvider>
40           <AppProvider>
41             <ToastProvider>
42               <AutoScanProvider>
43                 <Navigator ready={fontsLoaded || !!fontError} />
44               </AutoScanProvider>
45             </ToastProvider>
46           </AppProvider>
47         </CelebrationProvider>
48       </WebFrame>
49     </SafeAreaProvider>
50   );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด	ความหมาย
1–22	นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
24–25	สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
27	export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
28–35	โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
36–50	ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลูลอง → ข้อมูลกลาง (AppProvider) → ข้อความแจ้งเตือน (toast) → สแกนสลิปอัตโนมัติ (FR-4) → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
43	ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังกี่ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53–73

```

53  /** On a computer browser, show the app at phone width in the middle of the screen. */
54  function WebFrame({ children }: { children: ReactNode }) {
55    const theme = useTheme();
56    if (Platform.OS !== 'web') return <>{children}</>;
57    return (
58      <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg,
59        '#000000', 0.55) : theme.surfaceAlt }}>
60        <View
61          style={{
62            flex: 1,
63            width: '100%',
64            maxWidth: 440,
65            backgroundColor: theme.bg,
66            overflow: 'hidden',
67            boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
68          }}
69        >
70          {children}
71        </View>
72      </View>
73    );
74  }

```

```

70     </View>
71   </View>
72 );
73 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

53–54 WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือไว้ตรงกลางจอ

56 ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร ค้นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)

58 พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด

59–68 กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <>
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp=
{inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>
104           <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105         </Stack.Protected>
106         <Stack.Protected guard={needsOnboarding}>
107           <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108         </Stack.Protected>
109         <Stack.Protected guard={inApp}>
110           <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />

```

```

111         <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112         <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113         <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114         <Stack.Screen name="achievements" />
115         <Stack.Screen name="skins" />
116         <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117         <Stack.Screen name="goals" />
118         <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119         <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120         <Stack.Screen name="drafts" />
121         <Stack.Screen name="settings" />
122     </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo ใหม่ โปรไฟล์ตั้งค่าเสร็จไหม และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว้างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้าไฟล์) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่กับหน้าเก่า กดกลับก็หิบบนหน้าออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, พื้นตามืด, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตุมียาม: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พุดจด, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ ผลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center',
137       justifyContent: 'center' }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3}
139         color={theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         />
166     </View>
167   </View>
```

```

164         <Ionicons name="scan" size={26} color={theme.heroAccent} />
165     </Pressable>
166 </Animated.View>
167 </View>
168     <Text style={[type.micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4
    ]}]>สแกนสลิป</Text>
169 </View>
170     {routes.slice(half).map((r, i) => tab(r, i + half))}
171 </View>
172 );
173 }
174
175 export default function TabsLayout() {
176 // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177 useKlaSync();
178 return (
179     <Tabs tabBar={props} => <TabBar {...props} /> screenOptions={{ headerShown: false }}>
180         <Tabs.Screen name="index" />
181         <Tabs.Screen name="transactions" />
182         <Tabs.Screen name="runway" />
183         <Tabs.Screen name="coach" />
184     </Tabs>
185 );
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด	ความหมาย
134	วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย
135–169	ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระเพื่อม (PulseRing) 3 รอบ ตอนเปิดแอป และกระเพื่อมตลอดระหว่างสแกนอัตโนมัติ
144–147	แตะแล้วมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน
170	วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา
175–186	component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แทนของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้บันทึกการกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app
  settings and demo data.
3  * Phone: SQLite-backed store (survives
  app restarts).
4  * Web: see storage.web.ts (browser
  localStorage).

```

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the
  browser's localStorage, wrapped so it
  never throws. */
2 export const Storage = {
3   async getItem(key: string):
    Promise<string | null> {
4     try {

```

```

5  */
6  import KV from 'expo-sqlite/kv-store';
7
8  export const Storage = {
9    getItem: (key: string) =>
10     KV.getItem(key),
11    setItem: (key: string, value: string) =>
12     KV.setItem(key, value),
13    removeItem: (key: string) =>
14     KV.removeItem(key),
15  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.ts:1

```

5      return
6      window.localStorage.getItem(key);
7    } catch {
8      return null;
9    },
10   async setItem(key: string, value:
11     string): Promise<void> {
12     try {
13       window.localStorage.setItem(key,
14         value);
15     } catch {
16       // Private mode or storage full:
17       // keep working in memory.
18     }
19   },
20   async removeItem(key: string):
21     Promise<void> {
22     try {
23       window.localStorage.removeItem(key);
24     } catch {
25       // ignore
26     }
27   },
28 };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์โหมดส่วนตัวอาจห้ามเก็บข้อมูล คู่ไฟล์แบบนี้มีทั้งหมด 5 คู่:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้ใช้เลือกรูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอดรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

src/services/speech.ts

บรรทัด 1–24

```

1  /**
2   * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech

```

```

3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15
16 interface NativeSpeech {
17   start(options: Record<string, unknown>): void;
18   stop(): void;
19   abort(): void;
20   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
21   isRecognitionAvailable(): boolean;
22   addListener(event: string, listener: (e: never) => void): Subscription;
23 }
24
25 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ requireOptionalNativeModule = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ให้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดไม่มีโมดูลนี้ แต่ยังได้รับโค้ดใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะเดิ่งปิดแบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไมค์

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง

โค้ดใน src/ (TypeScript + React Native)



เว็บ

expo export --platform web รวมโค้ดเป็นไฟล์ JavaScript + HTML ในโฟลเดอร์ dist แล้ววางบน GitHub Pages

APK (Android)

expo prebuild สร้างโปรเจกต์ Android จาก app.json แล้ว Gradle build เป็นไฟล์ .apk

EAS Update

ส่งเฉพาะโค้ด JavaScript ใหม่ แอปที่ลงแล้วโหลดเองตอนเปิดแอป ใช้ตอนเปิดครั้งถัดไป

ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที่เรา push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": {"policy": "appVersion"}	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	รหัสแอป com.socratesandskeletons.mindpay

ค่าใน app.json	ความหมาย
"blockedPermissions"	บล็อกสิทธิ์กล้อง วัตถุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอร์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็กอัปเดตทุกครั้งที่เปิดแอป
"web": {"output": "single"}	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ `app.config.js` เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ `/Socrates-and-Skeletons - /` บน GitHub Pages
จึงต้องตั้ง `baseUrl` แต่ห้ามใส่ค่านี้นในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

- อ่านคอมเมนต์บนสุดของไฟล์ก่อน** ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
- ดูบรรทัด `import`** บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า `import` จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
- กด `Ctrl+Shift+O`** ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
- หาฟังก์ชันที่มี `export`** นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี `export` คือผู้ช่วยภายใน
- อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด** ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
- หาเทสต์ของมัน** กด `Shift+F12` ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย Satang	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น bkk เช่น <code>bkkDayKey</code> , <code>bkkToIso</code>	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ <code>'2026-10-09'</code>	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่า กันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
<code>useApp()</code>	หยิบข้อมูลกลาง (ไปรไฟล์ รายการ คำสั่ง <code>addTx</code> ฯลฯ)
<code>useMoney()</code>	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย Runway แบบร่าง
<code>useTheme()</code> แล้วตามด้วย <code>theme.ink</code> , <code>theme.primary</code>	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<code><T v="h3"></code> , <code><Money satang={...}></code> , <code><Card></code> , <code><Button></code>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
<code>space.lg</code> , <code>radius.md</code>	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
<code>toast({ message })</code>	ข้อความแจ้งเตือนสั้น ๆ ด้านล่างจอ
<code>router.push('/xxx')</code>	เปิดหน้า <code>src/app/xxx.tsx</code>
<code>accessibilityLabel=""</code>	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม

เห็นอะไร	แปลว่าอะไร
status === 'confirmed' หรือ confirmedOnly(tx)	นับเฉพาะรายการที่ยืนยันแล้ว
Reveal, usePressSpring, useReduceMotion	แอนิเมชัน (ค่อย ๆ โฟกัส, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส TC-12	รหัสเทสต์ ตรงกับในเอกสาร docs/TRACEABILITY.md
ไฟล์ชื่อ xxx.web.ts	ฉบับเว็บของไฟล์ xxx.ts
ชื่อแบบ snake_case เช่น amount_satang	ชื่อคอลัมน์ในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน src/data/repo.ts

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

src/domain/dates.ts

บรรทัด 1-51

```

1  /**
2   * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3   *
4   * Everything the user sees ("today", "this week", the date printed on a slip)
5   * is a Bangkok calendar day, so all grouping and range logic goes through
6   * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7   */
8  import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }

```

```

36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /\d{1,2}:\d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: ทุกอย่างที่ใช้ให้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลา ฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที ($86,400,000$)
13–17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบ สากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20–24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้ง นาฬิกาใน AppProvider (un A6)
26–28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30–35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือน ข้ามปีให้เอง
37–40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42–46	bkkToIso : เอาวันไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48–51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไร ไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะ อะไร	src/app/ชื่อหน้า.tsx (ตารางใน un A7)

อยากรู้	เปิดไฟล์
ตัวเลขนี้คิดยังไง	src/domain/: src/domain/summary.ts (ยอด), src/domain/runway.ts (Runway), src/domain/insights.ts (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	src/data/repo.ts , src/data/AppProvider.tsx
ตารางในฐานข้อมูลมีคอลัมน์อะไร	supabase/migrations/20260927000000_init.sql
AI ได้รับคำสั่งอะไร	supabase/functions/coach/index.ts , supabase/functions/parse-slip/index.ts
สี ขนาด ฟอนต์	src/ui/theme.ts , docs/DESIGN_SYSTEM.md
ฟีเจอร์นี้ตอบ requirement ข้อไหน มีทดสอบอะไร	docs/TRACEABILITY.md
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	docs/AI_USAGE_LOG.md
build / deploy อัตโนมัติ	.github/workflows/*.yaml

★ ถ้าอ่านแล้วงง

ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แถบ Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชีต แต่ละชีตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชั่น (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มินท์	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มินท์	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรกับเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล ลืมรหัสผ่าน

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ทให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ผังเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

โปรเจกต์ของทีมอยู่ภูมิภาค Singapore (ใกล้ไทย ตอบเร็ว) และใช้แพ็คเกจฟรี

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทีมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low
17   line, default #500
```

```

17 monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang >
0),
18 coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')),
-- FR-5
19 onboarded boolean not null default false,
20 created_at timestamptz not null default now(),
21 updated_at timestamptz not null default now()
22 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1-7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยคุณดูแลสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โพรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (฿500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมาฐานข้อมูลปฏิเสธเอง
17	งบต่อเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโค้ช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20-21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql

บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
ocr_confidence <= 1)),

```

```

41 review_flags text[] not null default '{}',
42 created_at timestamptz not null default now(),
43 updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_uidx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_uidx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตางค์ ต้องมากกว่า 0
32	รหัสหมวด
33–34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอลายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามือถือ 2 เครื่องสแกนสลิปใบเดียวกันพร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql

บรรทัด 55–65

```

55 -- -----
56 -- ai_usage: counts calls to the AI so one account cannot run up the bill.
57 -- Written only by the Edge Functions (service role), readable by the owner.
58 -- -----
59 create table if not exists public.ai_usage (
60   id bigserial primary key,
61   user_id uuid not null references auth.users (id) on delete cascade,

```



```

62 kind text not null check (kind in ('slip', 'coach')),
63 created_at timestamptz not null default now()
64 );
65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at
desc);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กันบัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ล็อกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครก็มีบัตรเข้าห้องได้ แต่แต่ละคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะโยนบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql

บรรทัด 67-87

```

67 -- -----
68 -- Row Level Security
69 -- -----
70 alter table public.profiles enable row level security;
71 alter table public.transactions enable row level security;
72 alter table public.ai_usage enable row level security;
73
74 drop policy if exists "own profile read" on public.profiles;
75 create policy "own profile read" on public.profiles
76   for select using (auth.uid() = id);
77 drop policy if exists "own profile update" on public.profiles;
78 create policy "own profile update" on public.profiles
79   for update using (auth.uid() = id) with check (auth.uid() = id);
80
81 drop policy if exists "own transactions" on public.transactions;
82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด	ความหมาย
70-72	เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง
74-76	policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รันไฟล์ซ้ำได้
77-79	แก้โปรไฟล์ได้เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ (กันแค่ id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น

บรรทัด	ความหมาย
81–83	รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง
85–87	ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรับตัวตรวจ Security Advisor ของ Supabase แล้วแก้จนไม่เหลือคำเตือนเรื่องตาราง ไฟล์ harden.sql เปลี่ยน auth.uid() เป็น (select auth.uid()) เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql
บรรทัด 1–25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7     new.updated_at := now();
8     return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17     for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21     for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25     for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เร็ว)
GitHub
VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql
บรรทัด 89–118

```

89 -- -----
90 -- Triggers
91 -- -----
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95     new.updated_at := now();
96     return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101     for each row execute function public.touch_updated_at();

```

```

102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104   for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110   insert into public.profiles (id, display_name)
111   values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112   on conflict (id) do nothing;
113   return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118   for each row execute function public.handle_new_user();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92–97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99–104	trigger = "ทุกครั้งก่อนแก้ไขในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107–114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116–118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใส่ให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคนที่เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยิงไข่: src/data/supabase.ts

src/data/supabase.ts

บรรทัด 1-38

```
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: คุญแจมาจากตัวแปรสภาพแวดล้อม (.env) คุญแจ publishable ปลอดภัยเพราะ RLS และคุญแจ AI ไม่อยู่ที่นี่
10	ตั้งที่เก็บการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และคุญแจจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง

บรรทัด	ความหมาย
23–26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้ไลบรารีอ่านลิงก์ในแถบที่อยู่เอง (AppProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31–36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว [src/app/welcome.tsx](#)

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแถวโปรไฟล์ให้ทันที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ `SIGNED_IN` แล้วสร้าง CloudRepo โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

Stack.Protected พาไปหน้าตั้งค่าเงิน กรอทยอดเงินตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

[src/app/welcome.tsx](#)

บรรทัด 148–172

```

148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);

```

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพากลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกินไป) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลิ้งรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;

```

```

31 update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32 remove(id: string): Promise<void>;
33 confirmMany(ids: string[]): Promise<void>;
34 listGoals(): Promise<SavingsGoal[]>;
35 insertGoal(input: GoalInput): Promise<SavingsGoal>;
36 updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37 removeGoal(id: string): Promise<void>;
38 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพีริเซนต์)
14-19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำสิ่งเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23-24	อ่านและแก้ไขไฟล์
25-29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปรอยืนยัน)
34-37	คำสั่งของกระปุกออม (FR-6)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพีริเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44-93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;

```

```

58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:44

บรรทัด	ความหมาย
44–59	TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (คั่นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
61–76	fromRow: แถวฐานข้อมูล → Transaction ของแอป ใช้ Number(. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว่าง
78–93	toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นค่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();

```



```

172     if (error) throw error;
173     return profileFromRow(data as ProfileRow);
174 },
175 async updateProfile(patch) {
176     const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
    userId).select().single();
177     if (error) throw error;
178     return profileFromRow(data as ProfileRow);
179 },
180 async listTransactions() {
181     // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182     const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183     const all: Transaction[] = [];
184     for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186             .from('transactions')
187             .select('*')
188             .gte('occurred_at', since)
189             .order('occurred_at', { ascending: false })
190             // A unique tiebreaker, so rows with the same time are never skipped or read twice
    across pages.
191             .order('id', { ascending: true })
192             .range(from, from + 999);
193         if (error) throw error;
194         all.push(...(data as TxRow[]).map(fromRow));
195         if (!data || data.length < 1000) break;
196     }
197     // Older confirmed money still counts in the balance: add it up (two columns only).
198     let carrySatang = 0;
199     for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201             .from('transactions')
202             .select('kind,amount_satang')
203             .eq('status', 'confirmed')
204             .lt('occurred_at', since)
205             .order('occurred_at', { ascending: false })
206             // A unique tiebreaker, so rows with the same time are never skipped or read twice
    across pages.
207             .order('id', { ascending: true })
208             .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211             carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
    Number(r.amount_satang);
212         }
213         if (!data || data.length < 1000) break;
214     }
215     return { txs: all, carrySatang };
216 },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้ไขโปรไฟล์แล้วขอแถวใหม่กลับมา

บรรทัด	ความหมาย
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ีกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ยอดยกมา: รายการที่ ยืนยันแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้ใช้แอปมาหลายปี

โมดกทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };
302       await save();
303     }
304     return cache;

```

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ `mindpay.demo.v2` (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่) บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินเริ่มต้น ฿3,000 เลื่อนสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยังไม่มีข้อมูล จะสร้างรายการตัวอย่าง (`buildSampleTransactions` ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้ แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as
300       // TOKEN_REFRESHED:
301       // that signs in too, but only while nothing is open (never over the demo or another
302       // account).
303       const signedIn =
304         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED'
305         && !activeUserRef.current);
306       if (signedIn && session && activeUserRef.current !== session.user.id) {
307         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
308         activateRepo(createCloudRepo(session.user.id), session.user.id);
309       }
310       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
311     }, 0);
312   });
313
314   const stopLinks = onIncomingLink((url) => {
315     handleAuthLink(url);
316   });

```

```

313     return () => {
314         cancelled = true;
315         sub?.data.subscription.unsubscribe();
316         stopLinks();
317     };
318 }, [activateRepo, clear, handleAuthLink]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:271`

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมา หลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย <code>setTimeout(..., 0)</code> เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

`src/data/AppProvider.tsx`

บรรทัด 127–167

```

127     const loadAll = useCallback(async (r: Repo) => {
128         const gen = ++loadGen.current;
129         localChanges.current = new Map();
130         setLoadError(null);
131         try {
132             // Goals never block the app: if they cannot be loaded, the rest still works.
133             const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
134             r.listGoals().catch(() => [] as SavingsGoal[])]);
135             if (gen !== loadGen.current || repoRef.current !== r) return;
136             // A slip saved by the automatic scan during the load is not lost when the list arrives.
137             const changes = localChanges.current;
138             const fresh = list.txs.filter((t) => !changes.has(t.id));
139             const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140             setProfile(p);
141             setTxs([...kept, ...fresh]);
142             setCarry(list.carrySatang);
143             setGoals(g);
144             setStatus('ready');
145         } catch (e) {
146             if (gen !== loadGen.current || repoRef.current !== r) return;
147             setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148             setStatus('ready');
149         }
150     }, []);
151
152     const activateRepo = useCallback(
153         async (r: Repo, id: string) => {

```

```

153     if (activeUserRef.current !== id) {
154         // Another account (or leaving the demo): never show or use the previous one's data
           meanwhile.
155         setProfile(null);
156         setTxs([]);
157         setCarry(0);
158         setGoals([]);
159     }
160     activeUserRef.current = id;
161     repoRef.current = r;
162     setRepo(r);
163     setUserId(id);
164     await loadAll(r);
165 },
166 [loadAll],
167 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง error เก่า
133	โหลด 3 อย่างพร้อมกัน (un A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านบน ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอวาดใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจํา Repo และโหลดข้อมูล

★ สรุป un A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโคชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี API key (กุญแจ) ไว้ระบุว่าใครเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น {"text": "สวัสดี"}

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้เราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโคชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI สัมแปลง)
5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ข้อยิ่งน้องกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ซื้อรองเท้า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังซื้อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ กุญแจลับ (Secret)

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้าขยับปากพร้อมซับไต้เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10  if (error) {
11    if (error instanceof FunctionsHttpError) {
12      const body = await (error.context as Response).json().catch(() => null);
13      if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่');
14      if (body?.error?.code === 'not_configured') {
15        throw new CoachError('โค้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย');
16      }
17      if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่');
18      if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19    }
20    throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้ง');
21  }
22  const text = (data as { text?: string })?.text;
```

```

23 if (!text) throw new CoachError('น้องคล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24 return text;
25 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/coach.ts:1

บรรทัด	ความหมาย
2	นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมาว่าผิดพลาด" มาใช้
5	สร้าง error ชนิดของตัวเอง ข้อความของมันคือสิ่งที่จะแสดงให้ผู้ใช้เห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้ว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวราวีแบบ token การเข้าสู่ระบบให้เอง
10–21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22–24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้น้องคล้า

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts

บรรทัด 1–26

```

1 // FR-5: the AI coach is น้องคล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องคล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "คล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22   Rules:
23   - Reply in Thai, at most 90 words.

```


- 22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot answer, say what is missing.
- 23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to the user.
- 24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
- 25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and after.
- 26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:1`

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก <code>_shared</code> มาใช้ (นามสกุล <code>.ts</code> ต้องเขียนเต็มใน Deno)
8	โควตาต่อวัน อ่านจาก <code>Secret COACH_DAILY_LIMIT</code> ถ้าไม่ได้ตั้งใช้ 60
10	ขนาดคำขอสองสูงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)
12-26	System prompt (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง · โครงสร้างคำตอบ: ตัวเลขบอกอะไร → สำคัญยัง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · ห้ามตำหนิ · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข <code>purchaseCheck</code> บอก Runway ก่อนและหลัง · ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน กู้ยืม ทรัพย์สิน

`supabase/functions/coach/index.ts`

บรรทัด 28-76

```

28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
'Request is too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({

```

```

53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่คว
60       }`,
61     ],
62   });
63   return json({ text: plainText(text) });
64 } catch (e) {
65   if (e instanceof NotConfiguredError) {
66     console.error('coach: AI key missing or refused', e.message);
67     return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68   }
69   if (e instanceof BusyError) {
70     console.warn('coach: AI busy', e.message);
71     return fail('busy', 'The AI service is busy, try again shortly', 503);
72   }
73   console.error('coach failed', e);
74   return fail('coach_error', 'The coach is unavailable right now', 502);
75 }
76 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32–33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36–42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49–51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52–62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของเงินช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64–75	จัดการ error ตามชนิด: คุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวอย่างกลาง: supabase/functions/_shared/common.ts

บรรทัด 17-83

```
17 /** Read one environment variable; a name the runtime refuses counts as "not set". */
18 const env = (name: string) => {
19   try {
20     return Deno.env.get(name);
21   } catch {
22     return undefined;
23   }
24 };
25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES);
26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES);
27
28 export const corsHeaders = {
29   'Access-Control-Allow-Origin': '*',
30   'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
31   'Access-Control-Allow-Methods': 'POST, OPTIONS',
32 };
33
34 export function json(body: unknown, status = 200) {
35   return new Response(JSON.stringify(body), {
36     status,
37     headers: { ...corsHeaders, 'Content-Type': 'application/json' },
38   });
39 }
40
41 /** Error body the app understands: { error: { code, message } } */
42 export function fail(code: string, message: string, status: number) {
43   return json({ error: { code, message } }, status);
44 }
45
46 /** Returns the signed-in user's id, or null when the request has no valid session. */
47 export async function requireUser(req: Request): Promise<string | null> {
48   const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49   if (!token) return null;
50   const { data, error } = await admin().auth.getUser(token);
51   if (error || !data.user) return null;
52   return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
57     {
58       auth: { persistSession: false },
59     });
60 }
61
62 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
63 export class NotConfiguredError extends Error {}
64
65 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
66 export class BusyError extends Error {}
67
68 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
69 export function envLimit(name: string, fallback: number): number {
70   const n = Number(Deno.env.get(name));
```

```

70 return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind,
81     p_limit: dailyLimit });
82   if (error) throw error;
83   return data === true;
84 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:17`

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเชิร์ฟเวอร์ (<code>Deno.env.get()</code>) ถ้าชื่อแปลจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ยอมรับ (ดู <code>readKey</code> ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	<code>json()</code> สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	<code>fail()</code> สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง <code>{ "error": { "code", "message" } }</code> ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ <code>coach.ts</code> ผังแอป)
46–53	<code>requireUser</code> : ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	<code>admin()</code> : client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเชิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านไควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	<code>takeQuota</code> : เรียกฟังก์ชัน SQL <code>take_ai_quota</code> ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในชั้นตอนเดียว

ทำไมนับไควตาต้อง "ทำในชั้นตอนเดียว"

`supabase/migrations/20261001000000_atomic_ai_quota.sql`

บรรทัด 1–44

```

1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''

```

```

12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21         from public.ai_usage
22         where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon,
    authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37     drop constraint if exists transactions_amount_max,
38     add constraint transactions_amount_max check (amount_satang <= 1000000000000);
39 alter table public.profiles
40     drop constraint if exists profiles_amounts_max,
41     add constraint profiles_amounts_max check (
42         abs(opening_balance_satang) <= 1000000000000
43         and (monthly_budget_satang is null or monthly_budget_satang <= 1000000000000)
44     );

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด	ความหมาย
1-5	คอมเมนต์เล่าบ๊วกเดิม: เดิมนับจำนวนครั้งวันนี้นับกับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กุญแจล็อก
7-14	สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิตงาน (slip/coach) และเพดาน คำนวณจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer)
16-18	ค่าที่ส่งมาผิด → ไม่อนุญาต
19	ล็อก ต่อผู้ใช้ต่อชนิตงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ล็อกปลดเองเมื่อจบคำสั่ง)
20-22	นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด
23-25	ถึงเพดานแล้ว → ไม่อนุญาต
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเพดานจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]):
  string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\"']|['\"]$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini:
  boolean }): AiProvider | null {
39   const forced = env.forced?.trim().toLowerCase();
40   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
41   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
42   if (env.hasClaude) return 'claude';
43   if (env.hasGemini) return 'gemini';
44   return null;
45 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด ความหมาย

1-2 โฟลน์นี้เป็นฟังก์ชันล้วน ไม่ใช้คำสั่งของ Deno จึงเอาไปเทสต์ด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนถือปวงออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีคีย์แจด้วย) ไม่จันมีคีย์แจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.ts

บรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req,
133     deadline);
134   // Metadata only: never log images, questions or replies.
135   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant:
136     req.variant ?? 'primary', ms: Date.now() - started }));
137   return result;
138 }
139
140 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and
141   retries). */
142 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
```

```

140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS,
144   left)) });
145   } catch (e) {
146     const name = (e as Error)?.name;
147     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed
148   out (${url.split('/')[2]})`);
149     throw e;
150   }
151 }
152
153 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
154
155 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model:
156   string }> {
157   const key = claudeKey();
158   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ??
159   CLAUDE_DEFAULT_MODEL;
160   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
161   const body: Record<string, unknown> = {
162     model,
163     max_tokens: req.maxTokens,
164     system: req.system,
165     messages: [
166       {
167         role: 'user',
168         content: req.content.map((c) =>
169           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type:
170   c.mediaType, data: c.data } },
171       ),
172     ],
173   };
174   if (req.schema) {
175     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
176   }
177   const res = await timedFetch(
178     'https://api.anthropic.com/v1/messages',
179     {
180       method: 'POST',
181       headers: {
182         'x-api-key': key,
183         'anthropic-version': '2023-06-01',
184         'content-type': 'application/json',
185       },
186       body: JSON.stringify(body),
187     },
188     deadline,
189   );
190   if (!res.ok) {
191     const detail = await res.text();
192     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
193     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
194     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
195     throw new Error(msg);
196   }
197   const data = await res.json();
198   const text = (data.content ?? []).
199     .filter((b: { type: string }) => b.type === 'text')

```



```

196     .map((b: { text: string }) => b.text)
197     .join('');
198     if (!text) throw new Error('Claude API returned no text');
199     return { text, model };
200 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:114

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลคำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูปคำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็นะ
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบถูกใจใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: คุญแจผิด → NotConfigured, 429/529 (ส่งถี่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทั่วไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คืนพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับคุญแจนี้ข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.ts

บรรทัด 75–93

```

75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';

```

บรรทัด	ความหมาย
87	กุญแจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กุญแจนี้ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

1. สร้างกุญแจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
2. เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)
3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีการขอ
5. ถ้าแก้ไขโค้ดใน `supabase/functions` ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ `parse-slip`)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้ใช้เช็ค (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้งานง่ายไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำอย่างไรในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจได้, ปุ่มลัด + ฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	บอกล่าพุดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามันใจแค่นี้	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังงี้" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ **กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง**

สี: Green + Gold

ทิศทางแบรนด์คือ **สีเขียว** (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ **สีทอง** ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

`src/ui/theme.ts`บรรทัด 1-21

1 /**

```

2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9  import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

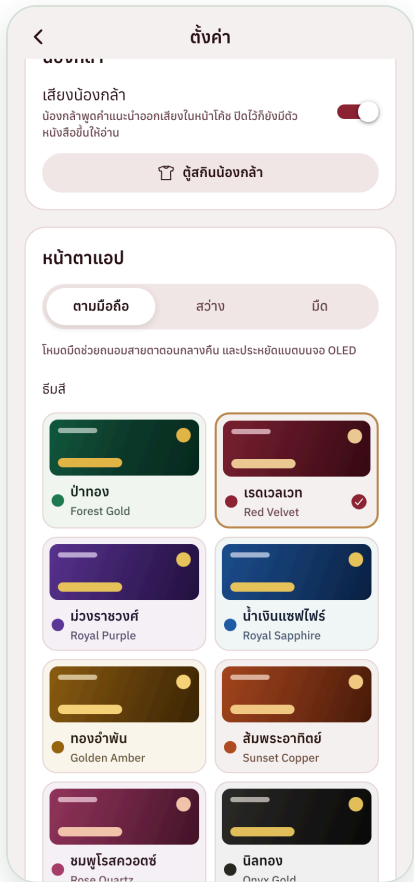
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ป่าทอง สว่าง	ป่าทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียม ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่ง que เลือกอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, สะวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดงอิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), เรดเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, สัมพระอาทิตย์, ชมพูโรสควอตซ์, นิลทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts บรรทัด 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน buildTheme สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (onAccent) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทงนั้น

หน้าจอเรียกแค่ const theme = useTheme() ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล WCAG AA กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย 4.5 : 1 (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาไม่ดีหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token accentInk ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน contrast() ในแอป

src/ui/theme.tsบรรทัด 261–279

```
261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
```

```

264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266 };
267 return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272     const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273     return (hi + 0.05) / (lo + 0.05);
274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278     return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35     for (const dark of [false, true]) {
36         it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is
used on`, () => {
37             const t = buildTheme(spec.key, dark);
38             const pairs: [string, string, string][] = [
39                 ['inkSoft on bg', t.inkSoft, t.bg],
40                 ['inkSoft on surface', t.inkSoft, t.surface],
41                 ['onPrimary on primary', t.onPrimary, t.primary],
42                 ['onAccent on accent', t.onAccent, t.accent],
43                 ['accentInk on surface', t.accentInk, t.surface],
44                 ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45                 ['good on goodSoft', t.good, t.goodSoft],
46                 ['watch on watchSoft', t.watch, t.watchSoft],
47                 ['critical on criticalSoft', t.critical, t.criticalSoft],
48                 ['good on bg', t.good, t.bg],
49                 ['watch on bg', t.watch, t.bg],
50                 ['critical on bg', t.critical, t.bg],
51                 ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52                 ['heroInk on heroTop', t.heroInk, t.heroTop],
53             ];
54             const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([n, a, b]) => `${n}
${contrast(a, b).toFixed(2)}`);
55             expect(low).toEqual([]);
56         });
57     }
58 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ TC-78 ตรวจสอบทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่มือ (บรรทัด 38–53) ถ้าคู่มือต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่มือพร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```
301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {
308   serif: 'NotoSerifThai_600SemiBold',
309   serifBold: 'NotoSerifThai_700Bold',
310   sans: 'Anuphan_400Regular',
311   sansMedium: 'Anuphan_500Medium',
312   sansSemi: 'Anuphan_600SemiBold',
313   sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below.
320  */
321 export const type = {
322   display: { fontSize: 38, lineHeight: 52 },
323   h1: { fontSize: 26, lineHeight: 38 },
324   h2: { fontSize: 20, lineHeight: 30 },
325   h3: { fontSize: 17, lineHeight: 26 },
326   body: { fontSize: 15, lineHeight: 24 },
327   small: { fontSize: 13, lineHeight: 20 },
328   micro: { fontSize: 11, lineHeight: 16 },
329 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:301

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว กันสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

src/ui/components.tsx

บรรทัด 261–338

```
261 export function Button({
262   label,
```

```

263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 }: {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 }) {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },
291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down
only,
305       // so buttons side by side never share a touch area).
306       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
307       accessibilityRole="button"
308       aria-disabled={!off}
309       aria-busy={!loading}
310       style={[
311         {
312           backgroundColor: colors.bg,
313           borderColor: colors.border,
314           borderWidth: 1.5,
315           borderRadius: radius.pill,
316           paddingVertical: small ? 8 : 12,
317           paddingHorizontal: small ? 14 : 20,
318           flexDirection: 'row',
319           alignItems: 'center',
320           justifyContent: 'center',
321           gap: 8,
322           opacity: off ? 0.5 : 1,

```



```

323         overflow: shine ? 'hidden' : undefined,
324     },
325     style,
326     { transform: [{ scale: press.scale }] },
327   ]}
328   >
329   {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ?
'rgba(255,255,255,0.45)' : 'rgba(255,255,255,0.28)'} /> : null}
330   {loading ? (
331     <ActivityIndicator color={colors.fg} />
332   ) : icon ? (
333     <Ionicons name={icon} size={small ? 16 : 19} color={colors.fg} />
334   ) : null}
335     <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color:
colors.fg }]}>{label}</Text>
336   </AnimatedPressable>
337 );
338 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/components.tsx:261`

บรรทัด	ความหมาย
261–283	props ของปุ่ม: ข้อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
284–285	สีของธีม และแอนิเมชันยุบตัวตอนกด (เหลือ 95%)
286–293	ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาวตัวขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
294	ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
297–300	กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นิ้วคนกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอว่าเป็นปุ่ม และสถานะปิด/คำสั่งโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	คำสั่งโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาถึงหนา

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่คำสั่งพิมพ์ให้พื้นที่บอร์ด
Card	กลุ่มเนื้อหา มี 3 โท: ปกติ, พิเศษ (alt), กองอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ

ชิ้นส่วน	ใช้เมื่อ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินลด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ชุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: น้องกล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำอย่างไร
คนตาบอด/สายตาเลือนราง	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"
แยกลีไม่ออก (ตาบอดสี)	สถานะไม่ใช้สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมารถจากอาการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มแดง, ตันไม้เงินโยก, พลุตอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

1. ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นแม่มือถือราคาประหยัด
2. ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
3. เคารพ "ลดการเคลื่อนไหว"
4. ไม่ใช้ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

1. ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha()
2. ตัวหนังสือใช้ <T> และเงินใช้ <Money>
3. ปุ่ม ชิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
4. ลองเปิดหน้านั้น 2–3 ธีม ทั้งสว่างและมืด

5. รัน `npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web`

i ทบทวนศึกษา UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จดให้") และแนวการออกแบบของ Hugging Face (มาสคอตตัวเดียวหลายอารมณ์, สีอุ้งบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพลุดอนทำสำเร็จ) แล้วออกแบบตัวละครน้องกล้ายของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเราทำการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทสต์

ทำไมต้องเขียนเทสต์

เวลาแก้ไขโค้ดจุดหนึ่ง อาจทำให้อีกจุดพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แย่ จะใช้เวลาหลายชั่วโมงและล้มบางจุดแน่นอน เทสต์อัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสต์ไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)
ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)
เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)
คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสต์อยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสต์ของ FR-6:

src/domain/__tests__/domain.test.ts

บรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35–36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสต์วันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสต์จะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38–58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายจ่าย หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

src/domain/__tests__/domain.test.ts บรรทัด 184–210

```

184   it('TC-14 computes days and depletion date', () => {
185       const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000,
now: NOW });
186       expect(r.days).toBe(15);
187       expect(r.depletionDay).toBe('2026-10-12');
188       expect(r.status).toBe('healthy');
189   });
190
191   it('TC-15 handles zero average without dividing by zero', () => {
192       const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now:
NOW });
193       expect(r.status).toBe('no_spending');
194       expect(r.days).toBeNull();
195       expect(r.depletionDay).toBeNull();
196   });
197
198   it('TC-16 flags balance at or below the low line', () => {
199       const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000,
now: NOW });
200       expect(r.status).toBe('below_floor');
201       expect(r.days).toBe(0);
202   });
203
204   it('TC-17 status thresholds', () => {
205       const at = (days: number) =>
206           computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang:
10_000, now: NOW }).status;
207       expect(at(6)).toBe('critical');
208       expect(at(7)).toBe('watch');
209       expect(at(14)).toBe('healthy');
210   });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เงินสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186–188	Assert: expect(ค่าจริง).toBe(ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191–196	TC-15 ทดสอบ กรณีพิเศษ: ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198–202	TC-16 ทดสอบ ขอบ: ยอดเท่ากับเงินสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"

บรรทัด	ความหมาย
204–210	TC-17 ทดสอบ ขอบของเกณฑ์ : 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเผลอแก้ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

บ๊ิกชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทิมจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือนปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บ๊ิกที่เจอตอนตรวจทั้ง repo (1–3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเซิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ [e2e/web-auth.mjs](#) (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เซิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน

`e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไม่ได้อยู่ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (`page.route`) เหมือนซ้อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29–60

```
29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)}):` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 },
46     deviceScaleFactor: 2, locale: 'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50     m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request
52     failed`, r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }
58 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
59 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false
60   }).first().waitFor({ state: 'visible', timeout }).then(() => true, () => false);
61 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false
62   }).first().waitFor({ state: 'hidden', timeout }).then(() => true, () => false);
63 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached',
64   timeout: 8000 }).catch(() => {});
65 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
66 /** Waits (up to `timeout`) until the element is visible. */
67 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(()
68   => true, () => false);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `e2e/web-auth.mjs:29`

บรรทัด ความหมาย

30–36 `check` (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions

บรรทัด	ความหมาย
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดักคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแถบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the
    coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has
    fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000
    }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x)
    < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
599     ? await page.evaluate(({ x, y }) => {
600       const el = document.elementFromPoint(x, y);
601       return el ? `${el.tagName} role=${el.getAttribute('role')}`
        label=${el.getAttribute('aria-label')}` : 'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';
604   if (box) {
605     // hover() waits until the slider itself receives the pointer (nothing on top of it).
606     await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607     await page.mouse.down();

```



```

608     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps:
8 });
609     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps:
8 });
610     await page.mouse.up();
611 }
612 const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613 await page.waitForTimeout(900);
614 await shot(page, '28-runway-slider');
615 check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' &&
(await visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under}
introWaitMs=${introMs}`);
616 // The arrow keys move it too (keyboard users on the web).
617 await slider.focus();
618 await page.keyboard.press('ArrowRight');
619 const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620 check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0)
+ 5), `value=${keyed}`);
621 await page.keyboard.press('Home');
622 await page.fill('#runway-price', '2000');
623 check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ
฿2,000')) && (await visible(page, 'ตอนนี้')));
624 await button(page, 'ถามโค้ชเรื่องนี้').click();
625 const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626 check('Runway: "ask the coach" opens the coach with the question ready',
asked.includes('฿2,000'), asked);
627 await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574–579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589–597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่เงินจะลากพลาด
604–611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612–615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจสอบว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617–620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622–623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624–626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ขั้วผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน [docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ตำหนิ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแกลเลอรี แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → ทดสอบ

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจสอบคุณภาพอื่น ๆ

คำสั่ง	ตรวจสอบอะไร
npm run typecheck	TypeScript ตรวจสอบนิพจน์ข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจสอบรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจสอบความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แจ้งเตือน 0 ปัญหาเรื่องตาราง)

คำสั่ง	ตรวจอะไร
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก่ครบพร้อมทดสอบ

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

วงจรทำงาน 1 รอบ

```

1 git pull                                # 1. ดึงของล่าสุดก่อนเริ่ม
2 git checkout -b fix-runway-text         # 2. แยก branch ใหม่ (ไม่บังคับ)
3 # ... แก้โค้ดใน VS Code ...
4 npm test                                # 3. รันเทสต์ให้ผ่านก่อน
5 git status                              # 4. ดูว่าแค่ไฟล์ไหนไปบ้าง
6 git add src/domain/runway.ts            # 5. เลือกไฟล์ที่จะเซฟ
  
```

```
7 git commit -m "Runway: say 'ประมาณ' in the date" # 6. เซฟพร้อมข้อความ
8 git push -u origin fix-runway-text # 7. ส่งขึ้น GitHub แล้วเปิด Pull Request
```

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่แถบ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก
พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: *"Runway: arrow keys on the web; steadier drag test"*, *"Audit follow-up: fix regressions found by independent review"* อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ github.com/2550expo-tech/Socrates-and-Skeletons - (สาธารณะ ใครก็อ่านได้) branch main มี **66 commit** ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บบน GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, บัญชี, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาคุกกี้ AI, อ่าน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจด (FR-3), บัญชีพุดได้, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แกะบีก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้ง push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งโค้ดเปลี่ยน ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ 2550expo-tech.github.io/Socrates-and-Skeletons/
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป

ไฟล์	ทำอะไร	ผลลัพธ์
e2e-webkit.yml	E2E ชุดเดียวกันบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่ล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ต่ออยู่

android-apk.yml

เทสหรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret EXPO_TOKEN และเทสผ่าน

e2e-webkit.yml

ตรวจบน Safari เพิ่ม

อ่าน web.yml ที่ละบรรทัด

.github/workflows/web.yml

บรรทัด 1-76

```

1 # Publishes the web version of MindPay to GitHub Pages on every push to main:
2 # https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3 # The web version shows the app at phone width, works with real accounts and
4 # demo data, and reads slips from photos the user uploads. Scanning the whole
5 # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6 # Add to Home Screen, and it opens full screen with the MindPay icon.
7 name: Deploy web app
8
9 on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21 jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env

```

```

37
38   - name: Export web build
39     env:
40       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41     run: npx expo export --platform web --output-dir dist
42
43   - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44     env:
45       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46     run: node scripts/web-home-screen.mjs dist
47
48   - name: Single-page app fallback for deep links
49     run: cp dist/index.html dist/404.html
50
51   - name: Unit tests
52     run: npm test
53
54   # Browser test of every sign-in path (sign up, email code, links, reset
55   # password, sign out) against a local stand-in for Supabase. If anything
56   # fails, the website is NOT published and the screenshots are attached.
57   - name: Install the test browser
58     run: npx playwright install --with-deps chromium
59
60   - name: Browser tests (sign-in flows)
61     run: node e2e/web-auth.mjs dist e2e-shots
62
63   - name: Keep screenshots when the browser tests fail
64     if: failure()
65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด	ความหมาย
1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอโฮมบน iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สคริปต์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่มาระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รับบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดีงโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)

บรรทัด	ความหมาย
28–31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35–36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38–41	build เว็บลงโพลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43–46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจอล็อกอิน" แล้วเปิดเต็มจอเหมือนแอป
48–49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51–52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทำงาน
57–61	ติดตั้งเบร่าวเซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63–68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวนโหลดดูว่าพังตรงไหน
70–76	ผ่านทุกด่านแล้วจึงเผยแพร่โพลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

<code>.github/workflows/android-apk.yml</code>	บรรทัด 12–90
<pre> 12 on: 13 push: 14 branches: [main] 15 # Editing documents only does not need a new APK. 16 paths-ignore: ['**/*.md', 'docs/**'] 17 workflow_dispatch: 18 19 permissions: 20 contents: write 21 22 # A build that already started runs to the end; newer pushes wait in line 23 # (only the newest waiting push is kept). 24 concurrency: 25 group: android-apk 26 cancel-in-progress: false 27 28 jobs: 29 apk: 30 runs-on: ubuntu-latest 31 timeout-minutes: 45 32 steps: 33 - uses: actions/checkout@v4 34 35 - uses: actions/setup-node@v4 36 with: 37 node-version: 22 38 cache: npm 39 40 - uses: actions/setup-java@v4 41 with: 42 distribution: temurin </pre>	


```

43     java-version: 17
44
45     - name: Install dependencies
46       run: npm ci
47
48     - name: Unit tests
49       run: npm test
50
51     - name: Typecheck
52       run: npx tsc --noEmit
53
54     - name: App settings (public Supabase URL and publishable key)
55       run: cp .env.example .env
56
57     - name: Generate the native Android project
58       run: npx expo prebuild --platform android --no-install
59
60     - name: Give Gradle enough memory
61       # The default Metaspace ran out during Android lint and the build hung until the job
timeout.
62       run: |
63         sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64         # The generated file has no trailing newline: start with one so the setting is not
glued onto the last line.
65         printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
        android/gradle.properties
66         tail -n 3 android/gradle.properties
67
68     - name: Build release APK
69       working-directory: android
70       # arm64 only: covers practically every Android phone from the last 7 years and builds
much faster.
71       # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to
produce the APK.
72       run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
        lintVitalAnalyzeRelease --stacktrace
73
74     - name: Rename APK
75       run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
76
77     - uses: actions/upload-artifact@v4
78       with:
79         name: mind-pay-apk
80         path: mind-pay.apk
81
82     - name: Publish to the "latest-apk" release
83       env:
84         GH_TOKEN: ${GITHUB_TOKEN}
85       run: |
86         gh release delete latest-apk --yes --cleanup-tag || true
87         gh release create latest-apk mind-pay.apk \
88           --title "MindPay (latest APK)" \
89           --notes "Built from ${GITHUB_SHA::7} on $(date -u +%Y-%m-%d %H:%M UTC'). Open this
        page on an Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90         --prerelease

```

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างโฟลเดอร์ android ทั้งโปรเจกต์จาก app.json (ทีมไม่เก็บโฟลเดอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์เล่าว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหน เมื่อไร

i APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและพีซีเซ็นได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

ค่าลับที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

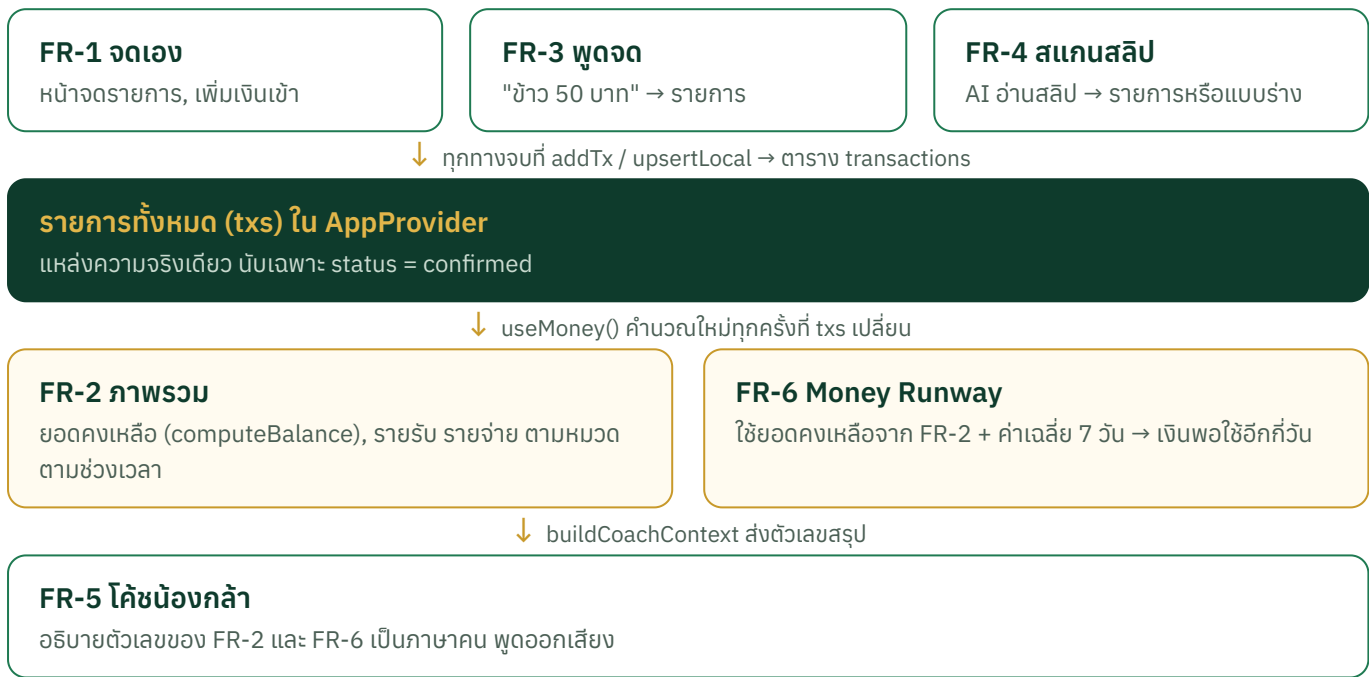
ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)

ไฟล์	มีอะไร
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ



ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จัด/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจดยรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจดยรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
<code>src/domain/types.ts</code>	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
<code>src/domain/money.ts</code>	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่
<code>src/domain/dates.ts</code>	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
<code>src/domain/summary.ts</code>	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
<code>src/domain/categories.ts</code>	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
<code>src/data/AppProvider.tsx</code>	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
<code>src/app/transaction.tsx</code>	FR-1 และ FR-4 (ตรวจสอบสลิป)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสต์ก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

- Q1** ทำไมเลือก Expo / React Native?

โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
- Q2** ทำไมเลือก Supabase?

ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในที่เดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
- Q3** ข้อมูลผู้ใช้ปลอดภัยยัง?

ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แวลบและคุกกี้ AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ออกรวม
- Q4** ทำไมเก็บเงินเป็นสตางค์?

ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
- Q5** ทำไมต้องจัดวันตามเวลาไทย?

ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเข้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
- Q6** ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม?

ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่มีข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
- Q7** แบบร่าง (draft) คืออะไร ทำไมต้องมี?

รายการจากสลิปที่ AI อ่านไม่มั่นใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
- Q8** ใช้ AI ตรงไหน และป้องกัน AI ผิดยังไง?

2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เหนือความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว

Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? กฎแฉ AI จะหลุดไปกับแอป Edge Function ถือกฎแฉไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยังไง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รันเทสก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ทหลุดระหว่างฟรีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยังไง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และรหัสเทส (TC/MT) ที่พิสูจน์
Q13	ใช้ AI ช่วยเขียนโค้ดไหม? ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกแถวบอกจุดประสงค์ ผลลัพธ์ สิ่งที่ทีมแก้หรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอนตามจริงว่าตัวเองตรวจอะไรไปแล้วบ้าง)
Q14	ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม? การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง
Q15	อะไรที่ยังไม่ได้ทำ (นอก MVP)? Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ท) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเซฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AuthProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.innk
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่

คำ	ความหมายแบบง่าย
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่ที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้ไขโครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น ไมค์ แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใช้ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบยังไฉ่
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-4 Automatic Slip Detection

สแกนสลิปในแอปพลิเคชันอัตโนมัติด้วย AI ภาคนี้เขียนในมุมมองของคนสร้าง FR-4 เอง ตั้งแต่ ปัญหาที่ยากแก่ หน้าจอ หลักการคำนวณ โค้ดจริงที่ละบรรทัด การทดสอบ ไปจนถึง คำถามที่อาจารย์น่าจะถาม

FR-4 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-4 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-4 **Automatic Gallery Slip Detection** คือ "ผู้ช่วยจดจากสลิป" ทุกครั้งที่โอนเงินด้วยแอปธนาคาร แอปจะเซฟสลิปไว้ใน แกลเลอรี MindPay **หาสลิปเหล่านั้นเอง** อ่านยอด วันที่ และชื่อผู้รับด้วย AI แล้ว **บันทึกเป็นรายการให้** โดยมีกติกาสำคัญ 3 ข้อจาก Requirement: **ตรวจเมื่อผู้ใช้เริ่มสแกน (และเมื่อเปิดแอป) · เกณฑ์ความมั่นใจ 80% · ตรวจรายการซ้ำ**

รูปในแกลเลอรี → มี QR สลิปใหม่ (บนมือถือ ฟรี) → ย่อรูป + ลายนิ้วมือ → AI อ่าน 2 ครั้ง (เชิร์ฟเวอร์) → ตรวจความมั่นใจ ≥ 0.8 และซ้ำ → บันทึก

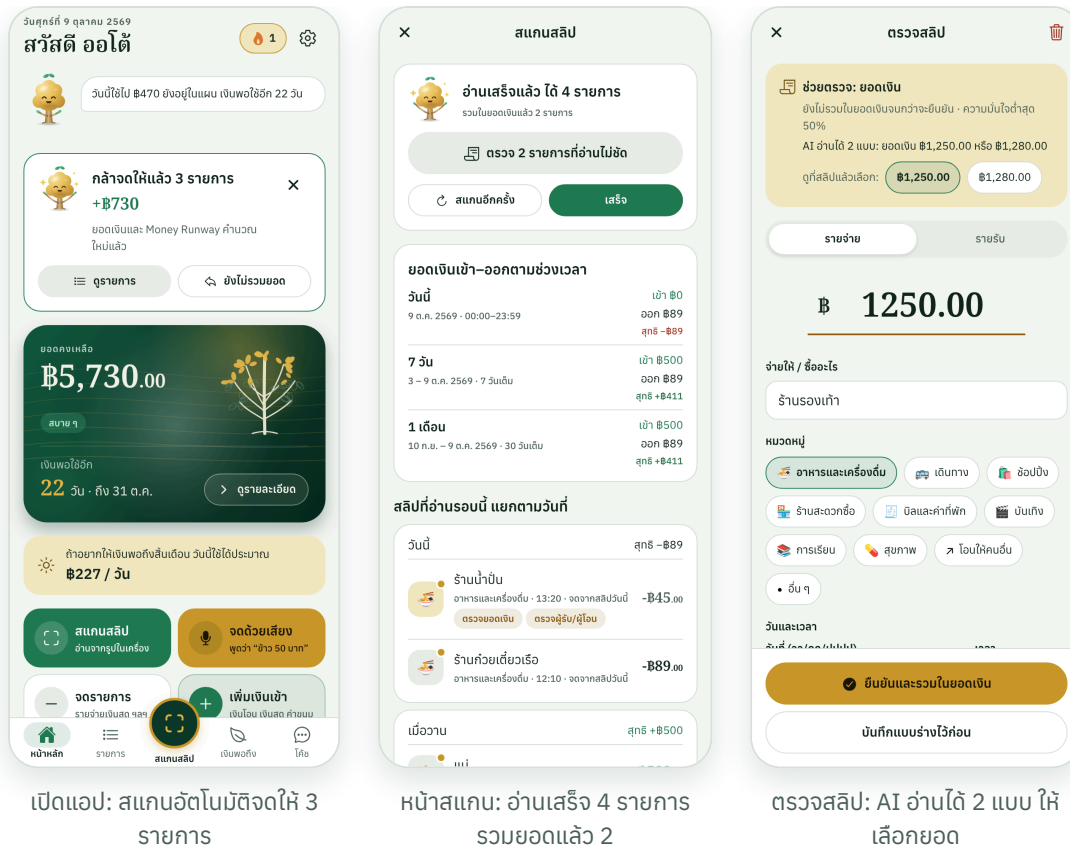
อ่านชัดทุกช่อง ($\geq 80\%$) → รวมในยอดทันที (ถ้าเปิด "รวมยอดทันทีเมื่ออ่านชัด") · ช่องใดไม่ชัด หรือ AI สองรอบเห็นไม่ตรงกัน → **แบบร่าง รอผู้ใช้ตรวจ** ไม่นับในยอดจนกว่าจะยืนยัน

อยู่ตรงไหนในแอป

ที่	ทำอะไร
เปิดแอป (Android)	สแกนอัตโนมัติ: หาสลิปใหม่ตั้งแต่ครั้งล่าสุด (ครั้งแรกย้อน 30 วัน) แสดงการ์ดบนหน้าหลัก "คล่าเจอให้แล้ว 3 รายการ +฿730" พร้อมปุ่ม "ยังไม่รวมยอด"
ปุ่มลัด "สแกนสลิป" / ปุ่มกลางแถบเมนู	หน้าสแกน: มือถือค้นแกลเลอรี 30 วันเอง (ย้อนได้ 1 ปี) · เว็บกด "เลือกรูปสลิป" ครั้งเดียว · แสดงความคืบหน้า ผลตามช่วงเวลา และแยกตามวัน
"สลิปรอยืนยัน"	แบบร่างทั้งหมด: กลุ่มพร้อมยืนยัน (กดทีเดียวทั้งหมด) และกลุ่มต้องตรวจ (แตะเพื่อแก้)
หน้า ตรวจสอบสลิป	ฟอร์มของ FR-1 ที่ไฮไลต์ช่องไม่ชัด และให้เลือกยอดเมื่อ AI อ่านได้ 2 แบบ
ตั้งค่า	สวิตช์ "หาสลิปใหม่ทุกครั้งที่เปิดแอป" และ "รวมยอดทันทีเมื่ออ่านชัด" · "ล้างประวัติรูปที่เคยสแกน"

4 ข้อที่ต้องจำ

- รูปออกจากมือถือเฉพาะรูปที่น่าจะเป็นสลิป:** สลิปธนาคารไทยมี "QR ตรวจสอบสลิป" มือถือหา QR เองฟรี (ไม่ใช่เน็ต) รูปอื่น ๆ เช่น รูปเพื่อน รูปอาหาร ไม่ถูกส่งไปไหน — ประหยัดเงินค่า AI และเป็นส่วนตัว
- 80% หรือรอตรวจ:** AI ให้ "ความมั่นใจ" 0–1 ของยอดเงิน วันที่ ผู้รับ ช่องใดต่ำกว่า 0.8 หรืออ่านไม่ได้ → ติดธง → แบบร่าง
- อ่าน 2 ครั้งด้วย AI 2 รุ่น** ที่ไม่เห็นคำตอบของกันและกัน ตรงกันจึงเชื่อ ไม่ตรงกันเก็บทั้ง 2 ค่าให้ผู้แตะเลือก
- กันซ้ำ 3 ชั้น + ฐานข้อมูล:** เลขอ้างอิงจาก QR, ลายนิ้วมือของรูป (SHA-256), ยอด+วัน+นาทียุกับผู้รับ และ unique index ในฐานข้อมูล



เปิดแอป: สแกนอัตโนมัติจัดให้ 3 รายการ

หน้าสแกน: อ่านเสร็จ 4 รายการ รวมยอดแล้ว 2

ตรวจสอบสลิป: AI อ่านได้ 2 แบบ ให้เลือกยอด

1 ภาพหน้าจอในเล่มนี้

ถ่ายจากเว็บเวอร์ชันจริงที่ **ล็อกอินบัญชีทดสอบ** กับ "เซิร์ฟเวอร์จำลอง" และ "เคลเลอร์จำลอง" ตัวเดียวกับที่เทส E2E ใช้ (e2e/fake-backend.mjs) เพราะการอ่านสลิปต้องมีเซิร์ฟเวอร์ (โหมดทดลองอ่านสลิปไม่ได้) ผลอ่านของ AI ถูกกำหนดไว้ล่วงหน้า แต่ทุกอย่างหลังจากนั้น (หา QR, ตรวจ 80%, ตรวจซ้ำ, บันทึก, หน้าจอ) เป็นโค้ดจริงทั้งหมด รูปสลิปเป็นรูปจำลอง (ไม่ใช่สลิปจริงของใคร) ที่มี QR จริงอยู่มุมขวาล่าง

ปัญหาที่ฉันอยากแก้

คนโอนเงินวันละหลายครั้ง แต่ไม่มีใครอยากพิมพ์ทุกรายการ

นักศึกษาส่วนใหญ่จ่ายเงินผ่าน mobile banking และ PromptPay ทุกครั้งที่โอน แอปธนาคาร เซฟสลิปลงแกลเลอรี่อยู่แล้ว ข้อมูลทุกอย่างที่ต้องจด (ยอด วันเวลา ร้าน) อยู่ในรูปนั้น ปัญหาคือไม่มีใครอยากเปิดสลิปแล้วพิมพ์ตามที่ละใบ FR-4 จึงให้แอป "อ่านแทน"

บทเรียนจาก prototype รุ่นแรก

ปัญหาที่เจอ	สาเหตุ	แก้ไขในเวอร์ชันนี้ยังง	เทสที่กัณ
สลิปเดือนกันยายนอ่านวันที่ไม่ ได้	OCR ทำจุดใน "ก.ย." หายหรือ ย้าย ("กย", "ก.ย")	เทียบชื่อเดือนแบบตัดจุดและช่องว่าง	TC-04
เปลี่ยนช่วงเวลาระหว่างสแกน แล้วรูปถูกข้าม	ควยยังใช้ช่วงเก่า และจับคู่ผลกับ รูปด้วยลำดับ	state machine ลือกช่วงตอนสแกน ใช้ runId และจับคู่ด้วย assetId	TC-27, TC-28
ข้อมูลตัวอย่างตายตัว (14 ก.ย.) จนเก่า	วันที่เขียนตายตัว	สร้างข้อมูลตัวอย่างย้อนจากวันนี้เสมอ	TC-33

เป้าหมายที่ฉันตั้งไว้ตอนออกแบบ

เป้าหมาย	ทำยังงในแอป
แม่นกว่าเร็ว ("ยอดผิดแยกว่าไม่มี คำตอบ")	เกณฑ์ 80%, AI อ่าน 2 รุ่น, ยอดเงินต้องตรงกับตัวเอง (ตัวเลข vs ที่พิมพ์บนสลิป), ไม่ใช่คำ ธรรมชาติ, วันที่อนาคตไม่เชื่อ
ไม่ต้องแตะอะไรเลย	เปิดแอปแล้วสแกนเอง ขอสิทธิ์รูปภาพครั้งเดียวตอนเปิดแอปครั้งแรก สลิปที่ซัดรวมยอดกันที่
ผู้ใช้คุมได้	ปุ่ม "ยังไม่รวมยอด" ย้อนการนับอัตโนมัติ, ปิดการสแกนอัตโนมัติ/การรวมยอดกันที่ได้ในตั้งค่า
ไม่ซ้ำ	กัน 3 ชั้นในแอป + unique index ในฐานข้อมูล (แม้สองเครื่องสแกนพร้อมกัน)
ประหยัดและเป็นส่วนตัว	ส่งเฉพาะรูปที่มี QR สลิป, ย่อรูปเหลือกว้าง 1,100 px, ไม่เก็บรูปบนเซิร์ฟเวอร์, โควตา 300 ใบ/ วัน/บัญชี
ไม่มีสลิปหลุด	เน็ตหลุดหยุดพร้อมบอกเหตุผล (ไม่เลื่อนช่วงเวลา), รูปที่อ่านไม่ได้ลองใหม่รอบหน้า, AI คิวเติม รอแล้วอ่านต่อเอง

Requirement และเงื่อนไขผ่าน

จาก docs/TRACEABILITY.md: Acceptance criteria → โค้ด → เทสต์ (FR ที่มีเงื่อนไขมากที่สุดของระบบ)

i Requirement FR-4

"ตรวจสอบเมื่อผู้ใช้เริ่ม Scan Gallery, เกณฑ์ความมั่นใจ 80%, ตรวจสอบรายการซ้ำ"

Acceptance criteria (ย่อ)	โค้ด	เทสต์
ทุกช่องสำคัญมั่นใจ $\geq 0.80 \rightarrow$ "พร้อมยืนยัน" ช่องใดต่ำกว่าหรือว่าง $\rightarrow$ "ต้องตรวจ"	slip.ts CONFIDENCE_THRESHOLD, normalizeReading	TC-19, TC-20
เฉพาะ status = confirmed เข้ายอดเงิน ได้มา 2 ทาง: ผู้ใช้กดยืนยัน หรือ (เปิดรวมยอดทันที) ทุกช่อง ≥ 0.80	slip.ts initialStatus, transaction.tsx, drafts.tsx	TC-09, TC-37, MT-06
อ่าน 2 ครั้งพร้อมกันด้วย AI 2 รุ่น ตรงกันจึงเข้ายอดได้ ไม่ตรงกัน $\rightarrow$ รอตรวจพร้อมแสดง 2 คำให้เลือก อ่านได้รอบเดียว $\rightarrow$ รอตรวจเสมอ	parse-slip, helpers.ts mergeReadings, similarName; slip.ts slipNote, amountChoices	TC-63, TC-64, E2E สองคำ
ยอดเงินตรวจกับตัวเอง (ตัวเลข vs ตามที่พิมพ์) และต้องไม่ต่ำกว่าค่าธรรมเนียม	helpers.ts normalizeReading	TC-62
ธนาคารจากรหัสใน QR (002 กรุงเทพ, 004 กสิกร, 006 กรุงเทพ, 014 SCB, ...) ไม่ใช่จากโลโก้	slip.ts BANK_BY_CODE, bankFromCode	TC-64, E2E QR
สลิป e-wallet ที่มี QR แบบของตัวเอง: รูปขนาดหน้าจอที่มี QR ใด ๆ ส่งให้ AI ตัดสิน รูปจากกล้องไม่ส่ง	processSlip.ts, slips.ts detectSlipQr	E2E auto (6 รูป ส่ง 3)
รายรับ/รายจ่ายจากชื่อบนสลิป: เรียนรู้ชื่อผู้ใช้จากบรรทัด "จาก" (เก็บเป็นลายนิ้วมือ) สลิปที่เพื่อนส่งมา = รายรับ โอนระหว่างบัญชีตัวเอง $\rightarrow$ รอตรวจ	slipNames.ts, services/myNames.ts	TC-65, TC-84
เปิดแอปแล้วหาสลิปใหม่อัตโนมัติ: ครั้งแรก 30 วันต่อไปนับจากครั้งล่าสุด ไม่ซ้ำภายใน 5 นาที ครั้งละ ≤ 150 รูป ย้อนการรวมยอดได้	autoScan.ts, AutoScanProvider.tsx, AutoScanBanner.tsx	TC-35, TC-36, MT-11, E2E auto
ขอสิทธิ์รูปภาพเองครั้งเดียวตอนเปิดแอปครั้งแรก ส่ง AI เฉพาะรูปที่มี QR สลิป	AutoScanProvider (askedGalleryOnce)	E2E auto
ไม่มีสลิปหลุด: เน็ตหลุดลองใหม่ 1 ครั้งหลัง 3 วินาทีแล้วหยุดบอกเหตุผล รูปอ่านไม่ได้ลองอีกรอบหน้า (รวม 2 ครั้ง)	nextAutoScanMark, MAX_AUTO_ATTEMPTS, useSlipScanner	TC-56, E2E offline
หน้าสแกนไม่ต้องเลือกช่วงก่อน (มือถือค้น 30 วันเอง เว็บกดเลือกรูปครั้งเดียว) สลิปบันทึกตามวันที่บนสลิป (ย้อนได้ 1 ปี) แสดงตามช่วงและตามวัน	scan.tsx, useSlipScanner.ts, PeriodSummary.tsx	E2E web scan

Acceptance criteria (ย่อ)	โค้ด	เทสต์
อ่านวันที่แบบไทย (พ.ศ. 2 หลัก, "ก.ย.") อ่านไม่ได้ หรืออนาคต → ไม่เดา	<code>dates.ts parseSlipDate</code>	TC-04-06, TC-21, TC-85
สลิปนอกช่วงเวลาถูกข้าม	<code>classifyCandidate</code>	TC-22
กันซ้ำ 3 ชั้น: เลขอ้างอิง (QR), hash ของรูป, ยอด+เวลา+ผู้รับ และ unique index	<code>slip.ts, migration</code>	TC-23, TC-24, TC-83
หา QR ให้เจอแม้เล็กอยู่มุมล่าง (ตัดดูครึ่งล่าง/มุมขวาล่าง เฉพาะรูปขนาดหน้าจอ) มือถือที่ไม่มีตัวอ่าน QR ได้ข้อความบอก	<code>slip.ts firstSlipQr, isScreenSized, SLIP_QR_REGIONS; slips.ts detectSlipQr</code>	TC-57, E2E QR
ตรวจจากรูปใหม่ไปเก่า (1 วัน → 7 วัน → 1 เดือน → 6 เดือน → 1 ปี) เปลี่ยนช่วงระหว่างสแกนไม่ได้ พักและสแกนต่อได้	<code>scanQueue.ts</code>	TC-26-30
ผลอ่านจาก AI ถูกทำความเข้าใจบนเซิร์ฟเวอร์ (ปี พ.ศ., คอมมา, ช่องว่างต้องมีความมั่นใจ 0)	<code>helpers.ts normalizeReading, pickProvider</code>	TC-38-42
key ของ AI จาก Supabase Secrets (ณชื่อพิมพ์ผิด) Gemini ติดลิมิตลองรันถัดไป แอปรอ 20/40 วินาทีแล้วอ่านต่อเอง	<code>helpers.ts readKey, geminiStep; slips.ts readSlip</code>	TC-54, TC-55, E2E busy

เส้นทางของสลิป 1 ใบ

จากรูปในแคลเลอริจันเป็นรายการในยอดเงิน: 3 ที่ (มือถือ เซิร์ฟเวอร์ ฐานข้อมูล) 8 ขั้นตอน

1. หารูป `src/services/gallery.ts`

รูปที่สร้างหลังจากเวลาที่กำหนด เรียงใหม่ไปเก่า ข้ามรูปแนวนอน (สลิปเป็นแนวตั้งเสมอ) ข้ามรูปที่เคยตรวจแล้ว



2. หา QR บนมือถือ `src/services/slips.ts detectSlipQr`

สแกนทั้งรูป ถ้าไม่เจอและเป็นรูปขนาดหน้าจอ ตัดดูครึ่งล่าง และมุมขวาล่าง · ไม่มี QR สลิป (และไม่มี QR อื่นในรูปขนาดหน้าจอ) → "ไม่ใช่ สลิป" รูปไม่ออกจากมือถือ



3. ย่อรูปและทำลายนิ้วมือ `prepareImage`

กว้างไม่เกิน 1,100 px JPEG ~150 KB · SHA-256 ของรูป → เคยเห็นแล้ว = ซ้ำ (ไม่ต้องส่ง AI)

↓ ส่งรูป (base64) ผ่าน HTTPS

4. เซิร์ฟเวอร์อ่าน 2 ครั้ง `supabase/functions/parse-slip/index.ts`

ตรวจผู้ใช้ ขนาด โควตา 300 ใบ/วัน → AI รุ่นหลัก + AI รุ่นตรวจ (พร้อมกัน) → ทำความสะอาดผลแต่ละรอบ → รวม 2 ผล (ตรงกัน = เชื่อ, ไม่ตรง = สง + เก็บ 2 ค่า)

↓ reading + check

5. แปลงเป็นผู้สมัคร `src/domain/slip.ts normalizeReading`

สตาจค์, วันไทย, เวลา, รายรับ/รายจ่ายจากชื่อ, ธนาการจาก QR, เลขอ้างอิงจาก QR, หมวด, ความมั่นใจต่ำสุด, สง



6. ตัดสิน `classifyCandidate`

ซ้ำ → ข้าม · เก่ากว่า 1 ปี → ข้าม · ไม่มีธง → ready · มีธง → needs_review



7. สถานะเริ่มต้น `initialStatus`

ready + เปิดรวมยอดทันที → confirmed · นอกนั้น → draft



8. บันทึก `repo.insert` (ตาราง transactions)

source = slip, slip_ref, slip_image_hash, ocr_confidence, review_flags, note (ธนาการ/2 ค่า) · unique index ปฏิเสธซ้ำอีกชั้น → ยอดเงิน (FR-2) และ Runway (FR-6) อัปเดต

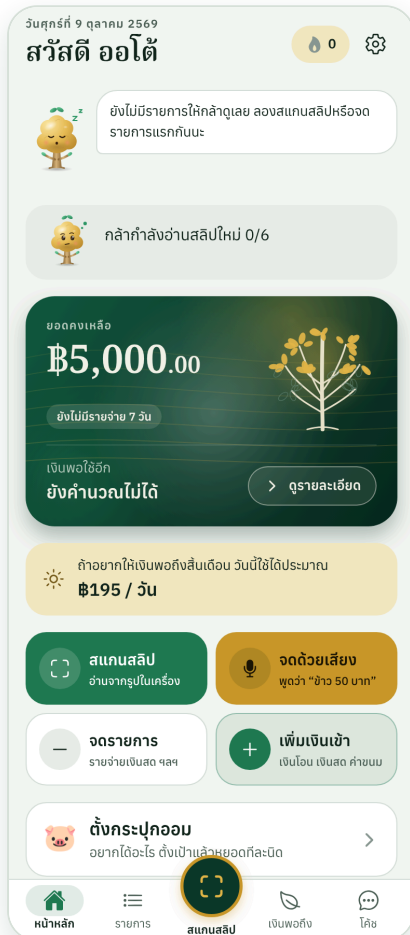
แผนที่โค้ดของ FR-4

ไฟล์	ชั้น	หน้าที่
<code>src/domain/slip.ts</code>	กฎ (ฟังก์ชัน ล้วน)	QR, ธนาการ, normalizeReading, ชำ, ตัดสิน, บันทึกข้อความ (405 บรรทัด)
<code>src/domain/slipNames.ts</code>	กฎ	ใครคือ "ฉัน" บนสลิป → รายรับ/รายจ่าย (119)
<code>src/domain/scanQueue.ts</code>	กฎ	state machine ของการสแกน (173)
<code>src/domain/autoScan.ts</code>	กฎ	ช่วงเวลาของการสแกนอัตโนมัติ (45)
<code>src/domain/dates.ts</code>	กฎ	parseSlipDate, parseSlipTime (บรรทัด 159–261)
<code>src/services/slips.ts</code>	มือถือ	หา QR, ย่อรูป, เรียกเชิร์ฟเวอร์, จำรูปที่ตรวจแล้ว (253)
<code>src/services/gallery.ts</code> , <code>src/services/gallery.web.ts</code>	มือถือ/เว็บ	สิทธิ์และค้นรูปในแกลเลอรี (60, 49)
<code>src/services/processSlip.ts</code>	มือถือ	ทำ 1 รูปตั้งแต่ต้นจนจบ ใช้ร่วมกันทั้งหน้าสแกนและสแกนอัตโนมัติ (108)
<code>src/services/useSlipScanner.ts</code>	มือถือ	ตัวควบคุมของหน้าสแกน (240)
<code>src/services/AutoScanProvider.tsx</code>	มือถือ	สแกนอัตโนมัติตอนเปิดแอป (321)
<code>src/services/myNames.ts</code> , <code>src/data/prefs.ts</code>	มือถือ	เก็บลายนิ้วมือชื่อ / ตั้งค่าการสแกน (42, 48)
<code>src/app/scan.tsx</code> , <code>src/app/drafts.tsx</code>	หน้าจอ	หน้าสแกน / สลิปรอยืนยัน (259, 96)
<code>src/ui/AutoScanBanner.tsx</code> , <code>src/ui/ScannerStage.tsx</code> , <code>src/ui/WaitNotice.tsx</code>	หน้าจอ	การ์ดบนหน้าหลัก / เวทแอนิเมชัน / นับถอยหลัง
<code>supabase/functions/parse-slip/index.ts</code>	เชิร์ฟเวอร์	ฟังก์ชันอ่านสลิป (154)
<code>supabase/functions/_shared/helpers.ts</code> , <code>supabase/functions/_shared/common.ts</code>	เชิร์ฟเวอร์	ทำความเข้าใจผล รวม 2 ผล เลือก AI โควตา
<code>supabase/migrations/20260927000000_init.sql</code>	ฐานข้อมูล	unique index กันซ้ำ

ทัวร์หน้าจอ: เปิดแอปแล้วกดตัวเอง

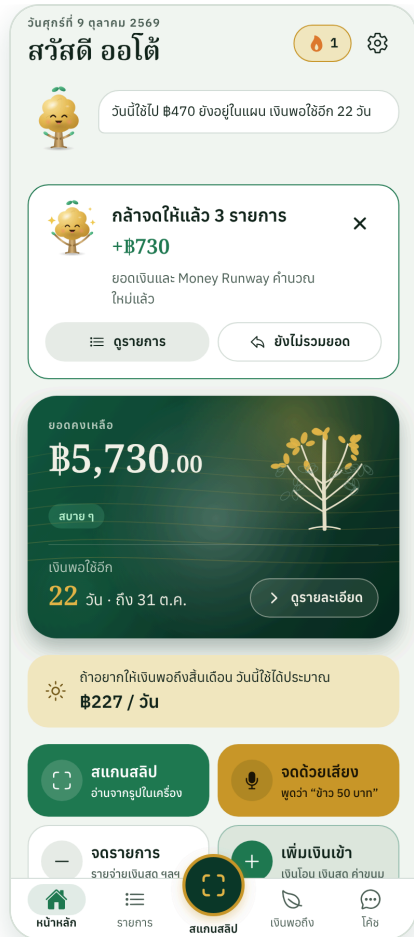
การสแกนอัตโนมัติ: ผู้ใช้ไม่ต้องแตะอะไรเลย แค่เปิดแอป

ภาพชุดนี้คือบัญชีใหม่ที่ตั้งยอดเงินเริ่มต้น ฿5,000 ในแกลเลอรีมีรูป 6 รูป (สลิป 3 ใบ รูปอื่น 3 รูป) พอเข้าหน้าหลักได้ประมาณ 1.8 วินาที (รอแอนิเมชันเปิดแอปจบก่อน) การ์ดของ FR-4 ก็โผล่ขึ้นมาเอง



กำลังอ่าน: ยอดยังเป็น ฿5,000 เพราะยังไม่มีรายการ

- 1 การ์ดสแกนอัตโนมัติ ([src/ui/AutoScanBanner.tsx](#)) พื้นสีอ่อน มีน้องกล้วยหน้า "ครุ่นคิด" และแสงสแกนวิ่งผ่านตัวกล้วย
- 2 ข้อความ "กล้วยกำลังอ่านสลิปใหม่ 0/6" = อ่านเสร็จแล้ว 0 จากที่เจอ 6 รูป (ก่อนรู้จำนวนจะขึ้นว่า "กล้วยกำลังหาสลิปใหม่ในแกลเลอรี...")
- 3 เมื่อจำนวนมากกว่า 0 จะมี แถบความคืบหน้าสีทองใต้ข้อความ และถ้า AI คิวเต็มจะมีบรรทัดนับถอยหลังเล็ก ๆ "AI มีคิวเยอะ รออีก ... วินาที"
- 4 ปุ่มสแกนกลางแถบเมนู มีวงคลื่นกระเพื่อมตลอดเวลาที่การสแกนอัตโนมัติทำงาน (ปกติกระเพื่อมแค่ 3 ครั้ง)
- 5 การ์ดยอดเงินยังเป็น ฿5,000 "ยังไม่มีรายการจ่าย 7 วัน" และ "เงินพอใช้อีก ยังคำนวณไม่ได้" เพราะ FR-6 ยังไม่มีข้อมูลรายการจ่ายให้คำนวณ
- 6 น้องกล้วยด้านบนยังพูดว่า "ยังไม่มีรายการให้กล้วยเลย ลองสแกนสลิปหรือจดยอดรายการแรกกันนะ"



อ่านเสร็จ: จดให้ 3 รายการ ยอดเงินและ Runway จำนวนใหม่ทันที

- 1 การ์ดเปลี่ยนเป็นขอบเขียว น้องกล้วยมูตใจ และมีฟลุกระดาษ (confetti) 1 ครั้ง
- 2 "กล้วยจืดให้แล้ว 3 รายการ" และยอดรวม $+\text{฿}730 = -350$ (ร้านข้าวมันไก่) $+1,200$ (พี่ชายโอนให้) -120 (ร้านชาวม) ตัวเลขเป็นสีเขียวเพราะรวมแล้วเป็นบวก
- 3 "ยอดเงินและ Money Runway จำนวนใหม่แล้ว"
- 4 ปุ่ม "ดูรายการ" ไปหน้ารายการ (ถ้ามีแบบร่างจะเป็นปุ่ม "ตรวจสอบสลิป" แทน)
- 5 ปุ่ม "ยังไม่รวมยอด" = ย้อนการนับอัตโนมัติ: ทั้ง 3 รายการ ถูกเปลี่ยนเป็นแบบร่าง ขึ้นข้อความ "ย้ายไปที่สลิปรอยืนยันแล้ว" ยอดเงินกลับเป็นเหมือนเดิม
- 6 ✕ ปิดการ์ด (รายการยังอยู่ แค่ซ่อนการ์ด)
- 7 ยอดคงเหลือ $\text{฿}5,730$ ($5,000 + 730$) ป้าย "สบาย ๆ" เงินพอใช้อีก 22 วัน ถึง 31 ต.ค. และ "วันนี้ใช้ได้ประมาณ $\text{฿}227/\text{วัน}$ " (FR-6 จำนวนใหม่ให้ทันที)
- 8 น้องกล้วยพูดว่า "วันนี้ใช้ไป $\text{฿}470$ ยังอยู่ในแผน..." ($350 + 120 = 470$ คือรายจ่ายของวันนี้) และเปลวไฟ (streak) ขึ้นเป็น 1

6 รูปในแกลเลอรี เกิดอะไรขึ้นกับแต่ละรูป

รูป (ใหม่ → เก่า)	ขนาด	หา QR บนมือถือ	ผล
slip-qr-2.jpg (10 นาทีก่อน)	1080×1920	QR สลิป ธนาคารสสส 014 (SCB)	ส่ง AI → ร้านข้าวมันไก่ $-\text{฿}350$ มั่นใจทุกช่อง → รวมยอดแล้ว
photo-1.jpg (20 นาที)	600×1000	ไม่มี QR	ไม่ใช่สลิป รูปไม่ออกจากเครื่อง
slip-qr-3.jpg (30 นาที)	1080×1920	QR สลิป สห 006 (Krungthai)	ส่ง AI → พี่ชาย $+\text{฿}1,200$ → รวมยอดแล้ว
photo-2.jpg (40 นาที)	600×1000	ไม่มี QR	ไม่ใช่สลิป
wallet-qr.jpg (50 นาที)	1080×1920	มี QR แต่ไม่ใช่ QR สลิปธนาคาร (เป็น QR ของ e-wallet)	รูปขนาดหน้าจอ + มี QR → ส่งให้ AI ตัดสิน → ร้านชาวม $-\text{฿}120$ (TrueMoney) → รวมยอดแล้ว
camera-qr.jpg (60 นาที)	3000×4000	QR แบบเดียวกับข้างบน	เป็น รูปจากกล้อง (ไม่ใช่ขนาดหน้าจอ) → ไม่ส่ง = ไม่ใช่สลิป

สรุป: เจอ 6 รูป **ส่งให้ AI แค่ 3 รูป** อีก 3 รูปไม่เคยออกจากมือถือ นี่คือสิ่งที่เกสต์ E2E "auto scan" ตรวจสอบทุกครั้ง (ส่งเซิร์ฟเวอร์ต้องเป็น 3 ครั้งพอดี)

การ์ดนี้มีได้ 5 หน้าตา

สถานะ (phase)	เกิดเมื่อ	สิ่งที่ผู้ใช้เห็น
idle	ยังไม่เริ่ม, ไม่มีสลิปใหม่, หรือกด × แล้ว	ไม่มีการ์ดเลย (ไม่มีอะไรใหม่ = เจียบ ไม่รบกวน)
needs_permission	ยังไม่ได้สิทธิ์เข้าถึงรูปภาพ	"ให้กำลังใจจากสลิปให้อัตโนมัติใหม่" + "แค่เปิดแอป กล้าจะหาสลิปใหม่ในแคเลเลอรี่ อ่านยอด แล้วคำนวณเงินให้ทันที รูปอื่นไม่ออกจากมือถือ" + ปุ่มทอง "อนุญาตเข้าถึงรูปภาพ"
scanning	กำลังหาและอ่าน	ภาพแรกด้านบน
done	อ่านจบและได้รายการอย่างน้อย 1	ภาพที่สอง หรือถ้ามีแต่แบบร่าง: "กล้าเจอสลิปใหม่ 2 ใบ" + "ต้องตรวจ 2 รายการที่อ่านไม่ชัด" + ปุ่ม "ตรวจสอบสลิป"
error	เน็ตหลุด, AI ชัดข้อง, โควตาเต็ม	น้องกล้าหน้ากังวล + เหตุผล เช่น "เชื่อมต่ออินเทอร์เน็ตไม่ได้ สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า" + "ก่อนหยุดกล้าจัดให้แล้ว 2 รายการ (รอตรวจ 1)"

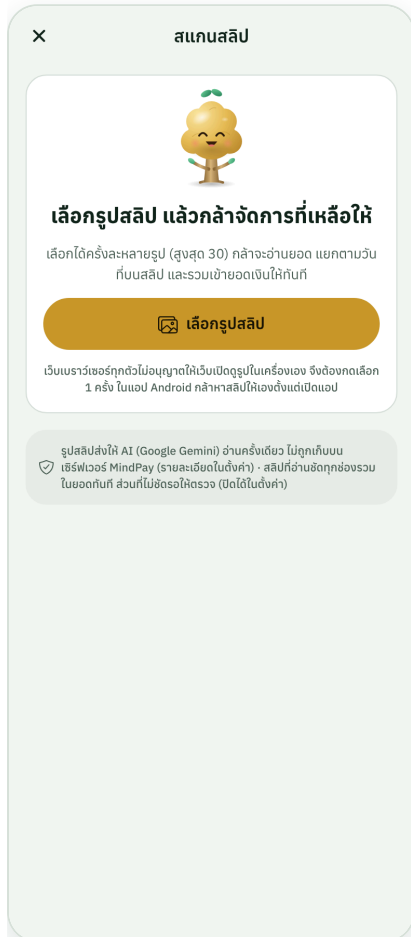
★ **ทำไมไม่ขอสิทธิ์ทุกครั้ง**

แอปขอสิทธิ์รูปภาพ **เองแค่ครั้งเดียว** ตอนเปิดแอปครั้งแรก (มือถือแสดงกล่องขออนุญาตของระบบ) ถ้าผู้ใช้ปฏิเสธ แอปจะไม่ถามซ้ำอีก แต่จะวางการ์ด needs_permission ไว้ให้กดเองเมื่อพร้อม การถามซ้ำทุกครั้งที่เปิดแอปเป็น UX ที่น่ารำคาญและ Android อาจบล็อกการถามถ้าถูกปฏิเสธหลายครั้ง

ทัวร์หน้าจอ: หน้าสแกนสลิป

เปิดจากปุ่มลัด "สแกนสลิป" หรือปุ่มกลมตรงกลางแถบเมนู

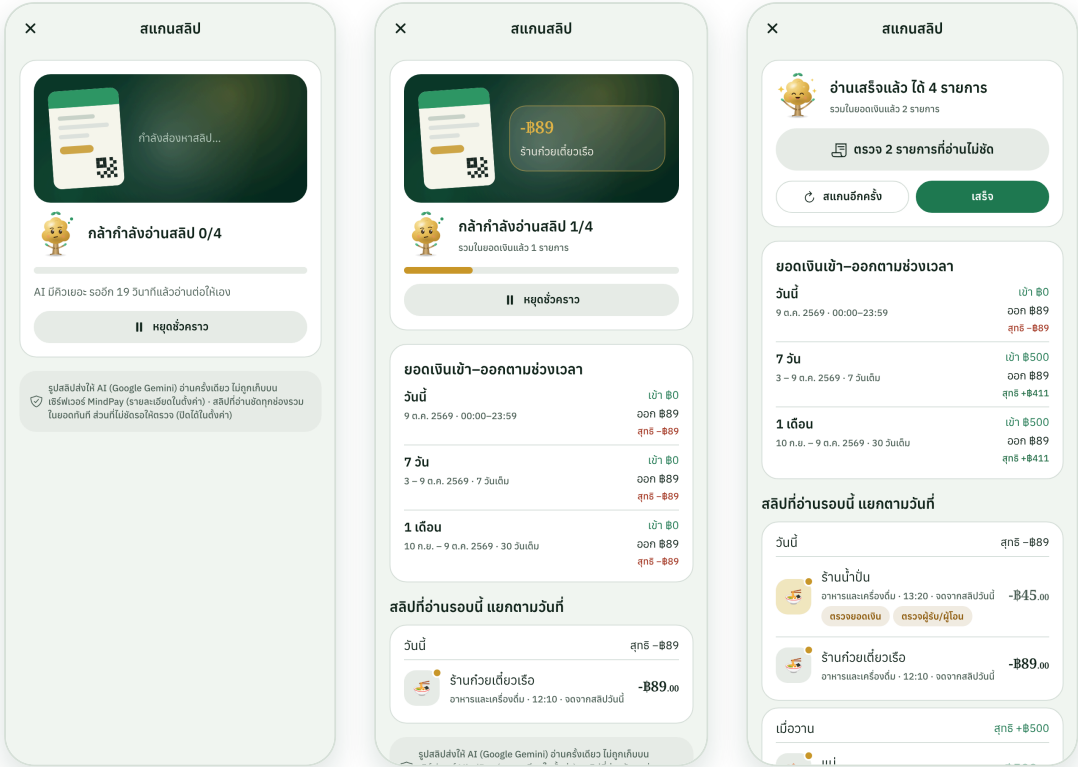
หน้าสแกน ([src/app/scan.tsx](#)) ทำงานต่างกันตามที่เปิด: แอป **Android** เปิดหน้าแล้วเริ่มนับแคลเลอร์ 30 วันล่าสุดทันที ส่วน **เว็บ** (iPhone, คอมพิวเตอร์) เบราวเซอร์ไม่อนุญาตให้เว็บรันรูปในเครื่องเอง จึงต้องกด "เลือกรูปสลิป" 1 ครั้ง หลังจากนั้นทุกอย่างอัตโนมัติเหมือนกัน



หน้าสแกนบนเว็บก่อนเลือกรูป

- 1 หัวหน้าจอ: ✕ ปิด และชื่อ "สแกนสลิป"
- 2 นื่องกล่าวหน้ายืม ขนาดใหญ่ (92 จุด)
- 3 หัวข้อ "เลือกรูปสลิป แล้วกล่าวจัดการที่เหลือให้" บอกทั้งสิ่งที่ต้องทำและสิ่งที่จะได้ ในประโยคเดียว
- 4 คำอธิบาย: เลือกได้ครั้งละหลายรูป (สูงสุด 30) อ่านยอด แยกตามวันที่บนสลิป รวมเข้ายอดเงินให้กับที่
- 5 ปุ่มทอง "เลือกรูปสลิป" (ไอคอนรูปภาพ) เปิดตัวเลือกรูปของระบบแบบเลือกได้หลายรูป
- 6 ตัวเล็กอธิบายเหตุผลว่าทำไมเว็บต้องกดเลือก และบอกว่าบนแอป Android ไม่ต้องกด
- 7 กล่องความเป็นส่วนตัว (ไอคอนโล่) อยู่ล่างสุดทุกสถานะ: รูปส่งให้ AI อ่านครั้งเดียว ไม่ถูกเก็บบนเซิร์ฟเวอร์ MindPay และสลิปที่ชดรวมยอดทันที (ปิดได้ในตั้งค่า)

ระหว่างอ่าน: คิว AI เต็ม แล้วอ่านต่อเอง



AI ตอบว่าคิวเต็ม: นับถอยหลัง 'รออีก 19 วินาทีแล้วอ่านต่อให้เอง'

อ่านใบแรกได้: -฿89 ร้านก๋วยเตี๋ยวเรือ เติ่งขึ้นข้างสลิป 1/4

อ่านครบ 4 รูป: ได้ 4 รายการ รวมยอดแล้ว 2

ส่วน	ภาพซ้าย (รอ)	ภาพกลาง (กำลังอ่าน)	ภาพขวา (เสร็จ)
เวทีสแกน (ScannerStage)	สลิปวาดด้วยโค้ด + แสงสแกน + "กำลังส่องหาสลิป..."	ยอดของสลิปล่าสุด -฿89 และชื่อ เติ่งขึ้นพร้อมประกาย	หายไป (อ่านจบแล้ว)
หัวข้อ	"กลัากำลังอ่านสลิป 0/4"	"กลักำลังอ่านสลิป 1/4"	"อ่านเสร็จแล้ว ได้ 4 รายการ"
บรรทัดเล็ก	(ว่าง)	"รวมในยอดเงินแล้ว 1 รายการ"	"รวมในยอดเงินแล้ว 2 รายการ"
แถบความคืบหน้า	0%	25% (1 จาก 4)	ช้อน
ปุ่ม	"หยุดชั่วคราว"	"หยุดชั่วคราว"	"ตรวจ 2 รายการที่อ่านไม่ชัด", "สแกนอีกครั้ง", "เสร็จ"

เวลารอไม่ได้สุ่ม: รอบแรกที่ AI ตอบว่าคิวเต็มแอปรอ 20 วินาที ถ้าคิวยังเต็มรอ 40 วินาที ถ้ายังเต็มอีกจึงหยุดและบอกผู้ใช้ (ค่าคงที่ `BUSY_WAITS_S = [20, 40]` ใน `src/services/slips.ts:201`) ภาพซ้ายเขียน 19 เพราะถ่ายหลังเริ่มรอไป 1 วินาที ตัวเลขนับลงทุกวินาที

4 รูปที่เลือก กลายเป็นอะไร

รูปที่เลือก	สิ่งที่ AI ตอบ (กำหนดไว้ในเซิร์ฟเวอร์จำลอง)	ผลในแอป
photo-1.jpg	ครั้งแรก "คิวเต็ม" → รอ 20 วินาที → อ่านใหม่ได้ ร้านก๋วยเตี๋ยวเรือ ฿89 ความมั่นใจ ยอด 0.98 วันที่ 0.95 ชื่อ 0.95	ทุกช่อง $\geq 0.8 \rightarrow$ รวมยอดแล้ว

รูปที่เลือก	สิ่งที่ AI ตอบ (กำหนดไว้ในเซิร์ฟเวอร์จำลอง)	ผลในแอป
photo-2.jpg	ร้านน้ำปั่น ฿45 ความมั่นใจ ยอด 0.5 วันที่ 0.9 ชื่อ 0.6	ธง 2 ช่อง → แบบร่าง "ตรวจยอดเงิน" "ตรวจผู้รับ/ผู้โอน"
photo-3.jpg	ร้านรองเท้า ฿1,250 แต่ AI 2 ตัวอ่านยอดไม่ตรงกัน : 1,250.00 กับ 1,280.00	ธงยอดเงิน → แบบร่าง และจำกัด 2 ค่าไว้ให้เลือก
slip-qr.jpg	แม่ +฿500 ความมั่นใจ 0.99 / 0.96 / 0.9	รวมยอดแล้ว · รูปนี้มี QR จริง เลขอ้างอิงและธนาคาร (KBank รหัส 004) จึงมาจาก QR ไม่ใช่จากคำตอบของ AI

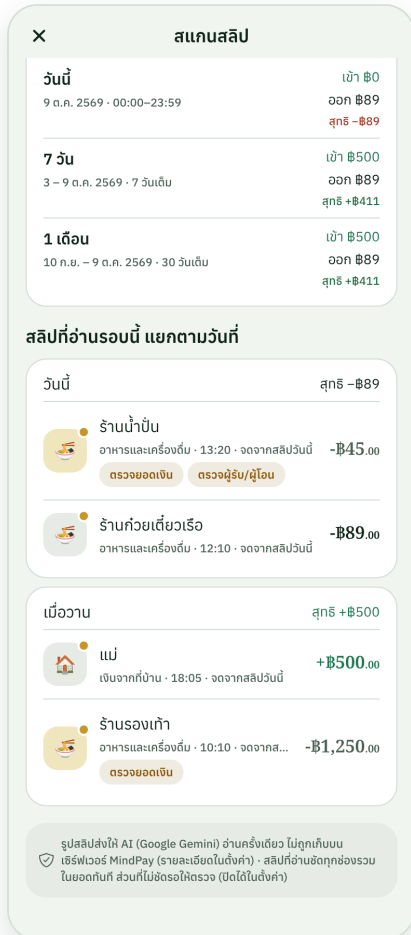
i บนเว็บ รูปที่ผู้ใช้เลือกเองถูกอ่านทุกรูป

photo-1, 2, 3 ไม่มี QR เลย แต่ก็ถูกส่งให้ AI เพราะ **ผู้ใช้เลือกเอง** = ผู้ใช้ยืนยันแล้วว่าเป็นสลิป (เช่น รูปถ่ายสลิปกระดาษ) กฎ "ต้องมี QR" ใช้กับรูปที่แอป **ค้นเอง** จากแกลเลอรีเท่านั้น ในโค้ดคือ

```
requireQr: requireQr && !picked ( src/services/useSlipScanner.ts:138 )
```

ทวิรห์หน้าจอ: ผลการสแกนตามช่วงเวลาและตามวัน

สลิปถูกบันทึกตามวันที่บนสลิป แล้วแสดงผลให้ดูได้ 2 มุม



เลื่อนลงในหน้าสแกน: ตามช่วงเวลา และแยกตามวัน

- 1 "ยอดเงินเข้า-ออกตามช่วงเวลา" (คอมโพเนนต์ `src/ui/PeriodSummary.tsx` ตัวเดียวกับ FR-2) วันนี้ 9 ต.ค. 2569 · 00:00–23:59: เข้า ฿0 ออก **฿89** สุทธิ -฿89
- 2 7 วัน (3 – 9 ต.ค. · 7 วันเต็ม): เข้า **฿500** ออก ฿89 สุทธิ **+฿411**
- 3 1 เดือน (10 ก.ย. – 9 ต.ค. · 30 วันเต็ม): เท่ากับ 7 วัน เพราะบัญชีนี้เพิ่งสร้าง
- 4 "สลิปที่อ่านรอบนี้ แยกตามวันที่" แต่ละวันเป็นการ์ด หัวการ์ดคือชื่อวัน แบบสัมพันธ์ (วันนี้ / เมื่อวาน / วันที่) และยอดสุทธิของวันนั้น
- 5 แลวกที่เป็น **แบบร่าง**: พื้นไอคอนสีทอง ยอดเงินสีเทา และป้าย "ตรวจสอบยอดเงิน" "ตรวจสอบผู้รับ/ผู้โอน"
- 6 แลวกที่ **รวมยอดแล้ว**: พื้นไอคอนสีเขียวอ่อน ยอดเงินสีปกติ (แดงรายจ่าย เขียวรายรับ)
- 7 จุดทอของ **มุมมองไอคอน** + ข้อความ "จดจากสลิปวันนี้" = รายการนี้ **ถูกจดวันนี้** (แม้วันที่บนสลิปจะเป็นเมื่อวาน เช่น แลวก "แม่")

! จุดที่คนมักงง: ทำไมออกแค่ ฿89 ทั้งที่อ่านรายจ่ายได้ 3 ใบ

รายจ่ายที่อ่านได้มี ฿89 + ฿45 + ฿1,250 แต่ **แบบร่างยังไม่นับในยอดใด ๆ** ทั้งยอดคงเหลือ (FR-2) ยอดตามช่วงเวลา ยอดสุทธิรายวัน และ Money Runway (FR-6) นี่คือการขุดข้อสำคัญที่สุดของ FR-4: **ข้อมูลที่ไม่แน่ใจต้องไม่ไปเปลี่ยนตัวเลขเงินของผู้ใช้** จนกว่าผู้ใช้จะยืนยัน ในโค้ด `groupByDay` ใช้

```
.filter((t) => t.status === 'confirmed') ก่อนรวมยอด (src/domain/summary.ts:123)
```

การ์ดวัน	รายการ	สุทธิที่แสดง	คิดยังง
วันนี้	ร้านน้ำปั่น -฿45 (แบบร่าง), ร้านก๋วยเตี๋ยวเรือ -฿89	-฿89	นับเฉพาะ -89
เมื่อวาน	แม่ +฿500, ร้านรองเท้าน้ำ -฿1,250 (แบบร่าง)	+฿500	นับเฉพาะ +500

บรรทัดสรุปตอนอ่านจบ บอกครบทุกรูป

ในการ์ดความคืบหน้า บรรทัดเล็กใต้หัวข้อต่อข้อความตามสถานะของแต่ละรูป แสดงเฉพาะสถานะที่มีจำนวนมากกว่า 0 ตัวอย่างถ้ามีครบทุกแบบ:

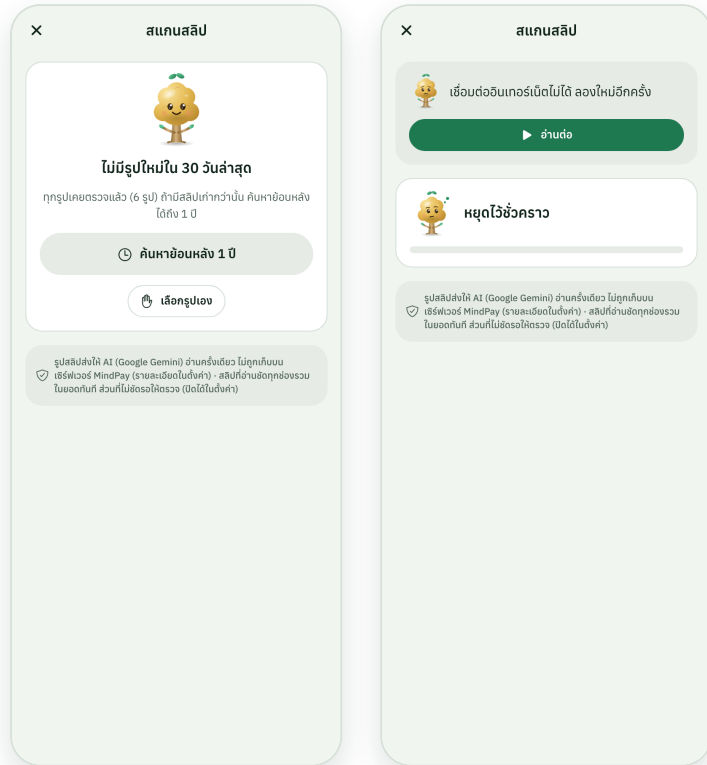
**รวมในยอดเงินแล้ว 3 รายการ · ซ้ำ 1 · รูปอื่น (ไม่มี QR สลิป) 12 · อ่านไม่ได้ 1 ·
เท่ากับ 1 ปี 2**

ผู้ใช้จึงรู้ว่าทุกรูปไปไหน ไม่มีรูปไหน "หายเงียบ"

และถ้าอ่านจบแล้ว **ไม่ได้รายการเลย** แต่มีรูปที่ไม่มี QR อยู่ หน้าสแกนจะช่วยเดาปัญหาให้: "มีสลิปในเครื่องแต่กล้าไม่เจอ? กล้า
หาสลิปจาก QR ตรวจสอบสลิปที่แอปธนาคารพิมพ์ไว้ ถ้าเป็นรูปถ่ายสลิปกระดาษ หรือรูปที่ตัด QR ออก ให้เลือกรูปนั่นเอง"
พร้อมปุ่ม "เลือกรูปเอง"

ทัวร์หน้าจอ: ไม่มีรูปใหม่ และเน็ตหลุด

สองสถานการณ์ที่ต้องบอกผู้ใช้ให้ชัด ว่า "ไม่มี" ต่างจาก "ยังไม่ได้ทำ"



เปิดหน้าสแกนหลังสแกนอัตโนมัติ
เสร็จ: 'ทุกรูปเคยตรวจแล้ว (6 รูป)'

เน็ตหลุดระหว่างอ่าน: หยุดไว้ชั่วคราว
พร้อมบอกเหตุผล และปุ่ม 'อ่านต่อ'

ไม่มีรูปใหม่ใน 30 วันล่าสุด

ภาพซ้ายใช้แกลเลอรีจำลองชุดเดียวกับการสแกนอัตโนมัติ (ทำตัวเหมือนแอปมือถือ) หลังสแกนอัตโนมัติจบ ทั้ง 6 รูปถูก "จำ" ไว้แล้ว รวมถึงรูปที่ไม่ใช้สลิป พอเปิดหน้าสแกน แอปค้น 30 วันล่าสุดได้ 6 รูป ตัดรูปที่เคยตรวจออกเหลือ 0 จึงแสดง:

- หัวข้อ "ไม่มีรูปใหม่ใน 30 วันล่าสุด" และ "ทุกรูปเคยตรวจแล้ว (6 รูป) ถ้ามีสลิปเก่ากว่านั้น ค้นหาย้อนหลังได้ถึง 1 ปี"
- ปุ่ม "ค้นหาย้อนหลัง 1 ปี" (ค้นใหม่ด้วยช่วง 1 ปี) และ "เลือกรูปเอง"

การ์ดนี้แสดงเฉพาะเมื่อ **ค้นเสร็จแล้วจริง ๆ** (ตัวแปร `searched`) ถ้ายังค้นอยู่จะเห็น "กลั้กำลังหาสลิปใหม่ในแกลเลอรี 30 วันล่าสุด..." แทน และถ้าการสแกนอัตโนมัติยังอ่านอยู่ หน้าสแกนจะรอให้จบก่อน: "กลั้กำลังอ่านสลิปใหม่ที่เจอตอนเปิดแอปอยู่ (2/6) เสร็จแล้วจะค้นต่อให้เลย" เพื่อไม่ให้ 2 ระบบอ่านรูปเดียวกันซ้ำ (เปลืองโควตา AI 2 เท่า)

เน็ตหลุดระหว่างอ่าน

1. ส่งรูปไม่สำเร็จ (ไม่มีเน็ต) → แอป **รอ 3 วินาทีแล้วลองรูปเดิมอีก 1 ครั้ง** (เพื่อเน็ตกระตุกแป๊บเดียว)
2. ยังไม่ได้ → **หยุดทั้งคิว** (ไม่ข้ามไปรูปถัดไป เพราะทุกรูปจะล้มเหมือนกัน) แสดงการ์ดน้องกลั้หน้ากังวล "เชื่อมต่ออินเทอร์เน็ตไม่ได้ ลองใหม่อีกครั้ง" + ปุ่มเขียว "อ่านต่อ"
3. การ์ดความคืบหน้าเปลี่ยนหัวข้อเป็น "หยุดไว้ชั่วคราว"

4. กด "อ่านต่อ" → รูปที่ค้างอยู่ถูกอ่านใหม่ตั้งแต่ต้น ไม่มีรูปไหนถูกนับว่า "อ่านไม่ได้" เพราะความผิดของเน็ต

≈ เปรียบเทียบ

เหมือนพนักงานส่งของที่เจอถนนปิด เขาไม่โยนพัสดุทิ้งแล้วไปส่งกล่องถัดไป (เพราะถนนก็ปิดสำหรับทุกกล่อง) แต่จอดรอ แจ้งลูกค้าว่าถนนปิด และพอถนนเปิดก็ส่งกล่องเดิมต่อ

ข้อความทั้งหมดที่ผู้ใช้อาจเห็นเมื่อมีปัญหา

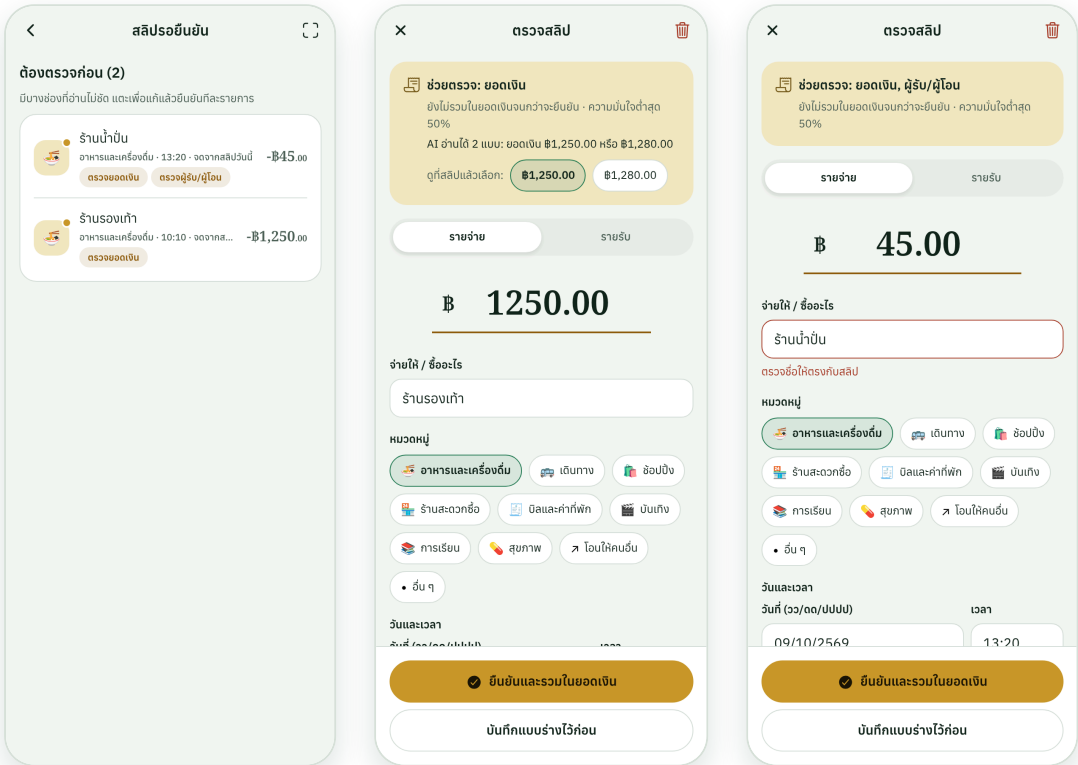
รหัสปัญหา	ข้อความ (จาก src/services/slips.ts:182-194)	หน้าสแกนทำอะไร
network	เชื่อมต่ออินเทอร์เน็ตไม่ได้ ลองใหม่อีกครั้ง	ลองซ้ำ 1 ครั้ง แล้วหยุดทั้งคิว
busy	ตอนนี้ AI อ่านสลิปมีคนใช้เยอะ ลองใหม่อีกสักครู่ สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า	รอ 20 และ 40 วินาทีก่อน แล้วจึงหยุด
quota	วันนี้อ่านสลิปครบโควตาแล้ว ลองใหม่อีกครั้ง	หยุด (300 ใบ/วัน/บัญชี)
unauthorized	เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง	หยุด
not_configured	ระบบอ่านสลิปยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ...	หยุด (ข้อความนี้สำหรับผู้ดูแลระบบ)
server	ระบบอ่านสลิปขัดข้องชั่วคราว สลิปที่เหลือจะอ่านต่อให้ภายหลัง	หยุด
cloud_required	ต้องเข้าสู่ระบบด้วยบัญชีจริงก่อน จึงจะอ่านสลิปได้	หยุด (โหมดทดลอง)
qr_unavailable	มือถือเครื่องนี้หา QR ของสลิปเองไม่ได้ (ต้องมี Google Play Services) ใช้ "เลือกรูปเอง" แทนได้	หยุด
reader_error	อ่านรูปนี้ไม่สำเร็จ	ไม่หยุด: นับรูปนี้ว่า "อ่านไม่ได้" แล้วไปรูปถัดไป
too_large	รูปใหญ่เกินไป	ไม่หยุด: รูปนี้ "อ่านไม่ได้" ไปรูปถัดไป

หลักการแบ่งคือ: ปัญหาที่ **เกิดกับทุกรูป** (เน็ต, AI, บัญชี) → หยุดและบอกผู้ใช้ · ปัญหาที่ **เกิดกับรูปเดียว** → ข้ามรูปนั้นไปรูปต่อไป

ทัวร์หน้าจอ: สลิปรอยืนยัน และหน้าตรวจสอบสลิป

ที่อยู่ของแบบร่าง และวิธีที่ผู้ใช้แก้ไขให้ถูกต้องภายในไม่กี่แตะ

เข้าหน้า "สลิปรอยืนยัน" (`src/app/drafts.tsx`) ได้ 4 ทาง: ปุ่ม "ตรวจ N รายการที่อ่านไม่ชัด" ในหน้าสแกน · ปุ่ม "ตรวจสลิป" บนการ์ดสแกนอัตโนมัติ · การ์ด "สลิปรอยืนยัน N รายการ · พร้อมยืนยัน X · ต้องตรวจ Y · ยังไม่รวมในยอดเงิน" บนหน้าหลัก · การ์ดเดียวกันบนหน้ารายการ



สลิปรอยืนยัน: 'ต้องตรวจก่อน (2)' และ 'ร้านรองเท้า': AI อ่านได้ 2 แบบให้แตะเลือก และ 'ร้านน้ำปั่น': ไฮไลต์ช่องยอดเงินและชื่อ

หน้าสลิปรอยืนยัน แบ่ง 2 กลุ่ม

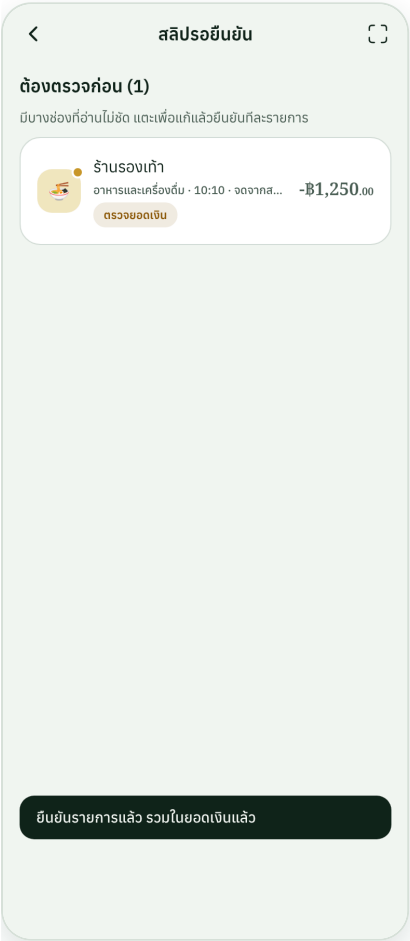
กลุ่ม	ใครอยู่กลุ่มนี้	ทำอะไรได้
พร้อมยืนยัน (N)	แบบร่างที่ไม่มีธงเลย (อ่านชัดทุกช่อง แต่ปิด "รวมยอดทันที" ไว้ หรือกด "ยังไม่รวมยอด")	บรรทัด "อ่านได้ครบทุกช่องด้วยความมั่นใจ 80% ขึ้นไป รวมแล้ว ±฿..." + ปุ่มทอง "ยืนยันทั้ง N รายการ" กดครั้งเดียวจบ (มีพลู + ข้อความ "ยืนยัน N รายการแล้ว")
ต้องตรวจก่อน (N)	แบบร่างที่มีธงอย่างน้อย 1 ช่อง	"มีบางช่องที่อ่านไม่ชัด แต่เพื่อแก้ไขแล้วยืนยันก็ละรายการ"
ไม่มีเลย	-	น้องกล้าเชียร์ "เคลียร์หมดแล้ว · ทุกรายการจากสลิปยืนยันเรียบร้อย เก่งมาก" + ปุ่ม "สแกนสลิปเพิ่ม"

ทำไมไม่มีปุ่ม "ยืนยันทั้งหมด" ในกลุ่มต้องตรวจ? เพราะนั่นคือการให้ผู้ใช้กดผ่านโดยไม่ดู ซึ่งทำลายจุดประสงค์ของเกณฑ์ 80% ทั้งหมด ข้อมูลที่ไม่ชัดต้องถูก มนุษย์ดูทีละรายการ

หน้าตรวจสอบสลิป = ฟอรัมของ FR-1 ที่มีกล่องช่วยตรวจ

แต่ละแบบร่างใด ๆ จะเปิด `src/app/transaction.tsx` (ฟอรัมเดียวกับจดรายการเอง) แต่เพราะรายการนี้เป็นแบบร่าง หน้าจอเลยเป็น "ตรวจสอบสลิป" และมีส่วนพิเศษ:

- 1. กล่องทองด้านบน: "ช่วยตรวจ: ยอดเงิน" (ชื่อช่องที่ติดธง คั่นด้วยจุลภาค) · "ยังไม่รวมในยอดเงินจนกว่าจะยืนยัน · ความมั่นใจต่ำสุด 50%" (ค่าต่ำสุดของทุกช่อง × 100)
- 2. ถ้า AI อ่านยอดได้ 2 แบบ: บรรทัด "AI อ่านได้ 2 แบบ: ยอดเงิน ฿1,250.00 หรือ ฿1,280.00" และชิป "ดูที่สลิปแล้วเลือก: [฿1,250.00] [฿1,280.00]" และชิปแล้วช่องยอดเงินเปลี่ยนตามทันที ชิปที่ตรงกับยอดปัจจุบันเป็นสีเขียว
- 3. ช่องที่ติดธงถูกไฮไลต์: ยอดเงินมีเส้นใต้สีน้ำตาลทอง ชื่อมีกรอบแดงและข้อความ "ตรวจสอบชื่อให้ตรงกับสลิป" วันที่ (ถ้าติดธง) มีข้อความ "อ่านวันที่บนสลิปไม่ชัด ตรวจให้ตรงก่อนยืนยัน"
- 4. ปุ่มล่าง 2 ปุ่ม: ทอง "ยืนยันและรวมในยอดเงิน" (สถานะ → confirmed และล้างธง) กับ "บันทึกแบบร่างไว้ก่อน" (เก็บที่แก้ไข ยังเป็นแบบร่าง)
- 5. ไอคอนถังขยะมุมขวาล่าง: ลบแบบร่าง (ข้อความยืนยัน: "แบบร่างจากสลิปนี้จะถูกลบ ถ้าสแกนรูปเดิมอีกครั้งจะอ่านใหม่ได้")



กดยืนยัน 'ร้านน้ำป่น' แล้ว: เหลือ 1 รายการ

หลังกด "ยืนยันและรวมในยอดเงิน" ในรายการร้านน้ำป่น:

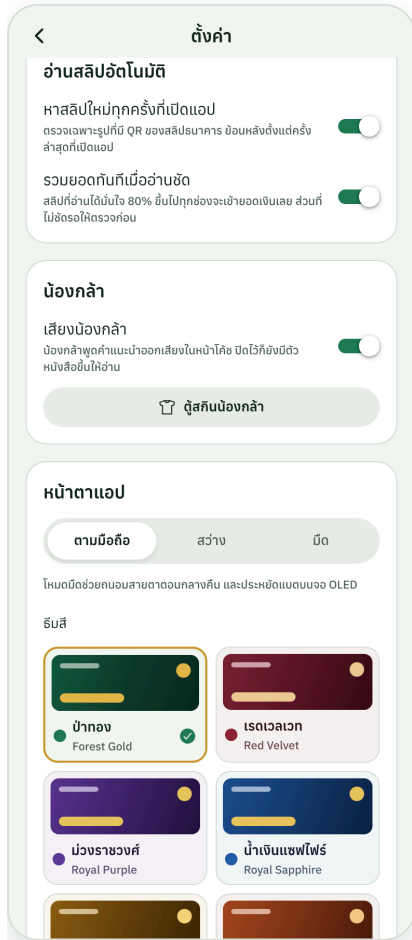
- ข้อความด้านล่างจอ "ยืนยันรายการแล้ว รวมในยอดเงินแล้ว"
- กลับมาหน้าสลิปรอยืนยันเอง เหลือ "ต้องตรวจก่อน (1)" คือร้านรองเท้า
- ยอดคงเหลือลดลง ฿45 ทันที และ FR-6 คำนวน Runway ใหม่ เพราะตอนนี้รายการนี้เป็น confirmed แล้ว

i ข้อสังเกตจากภาพ

ร้านรองเท้าถูกจัดหมวด "อาหารและเครื่องดื่ม" เพราะคำว่า "ร้าน" อยู่ในคำค้นของหมวดอาหาร (ฟังก์ชัน `suggestCategory` ของ FR-1) ผู้ใช้แก้หมวดได้ในหน้าตรวจ ฉะนั้นก็เรื่องนี้ไว้ในข้อจำกัดท้ายเล่ม

ทัวร์หน้าจอ: ตั้งค่าการสแกน และรูปทดสอบ

ผู้ใช้ต้องคุมระบบอัตโนมัติได้เสมอ

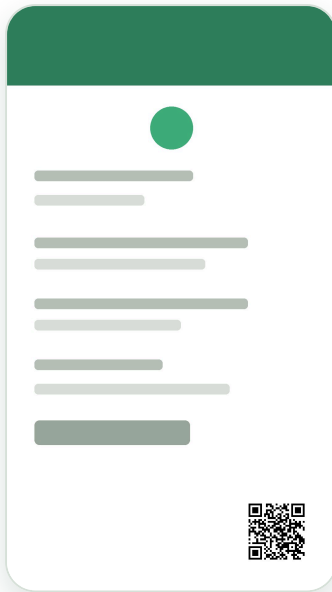


ตั้งค่า: การ์ด 'อ่านสลิปอัตโนมัติ'

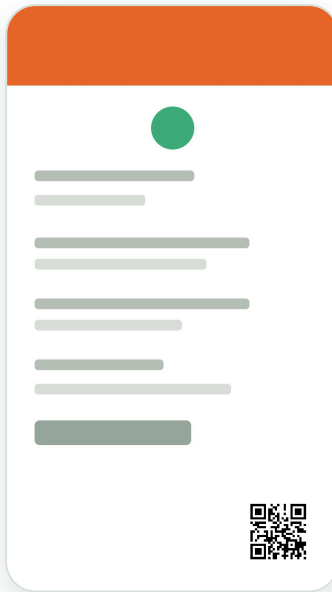
- 1 การ์ด "อ่านสลิปอัตโนมัติ" มีเฉพาะในแอปมือถือที่ล็อกอินบัญชีจริง (เว็บธรรมดาและโหมดทดลองจะไม่เห็นการ์ดนี้ เพราะสแกนอัตโนมัติไม่ได้)
- 2 สวิตช์ "หาสลิปใหม่ทุกครั้งที่เปิดแอป": "ตรวจสอบเฉพาะรูปที่มี QR ของสลิปธนาคาร ย้อนหลังตั้งแต่ครั้งล่าสุดที่เปิดแอป"
- 3 สวิตช์ "รวมยอดทันทีเมื่ออ่านชัด": "สลิปที่อ่านได้มั่นใจ 80% ขึ้นไปทุกช่องจะเข้ายอดเงินเลย ส่วนที่ไม่ชัดเจนให้ตรวจสอบ"
- 4 ค่าเริ่มต้นเปิดทั้งคู่ (`src/data/prefs.ts`) สวิตช์เป็นของ **มือถือเครื่องนี้** (เก็บในเครื่อง ไม่ได้เก็บในฐานข้อมูล) ส่วน "สแกนอัตโนมัติครั้งล่าสุดเมื่อไร" แยกตามบัญชี
- 5 เลื่อนลงไปที่การ์ด "ความเป็นส่วนตัว" มีปุ่ม **"ล้างประวัติรูปที่เคยสแกน"** → "สแกนครั้งหน้าจะตรวจสอบทุกรูปใหม่" (รูปที่บันทึกไปแล้วจะถูกจับว่าซ้ำ ไม่บันทึกซ้ำ)

ปิดสวิตช์นี้แล้ว	ผลในโค้ด
หาสลิปใหม่ทุกครั้งที่เปิดแอป	<code>shouldAutoScan</code> ตอบ false → ไม่มีการ์ดสแกนอัตโนมัติ ผู้ใช้ยังสแกนเองได้ที่หน้าสแกน
รวมยอดทันทีเมื่ออ่านชัด	<code>initialStatus</code> คืบ draft เสมอ → ทุกสลิป (แม้ชัด 100%) ไปรอที่ "พร้อมยืนยัน" ให้กดยืนยันเอง

รูปทดสอบที่ใช้ในเล่มนี้และในเทสต์



slip-qr.jpg: สลิปจำลอง
1080×1920 มี QR ตรวจสอบสลิป
'จริง' ตามมาตรฐานที่ธนาคาร
(ธนาคาร 004)



wallet-qr.jpg: แบบ e-wallet มี
QR แต่ข้อมูลข้างในไม่ใช่รูปแบบสลิป
ธนาคาร

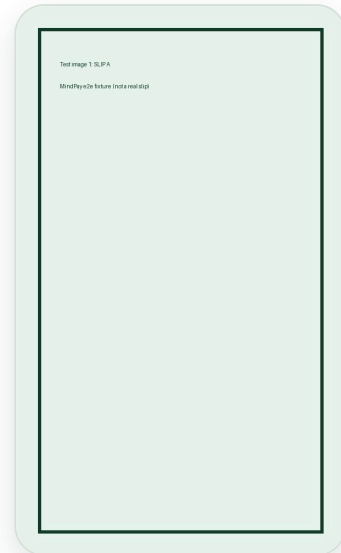


photo-1.jpg: รูปทั่วไป 600×1000
ไม่มี QR

รูปเหล่านี้อยู่ใน `e2e/fixtures/` ไม่ใช่สลิปของใคร แถบเทาคือ "ตัวหนังสือ" ปลอม สิ่งเดียวที่เป็นของจริงคือ QR ซึ่งมือถือ
ถอดรหัสได้จริง ส่วน "สิ่งที่ AI อ่าน" เซิร์ฟเวอร์จำลองตอบตามที่เทสต์กำหนด เพื่อให้ผลแน่นอนทุกครั้ง (ถ้าใช้ AI จริง ผลอาจ
ต่างกันเล็กน้อยทุกครั้ง และเปลืองเงิน)

★ ลองดู QR ในรูปด้วยตัวเอง

เปิดรูป slip-qr.jpg บนคอมแล้วสแกน QR ด้วยกล้องมือถือ จะได้ข้อความ

004100060000101030040220016271094231BTF056785102TH9104385A บทถัดไปจะแกะข้อความนี้
ทีละตัวอักษรว่าแต่ละส่วนคืออะไร

หลักการที่ 1: QR ตรวจสอบสลิป ด้านแรกที่ฟรีและแม่นยำที่สุด

ก่อนเรียก AI แม้แต่ครั้งเดียว มือถือรู้ได้เองว่ารูปไหนน่าจะเป็นสลิป

สลิปจากแอปธนาคารไทยแทบทุกแอปมี **QR เล็ก ๆ สำหรับตรวจสอบสลิป** (ร้านค้าใช้สแกนเช็คว่าสลิปจริงไหม) QR นี้ไม่ใช่ QR สำหรับจ่ายเงิน แต่เก็บ "บัตรประจำตัว" ของการโอนครั้งนั้น: ธนาคารที่โอน และ **เลขอ้างอิงที่ไม่ซ้ำกันเลยทั้งประเทศ** MindPay ใช้ QR นี้ 4 อย่าง:

ใช้ทำอะไร	ทำไมดีกว่าให้ AI อ่าน
1. ตัดสินว่ารูปไหนเป็นสลิป	มือถือถอด QR เองได้ ฟรี ไม่ใช้เน็ต รูปที่ไม่มี QR (รูปเพื่อน รูปอาหาร) ไม่ถูกส่งไปไหนเลย = ประหยัดค่า AI และเป็นส่วนตัว
2. เลขอ้างอิง สำหรับกันซ้ำ	ถอดจาก QR ได้ตรงทุกตัวอักษร AI อาจอ่านเลขยาว 20 ตัวผิดไป 1 ตัว
3. ธนาคาร	รหัสธนาคารใน QR มาจากธนาคารเอง แม่นกว่าเดาจากโลโก้
4. ตรวจสอบว่าข้อมูลถูกแก้ไขไหม	มีตัวเลขตรวจสอบ (CRC) ถ้าย้าย QR ถ้าข้อมูลถูกแก้ไข ตัวเลขจะไม่ตรง

ข้างใน QR มีอะไร: แยกทีละตัวอักษร

สแกน QR ของรูปทดสอบ slip-qr.jpg ได้ข้อความยาว 59 ตัวอักษรนี้:

0041000600000101030040220016271094231BTF056785102TH9104385A

ดูเหมือนตัวเลขมั่ว ๆ แต่จริง ๆ แล้วเป็นรูปแบบมาตรฐานชื่อ **TLV** ย่อมาจาก **Tag** (ป้ายบอกว่าเป็นข้อมูลอะไร 2 หลัก) · **Length** (ความยาว 2 หลัก) · **Value** (ตัวข้อมูล ยาวเท่าที่ L บอก) เรียงต่อกันไปเรื่อย ๆ (รูปแบบเดียวกับ QR พร้อมเพย์และบัตรเครดิตตามมาตรฐาน EMVCo) ระบายสีให้ดูแล้วเป็นแบบนี้:

0041000600000101030040220016271094231BTF056785102TH9104385A

เขียวเข้ม = Tag · ทอง = Length · เขียวอ่อน = Value · แดง = ตัวเลขตรวจสอบ (CRC)

ตำแหน่ง	Tag	Length	Value	ความหมาย
0-44	00	41	0006000001 0103004 022001627... 05678	ข้อมูลของการโอน (ซ้อนเป็น TLV อีกชั้น ดูตารางถัดไป)
45-50	51	02	TH	ประเทศ: ไทย
51-58	91	04	385A	CRC ตัวเลขตรวจสอบ 4 หลัก (เลขฐาน 16)

ค่าของ Tag 00 (ยาว 41 ตัว) ก็เป็น TLV อีกชั้นหนึ่ง และต่อได้:

000600000101030040220016271094231BTF05678

Tag ย่อย	Value	ความหมาย	MindPay ใช้อย่างไร
00	000001	รหัสบอกว่า "นี่คือ QR ตรวจสอบสลิป"	ต้องเป็น 000001 เท่านั้น ไม่จำเป็นต้องใช้ QR สลิป (เช่น QR พร้อมเพย์สำหรับจ่ายเงินจะไม่ผ่านตรงนี้)
01	004	รหัสธนาคารผู้โอน (ของธนาคารแห่งประเทศไทย)	004 = KBank → เขียนในบันทึก "สลิปจาก KBank"
02	016271094231BTF05678	เลขอ้างอิงการโอน (Transaction reference)	เก็บในช่อง slip_ref ใช้กันซ้ำ

★ ลองคิดตามโค้ด: `parseTlv` อ่านยังงี้

ฟังก์ชัน `parseTlv` (`src/domain/slip.ts:81`) มีตัวชี้ตำแหน่ง `i` เริ่มที่ 0 แล้ววนซ้ำ: อ่าน 2 ตัวเป็น Tag → อ่านอีก 2 ตัวเป็นความยาว → ตัดค่าออกมาตามความยาว → เลื่อน `i` ไป 4 + ความยาว

รอบ 1: `i = 0` → Tag "00" ยาว 41 → ค่าคือตัวที่ 4-44 → `i = 45` · รอบ 2: `i = 45` → Tag "51" ยาว 2 → "TH" → `i = 51` · รอบ 3: `i = 51` → Tag "91" ยาว 4 → "385A" → `i = 59` = ความยาวทั้งหมด → จบ ถ้ากลางทางตัวเลขไม่ครบ หรือความยาวไม่ตรง ฟังก์ชันคืน `null` ทันที (= ไม่ใช่ข้อมูลรูปแบบนี้)

CRC: ตัวเลขตรวจสอบท้าย QR

CRC (Cyclic Redundancy Check) คือตัวเลขที่ **คำนวณจากข้อมูลทั้งหมดก่อนหน้า** ถ้าข้อมูลถูกแก้ไขแม้แต่ตัวเดียว ผลคำนวณจะเปลี่ยน คนสร้าง QR คำนวณไว้แล้วแปะท้าย คนอ่านคำนวณซ้ำแล้วเทียบ ถ้าตรงกัน = ข้อมูลครบและไม่ถูกแก้ไข

≈ เหมือนเลขหลักสุดท้ายของบัตรประชาชน

เลขบัตรประชาชน 13 หลัก หลักที่ 13 คำนวณจาก 12 หลักแรก ถ้าพิมพ์ผิดไปหลักเดียว ระบบรู้ทันทีว่าเลขนี้ใช้ไม่ได้ CRC ทำหน้าที่เดียวกัน แต่สูตรซับซ้อนกว่าและได้ผลเป็นเลขฐาน 16 จำนวน 4 หลัก

MindPay คำนวณ CRC-16/CCITT-FALSE จากข้อความตั้งแต่ต้นจนถึง "9104" (55 ตัวแรก) ได้ 385A ตรงกับที่แปะไว้ → `crcValid: true` จินลองเปลี่ยนท้ายเป็น 0000 แล้วรันโค้ดจริง ได้ `crcValid: false` ตามคาด

QR ของรูปทดสอบทั้ง 3 ใบ (ผลจากการรันโค้ดจริง)

รูป	ธนาคาร (Tag 01)	เลขอ้างอิง (Tag 02)	CRC
slip-qr.jpg	004 → KBank	016271094231BTF05678	385A ✓

รูป	ธนาคาร (Tag 01)	เลขอ้างอิง (Tag 02)	CRC
slip-qr-2.jpg	014 → SCB	016272184455CKQ11220	0ECC ✓
slip-qr-3.jpg	006 → Krungthai	016273009912DMV30011	1D17 ✓
wallet-qr.jpg	ข้อความใน QR คือ WALLET-RECEIPT-20260928-000123 ไม่ใช่ TLV → <code>parseSlipQr</code> ตอบ null (เป็น "QR อื่น")		

รหัสธนาคาร 21 แห่งที่แอปรู้จัก

รหัส	ธนาคาร	รหัส	ธนาคาร	รหัส	ธนาคาร
002	Bangkok Bank	022	CIMB Thai	066	Islamic Bank
004	KBank	024	UOB	067	TISCO
006	Krungthai	025	Krungsri	069	KKP
011	ttb	030	GSB (ออมสิน)	070	ICBC Thai
014	SCB	031	HSBC	071	Thai Credit
017	Citibank	033	GHB (ธอส.)	073	LH Bank
020	Standard Chartered	034	BAAC (ธ.ก.ส.)	098	SME D Bank

`bankFromCode` เติม 0 ข้างหน้าให้ครบ 3 หลักก่อนค้น: "4" → "004" → KBank · "014" → SCB · "030" → GSB (ออมสิน) · "999" → null (ไม่รู้จัก ใช้ชื่อที่ AI อ่านแทน)

ปัญหา: QR บนสลิปเล็กมาก

ตัวสแกน QR ของ Android อ่านได้ดีเมื่อ QR **ใหญ่เต็มภาพ** แต่ QR บนสลิปเล็กและอยู่มุมล่าง สแกนทั้งรูปแล้วบางทีหาไม่เจอ วิธีแก้คือ "ซูม" ด้วยการตัดรูปเฉพาะส่วนที่ธนาคารชอบพิมพ์ QR แล้วสแกนใหม่ทีละส่วน:

ส่วนที่ตัด (SLIP_QR_REGIONS)	สัดส่วนของรูป	บนรูป 1080×1920 ได้เป็น พิกเซล	เหมาะกับ
ครึ่งล่าง	x 0, y 0.45, กว้าง 1, สูง 0.55	เริ่ม (0, 864) กว้าง 1080 สูง 1056	ธนาคารส่วนใหญ่
มุมขวาล่าง	x 0.45, y 0.55, กว้าง 0.55, สูง 0.45	เริ่ม (486, 1056) กว้าง 594 สูง 864	K PLUS, SCB EASY, Krungthai NEXT

การตัดใช้เวลา จึงทำ **เฉพาะรูปขนาดหน้าจอ** (สลิปที่แอปธนาคารเซฟ และภาพแคปหน้าจอ) ไม่ทำกับรูปจากกล้องซึ่งมีเป็นพัน ๆ รูป ฟังก์ชัน `isScreenSized` ใช้กฏง่าย ๆ: **กว้างไม่เกิน 1,600 px และสูง ≥ กว้าง** (ไม่รู้จักขนาด = ถือว่าใช่ เพื่อดูให้ละเอียดไว้ก่อน)

ขนาดรูป	isScreenSized	เหตุผล
1080×2400	true	ภาพแคปหน้าจอมือถือ
600×1000	true	แนวตั้ง กว้างไม่เกิน 1,600
3000×4000	false	รูปจากกล้อง (กว้างเกิน 1,600)
1920×1080	false	แนวนอน (สูงน้อยกว่ากว้าง)
ไม่รู้ขนาด	true	ดูให้ละเอียดไว้ก่อน ดีกว่าพลาดสลิป

ข้อยกเว้น: สลิป e-wallet

TrueMoney, ShopeePay และ e-wallet อื่น ๆ มี QR ของตัวเองที่ไม่ใช่รูปแบบ TLV ด้านบน ถ้ากฎ "ต้องมี QR สลิปธนาคาร" เข้มเกินไป สลิปเหล่านี้จะหลุดทั้งหมด จึงมีกฎเสริม: **รูปขนาดหน้าจอที่มี QR อะไรก็ได้** → ส่งให้ AI ตัดสินว่าเป็นสลิปไหม (AI ตอบ isSlip) ส่วนรูปจากกล้องที่บังเอิญมี QR (เช่น ถ่ายโปสเตอร์) ไม่ส่ง

ส่ง AI เมื่อ: เจอ QR สลิปธนาคาร · หรือ · (เจอ QR อื่น และ เป็นรูปขนาดหน้าจอ) · หรือ · ผู้ใช้เลือกรูปเอง

นอกนั้น = "ไม่ใช่สลิป" รูปไม่ออกจากมือถือ ([src/services/processSlip.ts:56](#))

! มือถือที่ไม่มีตัวสแกน QR

มือถือ Android บางรุ่นไม่มี Google Play Services จึงไม่มีตัวสแกน QR ถ้าไม่ระวัง ทุกรูปจะ "ไม่มี QR" และแอปจะบอกว่าไม่เจอสลิปเลยแบบเงียบ ๆ ซึ่งผิด โค้ดจึงแยกกรณีนี้ออกมา ([scannerWorks: false](#)) แล้วบอกผู้ใช้ตรง ๆ ว่า "มือถือเครื่องนี้หา QR ของสลิปเองไม่ได้ ... ใช้ 'เลือกรูปเอง' แทนได้"

หลักการที่ 2: เกณฑ์ความมั่นใจ 80%

หัวใจของ Requirement FR-4: AI ไม่แน่ใจ → มนุษย์ต้องดูก่อน

เวลา AI อ่านสลิป นอกจากคำตอบแล้ว ฉันให้มันบอก "**ความมั่นใจ**" (**confidence**) เป็นตัวเลข 0 ถึง 1 แยก 3 ช่อง: ยอดเงิน วันที่ และชื่อผู้รับ/ผู้โอน เช่น 0.98 = มั่นใจมาก 0.5 = ครึ่ง ๆ ในคำสั่ง (prompt) ฉันกำหนดชัดว่า **ต้องให้ต่ำกว่า 0.8** ถ้าตัวอักษรใดเบลอ ถูกตัด ถูกบัง เลิกเก็บไป หรืออ่านได้ 2 แบบ (1 กับ 7, 3 กับ 8, 5 กับ 6, 0 กับ 8, จุลภาคกับจุด) และให้ 0 เมื่ออ่านไม่ได้

ช่องใดความมั่นใจ < 0.80 หรือว่าง → ตัดตรงช่องนั้น · ไม่มีธงเลย = ready · มีธง = needs_review

โค้ดใช้ "น้อยกว่า 0.8" จึงตัดตรง: ฉันรันดูแล้ว 0.80 ทุกช่อง → ไม่มีธง (ผ่าน) · 0.799 → ตัดตรง · เทสต์ TC-20 ใช้ 0.79 ทดสอบตรง ขอบนี้ ส่วน TC-19 ทดสอบสลิปที่ขัดทุกช่อง

ธง 4 ชนิด และเงื่อนไขที่ทำให้ตัดตรง

ธง	เงื่อนไข	ตัดเมื่อ
amount	ตรวจยอดเงิน	อ่านยอดไม่ได้ หรือ ความมั่นใจยอด < 0.8
date	ตรวจวันที่	อ่านวันไม่ได้ · ความมั่นใจ < 0.8 · วันที่จากตัวหนังสือกับวันที่ที่ AI แปลงเองไม่ตรงกัน · วันที่เลยพ่วงนี้ไป (อนาคต)
counterparty	ตรวจผู้รับ/ผู้โอน	ไม่มีชื่อ หรือ ความมั่นใจชื่อ < 0.8
direction	ตรวจเงินเข้าหรือออก	AI ไม่รู้ว่าเงินเข้าหรือออก · หรือชื่อบนสลิปขัดกับที่ AI ตอบ (หลักการที่ 5)

จากรง ไปเป็นสถานะของรายการ

normalizeReading → ผู้สมัคร (candidate) พร้อมรายการธง



classifyCandidate

ซ้ำ → duplicate (ไม่บันทึก) · วันที่เก่ากว่า 1 ปี → out_of_range (ไม่บันทึก) · ไม่มีธง → ready · มีธง → needs_review



initialStatus

ready และ เปิด "รวมยอดทันทีเมื่ออ่านชัด" → **confirmed** (นับในยอดทันที) · นอกนั้นทั้งหมด → **draft** (รอผู้ใช้)

ตัวอย่างจริง 8 แบบ (รันโค้ดจริง วันที่ 9 ต.ค. 2569)

ตั้งต้นจากสลิปที่ดี: จ่าย ร้านกาแฟลอยช้าง ฿85.00 วันที่ "27 ก.ย. 69" เวลา 08:15 ความมั่นใจ ยอด 0.98 วันที่ 0.95 ชื่อ 0.90 แล้วเปลี่ยนทีละอย่าง:

เปลี่ยนอะไร	สิ่งที่ได้	ผล	สถานะเริ่มต้น
ไม่เปลี่ยน (สลิปดี)	-	ready · ความมั่นใจต่ำสุด 0.9 · หมวดอาหาร	confirmed
ความมั่นใจยอด 0.79	amount	needs_review	draft
AI แปลงวันที่เป็น 2026-09-21 แต่ตัวหนังสือคือ 27 ก.ย. 69	date	ใช้วันที่จากตัวหนังสือ (27 ก.ย.) แต่ติดธง	draft
วันที่ "5 ม.ค. 70" (อนาคต)	date	ไม่เชื่อวันที่ (dayKey = null) ใช้เวลาของรูปแบบ	draft
วันที่ "27 ก.ย. 67" (2 ปีก่อน)	-	out_of_range ไม่บันทึกอัตโนมัติ	-
ไม่มีชื่อผู้รับ	counterparty	หมวด "อื่น ๆ" (เดาไม่ได้)	draft
AI ตอบ direction = unknown	direction	needs_review	draft
รูปมี QR (ธนาคาร 004) แต่ AI อ่านโลโก้ว่า "Kasikorn"	-	ธนาคาร = KBank และเลขอ้างอิงจาก QR	confirmed

i ทำไม "พรุ่งนี้" ยังไม่นับว่าเป็นอนาคต

โค้ดถือว่าวันที่ **เลยพรุ่งนี้** ไปคืออนาคต (`dayKey > addDays(today, 1)`) ไม่ใช่เลยวันนี้ เพื่อเรื่องเขตเวลาและนาฬิกา มือถือเพี้ยนเล็กน้อย จันรันดู: วันนี้ 9 ต.ค. สลิปลงวันที่ 10 ต.ค. ไม่ติดธง แต่ 11 ต.ค. ติดธง date

≈ เปรียบเทียบ

เหมือนครูตรวจข้อสอบปรนัยด้วยเครื่อง ถ้าเครื่องเห็นว่ากระดาษคำตอบข้อไหนฝนจาง ๆ ไม่ชัด เครื่องจะไม่เดา แต่แยกกระดาษแผ่นนั้นออกมาให้ครูดูด้วยตาเอง เกณฑ์ 80% คือเส้นแบ่งว่า "ชัดพอให้เครื่องตัดสินใจ" หรือ "ต้องให้คนดู"

ทำไมเลือก 80%

- ตัวเลข 80% มาจาก Requirement ของทีม (Charter) จันจึงใส่ไว้เป็นค่าคงที่เดียว `CONFIDENCE_THRESHOLD = 0.8` ([src/domain/slip.ts:22](#)) ถ้าวันหนึ่งอยากเปลี่ยนเป็น 85% แก้ที่เดียวจบ
- สูงไป (เช่น 99%) = เกือบทุกสลิปเป็นแบบร่าง ผู้ใช้ต้องกดยืนยันเองทั้งหมด ระบบอัตโนมัติไม่มีประโยชน์
- ต่ำไป (เช่น 50%) = สลิปที่ AI ไม่แน่ใจจะเข้ายอดเงินเลย ตัวเลขเงินอาจผิดโดยผู้ใช้ไม่รู้
- ความมั่นใจที่แอปเก็บ (ocrConfidence) คือ **ค่าต่ำสุดของ 3 ช่อง** เพราะสลิปน่าเชื่อถือได้เท่ากับช่องที่อ่อนที่สุด หน้าตรวจสลิปแสดงเป็น "ความมั่นใจต่ำสุด 50%"

หลักการที่ 3: อ่าน 2 ครั้งด้วย AI 2 รุ่น

AI ก็มั่นใจผิดได้ วิธีจับคือให้อีกตัวอ่านแยกแล้วเทียบกัน

ปัญหาของการเชื่อ "ความมั่นใจ" ของ AI อย่างเดียวคือ **AI มั่นใจผิดได้** เช่น อ่าน 1,280 เป็น 1,250 แล้วบอกว่ามั่นใจ 0.97 วิธีแก้ของฉันทคือให้ AI **2 รุ่นที่ต่างกัน** อ่านรูปเดียวกัน **พร้อมกัน โดยไม่เห็นคำตอบของกันและกัน** แล้วเอามาเทียบ AI 2 รุ่นที่ต่างกันโอกาสอ่านผิด "แบบเดียวกันเป๊ะ" ต่ำมาก ถ้าตรงกันจึงเชื่อ ถ้าไม่ตรงก็รู้กันที่ว่าต้องให้คนดู

≈ เหมือนแคชเชียร์ 2 คนนับเงินถุงเดียวกัน

ธนาคารให้พนักงาน 2 คนนับเงินแยกกัน ถ้าได้ 10,000 เท่ากันก็จบ ถ้าคนหนึ่งได้ 10,000 อีกคนได้ 9,900 ต้องนับใหม่ ไม่มีใครเลือกเชื่อคนที่ "มั่นใจกว่า" เพราะความมั่นใจไม่ได้แปลว่าถูก

2 รุ่นที่ใช้ (เมื่อใช้ Gemini)

การอ่าน	ลำดับรุ่นที่ลอง (ถ้ารุ่นแรกใช้ไม่ได้ลองรุ่นถัดไป)
primary (อ่านหลัก)	gemini-3.5-flash-lite → gemini-3.1-flash-lite → gemini-flash-lite-latest → gemini-flash-latest
verify (อ่านตรวจ)	gemini-flash-latest → gemini-3.5-flash → gemini-3.1-flash-lite → gemini-flash-lite-latest

ตัวตรวจเริ่มจากรุ่นที่ **ใหญ่กว่า** และไม่ใช่รุ่นเดียวกับตัวหลัก AI แต่ละรุ่นมีโควตาฟรีของตัวเอง การอ่านพร้อมกัน 2 รุ่นจึงไม่ทำให้อ่านสลิปได้น้อยลงครึ่งหนึ่ง และในแอป สลิป 1 ใบนับโควตา (300 ใบ/วัน) แค่ 1 ครั้ง ถ้าเซิร์ฟเวอร์ตั้งให้ใช้ Claude แทนทั้ง 2 การอ่านจะใช้ claude-haiku-4-5 รุ่นเดียวกัน (เว้นแต่ผู้ดูแลตั้ง VERIFY_MODEL เป็นรุ่นอื่น) ซึ่งโค้ดรู้ตัวและจัดการเป็นกรณี "รุ่นเดียวกัน" ในตารางถัดไป

กฏการรวม 2 คำตอบ (mergeReadings)

ผลจริงจากการรันโค้ดเซิร์ฟเวอร์ สลิปตั้งต้น: ฿1,250.00 วันที่ 27 ก.ย. 69 ความมั่นใจ 0.97 / 0.96 / 0.95

สถานการณ์	ผลรวม	check ที่ส่งกลับ	ในแอป
ตรงกันทุกช่อง (ตัวที่ 2 มั่นใจยอด 0.99)	เก็บค่าที่ตรงกัน ความมั่นใจเอาตัวที่ สูงกว่า : 0.99 / 0.96 / 0.95	verified: true, reads: 2	ผ่าน 80% → รวมยอดได้
ยอดไม่ตรง: 1,250.00 กับ 1,280.00	ความมั่นใจยอดถูกกดเหลือ 0.5 เก็บยอดของตัวหลัก (1,250.00)	verified: false, disagree: amount [1250.00, 1280.00]	รงยอดเงิน + ชิปให้เลือก 2 ค่า
วันที่ไม่ตรง	ความมั่นใจวันที่ ≤ 0.5	disagree: date [ตัวหนังสือ 1, ตัวหนังสือ 2]	รงวันที่ + บันทึก 2 ค่า

สถานการณ์	ผลรวม	check ที่ส่งกลับ	ในแอป
ตัวหนึ่งว่าเงินออก อีกตัวว่าเงินเข้า	direction = unknown	disagree: direction	ธงเงินเข้าหรือออก
ชื่อต่างกันจริง	ความมั่นใจชื่อ ≤ 0.5	disagree: counterparty [ชื่อ 1, ชื่อ 2]	ธงผู้รับ/ผู้โอน
ชื่อเดียวกันแต่ตัวหนึ่งถูกปิดบาง "ร้านข้าวมันไก่ป่า***"	ถือว่าตรงกัน เลือก สะกดเต็ม "ร้านข้าวมันไก่ป่าแดง"	verified: true	ผ่าน
อ่านได้รอบเดียว (อีกตัวล้ม)	ทุกช่องถูกกดไม่เกิน 0.79	verified: false, reads: 1	ไม่ผ่าน 80% เสมอ → แบบร่าง
2 รอบเป็นรุ่นเดียวกัน	ทุกช่อง ≤ 0.79	verified: false, reads: 2	แบบร่าง (รุ่นเดียวกันผิดเหมือนกันได้)
ตัวหนึ่งว่าเป็นสลิป อีกตัวว่าไม่ใช่	เก็บไว้เป็นสลิป แต่ทุกช่อง ≤ 0.5	verified: false	แบบร่าง "AI 2 ตัวเห็นไม่ตรงกันว่าเป็นสลิปหรือไม่..."
ทั้งคู่ว่าไม่ใช่สลิป	ไม่ใช่สลิป	verified: true	นับเป็น "รูปอื่น"

i ตัวเลข 0.5 และ 0.79 มาจากไหน

ทั้งสองตัว **ต่ำกว่า 0.8 โดยตั้งใจ** เพื่อให้ช่องนั้นไม่ผ่านกฎของแอปแน่นอน UNSURE = 0.5 ใช้เมื่อ 2 ตัวขัดกัน (ไม่แน่ใจจริง) UNCHECKED_MAX = 0.79 ใช้เมื่อตรวจซ้ำไม่ได้ (อาจถูกก็ได้ แต่ยังไม่พยาน) เซิร์ฟเวอร์จึงไม่ต้อง "รู้" กฎ 80% ของแอปโดยตรง แค่ส่งค่าที่แน่ใจว่าต่ำพอ

ชื่อเดียวกันใหม่ (similarName)

ชื่อบนสลิปมักถูกปิดบางส่วน (เช่น "นาย สมชาย ใ***") และ AI 2 ตัวอาจเขียนต่างกันเล็กน้อย จึงเทียบแบบ "ยืดหยุ่น" 3 ชั้น หลังตัดคำนำหน้า ช่องว่าง จุด และเครื่องหมายปิดบังออก (nameLetters : "นาย สมชาย ใ***" → "สมชายใ"):

- ตัวอักษรเหมือนกันทุกตัว → ชื่อเดียวกัน
- ตัวสั้น (อย่างน้อย 3 ตัว) เป็น **จุดเริ่มต้น** ของตัวยาว → ชื่อเดียวกัน ("สมชายใ" กับ "สมชายใจดี")
- ต่างกันไม่เกิน 1 ตัวอักษรต่อทุก 5 ตัว (วัดด้วย edit distance = จำนวนครั้งที่ต้องแก้ไขให้เหมือนกัน)

ชื่อ 1	ชื่อ 2	ผล
นาย สมชาย ใ***	สมชาย ใจดี	เหมือน (ข้อ 2)
MR. SOMCHAI JAIDEE	Somchai Jaidee	เหมือน (ข้อ 1 หลังตัด MR. และตัวพิมพ์เล็ก)
ร้านกาแฟดอยช้าง	ร้านข้าวมันไก่ป่าแดง	ไม่เหมือน

จากเซิร์ฟเวอร์กลับมาที่แอป

เมื่อ check บอกว่ายอดไม่ตรง แอปเขียนบันทึกของรายการ (ฟังก์ชัน slipNote) จักรันจริงได้:

สลิปจาก SCB

AI อ่านได้ 2 แบบ: ยอดเงิน ฿1,250.00 หรือ ฿1,280.00 · ชื่อ ร้านก หรือ ร้านข

หน้าตรวจสลิปอ่านบรรทัดที่ขึ้นต้นด้วย "AI อ่านได้ 2 แบบ:" แล้วดึงยอดออกมาด้วย `amountChoices` ได้ [125000, 128000] (สตางค์) → กลายเป็นชิป [฿1,250.00] [฿1,280.00] ให้ผู้ใช้ "ดูที่สลิปแล้วเลือก"

บันทึกบรรทัดอื่นที่อาจเห็น: "อ่านได้รอบเดียว ช่วยตรวจกับสลิปอีกครั้ง" (reads = 1) · "AI 2 ตัวเห็นไม่ตรงกันว่าเป็นสลิปหรือไม่ ช่วยตรวจกับรูปอีกครั้ง" · "ดูเหมือนโอนระหว่างบัญชีของคุณเอง ถ้าใช่ ลบรายการนี้ได้ (ไม่ใช่รายรับหรือรายจ่าย)"

หลักการที่ 4: ยอดเงินต้องตรงกับตัวเอง

ทำความเข้าใจคำตอบของ AI บนเชิร์ฟเวอร์ ก่อนส่งกลับมือถือ

นอกจากอ่าน 2 รอบแล้ว ในแต่ละรอบฉันยังให้ AI เขียนยอดเงิน 2 แบบ: `amount` เป็นตัวเลขล้วน ("1250.00") และ `amountPrinted` แบบที่เห็นบนสลิปเป๊ะ ๆ ("1,250.00") ถ้า 2 ค่านี้ไม่ใช่ตัวเลขเดียวกัน แปลว่า AI สับสนเอง → **ความมั่นใจยอด = 0** กันที นอกจากนี่ถ้ายอดเท่ากับ **ค่าธรรมเนียม** (fee) พอดี แปลว่า AI น่าจะหยิบผิดบรรทัด → **ความมั่นใจยอด = 0** เหมือนกัน

ฟังก์ชัน `normalizeReading` ฝั่งเชิร์ฟเวอร์ทำอะไรบ้าง

AI ส่งมา	หลังทำความสะอาด	กฎ
<code>amount: "1,250.00 บาท"</code>	<code>"1250.00"</code>	ตัด ฿ บาท THB คอมมา ช่องว่าง เครื่องหมาย +/- ต้องเป็นตัวเลข ทศนิยมไม่เกิน 2 ตำแหน่ง มากกว่า 0 ไม่เกิน 1,000,000,000
<code>dateIso: "2569-09-27"</code>	<code>"2026-09-27"</code>	ปีเกิน 2400 = สัมแปลง พ.ศ. → ลบ 543
<code>time: "9.05"</code>	<code>"09:05"</code>	รับทั้ง : และ . ชั่วโมงไม่เกิน 23 นาทีไม่เกิน 59
<code>reference: "0152-ABC"</code>	<code>"0152ABC"</code>	เก็บเฉพาะตัวอักษรและตัวเลข
<code>counterparty: "ร้านป่าแดง"</code>	<code>"ร้านป่าแดง"</code>	ตัดช่องว่างหัวท้าย ยาวไม่เกิน 100 ตัว ("null" ที่เป็นข้อความ = null)
<code>amountPrinted "1,520.00" แต่ amount "1250.00"</code>	ความมั่นใจยอด 0	เขียน 2 แบบแล้วไม่ตรงกัน
<code>amount "15.00" และ fee "15.00"</code>	ความมั่นใจยอด 0	ยอดเท่ากับค่าธรรมเนียม
ช่องใดว่าง (null)	ความมั่นใจช่องนั้น 0	"ค่าว่างต้องไม่มีความมั่นใจสูง" ป้องกัน AI ที่ตอบ null แต่ให้ความมั่นใจ 0.9
<code>isSlip</code> ไม่ใช่ true	ค่าว่างทุกช่อง ความมั่นใจ 0	ไม่ใช่สลิป = ไม่มีข้อมูล

★ **ทำไมต้องทำความสะอาดบนเชิร์ฟเวอร์ ทั้งที่แอปก็ตรวจอีกรอบ**

หลัก **"อย่าเชื่อข้อมูลจากภายนอก"**: คำตอบของ AI คือข้อมูลจากภายนอกเหมือนสิ่งที่ผู้ใช้พิมพ์ ต้องตรวจสอบรูปแบบทุกครั้ง การทำบนเชิร์ฟเวอร์ทำให้ไม่ว่าจะใช้ AI ค่ายไหน (Gemini หรือ Claude) แอปได้ข้อมูลรูปแบบเดียวกันเสมอ และการรวม 2 คำตอบ (หลักการที่ 3) ทำงานบนข้อมูลที่สะอาดแล้ว ส่วนแอปตรวจซ้ำอีกชั้นตามกฎหมายของตัวเอง (80%, วันที่ไทย, ชื่อ) เป็นการป้องกัน 2 ชั้น

หลักการที่ 5: รายรับหรือรายจ่าย? ดูจากชื่อบนสลิป

สลิปที่เพื่อนส่งมาให้ หน้าตาเหมือนสลิปที่เราโอนเองทุกอย่าง

สมมติเพื่อนโอนค่าข้าวคืนให้ แล้วส่งสลิปมาทาง LINE เราเซฟรูปลงเครื่อง สลิปใบนั้นเขียนว่า "โอนเงินสำเร็จ" เหมือนสลิปที่เราโอนเอง AI ที่ดูแลรูปจะตอบว่า **รายจ่าย** ทั้งที่จริงคือ **รายรับ** ยอดเงินในแอปจะผิดไป 2 เท่าของยอดนั้น (ควร +100 กลายเป็น -100)

สลิปบอกได้แค่ **ผู้โอน** (จาก) และ **ผู้รับ** (ไปยัง) ถ้าแอปรู้ว่า "ฉันชื่ออะไร" ก็ตัดสินใจได้: ฉันเป็นผู้รับ = รายรับ · ฉันเป็นผู้โอน = รายจ่าย แต่แอปไม่ได้ถามชื่อบนบัญชีธนาคาร (และชื่อบนสลิปมักถูกปิดบางส่วน) ฉันจึงให้แอป **เรียนรู้เอง**

เรียนรู้ชื่อของฉันจากสลิปในเครื่อง

- สลิปส่วนใหญ่ในเครื่องคือ **สลิปที่เราโอนเอง** ชื่อในบรรทัด "จาก" ที่เจอบ่อย ๆ จึงน่าจะเป็นชื่อเรา
- ทุกครั้งที่ AI อ่านสลิปแล้วบอกว่าเป็นรายจ่ายและมีชื่อผู้โอน แอปนับชื่อนั้น +1 (`learnPayer` ใน `src/services/processSlip.ts:71`)
- ชื่อที่ถือว่า "เป็นฉัน" ต้องเจอเป็นผู้โอน **อย่างน้อย 2 ใบ** และ **อย่างน้อย 20%** ของสลิปที่นับ (`MINE_MIN_SLIPS = 2` , `MINE_MIN_SHARE = 0.2`)
- ชื่อไทยและชื่ออังกฤษนับแยกกัน (สลิปบางธนาคารพิมพ์อังกฤษ) และเก็บไว้ไม่เกิน 30 ชื่อที่พบบ่อยที่สุด

ไม่เก็บชื่อจริง เก็บแค่ "ลายนิ้วมือ"

ชื่อคนเป็นข้อมูลส่วนตัว แอปจึง **ไม่เก็บตัวชื่อ** แต่แปลงเป็นรหัส 8 ตัวด้วยฟังก์ชันแฮช FNV-1a (แฮช = สูตรที่แปลงข้อความเป็นตัวเลขสั้น ๆ ข้อความเดิมได้รหัสเดิมเสมอ แต่ดูรหัสแล้วไม่รู้ว่านี่เป็นชื่ออะไร) และเก็บเฉพาะในมือถือเครื่องนั้น แยกตามบัญชี (`mindpay.slipNames.<id>`)

ก่อนแฮช ชื่อถูกย่อให้เหลือ **ชื่อต้น + ตัวอักษรแรกของนามสกุล** และตัดคำนำหน้ากับเครื่องหมายปิดบังออก (`nameKey`) เพื่อให้ชื่อเดียวกันที่พิมพ์ต่างกันได้รหัสเดียวกัน ผลจริง:

ชื่อบนสลิป	ย่อเป็น	ลายนิ้วมือ
นาย สมชาย ใจดี	สมชาย ใ	90fbda2f
สมชาย ใ***	สมชาย ใ	90fbda2f (เหมือนกัน)
MR. SOMCHAI JAIDEE	somchai j	91fd7d61

ตัวอย่าง: เครื่องที่มีสลิป 6 ใบ

ผู้โอนของ 6 สลิป: นาย สมชาย ใจดี · นาย สมชาย ใ*** · MR SOMCHAI JAIDEE · MR SOMCHAI J · นาย สมชาย ใจดี · ร้านป่าแดง (ใบนี้อาจเป็นสลิปคืนเงินจากร้าน) ผลนับจริง:

ลายนิ้วมือ	มาจาก	จำนวน	สัดส่วน	เป็นเงินใหม่ (≥ 2 ใบ และ $\geq 20\%$)
90fbda2f	สมชาย ใจ (ไทย)	3	50%	✓ ใช่
91fd7d61	somchai j (อังกฤษ)	2	33%	✓ ใช่
9d07f9c8	ร้านป่าแดง	1	17%	× ไม่ใช่

ตัดสินใจรับ/รายจ่าย (decideDirection)

ผู้โอน เป็น เงิน?	ผู้รับ เป็น เงิน?	ผล	ตัวอย่างจริง
ใช่	ใช่	โอนระหว่างบัญชีตัวเอง → ตัดตรง "เงินเข้า หรือออก" + บันทึก "ดูเหมือนโอนระหว่าง บัญชีของคุณเอง..."	จาก นาย สมชาย ใจดี → MR. SOMCHAI JAIDEE
ไม่	ใช่	รายรับ จากผู้โอน	น.ส. มะลิ แสนต์ → นาย สมชาย ใจ*** = รายรับจาก "น.ส. มะลิ แสนต์" (ถ้ายังไม่รู้จักชื่อเงิน จะกลายเป็นรายจ่ายให้ "นาย สมชาย ใจ***" ตามที่ AI เห็น)
ใช่	ไม่	รายจ่ายให้ผู้รับ · ถ้า AI บอกว่า "เงินเข้า" = ขัดกัน → ตัดตรง	จาก นาย สมชาย ใจดี → ร้าน (AI ว่า income) = รายจ่าย + รง
ไม่	ไม่	เชื่อ AI · ถ้า AI ไม่รู้ (unknown) → ตัดตรง	ยังไม่เคยเรียนชื่อ = กรณีนี้เสมอ

★ ข้อดีของการเรียนรู้เอง

ผู้ใช้ไม่ต้องตั้งค่าอะไร ใช้ไปเรื่อย ๆ แอปจะแม่นยำขึ้นเอง และถ้าใช้หลายบัญชีในเครื่องเดียว แต่ละบัญชีมีชุดชื่อของตัวเอง
ข้อเสียคือช่วงแรก (ยังมีสลิปน้อยกว่า 2 ใบ) แอปยังแยกสลิปของเพื่อนไม่ได้ ซึ่งผู้ใช้แค่เป็นรายรับเองได้ในหน้าตรวจ

หลักการที่ 6: กันรายการซ้ำ 4 ชั้น

Requirement ข้อ 3: สลิปเดียวกันต้องไม่ถูกนับ 2 ครั้ง ไม่ว่าจะสแกนที่รอบ

สลิปซ้ำเกิดได้หลายทาง: สแกนแคสเลอริซ้ำ, เซฟสลิปเดียวกัน 2 รูป (รูปจากแอปธนาคาร + ภาพแคปหน้าจอ), เพื่อนส่งสลิปที่เราโอนกลับมาให้ดู, หรือเปิดแอป 2 เครื่องพร้อมกัน ถ้านับซ้ำ ยอดเงินจะผิดทันที ฉะนั้นจึงกันไว้หลายชั้น เรียงจากถูกที่สุดไปแพงที่สุด:

ชั้น 0: จำรูปที่เคยตรวจ (mindpay.scanned.<id>)

รูปที่เคยตรวจแล้วข้ามเลย ไม่ต้องหา QR ไม่ต้องย่อ (จำได้ 5,000 รูปล่าสุด) ชั้นนี้ไม่ได้ตัดสินใจซ้ำ แค่ไม่ทำงานซ้ำ



ชั้น 1: ลายนิ้วมือของรูป (SHA-256)

ย่อรูปแล้วแฮช ถ้าเคยมีรายการจากรูปที่แฮชเท่ากัน → ซ้ำ ตรวจ **ก่อนส่ง AI** จึงไม่เสียค่า AI เลย

↓ ส่ง AI อ่าน

ชั้น 2: เลขอ้างอิง (จาก QR ก่อน ไม่มีจึงใช้ที่ AI อ่าน)

ตัวพิมพ์ใหญ่ เฉพาะตัวอักษรและตัวเลข ยาว ≥ 6 ตัว เลขเดียวกัน = การโอนครั้งเดียวกัน แม้จะเป็นรูปคนละไฟล์



ชั้น 3: ยอด + วัน + นาที + ผู้รับ (เฉพาะสลิปที่ไม่มีเลขอ้างอิง)

เช่น "8500|2026-09-27|08:15" กับผู้รับ "ร้านกาแฟดอยช้าง"

↓ บันทึกลงฐานข้อมูล

ชั้น 4: unique index ในฐานข้อมูล

ฐานข้อมูลปฏิเสธรายการที่ (บัญชี, slip_ref) หรือ (บัญชี, slip_image_hash) ซ้ำ แม้ 2 เครื่องสแกนพร้อมกัน → แอปได้ DuplicateSlipError → นับเป็น "ซ้ำ"

supabase/migrations/20260927000000_init.sql

unique index 2 ตัว · บรรทัด 50-53

```
50 create unique index if not exists transactions_user_slip_ref_idx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_idx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:50

50-51 ห้ามมี 2 แถวที่ user_id และ slip_ref เหมือนกัน · where slip_ref is not null = ใช้เฉพาะแถวที่มีเลขอ้างอิง (รายการที่ตัวเองไม่มีเลข จึงไม่ถูกบังคับ)

52-53 แบบเดียวกันสำหรับลายนิ้วมือของรูป · มี user_id ด้วยเพราะ 2 คนอาจเซฟสลิปใบเดียวกัน (คนโอนกับคนรับ) ต่างคนต่างบันทึกได้

ตัวอย่างจริง (ร้านค้าจริง)

มีรายการเดิม: จ่ายร้านค้าแฟลดย่าง ฿85 วันที่ 27 ก.ย. 2569 เวลา 08:15 จากสลิป

สลิปที่อ่านใหม่	ซ้ำ?	ชั้นที่จับ
เลขอ้างอิงเดียวกัน (015270081500ABC123)	ซ้ำ	ชั้น 2
ไม่มีเลขอ้างอิง ยอด วัน นาทิ และร้านเดียวกัน	ซ้ำ	ชั้น 3
ไม่มีเลขอ้างอิง ร้านเดียวกัน ยอดเท่ากัน แต่เวลา 15:40	ไม่ซ้ำ	กาแฟแก้วที่ 2 ตอนบ่าย ไม่ใช่สลิปเดิม
ไม่มีเลขอ้างอิง ยอด วัน นาทิเดียวกัน แต่ผู้รับคือ "ร้านข้าว"	ไม่ซ้ำ	จ่าย 2 ร้านในนาทิตเดียวกันได้

i **ทำไมชั้น 3 ใช้เฉพาะสลิปที่ไม่มีเลขอ้างอิง**

ถ้าสลิปมีเลขอ้างอิงแล้วเลขไม่ซ้ำ แปลว่าเป็นการโอนคนละครั้งแน่นอน แม้ยอด เวลา และร้านจะเหมือนกัน (เช่น ชื่อน้ำ ฿20 สองขวดแยกบิลในนาทิตเดียวกัน) การใช้ชั้น 3 กับสลิปพวกนี้จะทำให้ "รายการจริง" หายไป ส่วนกรณีที่ชื่อผู้รับไม่มี (แอปตั้งชื่อเป็น "รายการจากสลิป") ถือว่าตรงกับทุกข้อ เพื่อกันซ้ำไว้ก่อน

หลักการที่ 7: วันที่บนสลิปแบบไทย

บิกใหญ่ของ prototype รุ่นแรก: สลิปเดือนกันยายนอ่านวันที่ไม่ได้

สลิปไทยพิมพ์วันที่หลายแบบมาก และส่วนใหญ่ใช้ ปี พ.ศ. แบบ 2 หลัก ("69" = 2569 = ค.ศ. 2026) ระบบ OCR/AI ยังชอบทำจุดใน "ก.ย." หายหรือย้าย จึงจึงเขียนตัวอ่านวันที่เอง (`parseSlipDate`) และ เชื้อตัวหนังสือบนสลิปก่อน วันที่ที่ AI แปลงมาให้ ถ้า 2 อย่างไม่ตรงกันติดธง

รูปแบบที่อ่านได้ (ผลจริง สมมติวันนี้ 9 ต.ค. 2569)

บนสลิป	อ่านได้	หมายเหตุ
27 ก.ย. 69	2026-09-27	แบบที่พบบ่อยที่สุด
27ก.ย.2569	2026-09-27	ไม่มีช่องว่าง ปี 4 หลัก
27 กย 69	2026-09-27	จุดหาย (บิกเดิมของ prototype)
27 กันยายน 2569	2026-09-27	ชื่อเดือนเต็ม
27/09/2569	2026-09-27	ตัวเลข
27-09-69	2026-09-27	ตัวเลข ชัด ปี 2 หลัก
2026-09-27	2026-09-27	รูปแบบ ISO
27 Sep 2026	2026-09-27	สลิปภาษาอังกฤษ ปี ค.ศ.
09 ต.ค. 69 14:05	2026-10-09	มีเวลาต่อท้าย (เวลาอ่านแยก)
31 ก.พ. 69	null	ไม่มีวันนี้จริง → ไม่เดา
5 ม.ค. 70	2027-01-05	อ่านได้ แต่ normalizeReading จะเห็นว่าเป็นอนาคตและติดธง

ปี 2 หลัก: พ.ศ. หรือ ค.ศ.?

"69" อาจหมายถึง พ.ศ. 2569 (= ค.ศ. 2026) หรือ ค.ศ. 2069 และ "26" อาจหมายถึง ค.ศ. 2026 หรือ พ.ศ. 2526 (= ค.ศ. 1983) ฟังก์ชัน `normalizeYear` ลองตีความทั้ง 2 แบบ ตัดแบบที่เกินปีหน้าถึง แล้วเลือกแบบที่ ใกล้ปีปัจจุบันที่สุด:

ปีบนสลิป	ตีความเป็น พ.ศ.	ตีความเป็น ค.ศ.	ผล
69	2026	2069 (อนาคตไกล ตัดทิ้ง)	2026
26	1983	2026	2026 (ใกล้กว่า)
70	2027 (ปีหน้า ยังรับ)	2070 (ตัดทิ้ง)	2027
99	2056 (ตัดทิ้ง)	2099 (ตัดทิ้ง)	null

ปีบนสลิป	ตีความเป็น พ.ศ.	ตีความเป็น ค.ศ.	ผล
2569	$\geq 2400 = \text{พ.ศ. เต็ม} \rightarrow \text{ลบ } 543$		2026
2026	$\geq 1900 = \text{ค.ศ. เต็ม}$		2026

เวลา และสิ่งที่เกิดเมื่ออ่านวันที่ไม่ได้

- `parseSlipTime`: "14:05 น." $\rightarrow$ 14:05 · "9.05" $\rightarrow$ 09:05 · "25:00" $\rightarrow$ null
- วันและเวลาถูกรวมเป็นเวลามาตรฐาน (UTC) ด้วย `bkkToIso` โดยถือว่าเป็นเวลาไทย (ไม่มีเวลา = 12:00)
- อ่านวันที่ไม่ได้หรือเป็นอนาคต $\rightarrow$ **ไม่เดา**: ใช้เวลาที่รูปถูกสร้างในแคลเลอรีแกน (`fallbackTimeMs`) และติดธงวันที่ให้ผู้ใช้ตรวจ
- วันที่ต่ำกว่า 1 ปี $\rightarrow$ `out_of_range` ไม่บันทึกอัตโนมัติ (ผู้ใช้จัดเองได้)

หลักการที่ 8: state machine ของการสแกน

ระบบที่มีปุ่มหยุด อ่านต่อ และงานที่ใช้เวลานาน ต้องมีสถานะที่ชัดเจน

State machine (เครื่องสถานะ) คือการออกแบบว่าระบบอยู่ได้ที่ "สถานะ" และจากสถานะหนึ่งไปอีกสถานะได้ด้วย "เหตุการณ์" อะไรบ้าง ไฟจราจรเป็นตัวอย่างที่ง่ายที่สุด: เขียว → เหลือง → แดง → เขียว ไม่มีทางกระโดดจากแดงไปเหลือง การสแกนก็เช่นกัน

idle (ว่าง)

ตั้งช่วงเวลา (setRange) หรือโหลดรูปเข้าคิว (load) ได้เฉพาะตอนนี้และตอน done

↓ start (ต้องมีรูปในคิว)

running (กำลังอ่าน)

itemStarted / itemFinished ทีละรูป · กด "หยุดชั่วคราว" → paused · รูปสุดท้ายเสร็จ → done

↕ pause / resume

paused (หยุดไว้)

รูปที่กำลัง "working" จะถูกอ่านใหม่เมื่อ resume · ถ้าไม่เหลือรูป resume แล้วเป็น done ทันที

↓

done (เสร็จ)

แสดงผลสรุป · เริ่มรอบใหม่ได้ (load / setRange)

บ๊ักใน prototype ที่ออกแบบให้เกิดไม่ได้

รุ่นแรก ถ้าผู้ใช้เปลี่ยนช่วงเวลาระหว่างสแกน คิวยังใช้ช่วงเก่าต่อ หรือข้ามรูปไปเลย และผลอ่านที่กลับมาซ้ำถูกจับคู่กับรูปผิดใบ เพราะจับคู่ด้วย "ลำดับในอาร์เรย์" รุ่นนี้กันด้วย 3 กฎ ([src/domain/scanQueue.ts](#)):

1. เปลี่ยนช่วงได้เฉพาะตอนไม่ได้สแกน (idle หรือ done) ระหว่าง running/paused คำสั่ง setRange ถูกเพิกเฉย
2. ทุกคิวมีเลขรอบ (runId) ใหม่ ผลที่กลับมาพร้อมเลขรอบเก่าถูกทิ้ง
3. จับคู่ผลกับรูปด้วย assetId (รหัสของรูป) ไม่ใช่ตำแหน่งในอาร์เรย์

ไล่ตามจริงทีละคำสั่ง (รันโค้ดจริง)

โหลดรูป A (เก่าสุด), C (ใหม่สุด), B และ A ซ้ำอีกครั้ง:

คำสั่ง	phase	runId	คิว	อธิบาย
load A, C, B, A	idle	1	C:queued B:queued A:queued	ตัด A ที่ซ้ำ เรียงใหม่ไปเก่า เลขรอบ +1
start	running	1	เหมือนเดิม	เริ่มอ่าน
setRange 1y	running	1	เหมือนเดิม	ถูกเพิกเฉย (กำลังสแกนอยู่)

คำสั่ง	phase	runId	คิว	อธิบาย
itemStarted C	running	1	C:working ...	รูปใหม่สุดก่อน
itemFinished C ready	running	1	C:ready B:queued A:queued	เสร็จ 1/3
itemStarted B	running	1	B:working	
pause	paused	1	B:working	หยุดกลางรูป B
ผลของรอบ 0 มาถึงช้า	paused	1	ไม่เปลี่ยน	ถูกทิ้ง เพราะ runId ไม่ตรง
resume	running	1	B:working	รูปถัดไปคือ B (อ่านใหม่)
itemFinished B needs_review	running	1	B:needs_review	เสร็จ 2/3
itemFinished A not_slip	done	1	C:ready B:needs_review A:not_slip	ไม่เหลือรูปในคิว → done อัตโนมัติ
setRange 1y	idle	2	(ว่าง)	ตอนนี้เปลี่ยนได้ ล้างคิว เลข รอบใหม่

★ ทำไมแยก state machine ออกมาเป็นฟังก์ชันล้วน

`scanReducer` รับ "สถานะเดิม + คำสั่ง" แล้วคืน "สถานะใหม่" ไม่แตะมือถือ ไม่แตะเน็ต จึงทดสอบได้ด้วย Vitest ล้วนๆ ในเสี้ยววินาที (TC-26 ถึง TC-30) ส่วนงานจริงอย่างหา QR หรือเรียก AI อยู่ใน `useSlipScanner` ซึ่งแค่ส่งคำสั่งเข้า reducer นี่คือหลัก แยกกฎออกจากงานภายนอก ที่ใช้ทั้งระบบ (ภาค ก)

หลักการที่ 9: สแกนอัตโนมัติ มองย้อนหลังแค่ไหน

เปิดแอปวันละหลายสิบครั้ง ต้องไม่อ่านรูปเดิมซ้ำและไม่พลาดรูปใหม่

ค่าคงที่ (src/domain/autoScan.ts)	ค่า	ความหมาย
FIRST_RUN_DAYS	30 วัน	ครั้งแรกมองย้อนหลัง 30 วัน
MAX_LOOKBACK_DAYS	30 วัน	อัตโนมัติไม่ย้อนเกิน 30 วัน (เผื่อว่านั้นใช้หน้าสแกน "ค้นหาย้อนหลัง 1 ปี")
OVERLAP_MS	1 ชั่วโมง	ย้อนกับรอบก่อน 1 ชม. เพื่อรูปที่ซิงค์เข้าแคเลเลอรี่ซ้ำ
MIN_INTERVAL_MS	5 นาที	เปิดแอปซ้ำภายใน 5 นาทีไม่สแกนใหม่
MAX_IMAGES_PER_RUN	150 รูป	รอบหนึ่งตรวจไม่เกิน 150 รูป ที่เหลือรอบหน้า
MAX_AUTO_ATTEMPTS	2 ครั้ง	รูปที่อ่านไม่ได้ลองรวม 2 ครั้ง แล้วปล่อยให้หน้าสแกน

ตัวอย่างจริง: เริ่มมองจากเวลาไหน (ตอนนี้ 9 ต.ค. 12:00 น.)

สถานการณ์	autoScanSince ตอบ	ทำไม
ไม่เคยสแกน	9 ก.ย. 12:00	ย้อน 30 วัน
สแกนล่าสุด 2 ชม. ก่อน (10:00)	9 ต.ค. 09:00	จากครั้งล่าสุด - 1 ชม.
สแกนล่าสุด 45 วันก่อน	9 ก.ย. 12:00	ไม่ย้อนเกิน 30 วัน

shouldAutoScan	ผล
สแกนล่าสุด 4 นาทีก่อน	false (ยังไม่ถึง 5 นาที)
สแกนล่าสุด 5 นาทีก่อน	true
ปิดสวิตช์ในตั้งค่า	false
ไม่เคยสแกน (lastAutoScanAt = 0)	true

เลื่อนเวลา "สแกนล่าสุด" เมื่อไร

นี่คือส่วนที่ละเอียดที่สุด: ถ้าเลื่อนเวลาไปข้างหน้าแล้วมีรูปที่ยังไม่ได้อ่าน รูปนั้นจะหลุดนอกหน้าต่างของรอบหน้า = **สลิปหาย** เจ็บ

กฎจึงเป็น:

- เลื่อน เฉพาะเมื่อทุกรูปในรอบนี้จัดการเสร็จ (ไม่ได้หยุดกลางทาง และรูปไม่เกิน 150)
- เน็ตหลุด / AI ชัดข้อง / โควตาเต็ม → หยุด **โดยไม่เลื่อน** รอบหน้าจะเห็นรูปเดิมอีก

- มีรูปที่อ่านไม่ได้และยังลองได้อีก → เลื่อนไปแค่ "เวลาของรูปนั้น + 1 ชม. - 1 มิลลิวินาที" (`nextAutoScanMark`) รอบหน้า ซึ่งย้อนกับ 1 ชม. จะยังเห็นรูปนั้น · ตัวอย่างจริง: เริ่ม 12:00 รูปที่ต้องลองใหม่ถ่ายตอน 09:00 → บันทึกเวลา 09:59:59.999 แทน 12:00

เมื่อมีปัญหาระหว่างสแกนอัตโนมัติ

ปัญหา	ทำอะไร
เน็ตหลุด	รอ 3 วินาทีลองรูปเดิมอีกครั้ง ไม่ได้ → หยุด "เชื่อมต่ออินเทอร์เน็ตไม่ได้ สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า"
AI ยุ่ง / เซิร์ฟเวอร์ขัดข้อง	หยุดและบอกเหตุผล แต่ถ้ารูปเดิมทำให้หยุดครบ 3 รอบ (<code>STALL_LIMIT</code>) แปลว่าน่าจะเป็นที่รูป (ใหญ่/อ่านนานจนหมดเวลา) → ปลอ่ยรูปนั้นให้หน้าสแกน รูปอื่นจะได้ไม่ติด
อ่านรูปนี้ไม่สำเร็จ	นับครั้ง ลองใหม่รอบหน้า (รวม 2 ครั้ง) · ถ้าอ่านไม่ได้ 2 รูปติดกัน น่าจะเป็นที่ระบบอ่าน → หยุด "ระบบอ่านสลิปขัดข้องชั่วคราว สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า"
โควตาเต็ม / หมดเซสชัน / ยังไม่ตั้ง key	หยุดและบอกเหตุผล (เกิดกับทุกรูปเหมือนกัน)
ผู้ใช้เปิดหน้าสแกน	สแกนอัตโนมัติหยุดเจียบ ๆ (<code>scanLock.manual</code>) 2 ระบบจะไม่อ่านรูปเดียวกันพร้อมกัน
ผู้ใช้สลับบัญชี	หยุดเจียบ ๆ ไม่บันทึกอะไรเพิ่ม (ตรวจ <code>stillValid</code> ก่อนบันทึกทุกครั้ง) และล้างการ์ดของบัญชีเก่าทันที

หลักการที่ 10: ต้นทุน ความเป็นส่วนตัว และความปลอดภัย

รูปในเคลเลอร์เป็นข้อมูลส่วนตัวที่สุดในมือถือ

ข้อมูล	อยู่ที่ไหน	ใครเห็น
รูปที่ไม่มี QR สลิป	มือถือเท่านั้น	ไม่มีใคร (ไม่ถูกส่งไปไหน)
รูปสลิป (ย่อแล้ว)	ส่งผ่าน HTTPS ไปเซิร์ฟเวอร์ → AI อ่าน → ทิ้ง ไม่เขียนลงฐานข้อมูลหรือที่เก็บไฟล์	ฟังก์ชัน parse-slip และผู้ให้บริการ AI (Google Gemini) ระหว่างอ่าน
ผลอ่าน (ยอด วัน ชื่อร้าน เลข อ้างอิง แหขรูป)	ตาราง transactions	เจ้าของบัญชีเท่านั้น (Row Level Security)
รายการรูปที่เคยตรวจ ลายนิ้วมือชื่อ ตั้งค่าการสแกน	มือถือเครื่องนั้น	ไม่ถูกส่งขึ้นเซิร์ฟเวอร์
key ของ AI	Supabase Secrets ฝั่งเซิร์ฟเวอร์	ไม่เคยอยู่ในแอป

ด่านป้องกันบนเซิร์ฟเวอร์ (ฟังก์ชัน parse-slip)

- ต้องล็อกอิน:** ตรวจ token ทุกครั้ง ไม่ผ่าน → 401 "unauthorized"
- ขนาด:** ดูขนาดจาก header ก่อนอ่านตัวข้อมูล (กันคนส่งไฟล์ยักษ์ให้หน่วยความจำเต็ม) และ base64 ต้องไม่เกิน 6,000,000 ตัวอักษร (ประมาณ 4.5 MB) · ชนิดรูปต้องเป็น JPEG, PNG หรือ WebP
- โควตา:** 300 สลิปต่อวันต่อบัญชี (ตั้งได้ด้วย SLIP_DAILY_LIMIT) นับในฐานข้อมูลแบบล๊อคต่อผู้ใช้ ยิ่งพร้อมกันหลายคำขอก็หลุดโควตาไม่ได้
- ไม่บันทึก log ที่มีข้อมูลส่วนตัว:** log เขียนแค่ "อ่านกี่รอบ ตรงกันไหม ใช้รุ่นอะไร ใช้เวลาที่ ms" ไม่มีรูป ไม่มีผลอ่าน
- เวลา:** เรียก AI แต่ละครั้งไม่เกิน 45 วินาที ทั้งคำขอไม่เกิน 110 วินาที (แพลตฟอร์มตัดที่ 150) เกินเวลาถือว่า "AI ยุ่ง" ให้แปปรอแล้วลองใหม่

ต้นทุน

- ส่งเฉพาะรูปที่น่าจะเป็นสลิป (ในตัวอย่างหน้าจอ 6 รูป ส่งแค่ 3)
- ย่อรูปให้กว้างไม่เกิน 1,100 px เป็น JPEG คุณภาพ 0.8 (ราว 150 KB) พอให้อ่านตัวเลขชัด แต่ส่งเร็วและใช้ token ของ AI น้อย
- รูปซ้ำ (แฮชเดิม) ถูกจับ **ก่อน** ส่ง AI · รูปที่เคยตรวจถูกข้ามตั้งแต่แรก
- ใช้ Gemini รุ่น flash-lite (เล็กและถูกที่สุด) เป็นตัวหลัก และช่วงทดลองใช้แบบฟรี ซึ่งหน้าตั้งค่าแจ้งผู้ใช้ตรง ๆ ว่า "Google อาจเก็บข้อมูลที่ส่งไปเพื่อปรับปรุง AI ตามเงื่อนไขของ Google"

“ ประโยคเดียวที่สรุปการออกแบบของ FR-4

"ทำทุกอย่างที่ทำงานมือถือได้ฟรีก่อน ส่งให้ AI เฉพาะที่จำเป็น ให้ AI 2 ตัวตรวจกันเอง และถ้ายังไม่แน่ใจ ให้คนตัดสิน
ตัวเลขเงินของผู้ใช้ต้องไม่เปลี่ยนจากข้อมูลที่ไม่น่าใจ"

อ่านโค้ด: slip.ts ตอนที่ 1 — ชนิดข้อมูลและ QR

src/domain/slip.ts คือ "สมอง" ของ FR-4 ยาว 405 บรรทัด เป็นฟังก์ชันล้วนทั้งไฟล์ ตอนนี้อ่านบรรทัด 1-217

ไฟล์นี้ไม่มีโค้ดที่แตะมือถือ เบ็ด หรือฐานข้อมูลเลย มีแต่ "กฎ" ที่รับข้อมูลเข้าแล้วคืนคำตอบ จึงทดสอบได้ทุกบรรทัดด้วย Vitest ถ้าเข้าใจไฟล์นี้ไฟล์เดียว ก็เข้าใจครึ่งหนึ่งของ FR-4 แล้ว

ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import (บรรทัด 1-20)

src/domain/slip.tsบรรทัด 1-20

```
1 /**
2  * FR-4 Automatic Gallery Slip Detection – the rules, separated from the phone APIs
3  * so every rule can be unit-tested.
4  *
5  * Pipeline for one image:
6  *   1. read the QR code on the slip (on the phone, free)           -> parseSlipQr
7  *   2. send the image to the slip reader (server + AI vision: Claude or Gemini) -> SlipReading
8  *   3. clean the reading and flag unsure fields                   -> normalizeReading
9  *   4. decide what to do with it                                 -> classifyCandidate
10 *
11 * CONFIDENCE RULE (Charter: 80%): a field whose confidence is below 0.8, or that
12 * is missing, is flagged. A slip with no flags is "ready" (one tap to confirm);
13 * any flag makes it "needs_review". Nothing from a slip is ever counted in a
14 * total until the user confirms it.
15 */
16 import { suggestCategory } from './categories';
17 import { addDays, bkkDayKey, isDayInRange, parseSlipDate, parseSlipTime } from './dates';
18 import { formatBaht, parseBahtToSatang } from './money';
19 import { decideDirection, EMPTY_NAME_STATS, type NameStats } from './slipNames';
20 import type { Transaction, TxKind, TxStatus } from './types';
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:1

บรรทัด	ความหมาย
1-3	ไฟล์นี้คือ "กฎ" ของ FR-4 แยกออกจากโค้ดที่เรียกใช้ฟีเจอร์ของมือถือ เพื่อให้ทุกกฎทดสอบได้
5-9	ท่อน 4 ขั้นของรูป 1 รูป: (1) อ่าน QR บนมือถือ ฟรี → parseSlipQr (2) ส่งรูปให้ตัวอ่านสลิปบนเซิร์ฟเวอร์ → SlipReading (3) ทำความสะอาดและตัดตรงช่องที่ไม่แน่ใจ → normalizeReading (4) ตัดสินว่าจะทำอะไร → classifyCandidate
11-14	กฎความมั่นใจ (Charter: 80%): ช่องที่ความมั่นใจต่ำกว่า 0.8 หรือว่าง ถูกตัดตรง ไม่มีธง = "ready" (แตะครั้งเดียว ยืนยัน) มีธง = "needs_review" และย้ำว่า ไม่มีอะไรจากสลิปถูกนับในยอดจนกว่าผู้ใช้ยืนยัน (ยกเว้นเมื่อเปิด "รวมยอดทันที" ซึ่งตัดสินในบรรทัด 403)
16	suggestCategory จาก FR-1 ใช้เดาหมวดจากชื่อผู้รับ
17	ฟังก์ชันวันที่: บวกวัน, วันแบบไทย (Bangkok), อยู่ในช่วงไหม, อ่านวันที่/เวลาบนสลิป
18	แสดงเงินเป็น "฿1,250.00" และแปลงข้อความเงินเป็นสตริง

บรรทัด	ความหมาย
19	ตัดสินรายรับ/รายจ่ายจากชื่อ (ไฟล์ slipNames.ts บกกัดไป)
20	<code>import type</code> = นำเข้าเฉพาะ "ชนิดข้อมูล" ไม่มีโค้ดจริงติดมา (หายไปตอนแปลงเป็น JavaScript)

ส่วนที่ 2: เกณฑ์ 80% และหน้าต่างของผลอ่าน (บรรทัด 22–57)

src/domain/slip.ts

บรรทัด 22–57

```

22 export const CONFIDENCE_THRESHOLD = 0.8;
23
24 /** What the slip reader (supabase/functions/parse-slip) returns for one image. */
25 export interface SlipReading {
26   isSlip: boolean;
27   direction: 'expense' | 'income' | 'unknown';
28   amount: string | number | null;
29   /** Date exactly as printed, e.g. "27 ก.ย. 69" */
30   dateText: string | null;
31   /** The reader's own conversion to YYYY-MM-DD (Gregorian), may be null */
32   dateIso: string | null;
33   time: string | null;
34   counterparty: string | null;
35   bank: string | null;
36   reference: string | null;
37   /** Payer and receiver as printed (newer readers only). */
38   fromName?: string | null;
39   toName?: string | null;
40   confidence: { amount: number; date: number; counterparty: number };
41 }
42
43 /** How the reader double-checked the slip (parse-slip reads every slip twice with two models).
44  */
45 export interface SlipCheck {
46   verified: boolean;
47   reads: number;
48   disagree: Partial<Record<'amount' | 'date' | 'direction' | 'counterparty', [string | null,
49     string | null]>>;
50 }
51
52 export type ReviewFlag = 'amount' | 'date' | 'counterparty' | 'direction';
53
54 export const FLAG_LABEL: Record<ReviewFlag, string> = {
55   amount: 'ยอดเงิน',
56   date: 'วันที่',
57   counterparty: 'ผู้รับ/ผู้โอน',
58   direction: 'เงินเข้าหรือออก',
59 };

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:22

บรรทัด	ความหมาย
22	<code>CONFIDENCE_THRESHOLD = 0.8</code> ตัวเลข 80% ของ Requirement อยู่ที่นี่ที่เดียวทั้งแอป <code>export</code> เพื่อให้ทดสอบและไฟล์อื่นอ้างถึงได้
24–25	<code>interface SlipReading</code> = สัญญาว่าเซิร์ฟเวอร์ (parse-slip) จะส่งข้อมูลหน้าต่างแบบนี้มาสำหรับ 1 รูป

บรรทัด	ความหมาย
26	<code>isSlip</code> AI ตัดสินว่าเป็นสลิปโอนเงินที่สำเร็จแล้วไหม
27	<code>direction</code> เป็นได้แค่ 3 ค่า: เงินออก / เงินเข้า / ไม่รู้ (ชนิดแบบ "ค่าใดค่าหนึ่งในนี้" เรียกว่า union type)
28	<code>amount</code> เป็นข้อความ ตัวเลข หรือ null ก็ได้ (เพื่อ AI ต่างค่ายตอบต่างแบบ) แอปจะแปลงเป็นสตริงเอง
29–32	<code>dateText</code> วันที่ตามที่พิมพ์ เช่น "27 ก.ย. 69" · <code>dateIso</code> วันที่ที่ AI แปลงเป็น ค.ศ. เอง อาจเป็น null
33–36	เวลา, ชื่ออีกฝ่าย, ธนาคาร, เลขอ้างอิง
37–39	<code>fromName</code> , <code>toName</code> ผู้โอนและผู้รับตามที่พิมพ์ เครื่องหมาย ? หลังชื่อ = ช่องนี้อาจไม่มีเลย (ตัวอ่านรุ่นเก่าไม่ส่งมา)
40	<code>confidence</code> ความมั่นใจ 3 ช่อง แต่ละช่องเป็นตัวเลข
43–48	SlipCheck = รายงานการอ่าน 2 รอบ: ตรงกันทุกช่องไหม (<code>verified</code>), อ่านได้กี่รอบ (<code>reads</code>), ช่องที่ไม่ตรงพร้อม 2 ค่า (<code>disagree</code>) <code>Partial<Record<...>></code> = อธิบายเจตที่อาจมีบางคีย์จาก 4 คีย์นี้ แต่ละคีย์เก็บคู่ [ค่าที่ 1, ค่าที่ 2]
50	ReviewFlag ธงมี 4 ชนิด
52–57	FLAG_LABEL ชื่อภาษาไทยของธง ใช้แสดงในป้าย "ตรวจยอดเงิน" และ "ช่วยตรวจ: ยอดเงิน, ผู้รับ/ผู้โอน" <code>Record<ReviewFlag, string></code> บังคับให้มีชื่อครบทุกธง ถ้าเพิ่มธงใหม่แต่ลืมใส่ชื่อ TypeScript จะแจ้ง error กันที่

ส่วนที่ 3: ผู้สมัคร (บรรทัด 59–74)

<code>src/domain/slip.ts</code>	บรรทัด 59–74
<pre> 59 export interface SlipCandidate { 60 kind: TxKind; 61 amountSatang: number null; 62 dayKey: string null; 63 time: string null; 64 counterparty: string null; 65 bank: string null; 66 ref: string null; 67 imageHash: string null; 68 categoryKey: string; 69 /** Lowest of the three field confidences */ 70 confidence: number; 71 flags: ReviewFlag[]; 72 /** Payer and receiver are both the user (money moved between the user's own accounts). */ 73 ownTransfer: boolean; 74 } </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → <code>src/domain/slip.ts:59</code>

บรรทัด	ความหมาย
59	SlipCandidate ("ผู้สมัคร") = ผลอ่านที่ทำความสะอาดแล้ว พร้อมจะเป็นรายการ แต่ยังไม่ได้ตัดสินว่าจะบันทึกไหม
60–61	ประเภท (income/expense) และยอดเป็น สตางค์ (จำนวนเต็ม) หรือ null ถ้าอ่านไม่ได้
62–63	<code>dayKey</code> วันที่รูปแบบ "YYYY-MM-DD" ตามเวลาไทย และเวลา "HH:MM"
64–67	อีกฝ่าย, ธนาคาร, เลขอ้างอิง (ใช้กันซ้ำ), แหขรูป (ใช้กันซ้ำ)

บรรทัด	ความหมาย
68	หมวดที่เดาได้
69–70	<code>confidence</code> = ค่า ต่ำสุด ของ 3 ช่อง (คอมเมนต์ <code>/** */</code> เหนือบรรทัดจะโชว์เป็นคำอธิบายเวลาเอาเมาส์ชี้ใน VS Code)
71	<code>flags</code> อาร์เรย์ของธง ว้าง = อ่านชัดทุกช่อง
72–73	<code>ownTransfer</code> ผู้โอนและผู้รับเป็นผู้ใช้ทั้งคู่ (โอนระหว่างบัญชีตัวเอง)

ส่วนที่ 4: `parseTlv` – แยกรูปแบบ Tag-Length-Value (บรรทัด 80–95)

<code>src/domain/slip.ts</code>	บรรทัด 80–95
<pre> 80 <i>/** Parse EMVCo-style TLV: 2-digit tag, 2-digit length, value. */</i> 81 export function parseTlv(data: string): Map<string, string> null { 82 const out = new Map<string, string>(); 83 let i = 0; 84 while (i < data.length) { 85 if (i + 4 > data.length) return null; 86 const tag = data.slice(i, i + 2); 87 const len = Number(data.slice(i + 2, i + 4)); 88 if (!/^d{2}\$/.test(tag) !Number.isInteger(len)) return null; 89 const value = data.slice(i + 4, i + 4 + len); 90 if (value.length !== len) return null; 91 out.set(tag, value); 92 i += 4 + len; 93 } 94 return out; 95 } </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → <code>src/domain/slip.ts:80</code>

บรรทัด	ความหมาย
81	รับข้อความ คืน <code>Map</code> (ตาราง Tag → Value) หรือ <code>null</code> ถ้าไม่ใช่รูปแบบนี้
82	สร้าง Map ว้าง <code>new Map<string, string>()</code> = คีย์เป็นข้อความ ค่าเป็นข้อความ
83	<code>let i = 0</code> ตัวชี้ตำแหน่ง ใช้ <code>let</code> เพราะค่าจะเปลี่ยน
84	วนจนกว่าตัวชี้จะถึงท้ายข้อความ
85	เหลือไม่ถึง 4 ตัว (Tag 2 + Length 2) = ข้อมูลขาด → <code>null</code>
86	<code>data.slice(i, i + 2)</code> ตัด 2 ตัวอักษรเป็น Tag (<code>slice(a, b)</code> = ตั้งแต่ตำแหน่ง a ถึงก่อน b)
87	อีก 2 ตัวถัดไปแปลงเป็นตัวเลข = ความยาวของค่า
88	Tag ต้องเป็นเลข 2 หลัก (<code>/^d{2}\$/.test(tag)</code>) และความยาวต้องเป็นจำนวนเต็ม ไม่ขึ้น <code>null</code> ("WALLET-RE..." ตกตรงนี้ เพราะ "WA" ไม่ใช่ตัวเลข)
89	ตัดค่าออกมาตามความยาว
90	ตัดได้ไม่ครบตามที่บอก (ข้อความสั้นกว่าที่ควร) → <code>null</code>

บรรทัด	ความหมาย
91	เก็บลง Map
92	เลื่อนตัวชี้ข้าม Tag + Length + ค่า
94	แกะครบทุกส่วนพอดี → คืน Map

★ ผลจริงเมื่อเรียก parseTlv กับ QR ของ slip-qr.jpg

```
[[["00", "000600000101030040220016271094231BTF05678"], ["51", "TH"],
["91", "385A"]] และเรียกซ้ำกับค่าของ Tag 00 ได้ [["00", "000001"], ["01", "004"],
["02", "016271094231BTF05678"]]
```

ส่วนที่ 5: crc16 — ตัวเลขตรวจสอบ (บรรทัด 97–107)

src/domain/slip.ts

บรรทัด 97–107

```
97 /** CRC-16/CCITT-FALSE, used by Thai QR payloads (tag 91). */
98 export function crc16(input: string): string {
99   let crc = 0xffff;
100   for (let i = 0; i < input.length; i++) {
101     crc ^= input.charCodeAt(i) << 8;
102     for (let b = 0; b < 8; b++) {
103       crc = crc & 0x8000 ? ((crc << 1) ^ 0x1021) & 0xffff : (crc << 1) & 0xffff;
104     }
105   }
106   return crc.toString(16).toUpperCase().padStart(4, '0');
107 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:97

ส่วนนี้เป็นสูตรคณิตศาสตร์มาตรฐานชื่อ CRC-16/CCITT-FALSE (ใช้ใน QR พร้อมเพย์ด้วย) ไม่ต้องจำสูตร แต่ควรอ่านออกว่าแต่ละบรรทัดทำอะไร:

บรรทัด	ความหมาย
99	เริ่มที่ 0xffff (เลขฐาน 16 = 65,535 = บิต 1 ถึง 16 ตัว) ตามมาตรฐานแบบ "FALSE"
100	วนทีละตัวอักษรของข้อความ
101	charCodeAt(i) = รหัสตัวเลขของตัวอักษร (เช่น "0" = 48) เลื่อนไปซ้าย 8 บิต (<< 8) แล้ว XOR (^=) เข้ากับ crc · XOR = "ต่างกันได้ 1 เหมือนกันได้ 0" ของแต่ละบิต
102	ทำซ้ำ 8 ครั้ง (ทีละบิตของตัวอักษร)
103	ถ้าบิตซ้ายสุด (0x8000) เป็น 1: เลื่อนซ้าย 1 บิตแล้ว XOR กับ 0x1021 (ตัวเลขประจำสูตรนี้) · ถ้าเป็น 0: เลื่อนซ้ายอย่างเดียว · & 0xffff ตัดให้เหลือ 16 บิตเสมอ
106	แปลงผลเป็นเลขฐาน 16 ตัวพิมพ์ใหญ่ เติม 0 ข้างหน้าให้ครบ 4 ตัว เช่น "385A", "0ECC"

≈ นึกภาพเครื่องปั่น

ใส่ตัวอักษรทีละตัวลงเครื่องปั่น เครื่องคลุกกับสิ่งที่อยู่ในโถจนผลลัพธ์ "ขึ้นกับทุกตัวอักษรและลำดับของมัน" เปลี่ยนตัวอักษรเดียว รสชาติ (ตัวเลข 4 หลัก) ก็เปลี่ยน คนสร้าง QR ปั่นไว้ก่อนแล้วแปะรสชาติท้าย QR เราปั่นซ้ำแล้วชิมเทียบ

ส่วนที่ 6: parseSlipQr — QR นี้เป็น QR ตรวจสอบสลิปไหม (บรรทัด 109–134)

src/domain/slip.ts

บรรทัด 109–134

```
109 export interface SlipQr {
110   sendingBank: string;
111   transRef: string;
112   crcValid: boolean;
113 }
114
115 /**
116  * Thai bank slips carry a "slip verification" QR. It holds the sending bank
117  * code and a transaction reference that is unique per transfer, which makes it
118  * the most reliable duplicate key we have. Payment QRs (PromptPay) are rejected.
119  */
120 export function parseSlipQr(data: string | null | undefined): SlipQr | null {
121   if (!data) return null;
122   const top = parseTlv(data.trim());
123   if (!top) return null;
124   const info = top.get('00');
125   if (!info) return null;
126   const sub = parseTlv(info);
127   if (!sub || sub.get('00') !== '000001') return null;
128   const bank = sub.get('01');
129   const ref = sub.get('02');
130   if (!bank || !ref) return null;
131   const crc = top.get('91');
132   const crcValid = !!crc && crc.toUpperCase() === crc16(data.slice(0, data.lastIndexOf('9104') + 4));
133   return { sendingBank: bank, transRef: ref, crcValid };
134 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:109

บรรทัด	ความหมาย
109–113	SlipQr สิ่งที่ได้จาก QR: รหัสธนาคาร, เลขอ้างอิง, CRC ถูกต้องไหม
115–119	คอมเมนต์: QR นี้เก็บรหัสธนาคารผู้โอนและเลขอ้างอิงที่ไม่ซ้ำต่อการโอน จึงเป็น "คีย์กันซ้ำที่น่าเชื่อถือที่สุด" และ QR สำหรับจ่ายเงิน (PromptPay) ต้องถูกปฏิเสธ
120	รับข้อความจาก QR (อาจเป็น null/undefined เมื่อสแกนไม่เจอ)
121	ไม่มีข้อความ → null (คืนเร็วแบบ "early return" ทำให้โค้ดข้างล่างไม่ต้องเช็คซ้ำ)
122–123	แกะชั้นนอก (ตัดช่องว่างหัวท้ายก่อน) แกะไม่ได้ → null
124–125	ต้องมี Tag 00 (ข้อมูลการโอน)

บรรทัด	ความหมาย
126–127	แกะชั้นในต่อ และ Tag ย่อย 00 ต้องเป็น '000001' พอดี · QR พร้อมเพย์มี Tag 00 = "01" (เวอร์ชันของ QR) แกะชั้นในไม่ได้ จึงตกตรงนี้
128–130	ดึงรหัสธนาคาร (Tag ย่อย 01) และเลขอ้างอิง (02) ขาดอันใดอันหนึ่ง → null
131	CRC ที่แปะมา (Tag 91)
132	<code>data.lastIndexOf('9104')</code> หาตำแหน่ง "9104" ตัวสุดท้าย (Tag 91 ยาว 04) แล้วคำนวณ CRC ของข้อความตั้งแต่ต้นจนจบ "9104" เทียบกับที่แปะมา (ตัวพิมพ์ใหญ่ทั้งคู่) · <code>!!crc</code> = แปลงเป็น true/false (ไม่มี CRC = false)
133	คืนผล ใช้ชื่อย่อ (shorthand) <code>{ ..., crcValid } = { crcValid: crcValid }</code>

! ข้อสังเกตที่ฉันทเเจตอนเขียนเล่มนี้

QR ที่ CRC ไม่ถูก (`crcValid: false`) ยังถูกคืนเป็น SlipQr ปกติ และตอนนี้ยังไม่มีโค้ดส่วนไหนใช้ค่า `crcValid` ตัดสินอะไร (มีแต่เกสที่ตรวจว่าคำนวณถูก) ผลคือ QR ที่ถูกแก้ข้อมูลก็还用เลขอ้างอิงได้ ฉันทจดไว้ในหัวข้อข้อจำกัดท้ายเล่ม พร้อมวิธีแก้

ส่วนที่ 7: รหัสธนาคาร (บรรทัด 136–168)

src/domain/slip.ts

บรรทัด 136–168

```

136 /**
137  * Bank codes in the slip QR (the Bank of Thailand's 3-digit codes). The QR is
138  * made by the sending bank, so its code names the bank more reliably than
139  * reading the logo on the picture.
140  */
141 export const BANK_BY_CODE: Record<string, string> = {
142   '002': 'Bangkok Bank',
143   '004': 'KBank',
144   '006': 'Krungthai',
145   '011': 'ttb',
146   '014': 'SCB',
147   '017': 'Citibank',
148   '020': 'Standard Chartered',
149   '022': 'CIMB Thai',
150   '024': 'UOB',
151   '025': 'Krungsri',
152   '030': 'GSB (ออมสิน)',
153   '031': 'HSBC',
154   '033': 'GHB (ธอส.)',
155   '034': 'BAAC (ธ.ก.ส.)',
156   '066': 'Islamic Bank',
157   '067': 'TISCO',
158   '069': 'KKP',
159   '070': 'ICBC Thai',
160   '071': 'Thai Credit',
161   '073': 'LH Bank',
162   '098': 'SME D Bank',
163 };
164
165 export function bankFromCode(code: string | null | undefined): string | null {
166   if (!code) return null;

```

```
167 return BANK_BY_CODE[code.padStart(3, '0').slice(-3)] ?? null;
168 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/slip.ts:136`

บรรทัด	ความหมาย
136–140	รหัส 3 หลักของธนาคารแห่งประเทศไทย QR สร้างโดยธนาคารผู้โอนเอง จึงบอกธนาคารได้แม่นยำกว่าอ่านโลโก้
141–163	BANK_BY_CODE ตาราง 21 ธนาคาร (Record<string, string> = อ็อบเจกต์ที่คีย์และค่าเป็นข้อความ) ใช้ชื่อสั้นที่คนไทยคุ้น และวงเล็บภาษาไทยสำหรับธนาคารรัฐ
165	bankFromCode รับรหัส คืนชื่อหรือ null
166	ไม่มีรหัส → null
167	<code>padStart(3, '0')</code> เติม 0 ข้างหน้าให้ครบ 3 ตัว ("4" → "004") · <code>.slice(-3)</code> เอา 3 ตัวท้าย (เพื่อมีเลขเกินมา เช่น "0004" → "004") · <code>?? null</code> = หาไม่เจอ (undefined) ให้เป็น null

ส่วนที่ 8: firstSlipQr (บรรทัด 170–177)

`src/domain/slip.ts`

บรรทัด 170–177

```
170 /** The first code in a picture that is a Thai slip-verification QR. */
171 export function firstSlipQr(codes: (string | null | undefined)[]): SlipQr | null {
172   for (const code of codes) {
173     const qr = parseSlipQr(code);
174     if (qr) return qr;
175   }
176   return null;
177 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/slip.ts:170`

บรรทัด	ความหมาย
171	รูปหนึ่งอาจมีหลาย QR (เช่น QR ตรวจสอบสลิป + QR โฆษณาของธนาคาร) รับเป็นอาร์เรย์ของข้อความ
172–175	วนทีละอัน อันแรกที <code>parseSlipQr</code> แกะได้ คืนทันที (QR อื่นถูกข้าม)
176	ไม่มีอันไหนเป็น QR สลิปเลย → null

ส่วนที่ 9: isScreenSized (บรรทัด 179–194)

`src/domain/slip.ts`

บรรทัด 179–194

```
179 export interface PictureSize {
180   width?: number | null;
181   height?: number | null;
182 }
183
184 /**
185  * Pictures about the size of a phone screen: slips saved by bank apps and
186  * screenshots, not camera photos (those are 1,500+ pixels wide). Only these get
187  * the closer look below, which keeps scanning a big gallery quick.
```

```

188 */
189 export function isScreenSized(size: PictureSize | null | undefined): boolean {
190   const w = size?.width ?? 0;
191   const h = size?.height ?? 0;
192   if (!w || !h) return true; // size unknown: look closer anyway
193   return w <= 1600 && h >= w;
194 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:179

บรรทัด ความหมาย

179–182 `PictureSize` กว้าง/สูง ซึ่งอาจไม่รู้ (? และ | null)

184–188 "ขนาดหน้าจอ" = สลิปที่แอปธนาคารเซฟ และภาพแคปหน้าจอ ไม่ใช่รูปกล้อง (กว้าง 1,500 px ขึ้นไป) เฉพาะรูปพวกนี้ได้ "ดูใกล้ ๆ" เพื่อให้สแกนแถบเลอริใหญ่ ๆ ได้เร็ว

190–191 `size?.width ?? 0` = ถ้า size ไม่มี หรือ width ไม่มี ใช้ 0 (?. = optional chaining ไม่พังถ้าตัวหน้าเป็น null)

192 ไม่รู้ขนาด (เป็น 0) → true ดูให้ละเอียดไว้ก่อน

193 กว้าง ≤ 1,600 และ สูง ≥ กว้าง (แนวตั้งหรือจัตุรัส)

ส่วนที่ 10: ส่วนของรูปที่ตัดมาสแกนใหม่ (บรรทัด 196–217)

src/domain/slip.ts

บรรทัด 196–217

```

196 /**
197  * Where banks print the verification QR on a slip, as fractions of the picture.
198  * Phone QR scanners (Android's in particular) read best when the code fills a
199  * large part of the image, and the slip QR is small, so when the whole picture
200  * shows no slip QR the scan looks again at these parts, one by one.
201  */
202 export const SLIP_QR_REGIONS: readonly { x: number; y: number; width: number; height: number }[]
203   = [
204     { x: 0, y: 0.45, width: 1, height: 0.55 }, // lower half (most banks)
205     { x: 0.45, y: 0.55, width: 0.55, height: 0.45 }, // lower right corner (K PLUS, SCB EASY,
206     // Krungthai NEXT...)
207   ];
208
209 /** A region in whole pixels, always inside the picture. */
210 export function regionInPixels(region: { x: number; y: number; width: number; height: number },
211   size: { width: number; height: number }) {
212   const originX = Math.max(0, Math.floor(region.x * size.width));
213   const originY = Math.max(0, Math.floor(region.y * size.height));
214   return {
215     originX,
216     originY,
217     width: Math.max(1, Math.min(size.width - originX, Math.round(region.width * size.width))),
218     height: Math.max(1, Math.min(size.height - originY, Math.round(region.height *
219     size.height))),
220   };
221 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:196

บรรทัด	ความหมาย
196–201	ตัวสแกนของ Android อ่านดีเมื่อ QR ใหญ่เต็มภาพ แต่ QR สลিপเล็ก ถ้าสแกนทั้งรูปไม่เจอ ให้ดูใหม่ทีละส่วน
202	<code>readonly</code> = อาร์เรย์นี้ห้ามแก้ไข (กันโค้ดส่วนอื่นเผลอเปลี่ยน) ทุกค่าเป็น สัดส่วน 0–1 ของรูป จึงใช้ได้กับรูปทุกขนาด
203	ครึ่งล่าง: เริ่มที่ 45% ของความสูง กว้างเต็ม สูง 55%
204	มุมขวาล่าง: เริ่ม 45% ของความกว้าง 55% ของความสูง (ตัวอย่างแอปที่พิมพ์ QR ตรงนี้อยู่ในคอมเมนต์)
208	regionInPixels แปลงสัดส่วนเป็นพิกเซลจริงของรูปนั้น
209–210	จุดเริ่ม = สัดส่วน × ขนาด ปิดลง (<code>Math.floor</code>) และไม่ติดลบ
211–216	กว้าง/สูง = สัดส่วน × ขนาด ปิดใกล้สุด แต่ ไม่เกินขอบรูป (<code>Math.min(ขนาด - จุดเริ่ม, ...)</code>) และอย่างน้อย 1 px · ตัวอย่าง 1080×1920 มุมขวาล่าง: $\text{originX} = \text{floor}(0.45 \times 1080) = 486$ · $\text{originY} = \text{floor}(0.55 \times 1920) = 1056$ · กว้าง = $\text{round}(0.55 \times 1080) = 594$ · สูง = $\text{round}(0.45 \times 1920) = 864$ ซึ่ง $486 + 594 = 1080$ พอดีขอบ

อ่านโค้ด: slip.ts ตอนที่ 2 — จากผลอ่านสู่การตัดสินใจ

บรรทัด 219–405: ทำความสะอาด ตัดธง เขียนบันทึก กั้นซ้ำ และตัดสินใจสถานะ

ส่วนที่ 1: normalizeRef และ clamp01 (บรรทัด 219–229)

src/domain/slip.ts

บรรทัด 219–229

```

219 // -----
220 // Reading -> candidate
221 // -----
222
223 export function normalizeRef(ref: string | null | undefined): string | null {
224   if (!ref) return null;
225   const r = ref.toUpperCase().replace(/[^A-Z0-9]/g, '');
226   return r.length >= 6 ? r : null;
227 }
228
229 const clamp01 = (n: unknown) => (typeof n === 'number' && Number.isFinite(n) ? Math.min(1,
    Math.max(0, n)) : 0);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:219

บรรทัด	ความหมาย
223	normalizeRef ทำเลออ้างอิงให้อยู่รูปแบบเดียว เพื่อให้เทียบกันได้
224	ไม่มีเลข → null
225	เป็นตัวพิมพ์ใหญ่ทั้งหมด แล้วลบทุกอย่างที่ไม่ใช่ A–Z หรือ 0–9 (<code>[^A-Z0-9]</code> = "ไม่ใช่ตัวเหล่านี้" ธง <code>g</code> = ลบทุกตัว)
226	สั้นกว่า 6 ตัว ถือว่าไม่ใช่เลขอ้างอิงจริง (เช่น AI อ่านได้แค่ "AB12") → null · ผลจริง: "0152-ABC" → "0152ABC" · "ab 12" → null · "tmn55501" → "TMN55501"
229	clamp01 ("หนีบ" ค่าให้อยู่ระหว่าง 0 ถึง 1): ถ้าเป็นตัวเลขจริง (<code>Number.isFinite</code> ไม่ใช่ NaN หรือ Infinity) บีบให้อยู่ในช่วง ไม่เกินได้ 0 · ใช้กับความมั่นใจที่ได้จากภายนอก เพื่อ AI ส่ง 1.2 หรือ "0.9" มา

ส่วนที่ 2: normalizeReading — หัวใจของ FR-4 (บรรทัด 231–277)

src/domain/slip.ts

บรรทัด 231–277

```

231 export function normalizeReading(
232   reading: SlipReading,
233   extra: { qr?: SlipQr | null; imageHash?: string | null; now?: Date; names?: NameStats } = {},
234 ): SlipCandidate {
235   const now = extra.now ?? new Date();
236   const today = bkkDayKey(now);
237   const flags: ReviewFlag[] = [];
238   const conf = {
239     amount: clamp01(reading.confidence?.amount),
240     date: clamp01(reading.confidence?.date),
241     counterparty: clamp01(reading.confidence?.counterparty),
242   };

```

```

243
244 const amountSatang = parseBahtToSatang(reading.amount);
245 if (amountSatang === null || conf.amount < CONFIDENCE_THRESHOLD) flags.push('amount');
246
247 // Trust our own parser of the printed text first; fall back to the reader's ISO date.
248 const fromText = parseSlipDate(reading.dateText, now);
249 const fromIso = parseSlipDate(reading.dateIso, now);
250 const dayKey = fromText ?? fromIso;
251 const disagree = !!fromText && !!fromIso && fromText !== fromIso;
252 const inFuture = !!dayKey && dayKey > addDays(today, 1);
253 if (!dayKey || conf.date < CONFIDENCE_THRESHOLD || disagree || inFuture) flags.push('date');
254
255 // Income or expense: from the names on the slip once the user's own name is known.
256 const dir = decideDirection(reading, extra.names ?? EMPTY_NAME_STATS);
257 const kind: TxKind = dir.kind;
258 const counterparty = dir.counterparty?.trim() || null;
259 if (!counterparty || conf.counterparty < CONFIDENCE_THRESHOLD) flags.push('counterparty');
260 if (dir.unsure) flags.push('direction');
261
262 return {
263   kind,
264   amountSatang,
265   dayKey: inFuture ? null : dayKey,
266   time: parseSlipTime(reading.time),
267   counterparty,
268   // The QR's bank code first: it comes from the bank itself.
269   bank: bankFromCode(extra.qr?.sendingBank) ?? (reading.bank?.trim() || null),
270   ref: normalizeRef(extra.qr?.transRef) ?? normalizeRef(reading.reference),
271   imageHash: extra.imageHash ?? null,
272   categoryKey: suggestCategory(counterparty, kind),
273   confidence: Math.min(conf.amount, conf.date, conf.counterparty),
274   flags,
275   ownTransfer: dir.ownTransfer,
276 };
277 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:231

บรรทัด	ความหมาย
231–234	รับผลอ่าน 1 ใบ และ "ของเสริม" (extra) ที่ไม่บังคับ: QR ที่มีสื่อถอดได้, แฮชรูป, เวลาปัจจุบัน (ให้ทดสอบตั้งเวลาเองได้), สลิปชื่อ · = {} ค่าเริ่มต้นเป็นอีอบเจกต์ว่าง · คืน SlipCandidate
235–236	เวลาตอนนี้ (ไม่ส่งมาใช้เวลาจริง) และ "วันนี้" แบบไทย
237	เริ่มด้วยรายการรองว่าง จะเติมเข้าไปที่ละอง (const แต่ push ได้ เพราะ const ห้ามเปลี่ยน "ตัวแปร" ไม่ได้ห้ามแก้ "ข้างใน" อาร์เรย์)
238–242	ความมั่นใจ 3 ช่องผ่าน clamp01 · reading.confidence?.amount ไม่พังถ้าไม่มี confidence เลย
244	ยอดเงิน: แปลงเป็นสตางค์ด้วยฟังก์ชันเดียวกับฟอร์มของ FR-1 ("85.00" → 8500)
245	อ่านไม่ได้ หรือ ความมั่นใจ < 0.8 → ธง amount
247–250	วันที่: แปลงทั้งตัวหนังสือ (fromText) และวันที่ที่ AI แปลงเอง (fromIso) ด้วยตัวอ่านของเราเอง แล้ว เชื่อมตัวหนังสือก่อน (fromText ?? fromIso = ใช้ตัวแรกถ้าไม่ใช่ null)
251	มีทั้งคู่แต่ไม่ตรงกัน = ชัดกัน

บรรทัด	ความหมาย
252	เลขพຽ່ງนี้ไป = อนาคต (เทียบข้อความ "YYYY-MM-DD" ด้วย > ได้ เพราะเรียงแบบนี้แล้วตัวอักษรเรียงตรงกับวันเวลา)
253	ไม่มีวัน · มั่นใจต่ำ · ขัดกัน · อนาคต → รง date (4 เจื่อนไขต่อด้วย = "หรือ")
255–257	รายรับหรือรายจ่าย: ถามฟังก์ชัน decideDirection (ใช้ชื้อบนสลิป + สกิตชี้อ) ไม่มีสกิตชี้อใช้คำว่าง
258	ชื้ออีกฝ่ายตัดช่องว่าง ถ้าว่างเปล่าเป็น null (?.trim() null : ข้อความว่าง "" ถือเป็น "เท็จ" จึงได้ null)
259	ไม่มีชื้อ หรือมั่นใจต่ำ → รง counterparty
260	ชื้อขัดกับที่ AI บอก หรือไม่รู้ทิศทาง → รง direction
262–276	ประกอบผู้สมัคร:
265	วันที่เป็นอนาคต → ทั้ง (null) ไม่เชื่อเลย (ผู้เรียกจะใช้เวลาของรูปแทน)
266	เวลา "HH:MM" หรือ null
268–269	ธนาคารจาก QR ก่อน (??) ไม่มีจึงใช้ที่ AI อ่าน (ตัดช่องว่าง ว่างเป็น null)
270	เลขอ้างอิงจาก QR ก่อน ไม่มีจึงใช้ที่ AI อ่าน (ทั้งคู่ผ่าน normalizeRef)
271	แฮชรูป (ไม่มีเป็น null)
272	เดาหมวดจากชื้ออีกฝ่าย (null ได้หมวด "อื่น ๆ")
273	ความมั่นใจรวม = ต่ำสุดของ 3 ช่อง
274–275	รงทั้งหมด และ ownTransfer

★ ไล่ด้วยมือ: สลิปดี ฿85.00 "27 ก.ย. 69" (ผลจริง)

amountSatang = 8500 ความมั่นใจ 0.98 ≥ 0.8 → ไม่มีรง · fromText = 2026-09-27, fromIso = 2026-09-27
 ตรงกัน ไม่ใช่อนาคต ความมั่นใจ 0.95 → ไม่มีรง · ไม่มีสกิตชี้อ AI บอก expense → รายจ่ายให้ "ร้านค้าแฟดอยช้าง"
 ความมั่นใจ 0.9 → ไม่มีรง · หมวด food (มีคำว่า "ร้าน" และ "กาแฟ") · ความมั่นใจรวม min(0.98, 0.95, 0.9) = **0.9**
 · รง [] → ready

ส่วนที่ 3: slipNote – บันทึกของรายการ (บรรทัด 279–303)

src/domain/slip.ts

บรรทัด 279–303

```

279 const bahtText = (v: string | null) => {
280   const satang = parseBahtToSatang(v);
281   return satang === null ? (v ?? 'อ่านไม่ได้') : formatBaht(satang);
282 };
283
284 /**
285  * The note saved with a slip: which bank made it, and how it was checked.
286  * When the two AI reads disagree, both values are written down so the user can
287  * pick the right one (the review screen offers the amounts as buttons).
288  */

```



```

289 export function slipNote(c: Pick<SlipCandidate, 'bank' | 'ownTransfer'>, check?: SlipCheck |
    null): string | null {
290   const lines: string[] = [];
291   if (c.bank) lines.push(`สลิปจาก ${c.bank}`);
292   const d = check?.disagree ?? {};
293   const parts: string[] = [];
294   if (d.amount) parts.push(`${bahtText(d.amount[0])} หรือ ${bahtText(d.amount[1])}`);
295   if (d.date) parts.push(`วันที่ ${d.date[0]} ?? 'อ่านไม่ได้' หรือ ${d.date[1]} ?? 'อ่านไม่ได้'`);
296   if (d.direction) parts.push('เงินเข้าหรือออก');
297   if (d.counterparty) parts.push(`ชื่อ ${d.counterparty[0]} ?? 'อ่านไม่ได้' หรือ ${d.counterparty[1]} ??
    'อ่านไม่ได้'`);
298   if (parts.length) lines.push(`${ALT_PREFIX} ${parts.join(' · ')} `);
299   else if (check && check.reads < 2) lines.push('อ่านได้รอบเดียว ช่วยตรวจสอบสลิปอีกครั้ง');
300   else if (check && !check.verified) lines.push('AI 2 ตัวเห็นไม่ตรงกันว่าเป็นสลิปหรือไม่ ช่วยตรวจสอบรูปอีกครั้ง');
301   if (c.ownTransfer) lines.push('ดูเหมือนโอนระหว่างบัญชีของคุณเอง ถ้าใช่ ลบรายการนี้ได้ (ไม่ใช่รายรับหรือรายจ่าย)');
302   return lines.length ? lines.join('\n') : null;
303 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/slip.ts:279`

บรรทัด	ความหมาย
279–282	bahtText แปลงยอดที่ AI อ่าน ("1250.00") เป็นข้อความเงินสวย ๆ ("฿1,250.00") ถ้าแปลงไม่ได้แสดงตามเดิม หรือ "อ่านไม่ได้" ถ้าไม่มีค่า
289	รับแค่ 2 ช่องของผู้สมัคร (<code>Pick<SlipCandidate, 'bank' 'ownTransfer'></code>) = ชนิดที่หยิบมาเฉพาะช่องที่ใช้ ทำให้เกสต์ส่งข้อมูลน้อย ๆ ได้ และ <code>check</code> ซึ่งอาจไม่มี
290	เก็บบรรทัดของบันทึกในอาร์เรย์ ค่อยต่อกันตอนท้าย
291	บรรทัดแรก: "สลิปจาก KBank" (ถ้ารู้ธนาคาร)
292	ช่องที่ AI 2 ตัวอ่านไม่ตรงกัน (ไม่มีใช้อีจอกตัวว่าง)
294–297	เขียนทีละช่องที่ขัดกัน: ยอดเงิน X หรือ Y · วันที่ X หรือ Y · เงินเข้าหรือออก · ชื่อ X หรือ Y
298	มีช่องที่ขัดกัน → บรรทัด "AI อ่านได้ 2 แบบ: ..." คั่นแต่ละช่องด้วย " · "
299	ไม่มีช่องขัด แต่อ่านได้รอบเดียว → "อ่านได้รอบเดียว ช่วยตรวจสอบสลิปอีกครั้ง"
300	อ่าน 2 รอบแต่ไม่ <code>verified</code> (เช่นตัวหนึ่งว่าไม่ใช่สลิป) → "AI 2 ตัวเห็นไม่ตรงกันว่าเป็นสลิปหรือไม่..."
301	โอนระหว่างบัญชีตัวเอง → บรรทัดแนะนำว่าลบได้ (ไม่ใช่รายรับหรือรายจ่าย)
302	ต่อบรรทัดด้วยขึ้นบรรทัดใหม่ (<code>\n</code>) ไม่มีอะไรเลย → <code>null</code> (ไม่เก็บบันทึกว่าง)

ส่วนที่ 4: amountChoices — ถึงยอด 2 แบบกลับจากบันทึก (บรรทัด 305–315)

`src/domain/slip.ts`

บรรทัด 305–315

```

305 /** Start of the note line that lists the two readings. */
306 export const ALT_PREFIX = 'AI อ่านได้ 2 แบบ: ';
307
308 /** The amounts offered in a note written by slipNote, in satang (for the review screen's
    buttons). */
309 export function amountChoices(note: string | null | undefined): number[] {
310   const line = note?.split('\n').find((l) => l.startsWith(ALT_PREFIX));

```

```

311 const m = line?.match(/ยอดเงิน (.+?) หรือ (.+?)(?: \.|\$)/);
312 if (!m) return [];
313 const out = [m[1], m[2]].map((v) => parseBahtToSatang(v.replace(/^[^d.,]/g, ''))).filter((v): v
    is number => v !== null);
314 return [...new Set(out)];
315 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:305

บรรทัด	ความหมาย
306	ALT_PREFIX ข้อความขึ้นต้นบรรทัด เขียนไว้ที่เดียว ทั้งตัวเขียน (slipNote) และตัวอ่าน (amountChoices, หน้าตรวจสลิป) ใช้ค่าเดียวกัน จะได้ไม่สะกดต่างกัน
310	แยกบันทึกเป็นบรรทัด หาบบรรทัดที่ขึ้นต้นด้วย "AI อ่านได้ 2 แบบ:"
311	Regular expression /ยอดเงิน (.+?) หรือ (.+?)(?: \. \\$)/ : หา "ยอดเงิน" แล้วจับข้อความ (กลุ่มที่ 1) จนถึง " หรือ " แล้วจับอีกกลุ่ม (2) จนถึง "." หรือสุดบรรทัด . .+? = จับให้สั้นที่สุดเท่าทำได้ . (?: ...) = วงเล็บที่ไม่เก็บค่า
312	ไม่เจอ (ไม่มีบรรทัดหรือไม่มีส่วนยอดเงิน) → อาร์เรย์ว่าง (หน้าจอลังไม่แสดงชิป)
313	ทั้ง 2 ค่า: ลบทุกอย่างที่ไม่ใช่ตัวเลข จุด จุลภาค ("฿1,250.00" → "1,250.00") แปลงเป็นสตริง ตัดตัวที่แปลงไม่ได้ทิ้ง . (v): v is number => บอก TypeScript ว่าหลังกรองแล้วเหลือแต่ number
314	new Set ตัดค่าซ้ำ แล้วกระจายกลับเป็นอาร์เรย์ · ผลจริง: [125000, 128000]

ส่วนที่ 5: ผลการตัดสินใจและดัชนีกันซ้ำ (บรรทัด 317–342)

src/domain/slip.ts

บรรทัด 317–342

```

317 // -----
318 // Decision
319 // -----
320
321 export type SlipOutcome = 'ready' | 'needs_review' | 'duplicate' | 'out_of_range' | 'not_slip';
322
323 export interface DuplicateIndex {
324   refs: Set<string>;
325   hashes: Set<string>;
326   /** Amount + day + minute -> the payees seen with it ('' = none read). */
327   composite: Map<string, Set<string>>;
328 }
329
330 /** Titles the app gives a slip when no payee was read: they say nothing about who was paid. */
331 const NO_PAYEE_TITLES = new Set(['รายการจากสลิป', 'โอนระหว่างบัญชีตัวเอง']);
332
333 const payeeKey = (who: string | null | undefined) => {
334   const w = (who ?? '').toLowerCase().replace(/\s+/g, '');
335   return NO_PAYEE_TITLES.has(w) ? '' : w;
336 };
337
338 /** Amount + day + minute, the part of the duplicate check that must always match. */
339 export function compositeKey(p: { amountSatang: number | null; dayKey: string | null; time:
    string | null }): string | null {
340   if (!p.amountSatang || !p.dayKey || !p.time) return null;
341   return `${p.amountSatang}|${p.dayKey}|${p.time}`;

```

342 }

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:317

บรรทัด	ความหมาย
321	SlipOutcome ผลการตัดสินใจมี 5 แบบ: พร้อม / ต้องตรวจ / ชำ / นอกช่วง / ไม่ใช่สลิป
323–328	DuplicateIndex "สมุดจำ" ของรายการที่มีแล้ว: ชุดเลขอ้างอิง, ชุดแฮชรูป และตาราง "ยอด วัน นาทีก" → ชุดชื่อผู้รับที่เคยเห็น (<code>Set</code> = กลุ่มที่ไม่มีค่าซ้ำ ค้นว่ามีใหม่ได้เร็วมาก)
331	ชื่อที่แอปตั้งเองเมื่ออ่านชื่อผู้รับไม่ได้ ไม่ได้บอกว่าจ่ายให้ใคร
333–336	payeeKey ทำชื่อให้เทียบง่าย: ตัวพิมพ์เล็ก ลบช่องว่างทั้งหมด ถ้าเป็นชื่อที่แอปตั้งเอง → "" (= ไม่รู้ผู้รับ)
339–342	compositeKey ต่อ ยอด วัน เวลา เป็นข้อความเดียว เช่น 8500 2026-09-27 08:15 ขาดอย่างใดอย่างหนึ่ง (หรือยอดเป็น 0) → null (ไม่มีข้อมูลพอจะเทียบ)

ส่วนที่ 6: สร้างและเติมสมุดจำ (บรรทัด 344–367)

src/domain/slip.ts

บรรทัด 344–367

```

344 export function buildDuplicateIndex(existing: Transaction[]): DuplicateIndex {
345   const idx: DuplicateIndex = { refs: new Set(), hashes: new Set(), composite: new Map() };
346   for (const t of existing) addToIndex(idx, t);
347   return idx;
348 }
349
350 export function addToIndex(
351   idx: DuplicateIndex,
352   t: Pick<Transaction, 'slipRef' | 'slipImageHash' | 'amountSatang' | 'occurredAt' | 'title' |
    'source'>,
353 ) {
354   if (t.slipRef) idx.refs.add(t.slipRef);
355   if (t.slipImageHash) idx.hashes.add(t.slipImageHash);
356   if (t.source === 'slip') {
357     const key = compositeKey({
358       amountSatang: t.amountSatang,
359       dayKey: bkkDayKey(t.occurredAt),
360       time: new Date(new Date(t.occurredAt).getTime() + 7 * 3600e3).toISOString().slice(11, 16),
361     });
362     if (!key) return;
363     const who = idx.composite.get(key) ?? new Set<string>();
364     who.add(payeeKey(t.title));
365     idx.composite.set(key, who);
366   }
367 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:344

บรรทัด	ความหมาย
344–348	buildDuplicateIndex สร้างสมุดว่าง แล้วเติมทุกรายการที่มีอยู่ (ทำครั้งเดียวตอนเริ่มสแกน)
350–353	addToIndex เติมรายการเดียว (เรียกซ้ำหลังบันทึกสลิปใหม่แต่ละใบ เพื่อให้ใบถัดไปในรอบเดียวกันเห็นด้วย) รับเฉพาะช่องที่ใช้ (<code>Pick</code>)

บรรทัด	ความหมาย
354–355	มีเลขอ้างอิง/แอช → ใส่ลงชุด (รายการทุกแหล่ง)
356	ส่วนยอด วัน นาทื ใช้เฉพาะรายการที่มาจากสลิป (รายการที่จัดเองไม่มีเวลาแม่ระดับนาทืแบบสลิป)
357–361	ทำคีย์จากรายการที่บันทึกแล้ว: วันไทยจาก occurredAt และเวลาไทย = เวลา UTC + 7 ชั่วโมง (7 * 3600e3 มิลลิวินาที) แล้วตัดตัวอักษรที่ 11–15 ของ ISO ("2026-09-27T08:15:00.000Z")
362	ทำคีย์ไม่ได้ → จบ
363–365	เอาชุดชื่อของคีย์นี้ (ไม่มีสร้างใหม่) เพิ่มชื่อผู้รับ แล้วเก็บกลับ

ส่วนที่ 7: isDuplicate (บรรทัด 369–386)

src/domain/slip.ts

บรรทัด 369–386

```

369 /**
370  * The same slip seen again: same reference, same image, or (for slips without a reference) the
371  * same amount, day and minute with the same payee. A payee missing on either side still matches,
372  * since the saved title may be the app's own "รายการจากสลิป".
373  */
374 export function isDuplicate(c: SlipCandidate, idx: DuplicateIndex): boolean {
375   if (c.ref && idx.refs.has(c.ref)) return true;
376   if (c.imageHash && idx.hashes.has(c.imageHash)) return true;
377   if (!c.ref) {
378     const key = compositeKey(c);
379     const seen = key ? idx.composite.get(key) : undefined;
380     if (seen) {
381       const who = c.ownTransfer ? '' : payeeKey(c.counterparty);
382       if (!who || seen.has('') || seen.has(who)) return true;
383     }
384   }
385   return false;
386 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:369

บรรทัด	ความหมาย
375	มีเลขอ้างอิงและเคยเห็น → ชำ
376	มีแอชรูปและเคยเห็น → ชำ
377	เฉพาะสลิปที่ไม่มีเลขอ้างอิง จึงเทียบ ยอด วัน นาทื
378–379	ทำคีย์ แล้วดูว่าเคยเห็นคีย์นี้ไหม (ได้ชุดชื่อผู้รับ)
381	ชื่อผู้รับของสลิปใหม่ (โอนเข้าบัญชีตัวเองถือว่าไม่รู้ผู้รับ)
382	ชำเมื่อ: สลิปใหม่ไม่รู้ผู้รับ · หรือรายการเดิมไม่รู้ผู้รับ · หรือชื่อตรงกัน ("ไม่รู้" ผังใดผังหนึ่งถือว่าตรงไว้ก่อน เพราะกันชำสำคัญกว่า)
385	ผ่านทุกด่าน → ไม่ชำ

ส่วนที่ 8: classifyCandidate และ initialStatus (บรรทัด 388–405)

src/domain/slip.ts

บรรทัด 388–405

```
388 export function classifyCandidate(  
389   c: SlipCandidate,  
390   opts: { range: { from: string; to: string }; index: DuplicateIndex },  
391 ): Exclude<SlipOutcome, 'not_slip'> {  
392   if (isDuplicate(c, opts.index)) return 'duplicate';  
393   if (c.dayKey && !isDayInRange(c.dayKey, opts.range)) return 'out_of_range';  
394   return c.flags.length === 0 ? 'ready' : 'needs_review';  
395 }  
396  
397 /**  
398  * How a newly read slip is saved.  
399  * Team decision (27 ก.ย. 2569): when "auto-confirm" is on, a slip whose every key  
400  * field was read with at least 80% confidence counts in the balance right away  
401  * (the user can undo). Anything unsure stays a draft until the user checks it.  
402  */  
403 export function initialStatus(outcome: 'ready' | 'needs_review', autoConfirm: boolean): TxStatus  
404 {  
405   return outcome === 'ready' && autoConfirm ? 'confirmed' : 'draft';  
406 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slip.ts:388

บรรทัด	ความหมาย
388–391	รับผู้สมัคร ช่วงวันที่ยอมรับ และสมุดจำ · คืนผลได้ 4 แบบ (Exclude<..., 'not_slip'> = ทุกแบบยกเว้น not_slip เพราะตัดสินใจไปแล้ว)
392	ซ้ำตรวจสอบอย่างอื่น ซ้ำแล้วไม่ต้องสนใจเรื่องอื่น
393	มีวันที่และอยู่นอกช่วง (สแกนจริงใช้ 1 ปี) → out_of_range · ไม่มีวันที่ไม่นับว่านอกช่วง (จะติดตรง date อยู่แล้ว)
394	ไม่มีธง → ready มีธง → needs_review (? : = if-else แบบสั้น)
397–402	คอมเมนต์บันทึก การตัดสินใจของทีม (27 ก.ย. 2569): เปิด "รวมยอดทันที" แล้วสลิปที่ชัดทุกช่อง ≥ 80% นับเข้ายอดเลย (ย้อนได้) ที่ไม่ชัดยังเป็นแบบร่าง
403–404	initialStatus: ready และ autoConfirm → confirmed นอกนั้น draft · บรรทัดเดียวนี้คือ "สวิตช์" ระหว่างแอปที่จัดให้อัตโนมัติ กับแอปที่ให้ยืนยันทุกใบ

i ทำไมเขียนการตัดสินใจของทีมไว้ในคอมเมนต์

คนที่มาอ่านโค้ดทีหลัง (รวมถึงอาจารย์) อาจสงสัยว่า "ทำไมไม่ให้ผู้ใช้ยืนยันทุกใบ ปลอดภัยกว่า" คอมเมนต์ตอบไว้ตรงจุดเลยว่าเป็นการตัดสินใจโดยตั้งใจ เมื่อไร และมีทางย้อน (ปุ่ม "ยังไม่รวมยอด" และสวิตช์ในตั้งค่า) นี่คือนิสัยที่ดี: คอมเมนต์อธิบาย "ทำไม" ส่วนโค้ดบอก "ทำอะไร"

อ่านโค้ด: slipNames.ts และ myNames.ts — ใครคือ "ฉัน"

กฎ (119 บรรทัด) กับที่เก็บข้อมูลบนมือถือ (42 บรรทัด) แยกไฟล์กันตามหลักของทั้งระบบ

ส่วนที่ 1: หัวไฟล์ ชนิดข้อมูล และค่าคงที่ (บรรทัด 1–28)

src/domain/slipNames.ts

บรรทัด 1–28

```

1 /**
2  * Who is "me" on a slip. A slip only shows a payer and a receiver; whether it
3  * is the user's expense or income depends on which of the two is the user. A
4  * slip a friend sent over LINE ("I paid you") looks exactly like the user's own
5  * payment slip, so the AI alone would call it an expense.
6  *
7  * The phone learns the user's own name(s) from the payer line of the slips in
8  * the gallery (most saved slips are the user's own payments; Thai, English and
9  * masked spellings are learned separately). Only short fingerprints of the
10 * names are kept, on this phone. Unit tests: __tests__/slipNames.test.ts.
11 */
12 import type { TxKind } from '../types';
13
14 export interface NameStats {
15   /** Fingerprint of a payer name -> how many slips showed it as the payer. */
16   counts: Record<string, number>;
17   /** Slips whose payer was counted. */
18   total: number;
19 }
20
21 export const EMPTY_NAME_STATS: NameStats = { counts: {}, total: 0 };
22
23 /** A name seen as payer at least this often, and on this share of slips, is the user. */
24 export const MINE_MIN_SLIPS = 2;
25 export const MINE_MIN_SHARE = 0.2;
26 const KEEP_NAMES = 30;
27
28 const TITLES = /^(?:นางสาว|นาง|นาย|น\..ส\..|ด\..ช\..|ด\..ญ\..|คุณ|mrs\..?|mr\..?|ms\..?|miss)\s*/i;

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/slipNames.ts:1

บรรทัด	ความหมาย
1–5	ปัญหา: สลิปบอกแค่ผู้โอนกับผู้รับ จะเป็นรายจ่ายหรือรายรับขึ้นกับว่า "ฉัน" อยู่ฝั่งไหน สลิปที่เพื่อนส่งมาทาง LINE หน้าตาเหมือนสลิปที่เราจ่ายเองทุกอย่าง AI อย่างเดียวจะตอบว่ารายจ่าย
7–10	วิธีแก้: เรียนชื่อของผู้ใช้จากบรรทัดผู้โอนของสลิปในเครื่อง (ไทย อังกฤษ และแบบปิดบาง นับแยกกัน) เก็บแค่ลายนิ้วมือสั้น ๆ บนมือถือ มีทดสอบใน slipNames.test.ts
14–19	NameStats: <code>counts</code> ลายนิ้วมือ → จำนวนสลิปที่ชื่อนี้เป็นผู้โอน และ <code>total</code> จำนวนสลิปที่นับทั้งหมด
21	สถิติว่าง (ยังไม่เคยเรียน)
23–25	เกณฑ์ "เป็นฉัน": อย่างน้อย 2 ใบ และ 20% ของสลิปที่นับ

บรรทัด	ความหมาย
26	จำไม่เกิน 30 ชื่อ (ไม่มี <code>export</code> = ใช้ภายในไฟล์นี้เท่านั้น)
28	TITLES คำนำหน้าที่ตัดทิ้ง: นางสาว นาง นาย น.ส. ด.ช. ด.ญ. คุณ Mrs Mr Ms Miss (<code>^</code> = ตัดข้อความ, <code>\.?</code> = มีจุดหรือไม่มีก็ได้, <code>ง</code> <code>i</code> = ไม่สนใจตัวพิมพ์เล็ก/ใหญ่) · ต้องเรียง "นางสาว" ก่อน "นาง" ไม่ขึ้น "นางสาว" จะถูกตัดแค่ "นาง" เหลือ "สาว..."

ส่วนที่ 2: ลายนิ้วมือและคีย์ของชื่อ (บรรทัด 30–58)

src/domain/slipNames.ts

บรรทัด 30–58

```

30 /** 32-bit FNV-1a, so names themselves are never stored. */
31 function fingerprint(s: string): string {
32   let h = 0x811c9dc5;
33   for (let i = 0; i < s.length; i++) {
34     h ^= s.charCodeAt(i);
35     h = Math.imul(h, 0x01000193) >>> 0;
36   }
37   return h.toString(16).padStart(8, '0');
38 }
39
40 /**
41  * First name plus the first letter of the last name, without title or masking:
42  * "นาย สมชาย ใจดี", "สมชาย ใจดี" and "น.ส. สมชาย ใจดี" give the same key;
43  * "MR. SOMCHAI JAIDEE" and "SOMCHAI J" give another. Null when there is no name.
44  */
45 export function nameKey(name: string | null | undefined): string | null {
46   const words = (name ?? '')
47     .trim()
48     .replace(TITLES, '')
49     .toLowerCase()
50     .replace(/[*\._]+|x{2,}/g, ' ')
51     .replace(/\.\/g, ' ')
52     .split(/\s+/)
53     .filter(Boolean);
54   if (!words.length) return null;
55   const first = words[0];
56   if (first.length < 2) return null;
57   return fingerprint(`${first}|${words[1]?.[0] ?? ''}`);
58 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slipNames.ts:30

บรรทัด	ความหมาย
31	fingerprint แฮชแบบ FNV-1a 32 บิต (สูตรแฮชที่ง่ายและเร็ว)
32	เริ่มที่ค่าคงที่ประจำสูตร <code>0x811c9dc5</code>
33–36	ทุกตัวอักษร: XOR รหัสตัวอักษรเข้าไป แล้วคูณด้วยจำนวนเฉพาะประจำสูตร <code>0x01000193</code> · <code>Math.imul</code> = คูณแบบจำนวนเต็ม 32 บิต · <code>>>> 0</code> = ทำให้เป็นเลขไม่ติดลบ
37	เป็นเลขฐาน 16 ยาว 8 ตัว เช่น "90fbda2f"
45	nameKey ทำชื่อให้เป็น "ชื่อต้น + อักษรแรกของนามสกุล" แล้วแฮช

บรรทัด	ความหมาย
46–47	ไม่มีชื่อใช้ "" แล้วตัดช่องว่างหัวท้าย
48	ตัดคำนำหน้า
49	ตัวพิมพ์เล็ก
50	เครื่องหมายปิดบัง (* • • _) และ "xx" ขึ้นไป (ธนาคารบางแห่งปิดด้วย x) → ช่องว่าง
51	จุด → ช่องว่าง ("MR." ตัดไปแล้ว แต่ชื่อย่ออย่าง "S." ยังมีได้)
52–53	แยกเป็นคำด้วยช่องว่าง และทิ้งคำว่าง (<code>filter(Boolean)</code>)
54	ไม่เหลือคำ → null
55–56	ชื่อต้นต้องยาวอย่างน้อย 2 ตัว (กันชื่อที่ถูกปิดจนเหลือตัวเดียว)
57	แฮช "ชื่อต้น อักษรแรกของคำที่ 2" (ไม่มีคำที่ 2 ใช้ "") · "นาย สมชาย ใจดี" → "สมชาย ใ" → 90fbda2f

ส่วนที่ 3: นับและตัดสิน (บรรทัด 60–79)

src/domain/slipNames.ts บรรทัด 60–79

```

60 /** Count one slip's payer (only slips that look like the user's own payments are counted). */
61 export function addPayer(stats: NameStats, payer: string | null | undefined): NameStats {
62   const key = nameKey(payer);
63   if (!key) return stats;
64   const counts = { ...stats.counts, [key]: (stats.counts[key] ?? 0) + 1 };
65   // Keep the most frequent names, but never drop the one just counted: otherwise, once the table
66   // is full, a new name (the user's own on a new phone) would be cut straight away every time.
67   const others = Object.entries(counts)
68     .filter(([k]) => k !== key)
69     .sort((a, b) => b[1] - a[1])
70     .slice(0, KEEP_NAMES - 1);
71   return { counts: Object.fromEntries([[key, counts[key]], ...others]), total: stats.total + 1 };
72 }
73
74 export function isMine(stats: NameStats, name: string | null | undefined): boolean {
75   const key = nameKey(name);
76   if (!key) return false;
77   const n = stats.counts[key] ?? 0;
78   return n >= MINE_MIN_SLIPS && n / Math.max(stats.total, 1) >= MINE_MIN_SHARE;
79 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/slipNames.ts:60

บรรทัด	ความหมาย
61–63	<code>addPayer</code> นับผู้โอน 1 ใบ ทำคีย์ไม่ได้ → ค้นสถิติเดิม
64	สร้างอ็อบเจกต์ ใหม่ (ไม่แก๊งของเดิม) โดยคัดลอก counts เดิม (...) แล้วบวก 1 ให้คีย์นี้ (<code>[key]:</code> = ใช้ค่าของตัวแปรเป็นชื่อคีย์)
65–66	คอมเมนต์: เก็บชื่อที่พบบ่อยที่สุด แต่ ห้ามทิ้งชื่อที่เพิ่งนับ ไม่จันเมื่อตารางเต็ม ชื่อใหม่ (เช่นชื่อเราบนมือถือเครื่องใหม่) จะถูกตัดทิ้งทุกครั้ง ไม่มีวันได้นับถึง 2

บรรทัด	ความหมาย
67-70	ชื่ออื่น ๆ: เอาคีย์นี้ออก เรียงจากมากไปน้อย เก็บ 29 อันดับแรก
71	รวมคีย์นี้ + 29 ชื่อ = 30 และ total + 1
74-79	isMine: ทำคีย์ หาจำนวน ต้อง ≥ 2 และ สัดส่วน $\geq 0.2 \cdot \text{Math.max}(\text{stats.total}, 1)$ กันหารด้วยศูนย์

ส่วนที่ 4: decideDirection (บรรทัด 81-119)

src/domain/slipNames.ts

บรรทัด 81-119

```

81 export interface Direction {
82   kind: TxKind;
83   /** The other party: the receiver of an expense, the sender of income. */
84   counterparty: string | null;
85   /** Both sides are the user: money moved between the user's own accounts. */
86   ownTransfer: boolean;
87   /** The names contradict the AI's reading: the user should check income/expense. */
88   unsure: boolean;
89 }
90
91 /**
92  * Income or expense, from the names on the slip when the user's name is known,
93  * otherwise from the AI's reading ("ได้รับเงิน" screens are income).
94  */
95 export function decideDirection(
96   reading: { direction: 'expense' | 'income' | 'unknown'; fromName?: string | null; toName?:
97     string | null; counterparty: string | null },
98   stats: NameStats,
99 ): Direction {
100   const fromMine = isMine(stats, reading.fromName);
101   const toMine = isMine(stats, reading.toName);
102   const aiKind: TxKind = reading.direction === 'income' ? 'income' : 'expense';
103   if (fromMine && toMine) {
104     return { kind: aiKind, counterparty: reading.toName ?? reading.counterparty, ownTransfer:
105       true, unsure: true };
106   }
107   if (toMine) {
108     // The user received this money, e.g. a slip a friend sent: income, from the payer.
109     return { kind: 'income', counterparty: reading.fromName ?? reading.counterparty, ownTransfer:
110       false, unsure: false };
111   }
112   if (fromMine) {
113     // The user paid. A "money received" reading contradicts that, so ask.
114     return {
115       kind: 'expense',
116       counterparty: reading.toName ?? reading.counterparty,
117       ownTransfer: false,
118       unsure: reading.direction === 'income',
119     };
120   }
121   return { kind: aiKind, counterparty: reading.counterparty, ownTransfer: false, unsure:
122     reading.direction === 'unknown' };
123 }

```

▶ เปิดใน VS Code (లేదు) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/slipNames.ts:81

บรรทัด	ความหมาย
81–89	Direction ผลการตัดสินใจ: ประเภท, อีกฝ่าย, โอนเข้าบัญชีตัวเองไหม, ไม่แน่ใจไหม
95–98	รับผลอ่าน (เฉพาะ 4 ช่องที่ใช้) และสถิติชื่อ
99–100	ผู้โอนเป็นฉันไหม ผู้รับเป็นฉันไหม
101	สิ่งที่ AI บอก: income ก็ income บอคนั้น (expense และ unknown) ถือเป็น expense ไว้ก่อน
102–104	ทั้ง 2 ผังเป็นฉัน → โอนระหว่างบัญชีตัวเอง ไม่แน่ใจ (ผู้ใช้ควรลบ เพราะเงินไม่ได้เข้าหรือออกจริง)
105–108	ผู้รับเป็นฉัน → รายรับ จากผู้โอน แน่ใจ (เช่น สลิปที่เพื่อนส่งมา)
109–117	ผู้โอนเป็นฉัน → รายจ่ายให้ผู้รับ · ถ้า AI บอกว่า "เงินเข้า" = ชัดเจน → ไม่แน่ใจ
118	ไม่รู้ว่าเป็นใคร → เชื่อ AI ใช้ counterparty ที่ AI ให้มา ไม่แน่ใจเฉพาะเมื่อ AI บอก unknown

ส่วนที่ 5: myNames.ts — เก็บสถิติบนมือถือ (บรรทัด 1–42)

src/services/myNames.ts

บรรทัด 1–42

```

1  /**
2   * The user's own name(s) as printed on slips, learned on this phone per
3   * account (see domain/slipNames.ts). Only fingerprints are stored.
4   */
5  import { Storage } from '../data/storage';
6  import { addPayer, EMPTY_NAME_STATS, type NameStats } from '../domain/slipNames';
7
8  const key = (userId: string) => `mindpay.slipNames.${userId}`;
9  const cache = new Map<string, NameStats>();
10
11 export async function loadNames(userId: string): Promise<NameStats> {
12   const hit = cache.get(userId);
13   if (hit) return hit;
14   let stats = EMPTY_NAME_STATS;
15   try {
16     const raw = await Storage.getItem(key(userId));
17     const parsed = raw ? (JSON.parse(raw) as Partial<NameStats>) : null;
18     if (parsed && typeof parsed.total === 'number' && parsed.counts && typeof parsed.counts ===
19 'object') {
20       stats = { counts: parsed.counts, total: parsed.total };
21     }
22   } catch {
23     // Start learning again.
24   }
25   cache.set(userId, stats);
26   return stats;
27 }
28 /** Count the payer of a slip that looks like the user's own payment. */
29 export async function learnPayer(userId: string, payer: string | null | undefined) {
30   const next = addPayer(await loadNames(userId), payer);
31   cache.set(userId, next);
32   try {
33     await Storage.setItem(key(userId), JSON.stringify(next));
34   } catch {
35     // Not critical: the name is learned again from the next slips.

```

```

36   }
37 }
38
39 export async function forgetNames(userId: string) {
40   cache.delete(userId);
41   await Storage.removeItem(key(userId)).catch(() => {});
42 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/myNames.ts:1

บรรทัด	ความหมาย
1–3	ชื่อของผู้ใช้ที่เรียนบนมือถือ แยกตามบัญชี เก็บแค่ลายนิ้วมือ
8	คีย์ที่เก็บ: mindpay.slipNames.<userId>
9	cache เก็บสถิติไว้ในหน่วยความจำ อ่านจากที่เก็บครั้งเดียว ครั้งต่อไปใช้ที่จำไว้ (เร็วกว่า เพราะสแกน 1 รอบเรียกหลายสิบครั้ง)
11–13	loadNames มีใน cache → คืบเลย
14–20	อ่านจาก Storage แปลง JSON และ ตรวจสอบรูปแบบ ก่อนเชื่อ (total เป็นตัวเลข counts เป็นอ็อบเจกต์) ข้อมูลเสียจะไม่ทำให้แอปพัง
21–23	อ่านพัง → เริ่มเรียนใหม่ (ไม่ใช่เรื่องร้ายแรง)
24–25	จำลง cache แล้วคืบ
29–37	learnPayer นับผู้โอนเพิ่ม (ใช้ addPayer ที่เป็นฟังก์ชันล่วน) อัปเดต cache แล้วบันทึก บันทึกไม่สำเร็จก็ไม่เป็นไร
39–42	forgetNames ลบทั้ง cache และที่เก็บ

★ แบ่งไฟล์แบบนี้ได้อะไร

slipNames.ts (กฎ) ไม่รู้ว่าข้อมูลเก็บที่ไหน ส่วน myNames.ts (บริการ) ไม่รู้ว่ากฎคืออะไร แค่เรียก addPayer เทสต์ของกฎจึงไม่ต้องจำลองที่เก็บข้อมูลเลย และถ้าวันหนึ่งย้ายที่เก็บไปฐานข้อมูล ก็แค่แก้ myNames.ts

อ่านโค้ด: dates.ts ส่วนอ่านวันที่บนสลิป

src/domain/dates.ts บรรทัด 159–261 (ส่วนบนของไฟล์อ่านไปแล้วในเล่ม FR-2)

ส่วนที่ 1: ชื่อเดือนแบบไม่มีจุด (บรรทัด 159–185)

src/domain/dates.ts

บรรทัด 159–185

```

159 // -----
160 // Parsing dates printed on bank slips
161 // -----
162
163 /**
164  * Month tokens with dots and spaces removed. OCR engines often drop or move the
165  * dots in "ก.ย." (this is exactly what broke September slips in the old
166  * prototype), so we compare letters only.
167  */
168 const MONTH_TOKENS: [string, number][] = [
169   ...THAI_MONTHS_SHORT.map((m, i) => [m.replace(/\./g, ''), i + 1] as [string, number]),
170   ...THAI_MONTHS_LONG.map((m, i) => [m, i + 1] as [string, number]),
171   ...['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec'].map(
172     (m, i) => [m, i + 1] as [string, number],
173   ),
174 ];
175
176 function monthFromToken(token: string): number | null {
177   const t = token.toLowerCase().replace(/[\.\s]/g, '');
178   if (!t) return null;
179   // Longest match first so "มีค" is not read as "มค".
180   const sorted = [...MONTH_TOKENS].sort((a, b) => b[0].length - a[0].length);
181   for (const [name, month] of sorted) {
182     if (t === name || (name.length >= 3 && t.startsWith(name))) return month;
183   }
184   return null;
185 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:159

บรรทัด	ความหมาย
163–167	คอมเมนต์อธิบายเพิ่มเติม: OCR ชอบทำจุดใน "ก.ย." หายหรือย้าย (นี่แหละที่ทำให้สลิปเดือนกันยายนพังใน prototype) จึงเทียบเฉพาะตัวอักษร
168–174	MONTH_TOKENS คู่ [คำ, เลขเดือน] 36 คู่: ชื่อย่อไทยตัดจุด ("กย" → 9), ชื่อเต็มไทย ("กันยายน" → 9), ชื่อย่ออังกฤษ ("sep" → 9) · as [string, number] บอก TypeScript ว่าเป็นคู่ 2 ช่อง
176	monthFromToken รับคำ คืนเลขเดือนหรือ null
177	ตัวพิมพ์เล็ก ลบจุดและช่องว่าง: "ก.ย." → "กย" · "Sep." → "sep"
179–180	เรียงคำจาก ยาวไปสั้น ก่อนค้น เพื่อให้ "มีค" (มี.ค.) ไม่ถูกจับเป็น "มค" (ม.ค.) ผิดเดือน
181–183	ตรงกันพอดี หรือ (คำยาว ≥ 3 ตัว และขึ้นต้นเหมือนกัน เช่น "september" ขึ้นต้นด้วย "sep") → เดือนนั้น

ส่วนที่ 2: normalizeYear และ validDay (บรรทัด 187–214)

src/domain/dates.ts

บรรทัด 187–214

```
187 /**
188  * Turn a year as printed on a slip into a Gregorian year.
189  * Thai slips usually print Buddhist Era years, often shortened: "69" = 2569 = 2026.
190  */
191 export function normalizeYear(raw: number, now: Date = new Date()): number | null {
192   const currentCe = Number(bkkDayKey(now).slice(0, 4));
193   let year: number;
194   if (raw >= 2400) year = raw - 543; // full BE: 2569
195   else if (raw >= 1900) year = raw; // full CE: 2026
196   else if (raw >= 0 && raw < 100) {
197     const asBe = 2500 + raw - 543; // "69" -> 2026
198     const asCe = 2000 + raw; // "26" -> 2026
199     // Prefer the reading that is not in the future and closest to today.
200     const candidates = [asBe, asCe].filter((y) => y <= currentCe + 1);
201     if (candidates.length === 0) return null;
202     year = candidates.reduce((best, y) =>
203       Math.abs(y - currentCe) < Math.abs(best - currentCe) ? y : best,
204     );
205   } else return null;
206   return year;
207 }
208
209 function validDay(y: number, m: number, d: number): string | null {
210   if (m < 1 || m > 12 || d < 1 || d > 31) return null;
211   const date = new Date(Date.UTC(y, m - 1, d));
212   if (date.getUTCMonth() !== m - 1) return null; // e.g. 31 ก.พ.
213   return date.toISOString().slice(0, 10);
214 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:187

บรรทัด	ความหมาย
192	ปี ค.ศ. ปัจจุบันตามเวลาไทย
194	≥ 2400 = พ.ศ. เต็ม → ลบ 543
195	≥ 1900 = ค.ศ. เต็ม ใช้ตามนั้น
196–198	ปี 2 หลัก: ลองเป็น พ.ศ. ($2500 + \text{ปี} - 543$) และ ค.ศ. ($2000 + \text{ปี}$)
200	ตัดตัวที่เกินปีหน้า (อนาคตไกลเกินไปไม่ได้)
201	ไม่เหลือเลย → null
202–204	<code>reduce</code> เลือกตัวที่ ห่างจากปัจจุบันน้อยที่สุด (<code>Math.abs</code> = ค่าสัมบูรณ์ ไม่สนบวกลบ)
205	ตัวเลขแปลก ๆ (เช่น 150) → null
209–214	<code>validDay</code> : เดือน 1–12 วัน 1–31 แล้วสร้างวันจริงด้วย <code>Date.UTC</code> ถ้า JavaScript "ปิดไปเดือนถัดไป" (31 ก.พ. กลายเป็น 3 มี.ค.) แปลว่าวันนั้นไม่มีจริง → null · คืน "YYYY-MM-DD"

ส่วนที่ 3: parseSlipDate และ parseSlipTime (บรรทัด 216–261)

src/domain/dates.ts

บรรทัด 216–261

```
216 /**
217  * Parse a date as printed on a Thai bank slip. Returns "YYYY-MM-DD" or null.
218  * Handles: "27 ก.ย. 69", "27 ก.ย. 2569", "27 กันยายน 2569", "27/09/2569",
219  * "27-09-69", "2026-09-27", "27 Sep 2026", and OCR noise like "27ก.ย.69" or "27 กย 69".
220  */
221 export function parseSlipDate(text: string | null | undefined, now: Date = new Date()): string |
  null {
222   if (!text) return null;
223   const s = text.trim().replace(/\s+/g, ' ');
224
225   // ISO: 2026-09-27
226   let m = s.match(/(\d{4})-(\d{1,2})-(\d{1,2})/);
227   if (m) {
228     const y = normalizeYear(Number(m[1]), now);
229     const day = y ? validDay(y, Number(m[2]), Number(m[3])) : null;
230     if (day) return day;
231   }
232
233   // Day + month name + year: 27 ก.ย. 69 / 27ก.ย.2569 / 27 Sep 2026. Tried before the numeric
  form,
234   // because a time printed with dots ("14.30.25", "12.10.25") looks like a numeric date.
235   m = s.match(/(\d{1,2})\s*([ก-ฃา-ซา-ซ][ก-ฃา-ซา-ซ.\s]{0,12}?)\s*(\d{2,4})(?!\d)/);
236   if (m) {
237     const month = monthFromToken(m[2]);
238     const y = normalizeYear(Number(m[3]), now);
239     const day = month && y ? validDay(y, month, Number(m[1])) : null;
240     if (day) return day;
241   }
242
243   // Numeric: 27/09/2569, 27-09-69, 27.09.69
244   for (const n of s.matchAll(/(\d{1,2})[/\.-](\d{1,2})[/\.-](\d{2,4})(?!\d)/g)) {
245     const y = normalizeYear(Number(n[3]), now);
246     const day = y ? validDay(y, Number(n[2]), Number(n[1])) : null;
247     if (day) return day;
248   }
249   return null;
250 }
251
252 /** Extract "HH:MM" from slip text like "08:15 u." */
253 export function parseSlipTime(text: string | null | undefined): string | null {
254   if (!text) return null;
255   const m = text.match(/(\d{1,2}):(\d{2})/);
256   if (!m) return null;
257   const h = Number(m[1]);
258   const min = Number(m[2]);
259   if (h > 23 || min > 59) return null;
260   return `${String(h).padStart(2, '0')}:${String(min).padStart(2, '0')}`;
261 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:216

บรรทัด ความหมาย

222–223 ไม่มีข้อความ → null · ตัดช่องว่างหัวท้าย และยุบช่องว่างหลายตัวเหลือตัวเดียว

บรรทัด	ความหมาย
225–231	แบบที่ 1 ISO: หา "ปี4หลัก-เดือน-วัน" ถ้าได้วันจริงคืนเลย
233–234	คอมเมนต์: ลองแบบชื่อเดือน ก่อน แบบตัวเลข เพราะเวลาที่พิมพ์ด้วยจุด เช่น "14.30.25" หน้าตาเหมือนวันที่ตัวเลข
235	แบบที่ 2 วัน + ชื่อเดือน + ปี: <code>(\d{1,2})</code> วัน 1–2 หลัก · <code>\s*</code> ช่องว่างหรือไม่มี · <code>([ก-๙a-zA-Z][ก-๙a-zA-Z.\s]{0,12}?)</code> ชื่อเดือน: เริ่มด้วยอักษรไทยหรืออังกฤษ ตามด้วยอักษร จุด หรือช่องว่างไม่เกิน 12 ตัว · <code>(\d{2,4})</code> ปี 2–4 หลัก · <code>(?!\\d)</code> ตามด้วยตัวเลขไม่ได้ (กันตัดปีกลางตัวเลขยาว)
236–241	แปลงชื่อเดือน แปลงปี ตรวจสอบวันจริง ได้ → คืน
243–248	แบบที่ 3 ตัวเลข วัน/เดือน/ปี คั่นด้วย / - หรือ . · <code>matchAll</code> ลองทุกจุดที่เข้ารูปแบบ (เพื่อจุดแรกเป็นเวลา) จุดแรกที่ได้วันจริงชนะ
249	ไม่เข้าแบบไหนเลย → null (ไม่เอา)
253–261	parseSlipTime: หา "ชั่วโมง:นาที" หรือ "ชั่วโมง.นาที" ชั่วโมงเกิน 23 หรือนาทีเกิน 59 → null · เติม 0 หน้าชั่วโมงหลักเดียว ("9.05" → "09:05")

★ ไล่ด้วยมือ: "27ก.ย.2569"

แบบที่ 1: ไม่มี "ปี4หลัก-" → ข้าม · แบบที่ 2: วัน = "27" · ชื่อเดือน = "ก.ย." (หยุดก่อนตัวเลข) · ปี = "2569" · `monthFromToken("ก.ย.")` → "กย" → 9 · `normalizeYear(2569)` → 2026 · `validDay(2026, 9, 27)` → **"2026-09-27"** ✓

อ่านโค้ด: scanQueue.ts — state machine ของการสแกน

173 บรรทัด ฟังก์ชันล้วนทั้งไฟล์ ทดสอบด้วย TC-26 ถึง TC-30

ส่วนที่ 1: คอมเมนต์และชนิดข้อมูล (บรรทัด 1–48)

src/domain/scanQueue.ts

บรรทัด 1–48

```

1  /**
2   * FR-4 scanner state machine (pure, testable).
3   *
4   * The old prototype had a bug: changing the time range while a scan was running
5   * could keep using the old range or skip images. This reducer prevents that by
6   * design:
7   *   - the range can only change when no scan is running (idle / done);
8   *   - every queue gets a new `runId`; results that arrive for an older run are ignored;
9   *   - results are matched by asset id, never by array position.
10  *
11  * Images are processed newest first, so progress naturally passes the
12  * 1 day -> 7 days -> 1 month -> 6 months -> 1 year milestones in that order.
13  */
14  import { RANGE_ORDER, rangeStartMs } from './dates';
15  import type { RangeKey } from './types';
16
17  export type ScanPhase = 'idle' | 'running' | 'paused' | 'done';
18
19  export type ItemStatus =
20    | 'queued'
21    | 'working'
22    | 'ready'
23    | 'needs_review'
24    | 'duplicate'
25    | 'out_of_range'
26    | 'not_slip'
27    | 'failed';
28
29  export interface ScanItem {
30    assetId: string;
31    /** Creation time of the photo (ms). */
32    createdAt: number;
33    status: ItemStatus;
34    /** Id of the draft transaction created from this slip, if any. */
35    txId?: string;
36    amountSatang?: number | null;
37    label?: string | null;
38    message?: string;
39    /** true when the slip was counted in the balance straight away (auto-confirm). */
40    confirmed?: boolean;
41  }
42
43  export interface ScanState {
44    phase: ScanPhase;
45    range: RangeKey;

```



```

46   runId: number;
47   items: ScanItem[];
48 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/scanQueue.ts:1`

บรรทัด	ความหมาย
4–9	บ๊ิกของ prototype: เปลี่ยนช่วงระหว่างสแกนแล้วใช้ช่วงเก่าหรือข้ามรูป reducer นี้กัน "โดยการออกแบบ" 3 ข้อ: เปลี่ยนช่วงได้เฉพาะตอนไม่สแกน · ทุกคิวมี runId ใหม่ ผลของรอบเก่าถูกทิ้ง · จับคู่ด้วย asset id ไม่ใช่ตำแหน่ง
11–12	อ่านรูปใหม่ไปเท่า ความคืบหน้าจึงผ่านหลัก 1 วัน → 7 วัน → 1 เดือน → 6 เดือน → 1 ปี ตามลำดับเอง
17	ScanPhase 4 สถานะของทั้งคิว
19–27	ItemStatus 8 สถานะของรูปแต่ละรูป: รอคิว, กำลังทำ, พร้อม, ต้องตรวจ, ช้า, นอกช่วง, ไม่ใช่สลิป, ล้มเหลว
29–41	ScanItem รูป 1 รูปในคิว: รหัสรูป, เวลาที่ถ่าย (มิลิวินาที), สถานะ และผลที่ได้ (รหัสรายการที่สร้าง, ยอด, ชื่อ, ข้อความ, บัปในยอดแล้วไหม)
43–48	ScanState สถานะทั้งหมด: phase, ช่วงเวลา, เลขรอบ, รายการรูป

ส่วนที่ 2: คำสั่ง (actions) และสถานะเริ่มต้น (บรรทัด 50–77)

`src/domain/scanQueue.ts`

บรรทัด 50–77

```

50 export type ScanAction =
51   | { type: 'setRange'; range: RangeKey }
52   | { type: 'load'; items: { assetId: string; createdAt: number }[] }
53   | { type: 'start' }
54   | { type: 'pause' }
55   | { type: 'resume' }
56   | { type: 'itemStarted'; runId: number; assetId: string }
57   | {
58     type: 'itemFinished';
59     runId: number;
60     assetId: string;
61     status: Exclude<ItemStatus, 'queued' | 'working'>;
62     txId?: string;
63     amountSatang?: number | null;
64     label?: string | null;
65     message?: string;
66     confirmed?: boolean;
67   }
68   | { type: 'reset' };
69
70 export const initialScanState = (range: RangeKey = '7d'): ScanState => ({
71   phase: 'idle',
72   range,
73   runId: 0,
74   items: [],
75 });
76
77 const canConfigure = (s: ScanState) => s.phase === 'idle' || s.phase === 'done';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/scanQueue.ts:50`

บรรทัด	ความหมาย
50–68	ScanAction คำสั่ง 8 แบบ แต่ละแบบมี <code>type</code> ต่างกัน (เรียกว่า discriminated union: TypeScript รู้ที่ type แล้วรู้ว่า คำสั่งนั้นมีช่องอะไรบ้าง)
51–52	<code>setRange</code> (เปลี่ยนช่วง) · <code>load</code> (ใส่รูปเข้าคิว)
53–55	<code>start</code> · <code>pause</code> · <code>resume</code>
56	<code>itemStarted</code> : เริ่มทำรูปนี้ (ต้องบอกเลขรอบและรหัสรูป)
57–67	<code>itemFinished</code> : รูปนี้เสร็จ พร้อมผล · สถานะผลห้ามเป็น <code>queued/working</code> (<code>Exclude</code>)
68	<code>reset</code> : ล้างทั้งหมด
70–75	initialScanState สถานะเริ่มต้น: <code>idle</code> , ช่วงที่กำหนด (ค่าเริ่ม 7 วัน), รอบ 0, คิวว่าง
77	canConfigure ตั้งค่าได้ไหม = <code>idle</code> หรือ <code>done</code> เท่านั้น

ส่วนที่ 3: scanReducer (บรรทัด 79–142)

src/domain/scanQueue.ts

บรรทัด 79–142

```

79 export function scanReducer(state: ScanState, action: ScanAction): ScanState {
80   switch (action.type) {
81     case 'setRange':
82       // Locked while a scan is running or paused.
83       if (!canConfigure(state) || state.range === action.range) return state;
84       return { ...state, phase: 'idle', range: action.range, items: [], runId: state.runId + 1 };
85
86     case 'load': {
87       if (!canConfigure(state)) return state;
88       const seen = new Set<string>();
89       const items = action.items
90         .filter((i) => (seen.has(i.assetId) ? false : (seen.add(i.assetId), true)))
91         .sort((a, b) => b.createdAt - a.createdAt)
92         .map((i) => ({ ...i, status: 'queued' as const }));
93       return { ...state, phase: 'idle', items, runId: state.runId + 1 };
94     }
95
96     case 'start':
97       if (state.phase !== 'idle' || nextQueued(state) === null) return state;
98       return { ...state, phase: 'running' };
99
100    case 'pause':
101      return state.phase === 'running' ? { ...state, phase: 'paused' } : state;
102
103    case 'resume':
104      if (state.phase !== 'paused') return state;
105      return nextQueued(state) === null ? { ...state, phase: 'done' } : { ...state, phase:
'running' };
106
107    case 'itemStarted':
108      if (action.runId !== state.runId) return state;
109      return {
110        ...state,
111        items: state.items.map((i) =>

```

```

112         i.assetId === action.assetId && i.status === 'queued' ? { ...i, status: 'working' } :
113         i,
114     });
115
116     case 'itemFinished': {
117         if (action.runId !== state.runId) return state; // stale result from an older run
118         let found = false;
119         const items = state.items.map((i) => {
120             if (i.assetId !== action.assetId || (i.status !== 'working' && i.status !== 'queued'))
121                 return i;
122             found = true;
123             return {
124                 ...i,
125                 status: action.status,
126                 txId: action.txId,
127                 amountSatang: action.amountSatang,
128                 label: action.label,
129                 message: action.message,
130                 confirmed: action.confirmed,
131             };
132         });
133         if (!found) return state;
134         const next = { ...state, items };
135         const remaining = items.some((i) => i.status === 'queued' || i.status === 'working');
136         if (!remaining) next.phase = 'done';
137         return next;
138     }
139     case 'reset':
140         return { ...initialScanState(state.range), runId: state.runId + 1 };
141 }
142 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/scanQueue.ts:79

บรรทัด	ความหมาย
79	reducer = ฟังก์ชันที่รับ (สถานะเดิม, คำสั่ง) คืนสถานะใหม่ ไม่แก้ไขของเดิม (สร้างอ็อบเจกต์ใหม่ทุกครั้งด้วย { ...state, ... }) React จึงรู้ว่ามีอะไรเปลี่ยน
80	switch แยกทำตามชนิดคำสั่ง
81–84	setRange : ตั้งค่าไม่ได้ (กำลังสแกน) หรือช่วงเดิม → คืนสถานะเดิม (ไม่มีอะไรเกิดขึ้น) · ไม่ขึ้นกลับเป็น idle ช่วงใหม่ ล้างคิว เลขรอบ +1
86–94	load : ตั้งค่าไม่ได้ → เพิกเฉย · ตัดรูปซ้ำด้วย Set (seen.has ? false : (seen.add, true) = ถ้าเคยเห็นถึง ไม่ขึ้นจำแล้วเก็บ) · เรียงใหม่ไปเก่า · ทุกรูปเป็น queued · idle และเลขรอบ +1
96–98	start : ต้องเป็น idle และมีรูปให้ทำ ไม่ขึ้นเพิกเฉย
100–101	pause : ได้เฉพาะตอน running
103–105	resume : ได้เฉพาะตอน paused ถ้าไม่เหลือรูป → done เลย ไม่ขึ้น running
107–114	itemStarted : เลขรอบไม่ตรง → กิ่ง · เปลี่ยนเฉพาะรูปที่รหัสตรงและยัง queued เป็น working
116–117	itemFinished : เลขรอบไม่ตรง = "ผลค้างจากรอบเก่า" → กิ่ง

บรรทัด	ความหมาย
118–131	หารูปที่รหัสตรง และยังไม่เสร็จ (working หรือ queued) ใส่ผลลงไป · found จำว่าเจอไหม
132	ไม่เจอ (เช่น ผลซ้ำ 2 ครั้ง) → สถานะเดิม
133–135	ไม่เหลือรูปที่ queued หรือ working → done อัตโนมัติ
139–140	reset: กลับสู่สถานะเริ่มต้น (ช่วงเดิม) แต่เลขรอบ +1 เพื่อให้ผลที่อาจยังค้างอยู่ถูกทิ้ง

i ไม่มี default ใน switch

ทุกชนิดคำสั่งมี case ครบ ถ้าวันหนึ่งเพิ่มคำสั่งใหม่ใน ScanAction แต่ลืมเขียน case ฟังก์ชันจะไม่ return ค่าในบางกรณี และ TypeScript (โหมด strict) จะเตือนว่า "ฟังก์ชันอาจคืน undefined" เป็นการใช้ระบบชนิดข้อมูลช่วยจับลิม

ส่วนที่ 4: ตัวช่วยอ่านสถานะ (บรรทัด 144–173)

src/domain/scanQueue.ts

บรรทัด 144–173

```

144 /** The next image to process, or null. Items left 'working' by a pause are retried. */
145 export function nextQueued(state: ScanState): ScanItem | null {
146   return state.items.find((i) => i.status === 'queued' || i.status === 'working') ?? null;
147 }
148
149 export type ScanCounts = Record<ItemStatus, number> & { total: number; finished: number };
150
151 export function scanCounts(state: ScanState): ScanCounts {
152   const c = {
153     queued: 0, working: 0, ready: 0, needs_review: 0, duplicate: 0,
154     out_of_range: 0, not_slip: 0, failed: 0, total: state.items.length, finished: 0,
155   };
156   for (const i of state.items) c[i.status] += 1;
157   c.finished = c.total - c.queued - c.working;
158   return c;
159 }
160
161 /**
162  * Which range milestones (1 จุ, 7 จุ, ...) up to the chosen range are fully checked.
163  * A milestone is complete when every photo newer than its start has been processed.
164  */
165 export function milestones(state: ScanState, now: Date = new Date()) {
166   const chosen = RANGE_ORDER.indexOf(state.range);
167   return RANGE_ORDER.slice(0, chosen + 1).map((range) => {
168     const start = rangeStartMs(range, now);
169     const inside = state.items.filter((i) => i.createdAt >= start);
170     const done = inside.every((i) => i.status !== 'queued' && i.status !== 'working');
171     return { range, total: inside.length, complete: state.items.length > 0 && done };
172   });
173 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/scanQueue.ts:144

บรรทัด	ความหมาย
145–147	nextQueued รูปถัดไปที่ต้องทำ = รูปแรกที่ queued หรือ working (รูปที่ค้าง working เพราะกดหยุดกลางทาง จะถูกทำใหม่) ไม่มี → null
149	ScanCounts = นับทุกสถานะ + total + finished (& = รวมชนิด 2 แบบเข้าด้วยกัน)
151–159	scanCounts เริ่มทุกตัวนับเป็น 0 วนทุกรูปบวก 1 ให้สถานะของมัน · finished = ทั้งหมด – รอคิว – กำลังทำ · หน้าสแกนใช้ค่านี้แสดง "1/4" และ "ซ้ำ 1 · รูปอื่น 12 ..."
165–173	milestones หลักไหนตรวจครบแล้ว: สำหรับทุกช่วงจนถึงช่วงที่เลือก หาเวลาเริ่มของช่วง เอารูปที่ใหม่กว่านั้น ครบเมื่อไม่มีรูปไหนยัง queued/working · มีเทสต์ TC-30 ยืนยันว่าหลักเต็มตามลำดับ 1 วัน → 7 วัน → ... แต่หน้าสแกนรุ่นปัจจุบัน (ที่ไม่ต้องเลือกช่วงก่อน) ไม่ได้เรียกใช้แล้ว เหลือจากหน้าจอรุ่นก่อนที่ให้เลือกช่วง

อ่านโค้ด: autoScan.ts และ prefs.ts

กฎของการสแกนอัตโนมัติ (45 บรรทัด) และที่เก็บตั้งค่า (48 บรรทัด)

ส่วนที่ 1: autoScan.ts ทั้งไฟล์ (บรรทัด 1–45)

src/domain/autoScan.ts

บรรทัด 1–45

```

1  /**
2   * FR-4 automatic scan when the app opens: which photos to look at.
3   * Pure rules, unit-tested in __tests__/autoScan.test.ts.
4   */
5  import { DAY_MS } from './dates';
6
7  /** First run looks back 30 days (like MeowJot), later runs start from the last run. */
8  export const FIRST_RUN_DAYS = 30;
9  /** Never look back more than 30 days automatically (older slips: use the scan screen). */
10 export const MAX_LOOKBACK_DAYS = 30;
11 /** Photos saved just before the last run are checked again, in case the gallery was still
    syncing. */
12 export const OVERLAP_MS = 60 * 60 * 1000;
13 /** Opening the app repeatedly within this time does not start another scan. */
14 export const MIN_INTERVAL_MS = 5 * 60 * 1000;
15 /** At most this many photos per run (only QR slips are sent to be read); the rest are picked up
    next time. */
16 export const MAX_IMAGES_PER_RUN = 150;
17
18 /** A photo the automatic scan could not read is tried this many times in all, then left for the
    scan screen. */
19 export const MAX_AUTO_ATTEMPTS = 2;
20
21 /**
22  * The time to save as "last automatic scan": normally when this run started,
23  * but early enough that the oldest photo to try again stays inside the next
24  * run's window (see autoScanSince).
25  */
26 export function nextAutoScanMark(startedAt: number, retryFromCreatedAt: number | null): number {
27   if (retryFromCreatedAt === null) return startedAt;
28   return Math.min(startedAt, retryFromCreatedAt + OVERLAP_MS - 1);
29 }
30
31 export function autoScanSince(lastAutoScanAt: number, nowMs: number): number {
32   const floor = nowMs - MAX_LOOKBACK_DAYS * DAY_MS;
33   if (!lastAutoScanAt) return nowMs - FIRST_RUN_DAYS * DAY_MS;
34   return Math.max(floor, lastAutoScanAt - OVERLAP_MS);
35 }
36
37 export function shouldAutoScan(params: {
38   enabled: boolean;
39   lastAutoScanAt: number;
40   nowMs: number;
41   alreadyRunning: boolean;
42 }): boolean {
43   if (!params.enabled || params.alreadyRunning) return false;
44   return params.nowMs - params.lastAutoScanAt >= MIN_INTERVAL_MS;

```

บรรทัด	ความหมาย
5	<code>DAY_MS</code> = จำนวนมิลลิวินาทีใน 1 วัน (86,400,000)
7–8	ครั้งแรกย้อน 30 วัน (คอมเมนต์บอกว่าเหมือนแอป MeowJot ที่ทีมศึกษา)
9–10	อัตโนมัติไม่ย้อนเกิน 30 วัน
11–12	ย้อนกับ 1 ชม. เพื่อเคลเลอร์ยังซิงค์รูปไม่เสร็จตอนรอบที่แล้ว (<code>60 * 60 * 1000</code> เขียนเป็นสูตรให้อ่านออกกว่าคือ 1 ชั่วโมง)
13–14	เปิดเข้าภายใน 5 นาทีไม่สแกน
15–16	รอบละไม่เกิน 150 รูป
18–19	รูปที่อ่านไม่ได้ ลองรวม 2 ครั้ง
26–29	nextAutoScanMark : ไม่มีรูปต้องลองใหม่ → บันทึกเวลาที่เริ่มรอบนี้ · มี → บันทึกเวลาที่ เร็วกว่า ระหว่าง "เวลาเริ่มรอบ" กับ "เวลารูปเก่าสุดที่ต้องลองใหม่ + 1 ชม. - 1 ms" เพื่อให้รอบหน้า (ซึ่งย้อนกับ 1 ชม.) ยังเห็นรูปนั้น
31–35	autoScanSince : เส้นต่ำสุด = ตอนนี้ - 30 วัน · ไม่เคยสแกน → ตอนนี้ - 30 วัน · เคย → ค่าที่ ใหม่กว่า ระหว่างเส้นต่ำสุดกับ (ครั้งล่าสุด - 1 ชม.)
37–45	shouldAutoScan : ปิดสวิตช์อยู่หรือกำลังสแกน → ไม่ · ห่างจากครั้งล่าสุดอย่างน้อย 5 นาที → สแกน (ไม่เคยสแกน <code>lastAutoScanAt = 0</code> จึงห่างมากเสมอ)

ส่วนที่ 2: prefs.ts — ตั้งค่าการสแกน (บรรทัด 1–48)

src/data/prefs.ts

บรรทัด 1–48

```

1  /**
2   * Per-device preferences for slip scanning. Kept on the phone (not in the
3   * database) because they describe how THIS phone's gallery is handled.
4   */
5  import { Storage } from './storage';
6
7  export interface ScanPrefs {
8    /** Check the gallery for new slips every time the app opens. */
9    autoScan: boolean;
10   /** Slips read with ≥ 80% confidence on every field count in the balance right away. */
11   autoConfirm: boolean;
12   /** When the last automatic scan started (ms). 0 = never. */
13   lastAutoScanAt: number;
14   /** The phone's photo permission was asked for once by itself (the first time the app opened).
15   */
16   askedGalleryOnce: boolean;
17 }
18 const KEY = 'mindpay.scanPrefs.v1';
19 export const DEFAULT_SCAN_PREFS: ScanPrefs = { autoScan: true, autoConfirm: true, lastAutoScanAt:
20   0, askedGalleryOnce: false };
21 /** Stored form: the switches belong to this phone, the scan window to each account on it. */

```

```

22 type Stored = Omit<ScanPrefs, 'lastAutoScanAt'> & { lastAutoScanBy?: Record<string, number> };
23
24 async function loadStored(): Promise<Stored> {
25   try {
26     const raw = await Storage.getItem(KEY);
27     const parsed = raw ? (JSON.parse(raw) as Partial<Stored>) : {};
28     const { autoScan, autoConfirm, askedGalleryOnce } = { ...DEFAULT_SCAN_PREFS, ...parsed };
29     return { autoScan, autoConfirm, askedGalleryOnce, lastAutoScanBy: parsed.lastAutoScanBy ?? {} };
30   } catch {
31     return { ...DEFAULT_SCAN_PREFS, lastAutoScanBy: {} };
32   }
33 }
34
35 /** This phone's scan settings, with the scan window of `userId` (another account starts with its
    own). */
36 export async function loadScanPrefs(userId?: string | null): Promise<ScanPrefs> {
37   const s = await loadStored();
38   return { autoScan: s.autoScan, autoConfirm: s.autoConfirm, askedGalleryOnce:
    s.askedGalleryOnce, lastAutoScanAt: (userId && s.lastAutoScanBy?.[userId]) || 0 };
39 }
40
41 export async function saveScanPrefs(patch: Partial<ScanPrefs>, userId?: string | null):
    Promise<ScanPrefs> {
42   const s = await loadStored();
43   const { lastAutoScanAt, ...rest } = patch;
44   const next: Stored = { ...s, ...rest };
45   if (lastAutoScanAt !== undefined && userId) next.lastAutoScanBy = { ...s.lastAutoScanBy,
    [userId]: lastAutoScanAt };
46   await Storage.setItem(KEY, JSON.stringify(next));
47   return loadScanPrefs(userId);
48 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/prefs.ts:1

บรรทัด	ความหมาย
1-4	ตั้งค่าเก็บบนมือถือ ไม่ใช่ฐานข้อมูล เพราะเป็นเรื่องของ แพลตฟอร์มมือถือเครื่องนี้
7-16	ScanPrefs 4 ช่อง: สแกนอัตโนมัติ, รวมยอดทันที, เวลาสแกนอัตโนมัติล่าสุด (0 = ไม่เคย), เคยขอสิทธิ์รูปภาพเองแล้วหรือยัง
18	คีย์ที่เก็บ มี ".v1" เพื่ออนาคตเปลี่ยนรูปแบบ จะได้รู้ว่าข้อมูลเก่าเป็นรุ่นไหน
19	ค่าเริ่มต้น: เปิดทั้งสองสวิตช์ ไม่เคยสแกน ยังไม่เคยขอสิทธิ์
21-22	รูปแบบที่เก็บจริง: สวิตช์เป็นของเครื่อง แต่ "สแกนล่าสุดเมื่อไร" แยกตามบัญชี (lastAutoScanBy: userId → เวลา) · Omit = ชดเชยเต็มตัดบางช่องออก
24-33	loadStored อ่าน JSON แล้ว ผสมกับค่าเริ่มต้น ({ ...DEFAULT, ...parsed } ช่องที่ไม่มีในข้อมูลเก่าได้ค่าเริ่มต้น) อ่านพังได้ค่าเริ่มต้นทั้งหมด
36-39	loadScanPrefs คำนวณค่าของเครื่อง + เวลาสแกนล่าสุด ของบัญชีนี้ (ไม่มีหรือไม่ได้ล็อกอิน → 0) บัญชีใหม่ในเครื่องเดิมจึงเริ่มย้อน 30 วันของตัวเอง
41-47	saveScanPrefs แยก lastAutoScanAt ออกจาก patch (destructuring const { lastAutoScanAt, ...rest } = patch) ส่วนที่เหลือรวมเข้ากับของเดิม ถ้ามีเวลาและรู้บัญชี บันทึกลงช่องของบัญชีนั้น แล้วคืนค่าที่อ่านใหม่

อ่านโค้ด: gallery.ts และ gallery.web.ts — ค้นรูปในเครื่อง

ไฟล์ชื่อเดียวกัน 2 เวอร์ชัน: มือถือใช้ .ts เว็บใช้ .web.ts (หลักเดียวกับ speech.ts ในเล่ม FR-3)

ส่วนที่ 1: สิทธิ์เข้าถึงรูปภาพบนมือถือ (บรรทัด 1–23)

src/services/gallery.ts

บรรทัด 1–23

```

1 /**
2  * FR-4 gallery access on the phone (Android / iOS).
3  * The web build uses gallery.web.ts instead, where only hand-picked photos are possible.
4  */
5 import { Asset, AssetField, getPermissionsAsync, MediaType, Query, requestPermissionsAsync } from
   'expo-media-library';
6
7 /** Whether this platform can search the whole photo library. */
8 export const galleryAvailable = true;
9
10 export type GalleryAccess = 'all' | 'limited' | 'denied' | 'blocked';
11
12 /** Current access without showing a prompt (used by the automatic scan on app open). */
13 export async function getGalleryAccess(): Promise<GalleryAccess> {
14   const res = await getPermissionsAsync(false, ['photo']);
15   if (res.granted) return res.accessPrivileges === 'limited' ? 'limited' : 'all';
16   return res.canAskAgain ? 'denied' : 'blocked';
17 }
18
19 export async function requestGalleryAccess(): Promise<GalleryAccess> {
20   const res = await requestPermissionsAsync(false, ['photo']);
21   if (res.granted) return res.accessPrivileges === 'limited' ? 'limited' : 'all';
22   return res.canAskAgain ? 'denied' : 'blocked';
23 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/services/gallery.ts:1

บรรทัด	ความหมาย
5	ใช้ไลบรารี expo-media-library (คลังรูปของมือถือ): Asset (รูป 1 รูป), Query (ตัวค้น), ฟังก์ชันสิทธิ์
8	galleryAvailable = true มือถือค้นคลังรูปได้ (ในเวอร์ชันเว็บค่านี้เป็น false ถ้าไม่มีแคลเลอร์ทดสอบ)
10	สิทธิ์ 4 แบบ: ทั้งหมด / บางรูป (iOS และ Android รุ่นใหม่ให้เลือก "เฉพาะรูปที่เลือก") / ปฏิเสธ (ถามใหม่ได้) / บล็อก (ถามไม่ได้แล้ว ต้องไปเปิดในตั้งค่ามือถือ)
12–17	getGalleryAccess ดูสิทธิ์ปัจจุบัน โดยไม่ขึ้นกล้องถาม (สแกนอัตโนมัติใช้ตัวนี้ จะได้ไม่รบกวน) · พารามิเตอร์ false, ['photo'] = ขอแค่รูปภาพ ไม่ขอเขียนและไม่ขอวิดีโอ
15	ได้สิทธิ์: ถ้าเป็นแบบ limited คืน 'limited' ไม่ขึ้น 'all'
16	ไม่ได้: ถามใหม่ได้ → 'denied' · ถามไม่ได้ → 'blocked'

ส่วนที่ 2: ค้นรูปตามเวลา (บรรทัด 25–60)

src/services/gallery.ts

บรรทัด 25–60

```

25 export interface GalleryImage {
26   assetId: string;
27   createdAt: number;
28   width: number | null;
29   height: number | null;
30 }
31
32 /**
33  * Photos created since `sinceMs`, newest first. Landscape photos are skipped:
34  * bank slips are always portrait, which removes most camera photos cheaply.
35  * Read page by page, so a busy gallery is not cut off at the newest few hundred.
36  */
37 export async function findGalleryImages(sinceMs: number, max = 20000): Promise<GalleryImage[]> {
38   const PAGE = 1000;
39   const out: GalleryImage[] = [];
40   for (let offset = 0; offset < max; offset += PAGE) {
41     const rows = await new Query()
42       .eq(AssetField.MEDIA_TYPE, MediaType.IMAGE)
43       .gte(AssetField.CREATION_TIME, sinceMs)
44       .orderBy({ key: AssetField.CREATION_TIME, ascending: false })
45       .offset(offset)
46       .limit(Math.min(PAGE, max - offset))
47       .exeForMetadata();
48     for (const r of rows) {
49       if (!r.width || !r.height || r.height >= r.width * 1.1) {
50         out.push({ assetId: r.id, createdAt: r.creationTime ?? sinceMs, width: r.width, height:
51           r.height });
52       }
53     }
54     if (rows.length < PAGE) break;
55   }
56   return out;
57 }
58 export function galleryUri(assetId: string): Promise<string> {
59   return new Asset(assetId).getUri();
60 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/services/gallery.ts:25

25–30 **GalleryImage** ข้อมูลที่ต้องใช้ของรูป: รหัส, เวลาที่สร้าง, กว้าง, สูง (ยังไม่โหลดตัวรูป ประหยัดหน่วยความจำ)

32–36 คอมเมนต์: รูปใหม่ไปเฝ้า **ข้ามรูปแนวนอน** (สลิปเป็นแนวตั้งเสมอ ตัดรูปจากกล้องส่วนใหญ่ออกได้ฟรี) และอ่านทีละหน้า เพื่อไม่ให้เกลเลอริที่มีรูปเยอะถูกตัดที่ไม่ที่ร้อยรูปแรก

37 รับเวลาเริ่ม และจำนวนสูงสุด (ค่าเริ่ม 20,000)

บรรทัด	ความหมาย
38	หน้าละ 1,000 รูป
40	วนทีละหน้า: offset 0, 1000, 2000, ...
41–47	สร้างคำค้นแบบต่อกันเป็นสาย (method chaining): เฉพาะรูปภาพ · สร้างตั้งแต่เวลาที่กำหนด (gte = มากกว่าหรือเท่ากับ) · เรียงใหม่ไปเก่า · ข้าม offset รูป · เอาไม่เกินหน้าละ 1,000 · <code>exeForMetadata()</code> ขอแค่ข้อมูลประกอบ ไม่โหลดภาพ
48–52	เก็บรูปที่ไม่รู้ขนาด หรือ สูง $\geq$ กว้าง $\times 1.1$ (แนวตั้งชัด ๆ) · ไม่มีเวลาสร้างใช้เวลาเริ่มแทน
53	หน้านี้ได้น้อยกว่า 1,000 = หมดแล้ว หยุดวน
58–60	<code>galleryUri</code> แปลงรหัสรูปเป็นท็อยูโฟล์ (uri) ที่ฟังก์ชันอื่นเปิดอ่านได้ ทำทีละรูปตอนจะใช้จริง

ส่วนที่ 3: เวอร์ชันเว็บ และ "แกลเลอรีทดสอบ" (บรรทัด 1–49)

src/services/gallery.web.ts

บรรทัด 1–49

```

1 /**
2  * Web version of gallery.ts. A browser cannot search the photo library, so
3  * on the web the scanner offers only "เลือกรูปเอง" (pick photos).
4  *
5  * Test hook: the browser test and the demo video set
6  * `window.__MINDPAY_TEST_GALLERY__` before the app loads, to stand in for a
7  * phone's photo library. That runs the same automatic-scan code as the
8  * Android app, so it can be tested and shown on a computer. A real visitor's
9  * browser never has it, and a page script cannot reach real photos anyway.
10 */
11 export type GalleryAccess = 'all' | 'limited' | 'denied' | 'blocked';
12
13 export interface GalleryImage {
14   assetId: string;
15   createdAt: number;
16   width: number | null;
17   height: number | null;
18 }
19
20 interface TestGallery {
21   images: { id: string; uri: string; createdAt: number; width: number; height: number }[];
22 }
23
24 const testGallery = (): TestGallery | undefined =>
25   typeof window === 'undefined' ? undefined : (window as unknown as { __MINDPAY_TEST_GALLERY__?:
26     TestGallery }).__MINDPAY_TEST_GALLERY__;
27
28 export const galleryAvailable = !!testGallery();
29
30 export async function getGalleryAccess(): Promise<GalleryAccess> {
31   return testGallery() ? 'all' : 'denied';
32 }
33
34 export async function requestGalleryAccess(): Promise<GalleryAccess> {
35   return testGallery() ? 'all' : 'denied';
36 }
37
38 export async function findGalleryImages(sinceMs = 0, max = 20000): Promise<GalleryImage[]> {

```

```

38 const gallery = testGallery();
39 if (!gallery) return [];
40 return gallery.images
41   .filter((i) => i.createdAt >= sinceMs && i.height >= i.width * 1.1)
42   .sort((a, b) => b.createdAt - a.createdAt)
43   .slice(0, max)
44   .map((i) => ({ assetId: i.id, createdAt: i.createdAt, width: i.width, height: i.height }));
45 }
46
47 export async function galleryUri(assetId: string): Promise<string> {
48   return testGallery()?.images.find((i) => i.id === assetId)?.uri ?? assetId;
49 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/gallery.web.ts:1

บรรทัด	ความหมาย
1–3	เบราว์เซอร์ค้นคลังรูปไม่ได้ บนเว็บจึงมีแค่ "เลือกรูปเอง"
5–9	ทางเข้าสำหรับเทสต์: เทสต์บนเบราว์เซอร์และวิดีโอเดโมตั้งค่า <code>window.__MINDPAY_TEST_GALLERY__</code> ก่อนเปิดแอป เพื่อจำลองคลังรูปของมือถือ ทำให้รันโค้ดสแกนอัตโนมัติ "ตัวเดียวกับ Android" บนคอมได้ · เบราว์เซอร์ของผู้ใช้จริงไม่มีค่านี้ และสคริปต์บนหน้าเว็บก็เข้าถึงรูปจริงไม่ได้อยู่แล้ว (ไม่ใช่ช่องโหว่)
11–18	ชนิดข้อมูลชื่อเดียวกับเวอร์ชันมือถือ ไฟล์อื่นจึง import ได้โดยไม่ต้องรู้ว่าเป็นเวอร์ชันไหน
24–25	อ่านแกเลอรีทดสอบจาก window (ตอนรันนอกเบราว์เซอร์ <code>window</code> ไม่มี → undefined)
27	มีแกเลอรีทดสอบ = ทำตัวเหมือนมือถือ (ไม่มี = เว็บปกติ)
29–35	สิทธิ์: มีแกเลอรีทดสอบ → 'all' ไม่จัน 'denied'
37–45	คัณรูป: กฎเดียวกับมือถือทุกข้อ (หลังเวลาที่กำหนด, แนวตั้ง × 1.1, ใหม่ไปเก่า, ไม่เกินจำนวนสูงสุด) เทสต์จึงทดสอบกฎจริง
47–49	ที่อยู่ของรูปทดสอบ (เซิร์ฟเวอร์ทดสอบส่งไฟล์จาก e2e/fixtures ให้)

อ่านโค้ด: slips.ts — หา QR ย่อรูป และเรียกตัวอ่าน

src/services/slips.ts 253 บรรทัด: งานฝั่งมือถือทั้งหมดของรูป 1 รูป

ส่วนที่ 1: หัวไฟล์และ import (บรรทัด 1–27)

src/services/slips.ts

บรรทัด 1–27

```

1  /**
2   * FR-4 on the phone: find photos in the gallery, spot slips, read them.
3   *
4   * Cost and privacy rule: a photo leaves the phone only if it looks like a slip.
5   * By default that means it carries a Thai slip-verification QR code, which is
6   * detected on the device for free. Photos the user picks by hand are always read.
7   */
8  import { scanFromURLAsync } from 'expo-camera';
9  import * as Crypto from 'expo-crypto';
10 import { File } from 'expo-file-system';
11 import { ImageManipulator, SaveFormat } from 'expo-image-manipulator';
12 import { Platform } from 'react-native';
13 import { Storage } from '../data/storage';
14 import { FunctionsHttpError } from '@supabase/supabase-js';
15 import {
16   firstSlipQr,
17   isScreenSized,
18   regionInPixels,
19   SLIP_QR_REGIONS,
20   type PictureSize,
21   type SlipCheck,
22   type SlipQr,
23   type SlipReading,
24 } from '../domain/slip';
25 import { supabase } from '../data/supabase';
26
27 export * from './gallery';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:1

บรรทัด	ความหมาย
4–6	กฎต้นกุนและความเป็นส่วนตัว: รูปออกจากมือถือก็ต่อเมื่อดูเหมือนสลิป = มี QR ตรวจสอบสลิป (ตรวจบนเครื่อง ฟรี) รูปที่ใช้เลือกเองอ่านเสมอ
8	<code>scanFromURLAsync</code> จาก expo-camera: สแกนบาร์โค้ด/QR จาก "ไฟล์รูป" (ไม่ต้องเปิดกล้อง)
9	expo-crypto: ทำ SHA-256
10	expo-file-system: ลบไฟล์ชั่วคราว
11	expo-image-manipulator: ย่อ ตัด และบันทึกรูป
13	Storage = ที่เก็บข้อมูลแบบคีย์-ค่าในเครื่อง (มือถือใช้ kv-store ของ expo-sqlite ส่วนเว็บใช้ localStorage ของเบราว์เซอร์ ผ่านไฟล์ storage.ts / storage.web.ts)

14 FunctionsHttpError ชีต error เมื่อ Edge Function ตอบสหัสผิดพลาด (4xx/5xx)

15–24 นำกฎจาก slip.ts มาใช้

27 export * from './gallery' ส่งต่อทุกอย่างของ gallery ออกไปด้วย ไฟล์อื่น import จาก slips ที่เดียวได้ทั้งหมด (และได้ .web อัตโนมัติบนเว็บ)

ส่วนที่ 2: จำรูปที่เคยตรวจ และจำนวนครั้งที่ล้มเหลว (บรรทัด 29–83)

src/services/slips.ts

บรรทัด 29–83

```

29 // Remember which photos were already checked, so a second scan skips them.
30 const scannedKey = (userId: string) => `mindpay.scanned.${userId}`;
31 // How many times the automatic scan failed to read each photo (see MAX_AUTO_ATTEMPTS).
32 const failuresKey = (userId: string) => `mindpay.scanFailures.${userId}`;
33
34 export async function loadScannedIds(userId: string): Promise<Set<string>> {
35   try {
36     const raw = await Storage.getItem(scannedKey(userId));
37     return new Set(raw ? (JSON.parse(raw) as string[]) : []);
38   } catch {
39     return new Set();
40   }
41 }
42
43 /**
44  * The manual scan (scan screen) is running: the automatic scan waits, so the two never read the
45  * same photos at the same time (which would spend the daily AI quota twice).
46  */
47 export const scanLock = { manual: false };
48
49 export async function rememberScanned(userId: string, ids: Set<string>) {
50   try {
51     // Merge with what is stored now: the other scan may have added photos since this one
52     // started.
53     const all = await loadScannedIds(userId);
54     for (const id of ids) all.add(id);
55     await Storage.setItem(scannedKey(userId), JSON.stringify([...all].slice(-5000)));
56   } catch {
57     // Not critical: worst case a photo is checked again and caught as a duplicate.
58   }
59 }
60
61 export async function forgetScanned(userId: string) {
62   await Storage.removeItem(scannedKey(userId)).catch(() => {});
63   await Storage.removeItem(failuresKey(userId)).catch(() => {});
64 }
65
66 export async function loadScanFailures(userId: string): Promise<Record<string, number>> {
67   try {
68     const raw = await Storage.getItem(failuresKey(userId));
69     const parsed = raw ? (JSON.parse(raw) as unknown) : null;
70     return parsed && typeof parsed === 'object' ? (parsed as Record<string, number>) : {};
71   } catch {
72     return {};
73   }
74 }

```

```

73 }
74
75 export async function saveScanFailures(userId: string, failures: Record<string, number>) {
76   const entries = Object.entries(failures).slice(-500);
77   try {
78     if (entries.length === 0) await Storage.removeItem(failuresKey(userId));
79     else await Storage.setItem(failuresKey(userId), JSON.stringify(Object.fromEntries(entries)));
80   } catch {
81     // Not critical: worst case a photo is tried once more.
82   }
83 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:29

บรรทัด	ความหมาย
30	คีย์รายการรูปที่เคยตรวจ แยกตามบัญชี
32	คีย์จำนวนครั้งที่สแกนอัตโนมัติอ่านรูปไม่ได้
34–41	loadScannedIds อ่าน JSON array เป็น Set อ่านพึงได้ Set ว่าง
43–47	scanLock อีอบเจกต์ธรรมดาที่ใช้ร่วมกันทั้งแอป: <code>manual: true</code> ระหว่างหน้าสแกนทำงาน การสแกนอัตโนมัติจะรอไม่อ่านรูปเดียวกันพร้อมกัน (ซึ่งจะเปลืองโควตา AI 2 เท่า)
49–58	rememberScanned อ่านของล่าสุดก่อนแล้วรวม (เพราะอีกระบบอาจเพิ่มรูปไปแล้วระหว่างนี้) เก็บ 5,000 อันล่าสุด (<code>slice(-5000)</code>) บันทึกฟังก์ชันไม่เป็นไร แยกสุดท้ายคือตรวจรูปซ้ำแล้วถูกจับว่าซ้ำ
60–63	forgetScanned (ปุ่ม "ล้างประวัติรูปที่เคยสแกน") ลบทั้ง 2 คีย์
65–73	loadScanFailures อ่านอีอบเจกต์ รหัสรูป → จำนวนครั้ง ตรวจสอบว่าเป็นอีอบเจกต์จริงก่อนเชื่อ
75–83	saveScanFailures เก็บไม่เกิน 500 รายการ ว่างเปล่าลบคีย์ทิ้งเลย

ส่วนที่ 3: detectSlipQr – หา QR บนมือถือ (บรรทัด 85–148)

src/services/slips.ts

บรรทัด 85–148

```

85 // -----
86 // One image
87 // -----
88
89 export interface QrCheck {
90   /** The slip-verification QR, when one was found. */
91   qr: SlipQr | null;
92   /** False when this phone cannot scan QR codes at all (Android without Google Play Services). */
93   scannerWorks: boolean;
94   /** Some other QR code was found (e-wallet slips such as TrueMoney use their own). */
95   otherQr?: boolean;
96 }
97
98 /** The QR codes in one picture, or 'unavailable' when the phone has no QR scanner. Never throws. */
99 async function scanCodes(uri: string): Promise<string[] | 'unavailable'> {
100   try {
101     const results = await scanFromURLAsync(uri, ['qr']);
102     return results.map((r) => r.data);

```

```

103 } catch (e) {
104     const message = e instanceof Error ? e.message : String(e);
105     return /MLKit|Google Play Services/i.test(message) ? 'unavailable' : [];
106 }
107 }
108
109 /** Remove a temporary picture made on the phone (the web keeps them in memory). */
110 function removeTemp(uri: string) {
111     if (Platform.OS === 'web') return;
112     try {
113         new File(uri).delete();
114     } catch {
115         // Already gone, or the system cleans the cache later.
116     }
117 }
118
119 /**
120  * Look for the slip-verification QR on the device. Never throws.
121  *
122  * The whole picture first. Android's scanner reads best when the code fills a
123  * large part of the image and a slip's QR is small, so for screen-sized
124  * pictures (saved slips, screenshots) it then looks again at the parts where
125  * banks print the QR (SLIP_QR_REGIONS). Camera photos skip that step.
126  */
127 export async function detectSlipQr(uri: string, size?: PictureSize): Promise<QrCheck> {
128     const whole = await scanCodes(uri);
129     if (whole === 'unavailable') return { qr: null, scannerWorks: false };
130     const found = firstSlipQr(whole);
131     const otherQr = whole.length > 0;
132     if (found || !isScreenSized(size)) return { qr: found, scannerWorks: true, otherQr: !found &&
otherQr };
133     try {
134         // Decode once; each part is cut from the decoded picture.
135         const full = await ImageManipulator.manipulate(uri).renderAsync();
136         for (const region of SLIP_QR_REGIONS) {
137             const part = await ImageManipulator.manipulate(full).crop(regionInPixels(region,
full)).renderAsync();
138             const saved = await part.saveAsync({ format: SaveFormat.JPEG, compress: 0.92 });
139             const codes = await scanCodes(saved.uri);
140             removeTemp(saved.uri);
141             const qr = codes === 'unavailable' ? null : firstSlipQr(codes);
142             if (qr) return { qr, scannerWorks: true };
143         }
144     } catch {
145         // The picture could not be cut; the answer from the whole picture stands.
146     }
147     return { qr: null, scannerWorks: true, otherQr };
148 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:85

บรรทัด ความหมาย

89–96 **QrCheck** ผลการหา: QR สลิป (หรือ null), ตัวสแกนใช้ได้ไหม (Android ไม่มี Google Play Services = ไม่มีตัวสแกน), เจอ QR อื่นไหม (e-wallet)

99–107 **scanCodes** สแกนทุก QR ในรูป ค้นข้อความของแต่ละ QR · ถ้า error มีคำว่า MLKit หรือ Google Play Services = เครื่องนี้ไม่มีตัวสแกน → 'unavailable' · error อื่น → [] (ไม่เจอ) · "Never throws" = ไม่โยน error ออกไปเลย

บรรทัด	ความหมาย
110–117	removeTemp ลบรูปชั่วคราวที่สร้างตอนตัด/ย่อ (บนเว็บอยู่ในหน่วยความจำ ไม่ต้องลบ) ลบไม่ได้ก็ปล่อย ระบบล้างแคชเองทีหลัง
127	detectSlipQr รับรูปและขนาด
128–129	สแกนทั้งรูปก่อน ไม่มีตัวสแกน → บอกตรง ๆ (scannerWorks: false)
130–131	หา QR สลิปในผล และจำว่ามี QR อะไรสักอย่างไหม
132	เจอ QR สลิป หรือ ไม่ใช่รูปขนาดหน้าจอ → ค้นเลย (otherQr = มี QR แต่ไม่ใช่ QR สลิป)
135	ถอดรหัสรูปเต็ม ครั้งเดียว แล้วตัดแต่ละส่วนจากตัวนี้ (เร็วกว่าเปิดไฟล์ใหม่ทุกครั้ง)
136–143	ทุกส่วนใน SLIP_QR_REGIONS: ตัดเป็นพิกเซล → บันทึก JPEG คุณภาพ 0.92 → สแกน → ลบไฟล์ชั่วคราว → เจอ QR สลิปคืนทันที
144–146	ตัดรูปไม่ได้ → ใช้ผลของทั้งรูป
147	ไม่เจอ QR สลิปเลย ค้น otherQr ตามผลทั้งรูป

ส่วนที่ 4: prepareImage — ย่อรูปและทำลายนิ้วมือ (บรรทัด 150–160)

src/services/slips.ts

บรรทัด 150–160

```

150 /** Shrink to at most 1100px wide JPEG (enough to read a slip, about 150 KB) and hash it. */
151 export async function prepareImage(uri: string, width?: number | null) {
152   const ctx = ImageManipulator.manipulate(uri);
153   if (!width || width > 1100) ctx.resize({ width: 1100 });
154   const ref = await ctx.renderAsync();
155   const saved = await ref.saveAsync({ format: SaveFormat.JPEG, compress: 0.8, base64: true });
156   removeTemp(saved.uri);
157   if (!saved.base64) throw new Error('image_prepare_failed');
158   const hash = await Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, saved.base64);
159   return { base64: saved.base64, hash };
160 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:150

บรรทัด	ความหมาย
153	ไม่รู้จักความกว้าง หรือกว้างเกิน 1,100 px → ย่อเหลือกว้าง 1,100 (ความสูงย่อตามสัดส่วน)
154–155	สร้างรูป บันทึกเป็น JPEG คุณภาพ 0.8 พร้อมขอ base64 (รูปในรูปแบบข้อความ ส่งใน JSON ได้)
156	ลบไฟล์ชั่วคราว (เรามี base64 แล้ว)
157	ไม่ได้ base64 → error
158	SHA-256 ของ base64 =ลายนิ้วมือของรูป (รูปเดียวกันย่อแบบเดียวกันได้แฮชเดียวกัน)
159	คืนทั้งรูปและแฮช

i base64 คืออะไร

ไฟล์รูปเป็นข้อมูลไบนารี (ไบนารี 0–255) ซึ่งใส่ใน JSON ตรง ๆ ไม่ได้ base64 แปลงทุก 3 ไบนารีเป็นตัวอักษรอ่านได้ 4 ตัว (A–Z a–z 0–9 + /) ขนาดจึงโตขึ้นประมาณ 1/3 รูป 150 KB กลายเป็นข้อความราว 200,000 ตัวอักษร เซิร์ฟเวอร์รับได้ไม่เกิน 6,000,000 ตัว

ส่วนที่ 5: ชนิดของปัญหาและความ (บรรทัด 162–204)

src/services/slips.ts

บรรทัด 162–204

```
162 export class SlipReaderError extends Error {
163   constructor(
164     public code:
165       | 'cloud_required'
166       | 'unauthorized'
167       | 'quota'
168       | 'too_large'
169       | 'not_configured'
170       | 'busy'
171       | 'network'
172       | 'reader_error'
173       | 'server'
174       | 'cancelled'
175       | 'qr_unavailable',
176     message: string,
177   ) {
178     super(message);
179   }
180 }
181
182 export const READER_MESSAGES: Record<SlipReaderError['code'], string> = {
183   cloud_required: 'ต้องเข้าสู่ระบบด้วยบัญชีจริงก่อน จึงจะอ่านสลิปได้',
184   unauthorized: 'เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง',
185   quota: 'วันนี้อ่านสลิปครบโควตาแล้ว ลองใหม่พรุ่งนี้',
186   too_large: 'รูปใหญ่เกินไป',
187   server: 'ระบบอ่านสลิปขัดข้องชั่วคราว สลิปที่เหลือจะอ่านต่อให้ภายหลัง',
188   cancelled: 'หยุดอ่านสลิปแล้ว',
189   not_configured: 'ระบบอ่านสลิปยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions →
  Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้)',
190   busy: 'ตอนนี้ AI อ่านสลิปมีคนใช้เยอะ ลองใหม่อีกสักครู่ สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า',
191   network: 'เชื่อมต่ออินเทอร์เน็ตไม่ได้ ลองใหม่อีกครั้ง',
192   reader_error: 'อ่านรูปนี้ไม่สำเร็จ',
193   qr_unavailable: 'มือถือเครื่องนี้หา QR ของสลิปเองไม่ได้ (ต้องมี Google Play Services) ใช้ "เลือกรูปเอง" แทนได้',
194 };
195
196 /**
197  * Seconds to wait before asking again when the AI is rate-limited. The free
198  * Gemini tier allows only a few requests per minute, so a first scan with many
199  * slips can hit the limit; waiting a little is better than stopping the scan.
200  */
201 export const BUSY_WAITS_S = [20, 40];
202
203 /** Shown while the reader waits for the AI's rate limit to reset. */
204 export const waitMessage = (seconds: number) => `AI มีคิวเยอะ รออีก ${seconds} วินาทีแล้วอ่านต่อให้เอง`;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:162

บรรทัด	ความหมาย
162	<code>SlipReaderError</code> สืบทอด (extends) จาก <code>Error</code> มาตรฐาน เพิ่มช่อง <code>code</code>
163–177	<code>public code: ...</code> ใน constructor = ประกาศและเก็บค่าในบรรทัดเดียว (shorthand ของ TypeScript) รหัสมี 11 แบบ
178	<code>super(message)</code> ส่งข้อความให้ <code>Error</code> ตัวแม่
182–194	<code>READER_MESSAGES</code> ข้อความภาษาไทยของทุกรหัส (<code>SlipReaderError['code']</code> = ชนิดของช่อง <code>code</code> ตัวเดียวกัน ขาดรหัสไหน TypeScript เตือน)
196–201	<code>BUSY_WAITS_S = [20, 40]</code> เมื่อ AI ตัดลิมิต (Gemini ฟรีอนุญาตไม่ที่ครั้งต่อนาที) รอ 20 แล้ว 40 วินาที ดึกว่าหยุดสแกนเลย
203–204	<code>waitMessage</code> สร้างข้อความ "AI มีคิวเยอะ รออีก N วินาทีแล้วอ่านต่อให้เอง"

ส่วนที่ 6: readSlip และ readSlipOnce – เรียกเชิร์ฟเวอร์ (บรรทัด 206–253)

src/services/slips.ts

บรรทัด 206–253

```

206 export interface SlipRead {
207   reading: SlipReading;
208   /** How the reader double-checked it (missing from older readers). */
209   check: SlipCheck | null;
210 }
211
212 /**
213  * Send the prepared image to the slip reader (Edge Function parse-slip).
214  * When the AI is busy, waits and tries again (onWait tells the screen).
215  */
216 export async function readSlip(base64: string, onWait?: (seconds: number) => void):
  Promise<SlipRead> {
217   for (let attempt = 0; ; attempt++) {
218     try {
219       return await readSlipOnce(base64);
220     } catch (e) {
221       const wait = BUSY_WAITS_S[attempt];
222       if (!(e instanceof SlipReaderError) || e.code !== 'busy' || wait === undefined) throw e;
223       onWait?.(wait);
224       await new Promise((resolve) => setTimeout(resolve, wait * 1000));
225     }
226   }
227 }
228
229 async function readSlipOnce(base64: string): Promise<SlipRead> {
230   if (!supabase) throw new SlipReaderError('cloud_required', READER_MESSAGES.cloud_required);
231   const { data, error } = await supabase.functions.invoke('parse-slip', {
232     body: { imageBase64: base64, mediaType: 'image/jpeg' },
233   });
234   if (error) {
235     let code: SlipReaderError['code'] = 'network';
236     if (error instanceof FunctionsHttpError) {
237       const body = await (error.context as Response).json().catch(() => null);
238       const c = body?.error?.code;
239       const status = (error.context as Response).status;
240       code =

```

```

241     c === 'unauthorized' || c === 'quota' || c === 'too_large' || c === 'not_configured' || c
    === 'busy'
242     ? c
243     : // A gateway error or timeout (no answer from the reader itself): the service is
    down, not this photo.
244     !c && status >= 500
245     ? 'server'
246     : 'reader_error';
247   }
248   throw new SlipReaderError(code, READER_MESSAGES[code]);
249 }
250 const { reading, check } = (data ?? {}) as { reading?: SlipReading; check?: SlipCheck };
251 if (!reading || typeof reading.isSlip !== 'boolean') throw new SlipReaderError('reader_error',
  READER_MESSAGES.reader_error);
252 return { reading, check: check && typeof check.verified === 'boolean' ? check : null };
253 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/slips.ts:206

บรรทัด	ความหมาย
206–210	SlipRead = ผลอ่าน + รายงานการตรวจ (ตัวอ่านรุ่นเก่าไม่มี check จึงเป็น null ได้)
216–217	วน for (let attempt = 0; ; attempt++) ไม่มีเงื่อนไขหยุด (ออกด้วย return หรือ throw)
219	ลองอ่าน สำเร็จ → ค้นเลย
221	เวลารอของครั้งนี้: ครั้งที่ 0 → 20, ครั้งที่ 1 → 40, ครั้งที่ 2 → undefined
222	ไม่ใช่ "AI ยุ่ง" หรือรอครบแล้ว → โยน error ต่อ
223	บอกหน้าจอกำลังรอที่วินาที (onWait?.()) = เรียกเมื่อมีคนส่งฟังก์ชันมา)
224	รอจริง (Promise + setTimeout) แล้ววนกลับไปลองใหม่
230	ไม่มี Supabase (โหมดทดลอง) → cloud_required
231–233	เรียก Edge Function parse-slip ส่ง JSON { imageBase64, mediaType } · supabase-js แบบ token ของผู้ใช้ให้เองอัตโนมัติ
234–235	มี error: ตั้งต้นว่าเป็นเน็ต (เรียกไม่ถึงเซิร์ฟเวอร์เลย)
236–239	ถ้าเซิร์ฟเวอร์ตอบรหัสผิดพลาด (FunctionsHttpError): อ่าน body JSON เอารหัส error.code และ HTTP status
240–246	แปลเป็นรหัสของแอป: รหัสที่รู้จัก 5 ตัวใช้ตามนั้น · ไม่มีรหัสและ status ≥ 500 = เกตเวย์ล่ม/หมดเวลา → 'server' (ระบบล่ม ไม่ใช่ความผิดของรูป) · นอกนั้น → 'reader_error' (รูปนี้มีปัญหา)
248	โยน SlipReaderError พร้อมข้อความไทย
250–251	สำเร็จ: ดึง reading และ check · reading ต้องมีและ isSlip ต้องเป็น boolean ไม่จั้นถือว่าอ่านไม่สำเร็จ (ไม่เชื่อข้อมูลที่หน้าตาผิด)
252	check ใช้ได้เมื่อ verified เป็น boolean ไม่จั้น null

บทที่ 29

อ่านโค้ด: processSlip.ts — รูป 1 รูป ตั้งแต่ต้นจน บันทึก

108 บรรทัด ใช้ร่วมกันทั้งหน้าสแกนและการสแกนอัตโนมัติ (เขียนครั้งเดียว ใช้ 2 ที่)

ส่วนที่ 1: หัวไฟล์และผลลัพธ์ (บรรทัด 1–31)

src/services/processSlip.ts

บรรทัด 1–31

```
1 /**
2  * FR-4: process ONE image into a transaction. Shared by the manual scan screen
3  * and the automatic scan that runs when the app opens.
4  *
5  * QR check (phone, free) -> shrink + hash -> slip reader -> rules -> save
6  */
7 import { bkkToIso } from '../domain/dates';
8 import {
9   addToIndex,
10  classifyCandidate,
11  initialStatus,
12  isScreenSized,
13  normalizeReading,
14  slipNote,
15  type DuplicateIndex,
16 } from '../domain/slip';
17 import { EMPTY_NAME_STATS } from '../domain/slipNames';
18 import type { ItemStatus } from '../domain/scanQueue';
19 import type { Transaction } from '../domain/types';
20 import { DuplicateSlipError, type Repo } from '../data/repo';
21 import { learnPayer, loadNames } from '../myNames';
22 import { detectSlipQr, prepareImage, READER_MESSAGES, readSlip, SlipReaderError } from './slips';
23
24 export interface ProcessResult {
25   status: Exclude<ItemStatus, 'queued' | 'working'>;
26   tx?: Transaction;
27   amountSatang?: number | null;
28   label?: string | null;
29   message?: string;
30   confirmed?: boolean;
31 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/processSlip.ts:1

บรรทัด	ความหมาย
5	สรุปย่อ: เช็ก QR (มือถือ ฟรี) → ย่อ + แชน → ตัวอ่านสลิป → กฎ → บันทึก
7–22	นำเข้ากฎ (slip.ts, slipNames.ts), repo ของ FR-1, ที่เก็บชื่อ และบริการใน slips.ts
24–31	ProcessResult : สถานะ (ทุกแบบยกเว้น queued/working), รายการที่บันทึก, ยอด, ชื่อ, ข้อความ, นับใบยอดแล้วไหม

ส่วนที่ 2: ข้อมูลเข้าและด้าน QR (บรรทัด 33–60)

src/services/processSlip.ts

บรรทัด 33–60

```
33 export async function processSlipImage(opts: {
34   uri: string;
35   width?: number | null;
36   height?: number | null;
37   /** Skip images without a Thai slip QR code (saves cost; photos never leave the phone). */
38   requireQr: boolean;
39   range: { from: string; to: string };
40   index: DuplicateIndex;
41   /** Checked right before saving: false when the account changed meanwhile (nothing is saved then). */
42   stillValid?: () => boolean;
43   repo: Repo;
44   autoConfirm: boolean;
45   /** Used as the transaction time when the slip date could not be read. */
46   fallbackTimeMs: number;
47   /** Called when the AI is busy and the reader waits before trying again. */
48   onWait?: (seconds: number) => void;
49   /** The signed-in account: its own name on slips is learned to tell income from expense. */
50   userId?: string | null;
51 }): Promise<ProcessResult> {
52   const size = { width: opts.width, height: opts.height };
53   const { qr, scannerWorks, otherQr } = await detectSlipQr(opts.uri, size);
54   // E-wallet slips (TrueMoney, ShopeePay...) carry their own kind of QR: a screen-sized
55   // picture with any QR code is read too, and the reader says whether it is a slip.
56   if (!qr && opts.requireQr && !(otherQr && isScreenSized(size))) {
57     // Without a working scanner every photo would look like "not a slip": say so instead.
58     if (!scannerWorks) throw new SlipReaderError('qr_unavailable',
59       READER_MESSAGES.qr_unavailable);
60     return { status: 'not_slip', message: 'ไม่พบ QR ของสลิป' };
61   }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/processSlip.ts:33

บรรทัด	ความหมาย
33–51	รับ "อ็อบเจกต์ตัวเลือก" ตัวเดียว (อ่านง่ายกว่าพารามิเตอร์ 11 ตัวเรียงกัน): uri และขนาดรูป · requireQr ต้องมี QR ใหม่ · range ช่วงวันที่ยอมรับ · index สมุดจำกันซ้ำ · stillValid ถามก่อนบันทึกว่ายังเป็นบัญชีเดิมไหม · repo ที่บันทึก · autoConfirm · fallbackTimeMs เวลาแทนเมื่ออ่านวันที่ไม่ได้ · onWait · userId
52–53	หา QR (ไม่เคยโยน error)
54–56	ด้านแรก: ไม่มี QR สลิป และ ต้องมี QR และ ไม่ใช่ (มี QR อื่น และ รูปขนาดหน้าจอ) → ไม่ผ่าน
57–58	ถ้าที่ไม่ผ่านเพราะ เครื่องไม่มีตัวสแกน → โยน qr_unavailable (บอกผู้ใช้ ไม่ใช่เจียบ)
59	ไม่ใช่สลิป: "ไม่พบ QR ของสลิป" (รูปไม่ออกจากเครื่อง)

ส่วนที่ 3: ย่อ อ่าน และตัดสินใจ (บรรทัด 62–78)

src/services/processSlip.ts

บรรทัด 62–78

```
62 const prepared = await prepareImage(opts.uri, opts.width);
```

```

63   if (opts.index.hashes.has(prepared.hash)) return { status: 'duplicate', message: 'รูปนี้เคยบันทึกแล้ว'
    };
64
65   const { reading, check } = await readSlip(prepared.base64, opts.onWait);
66   if (!reading.isSlip) return { status: 'not_slip', message: 'ไม่ใช่สลิปโอนเงิน' };
67
68   const names = opts.userId ? await loadNames(opts.userId) : EMPTY_NAME_STATS;
69   const c = normalizeReading(reading, { qr, imageHash: prepared.hash, names });
70   // Most saved slips are the user's own payments: their payer line teaches the user's name.
71   if (opts.userId && reading.direction === 'expense' && reading.fromName) await
    learnPayer(opts.userId, reading.fromName);
72   const outcome = classifyCandidate(c, { range: opts.range, index: opts.index });
73   const label = c.ownTransfer ? 'โอนระหว่างบัญชีตัวเอง' : (c.counterparty ?? 'รายการจากสลิป');
74   if (outcome === 'duplicate') return { status: 'duplicate', amountSatang: c.amountSatang, label,
    message: 'มีรายการนี้แล้ว' };
75   if (outcome === 'out_of_range') return { status: 'out_of_range', amountSatang: c.amountSatang,
    label, message: 'วันที่ยื่นสลิปต่ำกว่า 1 ปี จึงไม่บันทึกอัตโนมัติ' };
76   if (c.amountSatang === null) return { status: 'failed', label, message: 'อ่านยอดเงินไม่ได้ ลองจดย่อยการ
    นี้เอง' };
77
78   const status = initialStatus(outcome, opts.autoConfirm);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/processSlip.ts:62

บรรทัด	ความหมาย
62	ย่อรูปและทำแฮช
63	แฮชเคยเห็น → ซ้ำ ก่อนส่ง AI ("รูปนี้เคยบันทึกแล้ว") ประหยัคค่า AI
65	ส่งให้เซิร์ฟเวอร์อ่าน (รอและลองใหม่เองเมื่อ AI ยุ่ง)
66	AI บอกว่าไม่ใช่สลิป → "ไม่ใช่สลิปโอนเงิน"
68	โหลดสถิติชื่อของบัญชีนี้ (ไม่มีบัญชีใช้สถิติว่าง)
69	แปลงผลอ่านเป็นผู้สมัคร พร้อม QR และแฮช
70–71	สลิปที่ AI ว่าเป็นรายจ่ายและมีชื่อผู้โอน → เรียนชื่อผู้โอน (ส่วนใหญ่คือผู้ใช้เอง)
72	ตัดสินใจ (ซ้ำ / นอกช่วง / พร้อม / ต้องตรวจ)
73	ชื่อรายการ: โอนเข้าบัญชีตัวเอง → "โอนระหว่างบัญชีตัวเอง" · มีชื่ออีกฝ่าย → ใช้ชื่อนั้น · ไม่มี → "รายการจากสลิป"
74	ซ้ำ → "มีรายการนี้แล้ว"
75	นอกช่วง → "วันที่ยื่นสลิปต่ำกว่า 1 ปี จึงไม่บันทึกอัตโนมัติ"
76	ไม่มียอดเลย → failed "อ่านยอดเงินไม่ได้ ลองจดย่อยการนี้เอง" (รายการที่ไม่มียอดบันทึกไม่ได้ เพราะตารางบังคับยอด > 0)
78	สถานะเริ่มต้น (confirmed หรือ draft)

ส่วนที่ 4: บันทึก (บรรทัด 79–108)

src/services/processSlip.ts

บรรทัด 79–108

```

79   if (opts.stillValid && !opts.stillValid()) throw new SlipReaderError('cancelled', 'หยุดอ่านสลิปแล้ว');
80   try {
81     const tx = await opts.repo.insert({
82       kind: c.kind,
83       amountSatang: c.amountSatang,
84       categoryKey: c.categoryKey,
85       title: label,
86       note: slipNote(c, check),
87       occurredAt: c.dayKey ? bkkToIso(c.dayKey, c.time) : new
Date(opts.fallbackTimeMs).toISOString(),
88       source: 'slip',
89       status,
90       slipRef: c.ref,
91       slipImageHash: c.imageHash,
92       ocrConfidence: Math.round(c.confidence * 100) / 100,
93       reviewFlags: c.flags,
94     });
95     addToIndex(opts.index, tx);
96     return {
97       status: outcome,
98       tx,
99       amountSatang: c.amountSatang,
100      label,
101      confirmed: status === 'confirmed',
102      message: status === 'confirmed' ? 'รวมในยอดเงินแล้ว' : undefined,
103    };
104   } catch (e) {
105     if (e instanceof DuplicateSlipError) return { status: 'duplicate', amountSatang:
c.amountSatang, label, message: 'มีรายการนี้แล้ว' };
106     throw e;
107   }
108 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/processSlip.ts:79

บรรทัด	ความหมาย
79	เช็ครั้งสุดท้ายก่อนบันทึก: ถ้าระหว่างรอ AI ผู้ใช้สลับบัญชี → ยกเลิก (ไม่บันทึกสลิปของคนหนึ่งลงบัญชีของอีกคน)
81–94	repo.insert (ฟังก์ชันเดียวกับ FR-1): ประเภท, ยอด, หมวด, ชื่อ, บันทึก (slipNote), เวลา (วันจากสลิป + เวลา หรือ เวลาของรูป), source 'slip', สถานะ, slip_ref, slip_image_hash, ความมั่นใจ (ปัดทศนิยม 2 ตำแหน่ง), ธง
87	เวลา: มี dayKey → bkkToIso(วัน, เวลา) ไม่มี → เวลาที่รูปถูกสร้าง
92	Math.round(c.confidence * 100) / 100 = ปัดเป็นทศนิยม 2 ตำแหน่ง (0.8999 → 0.9)
95	เพิ่มรายการใหม่ลงสมุดจำ ใบถัดไปในรอบเดียวกันจะเห็น (สลิปเดียวกัน 2 รูปในรอบเดียวกันจะไม่หลุด)
96–103	คืนผล ถ้าบันทึกในยอดแล้วข้อความ "รวมในยอดเงินแล้ว"
104–106	ฐานข้อมูลปฏิเสธเพราะซ้ำ (unique index) → นับเป็น "ซ้ำ" ไม่ใช่ error
107	error อื่น → โยนต่อให้ผู้เรียกจัดการ (หยุดหรือข้าม)

อ่านโค้ด: useSlipScanner.ts — ตัวควบคุมของหน้าสแกน

hook 240 บรรทัดที่เชื่อม state machine (กฎ) เข้ากับงานจริงบนมือถือ ทีละรูป

Hook ใน React คือฟังก์ชันที่ชื่อขึ้นต้นด้วย `use` ซึ่งเก็บ state และทำงานต่อเนื่องให้หน้าจอลoad หน้าสแกนเรียก `useSlipScanner('1m')` แล้วได้ทุกอย่างที่ต้องใช้: สถานะ ปุ่มต่าง ๆ และผลของรอบนี้ หน้าจอจึงเหลือแค่หน้าที่ "แสดงผล"

ส่วนที่ 1: หัวไฟล์ import และค่าคงที่ (บรรทัด 1–38)

src/services/useSlipScanner.ts

บรรทัด 1–38

```
1 /**
2  * FR-4 scanner controller. Runs the pure state machine from
3  * src/domain/scanQueue.ts and does the phone work for one image at a time.
4  *
5  * Every slip is kept under the date printed on it (up to one year back); the
6  * user then looks at it by period (วันนี้ / 7 วัน / ...) instead of choosing a
7  * range before scanning. The range here only decides how far back in the
8  * gallery to look.
9  *
10 * The state lives in a ref as well as React state so the processing loop
11 * always sees the latest phase (pause takes effect before the next image).
12 */
13 import * as ImagePicker from 'expo-image-picker';
14 import { useCallback, useEffect, useRef, useState } from 'react';
15 import { rangeDays, rangeStartMs } from '../domain/dates';
16 import { buildDuplicateIndex, type DuplicateIndex } from '../domain/slip';
17 import { initialScanState, nextQueued, scanReducer, type ScanAction, type ScanItem, type
18   ScanState } from '../domain/scanQueue';
19 import type { RangeKey } from '../domain/types';
20 import { useApp } from '../data/AppProvider';
21 import { loadScanPrefs } from '../data/prefs';
22 import { processSlipImage } from '../processSlip';
23 import {
24   findGalleryImages,
25   galleryUri,
26   loadScannedIds,
27   rememberScanned,
28   scanLock,
29   requestGalleryAccess,
30   SlipReaderError,
31   type GalleryAccess,
32 } from './slips';
33 type Finished = Extract<ScanAction, { type: 'itemFinished' }>;
34
35 /** Slips older than this (by the date printed on them) are not recorded automatically. */
36 const SLIP_WINDOW: RangeKey = '1y';
37 /** Wait before trying an image again after a network error. */
38 const NETWORK_RETRY_MS = 3000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:1

บรรทัด	ความหมาย
2-3	รับ state machine จาก scanQueue.ts และทำงานฝั่งมือถือทีละรูป
5-8	ทุกสลิปเก็บตามวันที่ยื่นสลิป (ย้อนได้ 1 ปี) ผู้ใช้ดูผลตามช่วงทีหลัง ไม่ต้องเลือกช่วงก่อนสแกน ช่วงในไฟล์นี้มีไว้บอกแล้วว่า "ค้นเจอแล้วหรือยังไปแค่นี้"
10-11	สถานะถูกเก็บทั้งใน ref และ React state เพื่อให้ลูปที่กำลังทำงานเห็นค่าล่าสุดเสมอ (กดหยุดแล้วมีผลก่อนรูปถัดไป)
13	expo-image-picker: เปิดตัวเลือกรูปของระบบ
14	hook พื้นฐานของ React 4 ตัว
33	<code>Finished</code> = ชนิดของคำสั่ง <code>itemFinished</code> (<code>Extract</code> = ดึงสมาชิกที่ตรงเงื่อนไขออกจาก union)
36	SLIP_WINDOW = '1y' สลิปที่วันที่ยื่นสลิปต่ำกว่า 1 ปีไม่บันทึกอัตโนมัติ
38	เน็ตหลุด รอ 3,000 ms ก่อนลองรูปเดิมอีกครั้ง

≈ ref กับ state ต่างกันยังไง

state เหมือนกระดานหน้าห้อง: เขียนใหม่เมื่อไร ทุกคนในห้อง (หน้าจอ) เห็นและวาดใหม่ แต่คนที่ออกไปทำงานนอกห้อง (ลูปที่กำลังรอ AI) จะจำค่าตอนที่ออกไป · **ref** เหมือนโน้ตในกระเป๋: เปลี่ยนแล้วหน้าจอไม่วาดใหม่ แต่ใครหยิบมาดูก็เห็นค่าล่าสุดเสมอ ลูปสแกนจึงอ่านจาก `stateRef.current` ส่วนหน้าจออ่านจาก `state`

ส่วนที่ 2: state, ref และ apply (บรรทัด 40-80)

src/services/useSlipScanner.ts

บรรทัด 40-80

```

40 export function useSlipScanner(initialRange: RangeKey = '1m') {
41   const { repo, userId, txs, upsertLocal } = useApp();
42   const [state, setState] = useState<ScanState>(() => initialScanState(initialRange));
43   const stateRef = useRef(state);
44   const [access, setAccess] = useState<GalleryAccess | null>(null);
45   const [finding, setFinding] = useState(false);
46   /** A gallery search has finished at least once (to tell "nothing new" from "not searched yet"). */
47   const [searched, setSearched] = useState(false);
48   const [skippedKnown, setSkippedKnown] = useState(0);
49   const [requireQr, setRequireQr] = useState(true);
50   const [notice, setNotice] = useState<string | null>(null);
51   /** Set while the reader waits for the AI's rate limit (free tier): when it tries again (ms). */
52   const [waiting, setWaiting] = useState<number | null>(null);
53   /** Transactions recorded by this scan, to show them grouped by date. */
54   const [runTxIds, setRunTxIds] = useState<string[]>([]);
55
56   const txsRef = useRef(txs);
57   useEffect(() => {
58     txsRef.current = txs;
59   }, [txs]);
60   const indexRef = useRef<DuplicateIndex>(buildDuplicateIndex([]));
61   const pickedRef = useRef(new Map<string, { width: number | null; height: number | null }>());
62   /** Picture sizes of gallery photos in the queue (screen-sized ones get a closer QR check). */
63   const sizesRef = useRef(new Map<string, { width: number | null; height: number | null }>());

```

```

64 const scannedRef = useRef<Set<string>>(new Set());
65 const loopRef = useRef(false);
66 const mounted = useRef(true);
67
68 const apply = useCallback((a: ScanAction) => {
69   stateRef.current = scanReducer(stateRef.current, a);
70   if (mounted.current) setState(stateRef.current);
71 }, []);
72
73 useEffect(() => {
74   mounted.current = true;
75   return () => {
76     mounted.current = false;
77     // Leaving the screen pauses the scan; drafts already saved stay saved.
78     if (stateRef.current.phase === 'running') stateRef.current = scanReducer(stateRef.current,
79   { type: 'pause' });
80   };
81 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:40

บรรทัด	ความหมาย
41	จาก AppProvider: repo, userId, รายการทั้งหมด, ฟังก์ชันใส่รายการลงหน้าจอทันที
42–43	สถานะของคิว (state) และสำเนาใน ref · <code>useState(() => ...)</code> แบบส่งฟังก์ชัน = คำนวณค่าเริ่มต้นครั้งเดียว
44–54	state อื่น ๆ ของหน้าจอ: สิทธิรูปภาพ · กำลังค้นเกลเลอรีใหม่ · ค้นเสร็จอย่างน้อย 1 ครั้งหรือยัง (แยก "ไม่มีรูปใหม่" กับ "ยังไม่ได้ค้น") · จำนวนรูปที่ข้ามเพราะเคยตรวจ · ต้องมี QR ใหม่ · ข้อความแจ้งปัญหา · เวลาที่จะลองใหม่เมื่อ AI ยุ่ง · รหัสรายการที่บันทึกในรอบนี้
56–59	เก็บรายการล่าสุดใน ref ทุกครั้งที่เปลี่ยน (ลูปจะสร้างสมุดกันซ้ำจากคำนี้)
60	สมุดกันซ้ำ
61	รูปที่ผู้ใช้เลือกเอง: uri → ขนาด (Map)
63	ขนาดของรูปจากเกลเลอรีในคิว (รูปขนาดหน้าจอได้ดู QR ละเอียด)
64	ชุดรหัสรูปที่เคยตรวจ
65	<code>loopRef</code> ลูปกำลังทำงานอยู่ไหม (กันเริ่มลูปซ้อน 2 ตัว)
66	<code>mounted</code> หน้าจอยังเปิดอยู่ไหม (ปิดแล้วห้าม setState)
68–71	apply หัวใจของไฟล์: ส่งคำสั่งเข้า reducer ได้สถานะใหม่ เก็บใน ref กันที แล้วบอก React ให้วาดใหม่ (ถ้าหน้าจอยังเปิด) · <code>useCallback(..., [])</code> = สร้างฟังก์ชันครั้งเดียว ไม่สร้างใหม่ทุกครั้งที่วาดหน้าจอ
73–80	effect ที่ทำงานตอนเปิดหน้าจอ และ ฟังก์ชันที่ return คือ "ตอนปิด": ปิดหน้าจอระหว่างสแกน → สั่ง pause (แบบร่างที่บันทึกแล้วยังอยู่)

ส่วนที่ 3: ใส่รูปเข้าคิว 2 ทาง (บรรทัด 82–128)

src/services/useSlipScanner.ts

บรรทัด 82–128

```

82 const setRange = (range: RangeKey) => apply({ type: 'setRange', range });
83

```

```

84  /** Queue gallery photos taken in the last `range`, skipping ones checked before. Starts at
    once. */
85  const loadFromGallery = useCallback(async (range?: RangeKey) => {
86      setNotice(null);
87      const a = await requestGalleryAccess();
88      setAccess(a);
89      if (a === 'denied' || a === 'blocked') return;
90      if (range && range !== stateRef.current.range) apply({ type: 'setRange', range });
91      setFinding(true);
92      try {
93          const since = rangeStartMs(stateRef.current.range);
94          const [images, scanned] = await Promise.all([findGalleryImages(since), userId ?
loadScannedIds(userId) : new Set<string>()]);
95          scannedRef.current = scanned;
96          const fresh = images.filter((i) => !scanned.has(i.assetId));
97          setSkippedKnown(images.length - fresh.length);
98          pickedRef.current.clear();
99          sizesRef.current = new Map(fresh.map((i) => [i.assetId, { width: i.width, height: i.height
})));
100         setRunTxIds([]);
101         apply({ type: 'load', items: fresh.map((i) => ({ assetId: i.assetId, createdAt: i.createdAt
})) });
102         apply({ type: 'start' });
103     } catch {
104         setNotice('เปิดแกลเลอรีไม่สำเร็จ ลองใหม่อีกครั้ง');
105     } finally {
106         setFinding(false);
107         setSearched(true);
108     }
109 }, [apply, userId]);
110
111 /** Queue photos the user picks by hand (always read, QR or not). Starts at once. */
112 const loadPicked = useCallback(async () => {
113     setNotice(null);
114     const res = await ImagePicker.launchImageLibraryAsync({
115         mediaTypes: ['images'],
116         allowsMultipleSelection: true,
117         selectionLimit: 30,
118         orderedSelection: true,
119         quality: 1,
120     });
121     if (res.canceled || res.assets.length === 0) return;
122     pickedRef.current = new Map(res.assets.map((a) => [a.uri, { width: a.width ?? null, height:
a.height ?? null }]));
123     setSkippedKnown(0);
124     setRunTxIds([]);
125     const now = Date.now();
126     apply({ type: 'load', items: res.assets.map((a, i) => ({ assetId: a.uri, createdAt: now - i
})) });
127     apply({ type: 'start' });
128 }, [apply]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:82

บรรทัด ความหมาย

82 setRange แคลส่งคำสั่ง (reducer ตัดสินเองว่าเปลี่ยนได้ไหม)

85 loadFromGallery (มือถือ) รับช่วงที่ต้องการค้นได้

บรรทัด	ความหมาย
86–89	ล้างข้อความเดิม ขอสิทธิ์ (ขึ้นกล่องถามถ้ายังไม่เคย) ไม่ได้ → หยุด (หน้าจจะแสดงการ์ด "ยังไม่ได้อนุญาต...")
90	ถ้าขอช่วงใหม่ (เช่น "ค้นหาย้อนหลัง 1 ปี") ส่งคำสั่งเปลี่ยนช่วง
93	เวลาเริ่มของช่วง (เช่น 1 เดือน = 30 วันเต็มนับรวมวันนี้)
94	ทำ 2 อย่างพร้อมกัน ด้วย <code>Promise.all</code> : ค้นรูป และโหลดรายการรูปที่เคยตรวจ (ไม่มีบัญชีใช้ Set ว่าง)
96–97	ตัดรูปที่เคยตรวจ และนับจำนวนที่ตัดไว้แสดง "ทุกรูปเคยตรวจแล้ว (6 รูป)"
98–100	ล้างรูปที่เลือกเองของรอบก่อน เก็บขนาดของรูปในคิว ล้างผลรอบก่อน
101–102	ส่ง load (คิวใหม่ เลขรอบใหม่) แล้ว start กันก็ ไม่ต้องกดปุ่มเริ่ม
103–108	เปิดเคลเลอร์ฟัง → ข้อความ · ทุกกรณีปิด "กำลังค้น" และจำว่าค้นแล้ว (<code>finally</code> ทำเสมอไม่ว่าสำเร็จหรือพัง)
112	loadPicked (ทั้งเว็บและมือถือ) ให้ผู้ใช้เลือกรูปเอง
114–120	เปิดตัวเลือกรูป: เฉพาะรูปภาพ เลือกหลายรูป สูงสุด 30 ตามลำดับที่เลือก คุณภาพเต็ม (เดี่ยวย่อเอง)
121	ยกเลิกหรือไม่ได้เลือก → จบ
122	จำนวนของรูปที่เลือก (คีย์คือ uri)
125–126	ใช้ uri เป็นรหัสรูป และให้ "เวลา" ลดหลั่นตามลำดับ (<code>now - i</code>) เพื่อให้ reducer ที่เรียงใหม่ไปเท่า ทำตามลำดับที่ผู้ใช้เลือก
127	เริ่มกันก็

ส่วนที่ 4: processOne — ทำ 1 รูป (บรรทัด 130–163)

src/services/useSlipScanner.ts

บรรทัด 130–163

```

130  async function processOne(item: ScanItem): Promise<Omit<Finished, 'type' | 'runId' |
    'assetId'>> {
131    const picked = pickedRef.current.get(item.assetId);
132    const size = picked ?? sizesRef.current.get(item.assetId);
133    const prefs = await loadScanPrefs(userId);
134    const r = await processSlipImage({
135      uri: picked ? item.assetId : await galleryUri(item.assetId),
136      width: size?.width,
137      height: size?.height,
138      requireQr: requireQr && !picked,
139      range: rangeDays(SLIP_WINDOW),
140      index: indexRef.current,
141      repo: repo!,
142      autoConfirm: prefs.autoConfirm,
143      fallbackTimeMs: item.createdAt,
144      onWait: (seconds) => {
145        if (mounted.current) setWaiting(Date.now() + seconds * 1000);
146      },
147      userId,
148    });
149    if (mounted.current) setWaiting(null);
150    if (r.tx) {
151      upsertLocal(r.tx);
152      const id = r.tx.id;

```

```

153     if (mounted.current) setRunTxIds((ids) => [...ids, id]);
154   }
155   return {
156     status: r.status,
157     txId: r.tx?.id,
158     amountSatang: r.amountSatang,
159     label: r.label,
160     message: r.message,
161     confirmed: r.confirmed,
162   };
163 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:130

บรรทัด	ความหมาย
130	คืนผลที่พร้อมใส่ในคำสั่ง itemFinished (ไม่รวม type, runId, assetId ซึ่งลูปใส่เอง: Omit)
131–132	รูปที่เลือกเองหรือรูปจากแกลเลอรี หายขนาดจากที่จำไว้
133	อ่านตั้งค่าล่าสุดทุกรูป (ผู้ใช้อาจเพิ่งเปลี่ยนสวิตช์ "รวมยอดทันที")
134–148	เรียก processSlipImage (บกก่อน): uri (รูปแกลเลอรีต้องขอ uri ก่อน) · ต้องมี QR เฉพาะรูปที่ ไม่ได้ เลือกเอง · ช่วง 1 ปี · สมุดกินซ้ำ · onWait ตั้งเวลานับถอยหลังบนหน้าจอ
149	อ่านเสร็จ ล้างการนับถอยหลัง
150–154	ได้รายการ → ใส่ลงข้อมูลของแอปทันที (ยอดเงินหน้าหลักเปลี่ยนเลยโดยไม่ต้องโหลดใหม่) และเพิ่มรหัสลงผลของรอบนี้ · setRunTxIds((ids) => [...ids, id]) ใช้แบบฟังก์ชัน เพื่อเพิ่มต่อจากค่าล่าสุดเสมอ
155–162	คืนผลสำหรับ reducer

ส่วนที่ 5: runLoop – ลูปที่ทำที่ละรูปจนหมด (บรรทัด 165–211)

src/services/useSlipScanner.ts

บรรทัด 165–211

```

165 const runLoop = useCallback(async () => {
166   if (loopRef.current) return;
167   loopRef.current = true;
168   scanLock.manual = true;
169   indexRef.current = buildDuplicateIndex(txRef.current);
170   try {
171     while (stateRef.current.phase === 'running') {
172       const item = nextQueued(stateRef.current);
173       if (!item) break;
174       const runId = stateRef.current.runId;
175       apply({ type: 'itemStarted', runId, assetId: item.assetId });
176       try {
177         let result: Awaited<ReturnType<typeof processOne>>;
178         try {
179           result = await processOne(item);
180         } catch (e) {
181           if (!(e instanceof SlipReaderError && e.code === 'network')) throw e;
182           // A short drop in the connection: try this image once more before stopping.
183           await new Promise((resolve) => setTimeout(resolve, NETWORK_RETRY_MS));
184           result = await processOne(item);
185         }
186         apply({ type: 'itemFinished', runId, assetId: item.assetId, ...result });

```

```

187         if (!pickedRef.current.has(item.assetId) && userId)
scannedRef.current.add(item.assetId);
188     } catch (e) {
189         if (mounted.current) setWaiting(null);
190         if (e instanceof SlipReaderError && e.code !== 'reader_error' && e.code !==
'too_large') {
191             // Problems that affect every image (offline, not configured, AI busy, quota, signed
out):
192             // stop and tell the user. This image is retried on resume.
193             setNotice(e.message);
194             apply({ type: 'pause' });
195             break;
196         }
197         const message = e instanceof SlipReaderError ? e.message : 'เกิดข้อผิดพลาดกับรูปนี้';
198         apply({ type: 'itemFinished', runId, assetId: item.assetId, status: 'failed', message
});
199     }
200 }
201 } finally {
202     loopRef.current = false;
203     scanLock.manual = false;
204     if (userId) rememberScanned(userId, scannedRef.current);
205 }
206 // eslint-disable-next-line react-hooks/exhaustive-deps
207 }, [apply, userId, requireQr, repo]);
208
209 useEffect(() => {
210     if (state.phase === 'running') runLoop();
211 }, [state.phase, runLoop]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:165

บรรทัด	ความหมาย
166–167	มีรูปทำงานอยู่แล้ว → ไม่เริ่มตัวที่ 2 · ไม่ขึ้นจองว่ากำลังทำ
168	ล๊อค: บอกการสแกนอัตโนมัติให้รอ
169	สร้างสมุดกันข้าจากรายการล่าสุด
171	วนรอบใดที่ยัง running (อ่านจาก ref กดหยุดแล้วรูปเห็นทันทีก่อนรูปถัดไป)
172–173	รูปถัดไป ไม่มี → ออก
174–175	จำนวนรอบ ตอนเริ่มรูปนี้ แล้วส่ง itemStarted
178–185	ลองทำรูปนี้ · ถ้าพังเพราะ เน็ต → รอ 3 วินาทีแล้วลองอีกครั้ง (error อันไหนต่อ)
186	ส่ง itemFinished ด้วย เลขรอบที่จำไว้ ถ้าระหว่างนี้รอบเปลี่ยน (reset/load ใหม่) reducer จะทิ้งผลนี้
187	รูปจากแคชเลอริ (ไม่ใช่ที่เลือกเอง) → จำว่าตรวจแล้ว
188–196	ปัญหาที่เกิดกับทุกรูป (ถูกระหีบยกเว้น reader_error และ too_large): แสดงข้อความ สั่ง pause แล้วออกจากกลุ่ม รูปนี้ยังค้าง working จึงถูกทำใหม่ตอนกด "อ่านต่อ"
197–198	ปัญหาของรูปนี้รูปเดียว: บันทึกเป็น failed พร้อมเหตุผล แล้ววนไปรูปถัดไป
201–205	finally : ปลดรูป ปลดล๊อค บันทึกรายการรูปที่ตรวจแล้ว (ทำเสมอ แม้รูปจบเพราะ error)

บรรทัด ความหมาย

209–211 effect: ทุกครั้งที่ phase เป็น running ให้เริ่มลูป (ทั้งตอน start และ resume)

ส่วนที่ 6: สิ่งที่ hook ส่งให้หน้าจอ (บรรทัด 213–240)

src/services/useSlipScanner.ts

บรรทัด 213–240

```
213   return {
214     state,
215     access,
216     finding,
217     searched,
218     skippedKnown,
219     requireQr,
220     setRequireQr,
221     notice,
222     waiting,
223     runTxIds,
224     setRange,
225     loadFromGallery,
226     loadPicked,
227     start: () => apply({ type: 'start' }),
228     pause: () => apply({ type: 'pause' }),
229     resume: () => {
230       setNotice(null);
231       apply({ type: 'resume' });
232     },
233     reset: () => {
234       setNotice(null);
235       setSkippedKnown(0);
236       setRunTxIds([]);
237       apply({ type: 'reset' });
238     },
239   };
240 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/services/useSlipScanner.ts:213

บรรทัด ความหมาย

214–223 คำสำหรับแสดงผล: สถานะ สิทธิ์ กำลังค้น ค้นแล้ว จำนวนที่ข้าม ต้องมี QR ข้อความ เวลา รอ รหัสรายการของรอบนี้

224–228 คำสั่ง: เปลี่ยนช่วง ค้นเคลเลอร์ เลือกรูปเอง เริ่ม หยุด

229–232 **resume** ล้างข้อความปัญหา ก่อน แล้วส่ง resume

233–238 **reset** ล้างทุกอย่าง (เลขรอบใหม่ ผลของลูปเก่าที่อาจกลับมาจะถูกทิ้ง)

อ่านโค้ด: AutoScanProvider.tsx — สแกนอัตโนมัติตอนเปิดแอป

321 บรรทัด ไฟล์ที่ซับซ้อนที่สุดของ FR-4 เพราะต้องทำงานเองโดยไม่มีผู้ใช้คอยดู

Provider ครอบทั้งแอป (ใส่ไว้ใน `src/app/_layout.tsx`) ทำงานเบื้องหลังและแชร์สถานะผ่าน **Context** ให้ทุกหน้าจออ่านได้: การ์ดบนหน้าหลัก ปุ่มกลางแถบเมนู หน้าสแกน และหน้าตั้งค่า

ส่วนที่ 1: คอมเมนต์ออกแบบ และคำคงที่ (บรรทัด 1–51)

`src/services/AutoScanProvider.tsx`

บรรทัด 1–51

```
1 /**
2  * FR-4 automatic scan: when the app opens (and each time it comes back to the
3  * foreground, at most every 5 minutes), look for NEW slips in the gallery,
4  * read them, and update the balance right away.
5  *
6  * - Only photos with a Thai slip QR code are read (cost + privacy).
7  * - Slips read with ≥ 80% confidence on every field count immediately when
8  *   auto-confirm is on; unsure ones wait in "สลิปรอยืนยัน".
9  * - Never shows the permission prompt by itself: if access is missing, the home
10 *   screen offers a button instead.
11 * - Nothing is skipped for good by accident: when the phone is offline or the AI
12 *   is unavailable the run stops without moving its window forward, and a photo
13 *   that could not be read is tried again next run (MAX_AUTO_ATTEMPTS in all).
14 *   Two unreadable photos in a row look like the reader being down: the run stops
15 *   early (each still counts as one try). A photo that keeps stopping whole runs
16 *   (busy or timed-out reader) is left for the scan screen after STALL_LIMIT runs,
17 *   so one bad photo can never hold up the photos behind it.
18 * - Each account has its own window, and a run stops (saving nothing more) when
19 *   the account changes. It also waits while the scan screen is reading photos.
20 */
21 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
22 import { AppState } from 'react-native';
23 import { autoScanSince, MAX_AUTO_ATTEMPTS, MAX_IMAGES_PER_RUN, nextAutoScanMark, shouldAutoScan }
  from '../domain/autoScan';
24 import { rangeDays } from '../domain/dates';
25 import { buildDuplicateIndex } from '../domain/slip';
26 import type { Transaction } from '../domain/types';
27 import { useApp } from '../data/AppProvider';
28 import { DEFAULT_SCAN_PREFS, loadScanPrefs, saveScanPrefs, type ScanPrefs } from '../data/prefs';
29 import { processSlipImage } from '../processSlip';
30 import {
31   findGalleryImages,
32   galleryAvailable,
33   galleryUri,
34   getGalleryAccess,
35   loadScanFailures,
36   loadScannedIds,
37   rememberScanned,
38   saveScanFailures,
39   requestGalleryAccess,
```

```

40  scanLock,
41  SlipReaderError,
42 } from './slips';
43
44 /** Wait before trying a photo again after a network error. */
45 const NETWORK_RETRY_MS = 3000;
46 /** The automatic scan starts this long after the app is ready (the opening animation takes about
47    1.8 s). */
48 const START_DELAY_MS = 1800;
49 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
50 const READER_DOWN = 'ระบบอ่านสลิปขัดข้องชั่วคราว สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า';
51 /** Runs one photo may stop (busy or timed-out reader) before the automatic scan leaves it for
52    the scan screen. */
53 const STALL_LIMIT = 3;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:1

บรรทัด	ความหมาย
2-4	เมื่อเปิดแอป (และทุกครั้งที่กลับมาหน้าแอป อย่างมากทุก 5 นาที) หาสลิปใหม่ อ่าน และอัปเดตยอดทันที
6	อ่านเฉพาะรูปที่มี QR สลิปไทย (ต้นทุน + ความเป็นส่วนตัว)
7-8	สลิปที่ $\geq 80\%$ ทุกช่องนับทันทีเมื่อเปิดรวมยอดทันที ไม่ชดรอใน "สลิปรอยืนยัน"
9-10	ไม่ขึ้นกล้องขอสิทธิ์เอง (ยกเว้นครั้งแรกครั้งเดียว) ถ้าไม่มีสิทธิ์ หน้าหลักมีปุ่มให้กดแทน
11-17	ไม่มีอะไรถูกข้ามโดยบังเอิญ: ออฟไลน์หรือ AI ใช้ไม่ได้ → หยุดโดยไม่เลื่อนหน้าต่างเวลา · รูปที่อ่านไม่ได้ลองใหม่รอบหน้า (รวม MAX_AUTO_ATTEMPTS) · อ่านไม่ได้ 2 รูปติด = ตัวอ่านน่าจะล่ม หยุดก่อน · รูปที่ทำให้หยุดซ้ำ ๆ (STALL_LIMIT รอบ) ปลอ่ยให้หน้าสแกน รูปเดียวจะได้ไม่ขวางรูปข้างหลัง
18-19	แต่ละบัญชีมีหน้าต่างเวลาของตัวเอง สลับบัญชีแล้วรอบที่ค้างหยุด (ไม่บันทึกอะไรเพิ่ม) และรอระหว่างหน้าสแกนทำงาน
22	AppState ของ React Native บอกว่าแอปอยู่หน้าจอ (active) หรือถูกพับไว้ (background)
45	เน็ตหลุด รอ 3 วินาที
47	START_DELAY_MS = 1800 เริ่มหลังแอปพร้อม 1.8 วินาที (แอนิเมชันเปิดแอปใช้ราว 1.8 วินาที) การเปิดแอปจึงยังเร็ว
48	sleep ฟังก์ชันรอ ms ที่กำหนด
49	ข้อความเมื่อตัวอ่านน่าจะล่ม
51	STALL_LIMIT = 3

ส่วนที่ 2: ชนิดข้อมูล และ Context (บรรทัด 53-87)

src/services/AutoScanProvider.tsx

บรรทัด 53-87

```

53 export type AutoScanPhase = 'idle' | 'needs_permission' | 'scanning' | 'done' | 'error';
54
55 export interface AutoScanState {
56   phase: AutoScanPhase;
57   total: number;
58   processed: number;
59   /** Transactions counted in the balance by this run. */
60   confirmed: Transaction[];

```

```

61  /** Slips saved as drafts (need a check) by this run. */
62  drafts: number;
63  message: string | null;
64  /** While the AI is rate-limited: when the reader tries again (ms). */
65  waitUntil: number | null;
66 }
67
68 interface AutoScanContextValue {
69   state: AutoScanState;
70   prefs: ScanPrefs;
71   available: boolean;
72   setPrefs(patch: Partial<Omit<ScanPrefs, 'lastAutoScanAt'>>): Promise<void>;
73   grantAndRun(): Promise<void>;
74   runNow(): Promise<void>;
75   /** Move this run's auto-counted slips back to "รอยืนยัน". */
76   undoLast(): Promise<void>;
77   dismiss(): void;
78 }
79
80 const IDLE: AutoScanState = { phase: 'idle', total: 0, processed: 0, confirmed: [], drafts: 0,
  message: null, waitUntil: null };
81 const AutoScanContext = createContext<AutoScanContextValue | null>(null);
82
83 export function useAutoScan() {
84   const ctx = useContext(AutoScanContext);
85   if (!ctx) throw new Error('useAutoScan must be used inside AutoScanProvider');
86   return ctx;
87 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:53

บรรทัด	ความหมาย
53	5 phase ของการ์ด (ตรงกับตาราง 5 หน้าตาในบทกวีหน้าจอ)
55–66	AutoScanState : phase, จำนวนรูปในรอบ, ทำไปแล้ว, รายการที่ นับในยอดแล้ว (เก็บตัวรายการ เพราะปุ่ม "ยังไม่รวมยอด" ต้องรู้ว่าจะย้อนอันไหน), จำนวนแบบร่าง, ข้อความ, เวลาที่จะลองใหม่
68–78	สิ่งที่ Context ให้หน้าจอ: สถานะ ตั้งค่า ใช้ได้ไหม และคำสั่ง setPrefs, grantAndRun, runNow, undoLast, dismiss
80	สถานะว่าง (ใช้ซ้ำหลายที่)
81	สร้าง Context ค่าเริ่มเป็น null
83–87	useAutoScan hook สำหรับหน้าจอ ถ้าเรียกนอก Provider โยน error ที่อ่านเข้าใจทันที (ดีกว่าฟังก์ชันแบบงง ๆ ที่หลัง)

ส่วนที่ 3: Provider เริ่มต้น และการสลับบัญชี (บรรทัด 89–121)

src/services/AutoScanProvider.tsx

บรรทัด 89–121

```

89 export function AutoScanProvider({ children }: { children: ReactNode }) {
90   const { status, repo, userId, txs, upsertLocal, updateTx, profile } = useApp();
91   // Only once the user is in the app (not during sign-up or money setup).
92   const inApp = status === 'ready' && !!profile?.onboarded;
93   const [state, setState] = useState<AutoScanState>(IDLE);
94   const [prefs, setPrefsState] = useState<ScanPrefs>(DEFAULT_SCAN_PREFS);
95   const running = useRef(false);
96   const txsRef = useRef(txs);

```

```

97 // The app's actions change on every data change; read them through refs so a new
98 // transaction does not start the scan again.
99 const upsertRef = useRef(upsertLocal);
100 const updateTxRef = useRef(updateTx);
101 useEffect(() => {
102   txsRef.current = txs;
103   upsertRef.current = upsertLocal;
104   updateTxRef.current = updateTx;
105 }, [txs, upsertLocal, updateTx]);
106 /** Changes with the account: a run for another account stops and its results are not shown. */
107 const runGen = useRef(0);
108 // Another account: forget this card at once (adjusting state during render, no extra pass).
109 const [stateUser, setStateUser] = useState(userId);
110 if (stateUser !== userId) {
111   setStateUser(userId);
112   setState(IDLE);
113 }
114 useEffect(() => {
115   runGen.current += 1;
116   loadScanPrefs(userId)
117     .then(setPrefsState)
118     .catch(() => {});
119 }, [userId]);
120
121 const available = galleryAvailable && repo?.mode === 'cloud';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:89

บรรทัด	ความหมาย
90	ข้อมูลจาก AppProvider
92	inApp = พร้อมและตั้งค่าเงินเสร็จแล้ว (ไม่สแกนระหว่างสมัครหรือตั้งค่าเงิน)
93–95	สถานะการ์ด ตั้งค่า และธงกำลังสแกน (ref)
96–105	ฟังก์ชันของแอปเปลี่ยนทุกครั้งที่ข้อมูลเปลี่ยน จึงอ่านผ่าน ref เพื่อไม่ให้ "รายการใหม่ 1 รายการ" ทำให้การสแกนเริ่มใหม่
107	runGen เลขรุ่นของบัญชี เปลี่ยนเมื่อสลับบัญชี รอบที่เริ่มด้วยเลขเก่าจะรู้ตัวว่าต้องหยุด
109–113	บัญชีเปลี่ยน → ล้างการ์ดทันที ระหว่างวาดหน้าจอ (เทคนิคที่ React แนะนำ ไม่ต้องรอ effect อีกรอบ จึงไม่มีจังหวะที่การ์ดของบัญชีเก่าโผล่ในบัญชีใหม่)
114–119	ทุกครั้งที่บัญชีเปลี่ยน: เลขรุ่น +1 และโหลดตั้งค่าของบัญชีนั้น
121	available = ค้นแกลเลอรีได้ และ เป็นบัญชีจริง (โหมดทดลองอ่านสลิปไม่ได้)

ส่วนที่ 4: run — เริ่มรอบ และเลือกรูป (บรรทัด 123–158)

src/services/AutoScanProvider.tsx

บรรทัด 123–158

```

123 const run = useCallback(
124   async (p: ScanPrefs) => {
125     if (!repo || !userId || running.current || scanLock.manual) return;
126     running.current = true;
127     const gen = runGen.current;
128     const current = () => gen === runGen.current;
129     const startedAt = Date.now();

```

```

130     setState({ ...IDLE, phase: 'scanning' });
131     const confirmed: Transaction[] = [];
132     let drafts = 0;
133     let processed = 0;
134     let stoppedBy: string | null = null;
135     /** Stopped quietly: the account changed or the scan screen took over. */
136     let quiet = false;
137     /** Unreadable photos in a row (reset by any photo read fine). */
138     let unreadable: string[] = [];
139     try {
140         const [images, scanned, failures] = await Promise.all([
141             findGalleryImages(autoScanSince(p.lastAutoScanAt, startedAt)),
142             loadScannedIds(userId),
143             loadScanFailures(userId),
144         ]);
145         // Photos that failed MAX_AUTO_ATTEMPTS times are left for the scan screen.
146         const fresh = images.filter((i) => !scanned.has(i.assetId) && (failures[i.assetId] ?? 0)
147 < MAX_AUTO_ATTEMPTS);
148         const batch = fresh.slice(0, MAX_IMAGES_PER_RUN);
149         setState((s) => ({ ...s, total: batch.length }));
150         const index = buildDuplicateIndex(txsRef.current);
151         const range = rangeDays('1y');
152         /** Oldest photo to try again next run (it must stay inside the next run's window). */
153         let retryFrom: number | null = null;
154         for (const img of batch) {
155             if (!current() || scanLock.manual) {
156                 quiet = true;
157                 break;
158             }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:123

บรรทัด	ความหมาย
125	ไม่มี repo/บัญชี · กำลังสแกนอยู่ · หน้าสแกนทำงานอยู่ → ไม่เริ่ม
126–128	จองว่ากำลังสแกน จำนวนรุ่นตอนเริ่ม และสร้างฟังก์ชัน <code>current()</code> ถ้ามียังเป็นบัญชีเดิมไหม
129–130	จำเวลาเริ่ม และขึ้นการ์ด "กำลังหาสลิปใหม่..."
131–138	ตัวนับของรอบนี้: ที่นับในยอดแล้ว, แบบร่าง, ทำแล้ว, เหตุผลที่หยุด, หยุดเงียบไหม, รายการรูปที่อ่านไม่ได้ติดกัน
140–144	3 อย่างพร้อมกัน: ค้นรูปตั้งแต่เวลาที่ <code>autoScanSince</code> คำนวณ · รูปที่เคยตรวจ · จำนวนครั้งที่ล้มเหลว
146	ตัดรูปที่เคยตรวจ และรูปที่ล้มเหลวครบ 2 ครั้งแล้ว (ปล่อยให้หน้าสแกน)
147–148	เอาไม่เกิน 150 รูป และบอกการ์ดว่ามีกี่รูป (ตรงนี้เองที่ข้อความกลายเป็น "0/6")
149–150	สมุดกันช้า และช่วงวันที่ยอมรับ 1 ปี
152	<code>retryFrom</code> เวลาของรูปเก่าสุดที่ต้องลองใหม่อรอบหน้า

ส่วนที่ 5: ลูปที่ละรูป และการจัดการปัญหา (บรรทัด 154–225)

src/services/AutoScanProvider.tsx

บรรทัด 154–225

```

154     for (const img of batch) {

```

```

155     if (!current() || scanLock.manual) {
156         quiet = true;
157         break;
158     }
159     for (let attempt = 0; ; attempt++) {
160         try {
161             const r = await processSlipImage({
162                 uri: await galleryUri(img.assetId),
163                 width: img.width,
164                 height: img.height,
165                 requireQr: true,
166                 range,
167                 index,
168                 repo,
169                 autoConfirm: p.autoConfirm,
170                 fallbackTimeMs: img.createdAt,
171                 onWait: (seconds) => {
172                     if (current()) setState((s) => ({ ...s, waitUntil: Date.now() + seconds * 1000
173                 }));
174                 },
175                 userId,
176                 stillValid: current,
177             });
178             unreadable = [];
179             delete failures[`stall:${img.assetId}`];
180             // Saved for an account that is no longer open: do not show it in this one.
181             if (r.tx && current()) {
182                 upsertRef.current(r.tx);
183                 if (r.confirmed) confirmed.push(r.tx);
184                 else drafts += 1;
185             }
186             scanned.add(img.assetId);
187             delete failures[img.assetId];
188         } catch (e) {
189             const code = e instanceof SlipReaderError ? e.code : null;
190             if (code === 'cancelled') {
191                 quiet = true;
192                 break;
193             }
194             if (code === 'network' && attempt === 0) {
195                 await sleep(NETWORK_RETRY_MS); // a short drop in the connection: try this photo
196                 once more
197                 continue;
198             }
199             if (code === 'network') {
200                 // Offline: stop without moving the window, so nothing found this time is
201                 skipped.
202                 stoppedBy = 'เชื่อมต่ออินเทอร์เน็ตไม่ได้ สลิปที่เหลือจะอ่านต่อตอนเปิดแอปครั้งหน้า';
203             } else if (code === 'busy' || code === 'server') {
204                 // The reader is busy or down. Usually that passes; but if this same photo
205                 stopped
206                 // STALL_LIMIT runs, it is probably the photo (it times out): leave it for the
207                 scan screen.
208                 const key = `stall:${img.assetId}`;
209                 failures[key] = (failures[key] ?? 0) + 1;
210                 if (failures[key] >= STALL_LIMIT) {
211                     delete failures[key];
212                     failures[img.assetId] = MAX_AUTO_ATTEMPTS;
213                 } else stoppedBy = (e as Error).message;
214             } else if (code && code !== 'reader_error' && code !== 'too_large') {

```

```

210         stoppedBy = (e as Error).message; // affects every photo (not configured, quota,
        signed out)
211     } else {
212         // This photo could not be read: try it again next run, up to MAX_AUTO_ATTEMPTS
        times.
213         failures[img.assetId] = (failures[img.assetId] ?? 0) + 1;
214         if (failures[img.assetId] < MAX_AUTO_ATTEMPTS) retryFrom = Math.min(retryFrom ??
img.createdAt, img.createdAt);
215         if (code === 'reader_error') unreadable.push(img.assetId);
216         // Two in a row: more likely the reader is down than the photos; stop and try
        later.
217         if (unreadable.length >= 2) stoppedBy = READER_DOWN;
218     }
219 }
220 break;
221 }
222 if (stoppedBy || quiet) break;
223 processed += 1;
224 setState((s) => ({ ...s, processed, confirmed: [...confirmed], drafts, waitUntil: null
    })));
225 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:154

บรรทัด	ความหมาย
155–158	ก่อนทุกรูป: บัญชีเปลี่ยนหรือหน้าสแกนเริ่มทำงาน → หยุดเฉียบ
159	วนลองรูปนี้ (attempt 0, 1, ...) ออกด้วย break
161–176	ทำรูปนี้ด้วย processSlipImage: requireQr: true เสมอ (อัตโนมัติไม่ส่งรูปที่ไม่มี QR) · stillValid: current เช็ กบัญชีก่อนบันทึก · onWait ใส่เวลาลงการ์ด (เฉพาะถ้ายังเป็นบัญชีเดิม)
177–178	สำเร็จ: ล้างตัวนับอ่านไม่ได้ติดกัน และตัวนับ "ทำให้หยุด" ของรูปนี้
180–184	ได้รายการและยังเป็นบัญชีเดิม: ใส่ลงแอปทันที นับในยอด → เก็บไว้ (สำหรับปุ่มย้อน) ไม่จิ้นนับเป็นแบบร่าง
185–186	จำว่ารูปนี้ตรวจแล้ว และลบประวัติล้มเหลว
188	เกิดปัญหา: ดึงรหัส (ถ้าเป็น SlipReaderError)
189–192	ถูกยกเลิก (บัญชีเปลี่ยนก่อนบันทึก) → หยุดเฉียบ
193–196	เน็ตหลุดครั้งแรก → รอ 3 วินาที continue = ลองรูปเดิมอีกรอบ
197–199	เน็ตหลุดซ้ำ → หยุด โดยไม่เลื่อนหน้าต่าง (ข้อความบอกว่าจะอ่านต่อตอนเปิดแอปครั้งหน้า)
200–208	AI ยุ่ง/เซิร์ฟเวอร์ล่ม: นับ "รูปนี้ทำให้หยุด" (คีย์ stall: รหัสรูป) ครบ 3 → ตั้งให้รูปนี้ล้มเหลวครบ 2 ครั้ง (ปล่อยให้หน้า สแกน) แล้วไปรูปถัดไป · ไม่ครบ → หยุดพร้อมเหตุผล
209–210	รหัสอื่นที่เกิดกับทุกรูป (ยังไม่ตั้ง key, โควตา, หมดเซสชัน) → หยุดพร้อมเหตุผล
211–217	รูปนี้อ่านไม่ได้: นับครั้ง ยังไม่ครบ 2 → จำเวลาไว้ลองใหม่ · ถ้าเป็น reader_error เพิ่มเข้าตัวนับติดกัน ครบ 2 รูปติด → หยุด "ระบบอ่านสลิปขัดข้องชั่วคราว..."
220	ออกจากกลุ่มลองซ้ำ
222	มีเหตุให้หยุด → ออกจากกลุ่มรูปทั้งหมด

บรรทัด	ความหมาย
223–224	นับว่าทำแล้ว อัปเดตการ์ด (0/6 → 1/6 ...) และล้างการนับถอยหลัง

ส่วนที่ 6: จบรอบ (บรรทัด 227–250)

src/services/AutoScanProvider.tsx

บรรทัด 227–250

```

227     await rememberScanned(userId, scanned);
228     await saveScanFailures(userId, failures);
229     // Move the window forward only when everything in it was handled.
230     if (!stoppedBy && !quiet && fresh.length <= MAX_IMAGES_PER_RUN) {
231         const next = await saveScanPrefs({ lastAutoScanAt: nextAutoScanMark(startedAt,
retryFrom) }, userId).catch(() => null);
232         if (next && current()) setPrefsState(next);
233     }
234     if (!current()) return;
235     if (quiet) {
236         setState(confirmed.length + drafts > 0 ? { ...IDLE, phase: 'done', total: batch.length,
processed, confirmed, drafts } : IDLE);
237         return;
238     }
239     // Keep what was recorded before stopping, so the banner can still say so.
240     if (stoppedBy) setState({ ...IDLE, phase: 'error', message: stoppedBy, total:
batch.length, processed, confirmed, drafts });
241     else if (confirmed.length + drafts > 0) setState({ ...IDLE, phase: 'done', total:
batch.length, processed, confirmed, drafts });
242     else setState(IDLE); // nothing new: stay quiet
243 } catch {
244     if (current()) setState({ ...IDLE, phase: 'error', message: 'ตรวจแกลเลอรีอัตโนมัติไม่สำเร็จ ลองใหม่
ได้ทีป่วนสแกนสลิป' });
245 } finally {
246     running.current = false;
247 }
248 },
249 [repo, userId],
250 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:227

บรรทัด	ความหมาย
227–228	บันทึกรายการรูปที่ตรวจแล้ว และตัวนับลิมเหลว
229–233	เลื่อนหน้าต่างเวลาเฉพาะเมื่อจัดการครบ: ไม่ได้หยุดด้วยเหตุผล ไม่ได้หยุดเจียบ และรูปไม่เกิน 150 · เวลาที่บันทึกมาจาก nextAutoScanMark (เพื่อรูปที่ต้องลองใหม่)
234	บัญชีเปลี่ยนไปแล้ว → ไม่แตะการ์ด
235–238	หยุดเจียบ: มีรายการ → การ์ด done ไม่มี → ว่าง
240	หยุดด้วยเหตุผล → การ์ด error พร้อมสิ่งที่จดได้ก่อนหยุด
241	มีรายการ → การ์ด done
242	ไม่มีอะไรใหม่ → เจียบ (ไม่ต้องบอกผู้ใช้ทุกครั้งว่า "ไม่มีอะไร")

บรรทัด	ความหมาย
243–244	ฟังก์ชันไม่คาดคิด → การ์ด error "ตรวจเจอเลขรหัสบัตรไม่สำเร็จ ลองใหม่ได้ที่ปุ่มสแกนสลิป"
245–247	<code>finally</code> : ปลอดภัยล้างสแกนเสมอ
249	dependency ของ <code>useCallback</code> : สร้างฟังก์ชันใหม่เมื่อ <code>repo</code> หรือบัญชีเปลี่ยน

ส่วนที่ 7: maybeRun — ควรสแกนตอนนี้ไหม (บรรทัด 252–287)

src/services/AutoScanProvider.tsx

บรรทัด 252–287

```

252 const maybeRun = useCallback(async () => {
253   if (!available || running.current || scanLock.manual) return;
254   let p = await loadScanPrefs(userId);
255   setPrefsState(p);
256   if (!shouldAutoScan({ enabled: p.autoScan, lastAutoScanAt: p.lastAutoScanAt, nowMs:
Date.now(), alreadyRunning: running.current })) return;
257   let access = await getGalleryAccess().catch(() => 'denied' as const);
258   if (access === 'denied' && !p.asksGalleryOnce) {
259     // First time the app opens: ask for the photos right away (the phone shows its own
260     // permission dialog once), so scanning needs no tap at all from then on.
261     p = await saveScanPrefs({ asksGalleryOnce: true }, userId);
262     setPrefsState(p);
263     access = await requestGalleryAccess().catch(() => 'denied' as const);
264   }
265   if (access === 'denied' || access === 'blocked') {
266     setState({ ...IDLE, phase: 'needs_permission' });
267     return;
268   }
269   await run(p);
270 }, [available, run, userId]);
271
272 // On open (once the user is in the app) and whenever the app returns to the foreground.
273 useEffect(() => {
274   if (!inApp || !available) return;
275   // Start once the opening animation has finished and the first screen is on show,
276   // so opening the app stays instant (and the one-time photo prompt comes after it).
277   const first = setTimeout(() => {
278     maybeRun().catch(() => {});
279   }, START_DELAY_MS);
280   const sub = AppState.addEventListener('change', (s) => {
281     if (s === 'active') maybeRun().catch(() => {});
282   });
283   return () => {
284     clearTimeout(first);
285     sub.remove();
286   };
287 }, [inApp, available, maybeRun]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/AutoScanProvider.tsx:252

บรรทัด	ความหมาย
253	ใช้ไม่ได้ / กำลังสแกน / หน้าสแกนทำงาน → ไม่
254–256	โหลดตั้งค่าล่าสุด แล้วถาม <code>shouldAutoScan</code> (เปิดอยู่ไหม ห่างครั้งก่อน 5 นาทีไหม)

บรรทัด	ความหมาย
257	ดูสิทธิ์โดยไม่ถาม
258–264	ครั้งแรกที่เปิดแอป: ยังไม่มีสิทธิ์และยังไม่เคยขอ → จำว่าขอแล้ว แล้วขอเลย (มือถือขึ้นกล่องของระบบครั้งเดียว) จากนั้นการสแกนไม่ต้องแตะอะไรอีก
265–268	ยังไม่มีสิทธิ์ → การ์ด needs_permission แล้วจบ
269	มีสิทธิ์ → สแกน
273–274	effect: ทำเฉพาะเมื่ออยู่ในแอปแล้วและใช้งานได้
277–279	ตั้งเวลา 1.8 วินาทีแล้วลองสแกน (หลังแอนิเมชันเปิดแอป)
280–282	ฟังเหตุการณ์แอปกลับมาหน้าจอ (active) → ลองสแกนอีก (shouldAutoScan กันไม่ให้ถี่เกิน 5 นาที)
283–286	ตอนเลิกใช้: ยกเลิกตัวตั้งเวลาและเลิกฟังเหตุการณ์ (กันหน่วยความจำรั่ว)

ส่วนที่ 8: คำสั่งที่ให้หน้าจอ (บรรทัด 289–321)

src/services/AutoScanProvider.tsx

บรรทัด 289–321

```

289  const value = useMemo<AutoScanContextValue>(
290    () => ({
291      state,
292      prefs,
293      available,
294      async setPrefs(patch) {
295        setPrefsState(await saveScanPrefs(patch, userId));
296      },
297      async grantAndRun() {
298        const a = await requestGalleryAccess().catch(() => 'denied' as const);
299        if (a === 'denied' || a === 'blocked') {
300          setState({ ...IDLE, phase: 'needs_permission', message: 'ยังไม่ได้รับสิทธิ์เข้าถึงรูปภาพ เปิดได้ในการตั้ง
คำขอมือถือ' });
301          return;
302        }
303        await run(await loadScanPrefs(userId));
304      },
305      async runNow() {
306        await run(await loadScanPrefs(userId));
307      },
308      async undoLast() {
309        const ids = state.confirmed.map((t) => t.id);
310        for (const id of ids) await updateTxRef.current(id, { status: 'draft' });
311        setState(IDLE);
312      },
313      dismiss() {
314        setState(IDLE);
315      },
316    }),
317    [state, prefs, available, run, userId],
318  );
319
320  return <AutoScanContext.Provider value={value}>{children}</AutoScanContext.Provider>;

```

บรรทัด	ความหมาย
289	<code>useMemo</code> สร้างอ็อบเจกต์ใหม่เฉพาะเมื่อค่าข้างในเปลี่ยน (หน้าจอที่ใช้ Context จะได้ไม่วาดใหม่โดยไม่จำเป็น)
294–296	<code>setPrefs</code> (สวิตช์ในตั้งค่า) บันทึกแล้วอัปเดต
297–304	<code>grantAndRun</code> (ปุ่ม "อนุญาตเข้าถึงรูปภาพ") ขอสิทธิ์ ไม่ได้ → ข้อความ "ยังไม่ได้รับสิทธิ์... เปิดได้ในการตั้งค่าของมือถือ" ได้ → สแกนเลย
305–307	<code>runNow</code> สแกนทันที
308–312	<code>undoLast</code> (ปุ่ม "ยังไม่รวมยอด") เปลี่ยนทุกรายการที่รอบนี้นับในยอด กลับเป็น draft ที่ละรายการ แล้วซ่อนการ์ด
313–315	<code>dismiss</code> (×) ซ่อนการ์ด
320	ส่งค่าให้ทุกหน้าจอที่อยู่ข้างใน Provider

อ่านโค้ด: scan.tsx — หน้าสแกนสลิป

259 บรรทัด หน้าจอที่แสดงได้หลายสถานะที่สุดของ FR-4 แต่แทบไม่มี "กฎ" อยู่ในนี้เลย

ส่วนที่ 1: คอมเมนต์และ import (บรรทัด 1–37)

src/app/scan.tsx

บรรทัด 1–37

```
1 /**
2  * FR-4 Automatic Gallery Slip Detection.
3  *
4  * New flow (28 ก.ย. 2569, from user feedback): nothing to choose first.
5  *   - Phone app: opening this screen searches the gallery (last 30 days, new
6  *     photos only) and starts reading right away.
7  *   - Web (iPhone/computer): browsers cannot look through the photo library, so
8  *     one tap opens the picker; reading starts as soon as photos are chosen.
9  * Every slip is recorded under the date printed on it, and clear ones count
10 * in the balance at once (auto-confirm). The results are then shown by period
11 * (วันนี้ 00:00–23:59, 7 วัน, 1 เดือน) and by day.
12 */
13 import { router } from 'expo-router';
14 import { goBack } from '../ui/nav';
15 import { useEffect, useMemo, useRef } from 'react';
16 import { Linking, ScrollView, View } from 'react-native';
17 import { SafeAreaView } from 'react-native-safe-area-context';
18 import { useApp } from '../data/AppProvider';
19 import { BUDDY_NAME } from '../domain/buddy';
20 import { relativeDayLabel } from '../domain/dates';
21 import { formatBaht } from '../domain/money';
22 import { scanCounts } from '../domain/scanQueue';
23 import { groupByDay } from '../domain/summary';
24 import { useAutoScan } from '../services/AutoScanProvider';
25 import { galleryAvailable } from '../services/slips';
26 import { useSlipScanner } from '../services/useSlipScanner';
27 import { Buddy, BuddySays } from '../ui/Buddy';
28 import { Button, Card, Divider, IconButton, Ionicons, ProgressBar, Row, T } from
29   '../ui/components';
30 import { useToast } from '../ui/feedback';
31 import { useCelebrate, Reveal } from '../ui/effects';
32 import { PeriodSummary } from '../ui/PeriodSummary';
33 import { ScannerStage } from '../ui/ScannerStage';
34 import { TxRow } from '../ui/TxRow';
35 import { radius, space, useTheme } from '../ui/theme';
36 import { WaitNotice } from '../ui/WaitNotice';
37 import { HeaderDecor } from '../ui/halloween';
```

▶ เปิดใน VS Code (లేబు) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:1

บรรทัด ความหมาย

4 Flow ใหม่ (28 ก.ย. 2569 จากความเห็นผู้ใช้): ไม่ต้องเลือกอะไรก่อน (รุ่นก่อนให้เลือกช่วงเวลาก่อนสแกน)

บรรทัด	ความหมาย
5-8	แอปมือถือ: เปิดหน้าแล้วค้นเอกสาร 30 วัน (เฉพาะรูปใหม่) และเริ่มอ่านทันที · เว็บ: เบราว์เซอร์ค้นคลังรูปไม่ได้ แต่ละครั้ง เดียวเปิดตัวเลือก เลือกแล้วเริ่มอ่านทันที
9-11	ทุกสลิปบันทึกตามวันที่บนสลิป ที่จัดนับในยอดทันที แล้วแสดงผลตามช่วง (วันนี้ 00:00-23:59, 7 วัน, 1 เดือน) และตาม วัน
13-36	import: router, ข้อมูลแอป, ชื่อน้องกล้า, ชื่อวันแบบสัมพัทธ์, รูปแบบเงิน, ตัวนับคิว, groupByDay (FR-1), การสแกน อัตโนมัติ, galleryAvailable, hook ของหน้าสแกน และคอมโพเนนต์ UI

ส่วนที่ 2: state และการเริ่มค้นอัตโนมัติ (บรรทัด 38-61)

src/app/scan.tsx

บรรทัด 38-61

```

38 export default function Scan() {
39   const theme = useTheme();
40   const toast = useToast();
41   const { repo, txs, confirmTxs } = useApp();
42   const s = useSlipScanner('1m');
43   const auto = useAutoScan();
44   // The automatic scan (on app open) may still be reading: wait for it, then skip what it read.
45   const autoBusy = auto.state.phase === 'scanning';
46   const { state } = s;
47   const counts = scanCounts(state);
48   const running = state.phase === 'running';
49   const paused = state.phase === 'paused';
50   const done = state.phase === 'done';
51   const demo = repo?.mode === 'demo';
52   const started = useRef(false);
53
54   // Phone app: start searching as soon as the screen opens (after the first frame).
55   useEffect(() => {
56     if (started.current || demo || !galleryAvailable || !repo || autoBusy) return;
57     started.current = true;
58     const t = setTimeout(() => s.loadFromGallery('1m'), 250);
59     return () => clearTimeout(t);
60     // eslint-disable-next-line react-hooks/exhaustive-deps
61   }, [demo, repo, autoBusy]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:38

บรรทัด	ความหมาย
41	จาก AppProvider: repo, รายการทั้งหมด, ฟังก์ชันยืนยันหลายรายการ
42	ตัวควบคุมของหน้านี้ ค่าเริ่ม "ค้น 1 เดือน"
43-45	สถานะของการสแกนอัตโนมัติ ถ้ากำลังอ่าน (autoBusy) หน้านี้ต้องรอ แล้วค่อยค้นต่อ (รูปที่การสแกนอัตโนมัติอ่าน แล้วจะถูกข้าม)
47-50	ตัวนับ และตัวแปรบูสอ่านง่าย: running / paused / done
51	อยู่ในโหมดทดลองใหม่
52	<code>started</code> ref กันการเริ่มซ้ำ

บรรทัด	ความหมาย
55–61	มือถือ: เริ่มต้นเมื่อเปิดหน้า · ไม่ทำถ้า: เริ่มไปแล้ว / โหมดทดลอง / คัดแคลเลอร์ไม่ได้ (เว็บ) / ยังไม่มี repo / สแกนอัตโนมัติยังทำงาน · หน่วง 250 ms ให้หน้าจาวาดเฟรมแรกก่อน (เปิดหน้าสลับ) · return ยกเลิกตัวตั้งเวลาถ้าออกจากหน้าก่อน · effect ทำงานใหม่เมื่อ autoBusy เปลี่ยน = พอสแกนอัตโนมัติเสร็จก็เริ่มเองทันที
60	คอมเมนต์ปิดคำเตือนของ ESLint (กฎ exhaustive-deps) อย่างตั้งใจ: s เปลี่ยนทุกครั้งที่วาด ถ้าใส่ใน dependency จะเริ่มต้นซ้ำ

ส่วนที่ 3: ข้อมูลที่คำนวณจากผลของรอบนี้ (บรรทัด 63–90)

src/app/scan.tsx	บรรทัด 63–90
<pre> 63 const recorded = useMemo(() => { 64 const ids = new Set(s.runTxIds); 65 return txs.filter((t) => ids.has(t.id)); 66 }, [txs, s.runTxIds]); 67 const byDay = useMemo(() => groupByDay(recorded), [recorded]); 68 const drafts = recorded.filter((t) => t.status === 'draft'); 69 const readyIds = drafts.filter((t) => t.reviewFlags.length === 0).map((t) => t.id); 70 const counted = recorded.filter((t) => t.status === 'confirmed').length; 71 const latest = recorded.length > 0 ? recorded.find((t) => t.id === s.runTxIds[s.runTxIds.length - 1]) ?? null : null; 72 73 // Confetti once per scan that found slips. 74 const celebrate = useCelebrate(); 75 const foundSlips = done && recorded.length > 0; 76 useEffect(() => { 77 if (foundSlips) celebrate(); 78 }, [foundSlips, state.runId, celebrate]); 79 80 async function confirmReady() { 81 try { 82 await confirmTxs(readyIds); 83 toast({ message: `ยืนยัน \${readyIds.length} รายการแล้ว` }); 84 } catch { 85 toast({ message: 'ยืนยันไม่สำเร็จ ลองอีกครั้ง', tone: 'error' }); 86 } 87 } 88 89 const idle = state.phase === 'idle' && state.items.length === 0; 90 const nothingNew = (state.phase === 'idle' done) && state.items.length === 0 && !s.finding && s.searched; </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:63

บรรทัด	ความหมาย
63–66	recorded = รายการที่รอบนี้นับถึง ถึงจาก txs (ข้อมูลล่าสุดของแอป) ไม่ใช่จากผลของคิว ถ้าผู้ใช้ยืนยันหรือแก้รายการหน้านี้เห็นสถานะใหม่ทันที · useMemo คำนวณใหม่เฉพาะเมื่อ txs หรือ runTxIds เปลี่ยน
67	จัดกลุ่มตามวัน (ฟังก์ชันเดียวกับหน้ารายการของ FR-1)
68–70	แบบร่าง · แบบร่างที่ไม่มีธง (พร้อมยืนยัน) · จำนวนที่นับในยอดแล้ว
71	latest รายการล่าสุดของรอบ (ไว้ใช้รอบเวกทีสแกน "-฿89 ร้านก๋วยเตี๋ยวเรือ")

บรรทัด	ความหมาย
74–78	พลกระดาษ 1 ครั้งต่อรอบที่เจอสลิป (dependency มี runId = รอบใหม่ได้พลใหม่)
80–87	confirmReady ยืนยันทุกแบบร่างที่พร้อม แล้วขึ้นข้อความ "ยืนยัน N รายการแล้ว" พลาด → ข้อความ error
89	idle = ยังไม่เริ่มและคิ่วว่าง (ใช้กับการ์ดเว็บ "เลือกรูปสลิป")
90	nothingNew = (ว่างหรือเสร็จ) และคิ่วว่าง และไม่ได้กำลังค้น และ ค้นเสร็จแล้วจริง

ส่วนที่ 4: หัวหน้าจอ โหมดทดลอง และการ์ดของเว็บ (บรรทัด 92–118)

src/app/scan.tsx	บรรทัด 92–118
<pre> 92 return (93 <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1, backgroundColor: theme.bg }}> 94 <Row justify="space-between" style={{ paddingHorizontal: space.sm, paddingTop: space.sm }}> 95 <IconButton icon="close" label="ปิด" onPress={() => goBack()} /> 96 <T v="h3">สแกนสลิป</T> 97 <HeaderDecor /> 98 </Row> 99 100 <ScrollView contentContainerStyle={{ padding: space.lg, gap: space.lg, paddingBottom: space.xxxl }}> 101 {demo ? (102 <BuddySays mood="calm">โหมดทดลองยังอ่านสลิปไม่ได้นะ เพราะต้องใช้เชิรฟ์เวอร์ เข้าสู่ระบบด้วยบัญชีจริง แล้ว{BUDDY_NAME}จะอ่านให้เลย</BuddySays> 103) : null} 104 105 {/* Web: one tap to choose photos, then everything else is automatic */} 106 {!galleryAvailable && idle && !demo ? (107 <Card style={{ gap: space.md, alignItems: 'center' }}> 108 <Buddy mood="happy" size={92} /> 109 <T v="h2" center>เลือกรูปสลิป แล้ว{BUDDY_NAME}จัดการที่เหลือให้</T> 110 <T v="small" center> เลือกได้ครั้งละหลายรูป (สูงสุด 30) {BUDDY_NAME}จะอ่านยอด แยกตามวันที่บนสลิป และรวมเข้ายอดเงินให้กับที่ 111 </T> 112 <Button label="เลือกรูปสลิป" kind="gold" icon="images-outline" onPress={s.loadPicked} style={{ alignSelf: 'stretch' }} /> 113 <T v="micro" center> เว็บเบราว์เซอร์ทุกตัวไม่อนุญาตให้เว็บเปิดดูรูปในเครื่องเอง จึงต้องกดเลือก 1 ครั้ง ในแอป Android {BUDDY_NAME}หา สลิปให้เองตั้งแต่เปิดแอป 114 </T> 115 </Card> 116) : null} </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:92

บรรทัด	ความหมาย
93	SafeAreaView เว้นขอบบน/ล่างของจอ (รอยบาก แถบปุ่ม)
94–98	แถวหัว: ปุ่ม × (ย้อนกลับ) ชื่อ "สแกนสลิป" และของตกแต่งตามเทศกาล (HeaderDecor)
100	เนื้อหาเลื่อนได้ ระยะห่างตาม design token (space.lg, space.xxxl)
101–103	โหมดทดลอง: นึกกลับบอกว่าต้องใช้เชิรฟ์เวอร์ ให้เข้าสู่ระบบด้วยบัญชีจริง

106–118 **การ์ดเว็บ** (ค้นคลังรูปไม่ได้ + ว่าง + ไม่ใช่โหมดทดลอง): นื่องคล้าย 92 · หัวข้อ · คำอธิบาย "สูงสุด 30" · ปุ่มทวง "เลือกรูปสลิป" เรียก `s.loadPicked` · ตัวเล็กรายละเอียดเหตุการณ์เบราว์เซอร์ (ภาพแรกของบทหน้าสแกน)

ส่วนที่ 5: สถานะของมือถือ และข้อความปัญหา (บรรทัด 120–166)

src/app/scan.tsx

บรรทัด 120–166

```
120      {/* Phone: the automatic scan from opening the app is still running */}
121      {galleryAvailable && autoBusy && !s.searched && !s.finding ? (
122        <BuddySays mood="thinking">
123          {`${BUDDY_NAME}กำลังอ่านสลิปใหม่ที่เจอตอนเปิดแอปอยู่${auto.state.total ? `
(${auto.state.processed}/${auto.state.total})` : ''} เสร็จแล้วจะคืนต่อให้เลย`}
124        </BuddySays>
125      ) : null}
126
127      {/* Phone: searching the gallery */}
128      {galleryAvailable && s.finding ? (
129        <BuddySays mood="thinking">{BUDDY_NAME}กำลังหาสลิปใหม่ในแกลเลอรี 30 วันล่าสุด...</BuddySays>
130      ) : null}
131
132      {s.access === 'denied' || s.access === 'blocked' ? (
133        <Card tone="alt">
134          <T v="h3">ยังไม่ได้อนุญาตให้เข้าถึงรูปภาพ</T>
135          <T v="small">MindPay ต้องเห็นรูปในแกลเลอรีเพื่อหาสลิปเอง ถ้าไม่อยากให้สิทธิ์ ยังกด "เลือกรูปเอง" ได้</T>
136          {s.access === 'blocked' ? (
137            <Button label="เปิดการตั้งค่า" kind="soft" small onPress={() => Linking.openSettings()}
138          />
139            ) : (
140              <Button label="ขอสิทธิ์อีกครั้ง" kind="soft" small onPress={() => s.loadFromGallery('1m')}
141            />
142            )}
143          <Button label="เลือกรูปเอง" kind="soft" small icon="hand-left-outline" onPress=
144            {s.loadPicked} />
145          </Card>
146        ) : null}
147      {s.access === 'limited' ? <T v="small">คุณอนุญาตเฉพาะบางรูป MindPay จะตรวจได้เฉพาะรูปที่เลือกไว้</T> :
148      null}
149
150      {galleryAvailable && nothingNew && s.access !== 'denied' && s.access !== 'blocked' &&
151      !demo ? (
152        <Card style={{ gap: space.md, alignItems: 'center' }}>
153          <Buddy mood="calm" size={84} />
154          <T v="h3" center>ไม่มีรูปใหม่ใน 30 วันล่าสุด</T>
155          <T v="small" center>
156            {s.skippedKnown > 0 ? `ถูกรูปเคยตรวจแล้ว (${s.skippedKnown} รูป) ` : ''}ถ้ามีสลิปเก่ากว่านั้น
157            ค้นหาย้อนหลังได้ถึง 1 ปี
158          </T>
159          <Button label="ค้นหาย้อนหลัง 1 ปี" kind="soft" icon="time-outline" onPress={() =>
160            s.loadFromGallery('1y')} style={{ alignSelf: 'stretch' }} />
161          <Button label="เลือกรูปเอง" kind="ghost" small icon="hand-left-outline" onPress=
162            {s.loadPicked} />
163          </Card>
164        ) : null}
165
166      {s.notice ? (
167        <Card tone="alt">
```



```

160         <Row gap={space.sm}>
161             <Buddy mood="worried" size={40} still />
162             <T v="body" style={{ flex: 1 }}>{s.notice}</T>
163         </Row>
164         {paused ? <Button label="อ่านต่อ" icon="play" small onPress={s.resume} /> : null}
165     </Card>
166 ) : null}

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:120

บรรทัด	ความหมาย
121–125	สแกนอัตโนมัติยังอ่านอยู่ (และหน้านี้ยังไม่ได้ค้น): "กลัคำล้งอ่านสลิปใหม่ที่เจอตอนเปิดแอปอยู่ (2/6) เสร็จแล้วจะค้นต่อให้เลย" (ใส่ตัวเลขเฉพาะเมื่อรู้จำนวนแล้ว)
128–130	กำลังค้นแคลเลอร์: "กลัคำล้งหาสลิปใหม่ในแคลเลอร์ 30 วันล่าสุด..."
132–143	ไม่ได้สิทธิ์: การ์ด "ยังไม่ได้อนุญาตให้เข้าถึงรูปภาพ" · ถ้าถูกล็อก → ปุ่ม "เปิดการตั้งค่า" (<code>Linking.openSettings()</code> เปิดหน้าตั้งค่าของแอปในมือถือ) · ถ้ามใหม่ได้ → ปุ่ม "ขอสิทธิ์อีกครั้ง" · และ "เลือกรูปเอง" เสมอ (ไม่ให้สิทธิ์ก็ยังใช้ได้)
144	อนุญาตบางรูป: บอกว่าตรวจได้เฉพาะรูปที่เลือกไว้
146–156	ไม่มีรูปใหม่: "ไม่มีรูปใหม่ใน 30 วันล่าสุด" + (ถ้าข้ามรูปที่เคยตรวจ) "ทุกรูปเคยตรวจแล้ว (6 รูป)" + ปุ่ม "ค้นหาย้อนหลัง 1 ปี" (เรียก <code>loadFromGallery('1y')</code>) และ "เลือกรูปเอง"
158–166	ข้อความปัญหา (เช่น เน็ตหลุด): น้องกลัหน้ากังวล (<code>still</code> = ไม่ขยับ) + ข้อความ · ถ้าหยุดอยู่ มีปุ่ม "อ่านต่อ"

ส่วนที่ 6: การ์ดความคืบหน้า (บรรทัด 168–218)

src/app/scan.tsx

บรรทัด 168–218

```

168     {/* Progress */}
169     {state.items.length > 0 ? (
170         <Card style={{ gap: space.md }}>
171             {running || (paused && recorded.length > 0) ? <ScannerStage active={running} latest=
{latest} /> : null}
172             <Row gap={space.md}>
173                 <Buddy mood={done ? (counted > 0 ? 'cheer' : 'calm') : 'thinking'} size={52} />
174                 <View style={{ flex: 1, gap: 4 }}>
175                     <T v="h3">
176                         {done
177                             ? recorded.length > 0
178                             ? `อ่านเสร็จแล้ว ได้ ${recorded.length} รายการ`
179                             : 'อ่านเสร็จแล้ว ไม่พบสลิปใหม่'
180                             : paused
181                             ? 'หยุดไว้ชั่วคราว'
182                             : `${BUDDY_NAME}กำลังอ่านสลิป ${counts.finished}/${counts.total}`}
183                     </T>
184                     <T v="micro">
185                         {counted > 0 ? `รวมในยอดเงินแล้ว ${counted} รายการ` : ''}
186                         {counts.duplicate > 0 ? `${counted > 0 ? ' ' : ''}ซ้ำ ${counts.duplicate}` : ''}
187                         {counts.not_slip > 0 ? ` ` รุปอื่น (ไม่มี QR สลิป) ${counts.not_slip}` : ''}
188                         {counts.failed > 0 ? ` ` อ่านไม่ได้ ${counts.failed}` : ''}
189                         {counts.out_of_range > 0 ? ` ` เก่ากว่า 1 ปี ${counts.out_of_range}` : ''}
190                     </T>

```

```

191         </View>
192     </Row>
193     {!done ? <ProgressBar value={counts.total ? counts.finished / counts.total : 0}
color={theme.accent} /> : null}
194     {done && recorded.length === 0 && counts.not_slip > 0 && galleryAvailable ? (
195         <View style={{ backgroundColor: theme.surfaceAlt, borderRadius: radius.lg, padding:
space.md, gap: space.sm }}>
196             <T v="small">
197                 มีสลิปในเครื่องแต่{BUDDY_NAME}ไม่เจอ? {BUDDY_NAME}หาสลิปจาก QR ตรวจสอบสลิปที่แอปธนาคารพิมพ์ไว้ ถ้า
เป็นรูปถ่ายสลิปกระดาษ หรือรูปที่ตัด QR ออก ให้เลือกรูปนั้นเอง
198             </T>
199             <Button label="เลือกรูปเอง" kind="soft" small icon="hand-left-outline" onPress=
{s.loadPicked} />
200         </View>
201     ) : null}
202     {running && s.waiting ? <WaitNotice until={s.waiting} /> : null}
203     {running ? <Button label="หยุดชั่วคราว" kind="soft" small icon="pause" onPress={s.pause}
/> : null}
204     {paused && !s.notice ? <Button label="อ่านต่อ" small icon="play" onPress={s.resume} /> :
null}
205     {done && readyIds.length > 0 ? (
206         <Button label={`ยืนยัน ${readyIds.length} รายการที่อ่านชัด`} kind="gold" icon="checkmark-
done" onPress={confirmReady} />
207     ) : null}
208     {done && drafts.length - readyIds.length > 0 ? (
209         <Button label={`ตรวจ ${drafts.length - readyIds.length} รายการที่อ่านไม่ชัด`} kind="soft"
icon="receipt-outline" onPress={() => router.dismissTo('/drafts')} />
210     ) : null}
211     {done ? (
212         <Row gap={space.sm}>
213             <Button label="สแกนอีกครั้ง" kind="ghost" small icon="refresh" onPress={() =>
(galleryAvailable ? s.loadFromGallery('1m') : s.loadPicked())} style={{ flex: 1 }} />
214             <Button label="เสร็จ" small onPress={() => goBack()} style={{ flex: 1 }} />
215         </Row>
216     ) : null}
217 </Card>
218 ) : null}

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/scan.tsx:168

บรรทัด	ความหมาย
169	แสดงเมื่อมีรูปในคิว
171	เวทีสแกน: ระหว่างอ่าน หรือหยุดแต่มีผลแล้ว · แสงสแกนวิ่งเฉพาะตอน running
173	หน้าน้องกล่า: เสร็จและมีที่นับในยอด → เชียร์ · เสร็จแต่ไม่มี → สงบ · ระหว่างทำ → คิด
175–183	หัวข้อ 4 แบบ (ternary ข้อ): เสร็จ + ได้รายการ → "อ่านเสร็จแล้ว ได้ N รายการ" · เสร็จ + ไม่ได้ → "อ่านเสร็จแล้ว ไม่พบสลิปใหม่" · หยุด → "หยุดไว้ชั่วคราว" · ระหว่างทำ → "กล่ากำลังอ่านสลิป 1/4"
184–190	บรรทัดสรุป ต่อข้อความเฉพาะที่มีจำนวน > 0: รวมในยอดแล้ว · ซ้ำ · รูปอื่น (ไม่มี QR สลิป) · อ่านไม่ได้ · เก่ากว่า 1 ปี
193	แถบความคืบหน้า = เสร็จ / ทั้งหมด (สีทอง accent) ซ่อนเมื่อเสร็จ
194–201	เสร็จแล้วไม่ได้รายการเลยแต่มีรูปที่ไม่มี QR (บนมือถือ): ช่วยบอกสาเหตุที่เป็นไปได้ + ปุ่ม "เลือกรูปเอง"
202	AI ยุ่ง → นับถอยหลัง

บรรทัด	ความหมาย
203–204	ปุ่ม "หยุดชั่วคราว" (ระหว่างทำ) / "อ่านต่อ" (หยุดอยู่และไม่มีการ์ดข้อความซึ่งมีปุ่มของตัวเองแล้ว)
205–207	เสร็จและมีที่พร้อมยืนยัน → ปุ่มทอง "ยืนยัน N รายการที่อ่านชัด" (เกิดเมื่อปิด "รวมยอดกันที")
208–210	เสร็จและมีที่ต้องตรวจ → "ตรวจ N รายการที่อ่านไม่ชัด" · <code>router.dismissTo('/drafts')</code> ปิดหน้าสแกนแล้วไปหน้าสลิปรอยืนยัน (กดย้อนกลับจะไม่กลับมาหน้าสแกนที่เสร็จแล้ว)
211–216	เสร็จ → "สแกนอีกครั้ง" (มีสื่อค้นใหม่ เว็บเปิดตัวเลือก) และ "เสร็จ"

ส่วนที่ 7: ผลตามช่วงและตามวัน และกล่องความเป็นส่วนตัว (บรรทัด 220–259)

src/app/scan.tsx

บรรทัด 220–259

```

220     { /* Results by period, then by day */
221     {recorded.length > 0 ? (
222         <>
223         <PeriodSummary txs={txs} title="ยอดเงินเข้า-ออกตามช่วงเวลา" />
224         <View style={{ gap: space.sm }}>
225         <T v="h3">สลิปที่อ่านรอบนี้ แยกตามวันที่</T>
226         {byDay.map((g, i) => (
227             <Reveal key={g.day} index={i}>
228             <Card style={{ paddingVertical: space.sm, gap: 0 }}>
229             <Row justify="space-between" style={{ paddingVertical: space.xs }}>
230             <T v="body">{relativeDayLabel(g.day)}</T>
231             <T v="small" color={g.netSatang >= 0 ? theme.income : theme.expense}>
232             สุทธิ {formatBaht(g.netSatang, { sign: true, decimals: false })}
233             </T>
234             </Row>
235             {g.items.map((t) => (
236                 <View key={t.id}>
237                 <Divider />
238                 <TxRow tx={t} />
239                 </View>
240             )))
241             </Card>
242             </Reveal>
243         )]}
244         </View>
245     </>
246     ) : null}
247
248     <View style={{ backgroundColor: theme.surfaceAlt, borderRadius: radius.lg, padding:
space.md, gap: 4 }}>
249     <Row gap={6}>
250     <Icons name="shield-checkmark-outline" size={16} color={theme.inkSoft} />
251     <T v="micro" style={{ flex: 1 }}>
252     รุปสลิปส่งให้ AI (Google Gemini) อ่านครั้งเดียว ไม่ถูกเก็บบนเซิร์ฟเวอร์ MindPay (รายละเอียดในตั้งค่า) · สลิปที่
อ่านชัดทุกช่องรวมในยอดกันที ส่วนที่ไม่ชัดรอให้ตรวจ (ปิดได้ในตั้งค่า)
253     </T>
254     </Row>
255     </View>
256 </ScrollView>
257 </SafeAreaView>
258 );

```

บรรทัด	ความหมาย
221	แสดงเมื่อรอบนี้บันทึกได้อย่างน้อย 1 รายการ · <code><>...</></code> = Fragment ห่อหลายชั้นโดยไม่เพิ่มกล่อง
223	PeriodSummary ได้ รายการทั้งหมด ของผู้ใช้ (ไม่ใช่แค่รอบนี้) จึงเป็นยอดจริงของวันนี้ 7 วัน 1 เดือน และนับเฉพาะ confirmed
225	หัวข้อ "สลิปที่อ่านรอบนี้ แยกตามวันที่"
226–243	ทุกกลุ่มวัน: การ์ดพร้อมแอนิเมชันโฟลต์ละใบ (<code>Reveal index={i}</code>) หัวการ์ด = ชื่อวัน + "สุทธิ ±฿" (สีเขียวถ้า ≥ 0) แล้วทุกรายการเป็น TxRow คั่นด้วยเส้น · <code>key</code> ช่วย React จำว่าแถวไหนคือแถวไหน
248–255	กล่องความเป็นส่วนตัว (ไอคอนโล่) แสดงทุกสถานะ

★ สิ่งที่ไม่ "มี" ในไฟล์นี้

ไม่มีการคำนวณยอด ไม่มีการตัดสินใจว่าสลิปชัดไหม ไม่มีการเรียกเซิร์ฟเวอร์ ทั้งหมดอยู่ใน slip.ts, scanQueue.ts, processSlip.ts และ useSlipScanner.ts หน้าจอแค่ "ถามสถานะแล้ววาด" นี่คือเหตุผลที่หน้าจอ 259 บรรทัดนี้อ่านง่ายทั้งที่มีหลายสถานะ

อ่านโค้ด: drafts.tsx และส่วนตรวจสอบสลิปใน transaction.tsx

หน้าสลิปรอยืนยัน (96 บรรทัด) และส่วนพิเศษของฟอร์ม FR-1 เมื่อเปิดแบบร่าง

ส่วนที่ 1: drafts.tsx ทรรกะ (บรรทัด 1–42)

src/app/drafts.tsx

บรรทัด 1–42

```

1  /**
2   * Review all slip drafts. "Ready" drafts can be confirmed together;
3   * drafts with unsure fields open the form with those fields highlighted.
4   */
5  import { router } from 'expo-router';
6  import { goBack } from '../ui/nav';
7  import { useState } from 'react';
8  import { ScrollView, View } from 'react-native';
9  import { SafeAreaView } from 'react-native-safe-area-context';
10 import { useApp, useMoney } from '../data/AppProvider';
11 import { formatBaht } from '../domain/money';
12 import { Buddy } from '../ui/Buddy';
13 import { Button, Card, Divider, EmptyState, IconButton, Row, T } from '../ui/components';
14 import { Reveal, useCelebrate } from '../ui/effects';
15 import { useToast } from '../ui/feedback';
16 import { TxRow } from '../ui/TxRow';
17 import { space, useTheme } from '../ui/theme';
18
19 export default function Drafts() {
20   const theme = useTheme();
21   const toast = useToast();
22   const { confirmTx } = useApp();
23   const celebrate = useCelebrate();
24   const { drafts } = useMoney();
25   const [busy, setBusy] = useState(false);
26   const sorted = [...drafts].sort((a, b) => b.occurredAt.localeCompare(a.occurredAt));
27   const ready = sorted.filter((d) => d.reviewFlags.length === 0);
28   const review = sorted.filter((d) => d.reviewFlags.length > 0);
29   const readyTotal = ready.reduce((s, d) => s + (d.kind === 'expense' ? -d.amountSatang :
30     d.amountSatang), 0);
31
32   async function confirmAll() {
33     setBusy(true);
34     try {
35       await confirmTx(ready.map((d) => d.id));
36       celebrate();
37       toast({ message: `ยืนยัน ${ready.length} รายการแล้ว` });
38     } catch {
39       toast({ message: 'ยืนยันไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
40     } finally {
41       setBusy(false);
42     }
43   }
44 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/drafts.tsx:1

บรรทัด	ความหมาย
2-3	แบบร่างที่ "พร้อม" ยืนยันพร้อมกันได้ ที่ไม่แน่ใจเปิดฟอร์มพร้อมไฮไลต์ช่อง
22	ฟังก์ชันยืนยันหลายรายการ (ฐานข้อมูลทำครั้งเดียว: <code>update({ status: 'confirmed', review_flags: [] }).in('id', ids)</code>)
24	แบบร่างทั้งหมดจาก useMoney (FR-2)
26	เรียงใหม่ไปเท่า ([...drafts] คัดลอกก่อน เพราะ sort แก้อาร์เรย์เดิม)
27-28	แยก 2 กลุ่ม: ไม่มีธง / มีธง
29	ยอดรวมของกลุ่มพร้อม (รายจ่ายลบ รายรับบวก)
31-42	confirmAll: ตั้ง busy (ปุ่มขึ้นตัวหมุน) ยืนยัน → พลั + "ยืนยัน N รายการแล้ว" พลาด → ข้อความ error · finally Unload busy

ส่วนที่ 2: drafts.tsx หน้าจอ (บรรทัด 44-96)

src/app/drafts.tsx

บรรทัด 44-96

```

44  return (
45    <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1, backgroundColor: theme.bg }}>
46      <Row justify="space-between" style={{ paddingHorizontal: space.sm, paddingTop: space.sm }}>
47        <IconButton icon="chevron-back" label="กลับ" onPress={() => goBack()} />
48        <T v="h3">สลิปรอยืนยัน</T>
49        <IconButton icon="scan-outline" label="สแกนเพิ่ม" onPress={() => router.push('/scan')} />
50      </Row>
51      <ScrollView contentContainerStyle={{ padding: space.lg, gap: space.lg, paddingBottom:
space.xxxl }}>
52        {sorted.length === 0 ? (
53          <EmptyState
54            icon="checkmark-done-circle-outline"
55            art={<Buddy mood="cheer" size={96} />}
56            title="เคลียร์หมดแล้ว"
57            body="ทุกรายการจากสลิปยืนยันเรียบร้อยแล้ว เก่งมาก"
58            action="สแกนสลิปเพิ่ม"
59            onAction={() => router.push('/scan')}
60          />
61        ) : null}
62
63        {ready.length > 0 ? (
64          <View style={{ gap: space.sm }}>
65            <T v="h3">พร้อมยืนยัน ({ready.length})</T>
66            <T v="small">อ่านได้ครบทุกช่องด้วยความมั่นใจ 80% ขึ้นไป รวมแล้ว {formatBaht(readyTotal, { sign:
true })}</T>
67          <Card style={{ paddingVertical: space.xs, gap: 0 }}>
68            {ready.map((t, i) => (
69              <Reveal key={t.id} index={i} from={10}>
70                {i > 0 ? <Divider /> : null}
71                <TxRow tx={t} />
72              </Reveal>
73            ))}
74          </Card>
75          <Button label={`ยืนยันทั้ง ${ready.length} รายการ`} kind="gold" icon="checkmark-done"
shine onPress={confirmAll} loading={busy} />
76        </View>

```

```

77     ) : null}
78
79     {review.length > 0 ? (
80       <View style={{ gap: space.sm }}>
81         <T v="h3">ต้องตรวจสอบ ({review.length})</T>
82         <T v="small">มีบางช่องที่อ่านไม่ชัด แต่เพื่อแก้แล้วยืนยันทีละรายการ</T>
83         <Card style={{ paddingVertical: space.xs, gap: 0 }}>
84           {review.map((t, i) => (
85             <Reveal key={t.id} index={i} from={10}>
86               {i > 0 ? <Divider /> : null}
87               <TxRow tx={t} />
88             </Reveal>
89           ))}
90         </Card>
91       </View>
92     ) : null}
93   </ScrollView>
94 </SafeAreaView>
95 );
96 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/drafts.tsx:44

บรรทัด	ความหมาย
46–50	หัว: ย้อนกลับ · "สลิปรอยืนยัน" · ไอคอนสแกน (สแกนเพิ่ม)
52–61	ไม่มีแบบร่าง → EmptyState น้องกล้าเชียร์ "เคลียร์หมดแล้ว" + ปุ่ม "สแกนสลิปเพิ่ม"
63–77	กลุ่ม พร้อมยืนยัน (N) : คำอธิบาย 80% + ยอดรวม · การ์ดรายการ (ไฟล์ทีละแถว) · ปุ่มทงมีประกาย (shine) "ยืนยันทั้ง N รายการ"
79–92	กลุ่ม ต้องตรวจสอบ (N) : คำอธิบาย + การ์ดรายการ (แต่ละ TxRow แล้วเปิดฟอร์ม) ไม่มีปุ่มยืนยันรวม

ส่วนที่ 3: transaction.tsx เตรียมข้อมูลของแบบร่าง (บรรทัด 56–60 และ 157)

src/app/transaction.tsx

บรรทัด 56–60

```

56 const cats = categoriesFor(kind);
57 const pickerDate = new Date(bkkToIso(day, time));
58 // The two AI reads disagreed: show both, and offer the amounts as buttons.
59 const altLine = isDraft ? (existing?.note ?? '').split('\n').find((l) =>
60   l.startsWith(ALT_PREFIX)) : undefined;
60 const choices = isDraft && flags.includes('amount') ? amountChoices(existing?.note) : [];

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:56

บรรทัด	ความหมาย
59	altLine: ถ้าเป็นแบบร่าง หาบรรทัดในบันทึกที่ขึ้นต้นด้วย "AI อ่านได้ 2 แบบ:"
60	choices: แบบร่างที่ติดธงยอดเงิน → ดึงยอด 2 ค่าจากบันทึก

และที่บรรทัด 157: `const flagged = (f) => isDraft && flags.includes(f)` ฟังก์ชันสั้นที่ใช้ถามว่า "ช่องนี้ต้องไฮไลต์ไหม" ใช้กับช่องยอดเงิน (บรรทัด 241 `flagged={flagged('amount')}`), ช่องชื่อ (บรรทัด 265 ข้อความ "ตรวจสอบชื่อ

ให้ตรงกับสลิป") และวันที่ (บรรทัด 319, 335)

ส่วนที่ 4: transaction.tsx กล้อง "ช่วยตรวจ" (บรรทัด 194–230)

src/app/transaction.tsx

บรรทัด 194–230

```
194      {isDraft ? (
195        <Card tone="accent">
196          <Row gap={space.sm} align="flex-start">
197            <Ionicons name="receipt-outline" size={20} color={theme.accentInk} />
198            <View style={{ flex: 1, gap: 4 }}>
199              <T v="body" style={{ fontFamily: fonts.sansSemi }}>
200                {flags.length === 0 ? 'อ่านสลิปได้ครบ ตรวจอีกครั้งแล้วกดยืนยัน' : `ช่วยตรวจ:
201                ${flags.map((f) => FLAG_LABEL[f]).join(', ')}`}
202              </T>
203              <T v="small">
204                ยังไม่รวมในยอดเงินจนกว่าจะยืนยัน
205                {existing?.ocrConfidence !== null ? ` · ความมั่นใจต่ำสุด
206                ${Math.round(existing.ocrConfidence * 100)}%` : ''}
207              </T>
208              {altLine ? (
209                <T v="small" color={theme.ink}>
210                  {altLine}
211                </T>
212              ) : null}
213              {choices.length > 1 ? (
214                <Row gap={space.sm} style={{ flexWrap: 'wrap', marginTop: 4 }}>
215                  <T v="small">ดูที่สลิปแล้วเลือก:</T>
216                  {choices.map((c) => (
217                    <Chip
218                      key={c}
219                      label={formatBaht(c)}
220                      selected={parseBahtToSatang(amount) === c}
221                      onPress={() => {
222                        setAmount(satangToInput(c));
223                        setAmountError(null);
224                      }}
225                    />
226                  ))}
227                </Row>
228              ) : null}
229            </View>
230          </Row>
231        </Card>
232      ) : null}
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:194

บรรทัด	ความหมาย
194–195	เฉพาะแบบร่าง: การ์ดสีทอง (tone accent)
197	ไอคอนใบเสร็จ
199–201	ไม่มีธง → "อ่านสลิปได้ครบ ตรวจอีกครั้งแล้วกดยืนยัน" · มีธง → "ช่วยตรวจ: " + ชื่อธงคั่นด้วยจุลภาค
202–205	"ยังไม่รวมในยอดเงินจนกว่าจะยืนยัน" + (ถ้ามี) " · ความมั่นใจต่ำสุด 50%" · <code>!== null</code> = ไม่ใช่ทั้ง <code>null</code> และ <code>undefined</code> (รายการที่ตัวเองไม่มีค่านี้)

บรรทัด	ความหมาย
206–210	มีบรรทัด 2 แบบ → แสดงตามบันทึก
211–226	มีตัวเลือกมากกว่า 1 → "ดูที่สลิปแล้วเลือก:" + ชิปทุกค่า (฿1,250.00) · ชิปที่ตรงกับยอดในช่องตอนนี้เป็นสีเลือก · และ → ใส่ยอดนั้นลงช่องและล้าง error

ส่วนที่ 5: transaction.tsx ปุ่มยืนยัน (บรรทัด 341–350 และ 106–111)

src/app/transaction.tsx	บรรทัด 341–350
<pre> 341 <View style={{ padding: space.lg, gap: space.sm, borderTopWidth: 1, borderTopColor: theme.line, backgroundColor: theme.surface }}> 342 {isDraft ? (343 <> 344 <Button label="ยืนยันและรวมในยอดเงิน" kind="gold" icon="checkmark-circle" onPress={() => save(true)} loading={busy} /> 345 <Button label="บันทึกแบบร่างไว้ก่อน" kind="ghost" onPress={() => save(false)} disabled= {busy} /> 346 </> 347) : (348 <Button label={existing ? 'บันทึกการแก้ไข' : 'บันทึกรายการ'} icon="checkmark" onPress={() => save(true)} loading={busy} /> 349)} 350 </View> </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:341	

บรรทัด	ความหมาย
342–346	แบบร่าง: ปุ่มทอง "ยืนยันและรวมในยอดเงิน" เรียก <code>save(true)</code> · ปุ่มโปร่ง "บันทึกแบบร่างไว้ก่อน" เรียก <code>save(false)</code>
347–349	รายการปกติ: ปุ่มเดียว "บันทึกการแก้ไข" หรือ "บันทึกรายการ"

src/app/transaction.tsx	บรรทัด 106–111
<pre> 106 if (existing) { 107 await updateTx(existing.id, { 108 ...input, 109 ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}), 110 }); 111 toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' }); </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:106	

บรรทัด	ความหมาย
107–110	แก้ไขรายการเดิมด้วยค่าที่ผู้ใช้ตรวจสอบแล้ว และถ้า confirm: <code>status = confirmed</code> และล้างธง · <code>...(เงื่อนไข ? { ... } : {})</code> = เพิ่มช่องเฉพาะเมื่อเงื่อนไขจริง
111	ข้อความ "ยืนยันรายการแล้ว รวมในยอดเงินแล้ว" (ภาพสุดท้ายของบทหน้าจอ)

i ถ้าผู้ใช้แกล้งกดจนไปซ้ำกับสลิปอื่น

ฐานข้อมูลยังมี unique index คมอยู่ ถ้าการแก้ทำให้ slip_ref ซนกัน repo จะโยน DuplicateSlipError แล้วฟอร์มขึ้นข้อความ "มีรายการจากสลิปนี้อยู่แล้ว" (บรทัด 126) แทนข้อความ error ที่ทั่วไป

ส่วนที่ 6: TxRow แสดงตรงเป็นป้าย (บรทัด 57–72)

src/ui/TxRow.tsx

บรรทัด 57–72

```
57     <Row gap={6}>
58       <T v="micro" numberOfLines={1}>
59         {cat.label}
60         {showTime ? ` · ${bkkTime(tx.occurredAt)}` : ''}
61         {tx.source === 'slip' ? (fresh ? ' · จดจากสลิปวันนี้' : ' · จากสลิป') : ''}
62       </T>
63     </Row>
64     {draft ? (
65       <Row gap={4} style={{ flexWrap: 'wrap', marginTop: 2 }}>
66         {tx.reviewFlags.length === 0 ? (
67           <Badge label="พร้อมยืนยัน" tone="good" />
68         ) : (
69           tx.reviewFlags.map((f) => <Badge key={f} label={`ตรวจ${FLAG_LABEL[f as ReviewFlag]
70             ?? f}`} tone="watch" />
71         )}
72       </Row>
73     ) : null}
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/TxRow.tsx:57

บรรทัด ความหมาย

58–62 บรรทัดเล็ก: หมวด · เวลา · "จดจากสลิปวันนี้" (บันทึกวันนี้) หรือ "จากสลิป"

64–71 แบบร่าง: ไม่มีธง → ป้ายเขียว "พร้อมยืนยัน" · มีธง → ป้ายส้มทอง "ตรวจ" + ชื่อธง ต่อธงละป้าย ("ตรวจยอดเงิน", "ตรวจผู้รับ/ผู้โอน")

อ่านโค้ด: AutoScanBanner, ScannerStage และ WaitNotice

3 คอมโพเนนต์เล็กที่ทำให้ FR-4 "รู้สึก" มีชีวิต

ส่วนที่ 1: AutoScanBanner ระหว่างสแกน (บรรทัด 1-48)

src/ui/AutoScanBanner.tsx

บรรทัด 1-48

```
1 /**
2  * Home-screen card for the automatic slip scan (FR-4): progress while it runs,
3  * then what changed, with a way to review or undo.
4  */
5 import { router } from 'expo-router';
6 import { useEffect } from 'react';
7 import { View } from 'react-native';
8 import { BUDDY_NAME } from '../domain/buddy';
9 import { useAutoScan } from '../services/AutoScanProvider';
10 import { Buddy } from './Buddy';
11 import { Button, Card, IconButton, Money, ProgressBar, Row, T } from './components';
12 import { Reveal, ScanBeam, useCelebrate } from './effects';
13 import { useToast } from './feedback';
14 import { space, useTheme } from './theme';
15 import { WaitNotice } from './WaitNotice';
16
17 export function AutoScanBanner() {
18   const theme = useTheme();
19   const toast = useToast();
20   const { state, available, grantAndRun, undoLast, dismiss } = useAutoScan();
21   const celebrate = useCelebrate();
22   const counted = state.phase === 'done' ? state.confirmed.length : 0;
23   useEffect(() => {
24     if (counted > 0) celebrate();
25   }, [counted, celebrate]);
26   if (!available || state.phase === 'idle') return null;
27
28   if (state.phase === 'scanning') {
29     return (
30       <Reveal from={-10}>
31         <Card tone="alt" style={{ paddingVertical: space.md, overflow: 'hidden' }}>
32           <Row gap={space.md}>
33             <View>
34               <Buddy mood="thinking" size={48} />
35               <ScanBeam height={48} active />
36             </View>
37             <View style={{ flex: 1, gap: 6 }}>
38               <T v="body">
39                 {state.total === 0 ? ` ${BUDDY_NAME} กำลังหาสลิปใหม่ในแคทเลอรี...` : ` ${BUDDY_NAME} กำลังอ่านสลิป
ใหม่ ${state.processed}/${state.total} `}
40               </T>
41               {state.total > 0 ? <ProgressBar value={state.processed / state.total} color=
{theme.accent} /> : null}
42               {state.waitUntil ? <WaitNotice until={state.waitUntil} micro /> : null}
43             </View>

```

```

44     </Row>
45   </Card>
46   </Reveal>
47 );
48 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/AutoScanBanner.tsx:1

บรรทัด	ความหมาย
20	อ่านสถานะและคำสั่งจาก Context ของการสแกนอัตโนมัติ
22–25	ตอนเสร็จและมีที่นับในยอด → พลุกกระดาษ (effect ทำงานเมื่อจำนวนเปลี่ยน)
26	ใช้ไม่ได้ (เว็บ, โหมดทดลอง) หรือว่าง → ไม่แสดงอะไร (return null)
28–47	scanning: การ์ดพื้นอ่อนเลื่อนลงมา (Reveal from -10) · นื่องคล้ายรุ่นคิด + แสงสแกนกับตัว · ยังไม่รู้จำนวน → "คล้ายกำลังหาสลิปใหม่ในแคสเลอร์..." รู้แล้ว → "คล้ายกำลังอ่านสลิปใหม่ 0/6" + แถบ · AI ยุ่ง → นับถอยหลังตัวเล็ก

ส่วนที่ 2: ขอสถิติ และเมื่อมีปัญหา (บรรทัด 50–87)

src/ui/AutoScanBanner.tsx

บรรทัด 50–87

```

50  if (state.phase === 'needs_permission') {
51    return (
52      <Card style={{ borderColor: theme.accent, borderWidth: 1.5 }}>
53        <Row gap={space.md} align="flex-start">
54          <Buddy mood="happy" size={52} />
55          <View style={{ flex: 1, gap: 4 }}>
56            <T v="h3">ให้{BUDDY_NAME}จดจากสลิปให้อัตโนมัติใหม่</T>
57            <T v="small">
58              {state.message ?? 'แค่เปิดแอป คล้ายจะหาสลิปใหม่ในแคสเลอร์ อ่านยอด แล้วคำนวณเงินให้ทันที รูปอื่นไม่ออกจากมือ
      ถือ'}
59            </T>
60          </View>
61          <IconButton icon="close" label="ปิด" onPress={dismiss} />
62        </Row>
63        <Button label="อนุญาตเข้าถึงรูปภาพ" kind="gold" icon="lock-open-outline" onPress={() =>
      grantAndRun()} />
64      </Card>
65    );
66  }
67
68  if (state.phase === 'error') {
69    const recorded = state.confirmed.length + state.drafts;
70    return (
71      <Card tone="alt" style={{ paddingVertical: space.md }}>
72        <Row gap={space.md}>
73          <Buddy mood="worried" size={44} still />
74          <View style={{ flex: 1, gap: 2 }}>
75            <T v="small">{state.message}</T>
76            {recorded > 0 ? (
77              <T v="micro">
78                ก่อนหยุด{BUDDY_NAME}จดให้แล้ว {recorded} รายการ
79                {state.drafts > 0 ? ` (รอตรวจ ${state.drafts})` : ''}
80              </T>
81            ) : null}
82          </View>

```

```

83         <IconButton icon="close" label="ปิด" onPress={dismiss} />
84     </Row>
85 </Card>
86 );
87 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/AutoScanBanner.tsx:50

บรรทัด ความหมาย

50–66 **needs_permission:** การ์ดขออนุญาต น้องกล้วยยิ้ม "ให้กล้องจากสลิปให้อัตโนมัติใหม่" + เหตุผล (หรือข้อความที่ Provider ส่งมา) + ✕ + ปุ่มทอง "อนุญาตเข้าถึงรูปภาพ" (grantAndRun)

68–87 **error:** น้องกล้วยกังวล + เหตุผล · ถ้าจอดได้บ้างก่อนหยุด: "ก่อนหยุดกล้องให้แล้ว N รายการ (สอตรวจ M)" + ✕

ส่วนที่ 3: เมื่อเสร็จ (บรรทัด 89–138)

src/ui/AutoScanBanner.tsx

บรรทัด 89–138

```

89 // done
90 const total = state.confirmed.reduce((s, t) => s + (t.kind === 'income' ? t.amountSatang : -
t.amountSatang), 0);
91 return (
92   <Reveal zoom>
93     <Card style={{ borderColor: theme.primary, borderWidth: 1.5 }}>
94       <Row gap={space.md} align="flex-start">
95         <Buddy mood={state.confirmed.length > 0 ? 'cheer' : 'thinking'} size={52} />
96         <View style={{ flex: 1, gap: 4 }}>
97           {state.confirmed.length > 0 ? (
98             <>
99               <T v="h3">
100                 {BUDDY_NAME}จอดให้แล้ว {state.confirmed.length} รายการ
101               </T>
102               <Money satang={total} sign decimals={false} size="h2" color={total >= 0 ?
theme.income : theme.ink} />
103             </>
104           ) : (
105             <T v="h3">{BUDDY_NAME}เจอสลิปใหม่ {state.drafts} ใบ</T>
106           )}
107           <T v="small">
108             {state.confirmed.length > 0 ? 'ยอดเงินและ Money Runway คำนวณใหม่แล้ว' : ''}
109             {state.drafts > 0 ? ` ${state.confirmed.length > 0 ? ' ' : ''}ต้องตรวจ ${state.drafts}
รายการที่อ่านไม่ชัด` : ''}
110           </T>
111         </View>
112         <IconButton icon="close" label="ปิด" onPress={dismiss} />
113       </Row>
114       <Row gap={space.sm}>
115         {state.drafts > 0 ? (
116           <Button label="ตรวจสอบสลิป" small kind="soft" icon="receipt-outline" onPress={() =>
router.push('/drafts')} style={{ flex: 1 }} />
117         ) : (
118           <Button label="ดูรายการ" small kind="soft" icon="list-outline" onPress={() =>
router.navigate('/transactions')} style={{ flex: 1 }} />
119         )}
120         {state.confirmed.length > 0 ? (
121           <Button
122             label="ยังไม่รวมยอด"

```

```

123      small
124      kind="ghost"
125      icon="arrow-undo-outline"
126      onPress={() =>
127        undoLast()
128        .then(() => toast({ message: 'ย้ายไปที่สลิปโปรยีนยันแล้ว' }))
129        .catch(() => toast({ message: 'ทำไม่สำเร็จ ลองอีกครั้ง', tone: 'error' })))
130      }
131      style={{ flex: 1 }}
132    />
133  ) : null}
134 </Row>
135 </Card>
136 </Reveal>
137 );
138 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/AutoScanBanner.tsx:89`

บรรทัด	ความหมาย
90	ยอดรวมของที่นับในยอด: รายรับบวก รายจ่ายลบ ($-350 + 1,200 - 120 = +730$)
92-93	การ์ดชุมเข้า ขอบเขียว
95	มีที่นับในยอด → น้องกล้าเชียร์ ไม่จิ้นครุ่นคิด
97-103	"กล้าจดให้แล้ว N รายการ" + ยอดรวมตัวใหญ่ (เขียวถ้า ≥ 0 ไม่มีทศนิยม มีเครื่องหมาย)
104-106	มีแต่แบบร่าง → "กล้าเจอสลิปใหม่ N ใบ"
107-110	บรรทัดเล็ก: "ยอดเงินและ Money Runway คำนวณใหม่แล้ว" และ/หรือ "ต้องตรวจ N รายการที่อ่านไม่ชัด"
114-119	มีแบบร่าง → ปุ่ม "ตรวจสอบสลิป" ไปหน้าสลิปโปรยีนยัน · ไม่มี → "ดูรายการ" ไปเก็บรายการ (<code>navigate</code> = สลับไปเก็บที่มีอยู่แล้ว)
120-133	มีที่นับในยอด → ปุ่ม "ยังไม่รวมยอด" เรียก <code>undoLast</code> แล้วขึ้นข้อความ "ย้ายไปที่สลิปโปรยีนยันแล้ว" พลาด → "ทำไม่สำเร็จ ลองอีกครั้ง" (<code>.then().catch()</code> = จัดการ Promise แบบต่อสาย)

ส่วนที่ 4: ScannerStage เวทีสแกน (บรรทัด 1-71)

`src/ui/ScannerStage.tsx`

บรรทัด 1-71

```

1  /**
2   * The little stage shown while slips are being read (FR-4): a slip under a
3   * sweeping scan beam, and each newly read slip's amount popping up beside it
4   * with a few sparkles. Everything stops when reading stops.
5   */
6  import { LinearGradient } from 'expo-linear-gradient';
7  import { StyleSheet, View } from 'react-native';
8  import Svg, { G, Rect } from 'react-native-svg';
9  import type { Transaction } from '../domain/types';
10 import { Aurora, Reveal, ScanBeam, Sparkles } from '../effects';
11 import { Money, T } from '../components';
12 import { alpha, palette, radius, space, useTheme } from '../theme';
13
14 const PAPER_W = 84;
15 const PAPER_H = 116;

```

```

16
17 /** A plain slip drawn in code (no bank's look): header, lines, amount and a small QR in the
corner. */
18 function SlipPaper() {
19   const qr = [
20     [0, 0], [1, 0], [2, 0], [0, 1], [2, 1], [0, 2], [1, 2], [2, 2],
21     [4, 0], [5, 1], [4, 2], [3, 3], [5, 3], [0, 4], [2, 4], [4, 4], [1, 5], [3, 5], [5, 5],
22   ];
23   return (
24     <Svg width={PAPER_W} height={PAPER_H} viewBox={`0 0 ${PAPER_W} ${PAPER_H}`}>
25       <Rect x={0} y={0} width={PAPER_W} height={PAPER_H} rx={10} fill="#F7F5EE" />
26       <Rect x={0} y={0} width={PAPER_W} height={20} rx={10} fill="#2E9A68" />
27       <Rect x={0} y={10} width={PAPER_W} height={10} fill="#2E9A68" />
28       <Rect x={10} y={30} width={44} height={5} rx={2.5} fill="#C9D3CB" />
29       <Rect x={10} y={41} width={60} height={5} rx={2.5} fill="#DDE3DD" />
30       <Rect x={10} y={52} width={36} height={5} rx={2.5} fill="#DDE3DD" />
31       <Rect x={10} y={66} width={40} height={9} rx={4.5} fill={palette.gold} opacity={0.85} />
32     <G>
33       {qr.map(([x, y]) => (
34         <Rect key={`-${x}-${y}`} x={52 + x * 4} y={82 + y * 4} width={4} height={4}
35         fill="#1D2A23" />
36       ))}
37     </G>
38   </Svg>
39 );
40
41 export function ScannerStage({ active, latest }: { active: boolean; latest: Transaction | null })
42 {
43   const theme = useTheme();
44   const sign = latest ? (latest.kind === 'income' ? 1 : -1) : 1;
45   return (
46     <View style={{ height: 152, borderRadius: radius.lg, overflow: 'hidden' }}
47     accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
48       <LinearGradient colors={[theme.hero, theme.heroDeep]} start={{ x: 0, y: 0 }} end={{ x: 1,
49       y: 1 }} style={StyleSheet.absoluteFill} />
50       <Aurora cycles={0} strength={0.28} seed={21} />
51       <View style={{ position: 'absolute', left: space.xl, top: 18, width: PAPER_W, height:
52       PAPER_H, transform: [{ rotate: '-4deg' }] }}>
53         <SlipPaper />
54         <ScanBeam height={PAPER_H} active={active} />
55       </View>
56       <Sparkles trigger={latest?.id} count={7} cycles={1} seed={5} area={{ top: 10, bottom: 80 }}
57       />
58       <View style={{ position: 'absolute', left: PAPER_W + space.xl * 2, right: space.lg, top: 0,
59       bottom: 0, justifyContent: 'center' }}>
60         {latest ? (
61           <Reveal key={latest.id} zoom from={14}>
62             <View style={{ backgroundColor: alpha(theme.heroInk, 0.12), borderRadius: radius.lg,
63             borderWidth: 1, borderColor: alpha(theme.heroAccent, 0.45), padding: space.md, gap: 2 }}>
64               <Money satang={sign * latest.amountSatang} sign decimals={false} size="h2" color=
65               {sign > 0 ? theme.heroIncome : theme.heroAccent} />
66               <T v="small" numberOfLines={1} color={theme.heroInk}>
67                 {latest.title}
68               </T>
69             </View>
69           </Reveal>
70         ) : (
71           <T v="small" color={theme.heroInkSoft}>
72             {active ? 'กำลังสแกนสลิป...' : 'พร้อมอ่านสลิป'}
73           </T>
74         )
75       </View>
76     </View>
77   );
78 }

```

```

67     })
68     </View>
69   </View>
70 );
71 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/ScannerStage.tsx:1

บรรทัด	ความหมาย
1–5	เวทีกเล็ก ๆ ระหว่างอ่าน: สลิปใต้แสงสแกน และยอดของสลิปที่เพิ่งอ่านได้ขึ้นข้าง ๆ พร้อมประกาย หยุดทุกอย่างเมื่อหยุดอ่าน
14–15	ขนาดกระดาษสลิปที่วาด 84 × 116
17–39	SlipPaper สลิปที่ วาดด้วยโค้ด (SVG) ไม่ใช่หน้าตาของธนาคารไหน: พื้นครีม หัวเขียว เส้นแทนตัวหนังสือ แถบทองแทนยอดเงิน และ QR เล็ก ๆ มุมขวาล่าง (สีเหลี่ยม 19 ช่องจากพิกัดในอาร์เรย์ qr แต่ละช่อง 4 × 4)
41–43	รับ active (แสงวิ่งไหม) และ latest (รายการล่าสุด) · เครื่องหมายบวก/ลบตามประเภท
45	กล่องสูง 152 มุมโค้ง · <code>accessibilityElementsHidden</code> ช้อนจากโปรแกรมอ่านหน้าจอ (เป็นแค่ของตกแต่ง ข้อมูลจริงอยู่ในข้อความใต้เวทีก)
46–47	พื้นไล่สีเขียวเหมือนการ์ดยอดเงิน + แสงออโรราจาง ๆ
48–51	กระดาษสลิปเอียง -4 องศา และแสงสแกนสูงเท่ากระดาษ
52	ประกาย 7 จุด เล่นทุกครั้งที่ latest เปลี่ยน (trigger = id ของรายการ)
54–62	มีรายการล่าสุด → กล่องโปร่ง ยอดตัวใหญ่ (รายจ่ายสีทอง รายรับสีเขียว) + ชื่อ 1 บรรทัด · <code>key={latest.id}</code> ทำให้ Reveal เล่นแอนิเมชันใหม่ทุกสลิป
63–67	ยังไม่มี → "กำลังส่องหาสลิป..." (หรือ "พร้อมอ่านสลิป")

ส่วนที่ 5: WaitNotice นับถอยหลัง (บรรทัด 1–21)

src/ui/WaitNotice.tsx

บรรทัด 1–21

```

1  /**
2   * "AI มีคิวเยอะ รออีก 17 วินาที..." with a live countdown, shown while the slip
3   * reader waits for the free AI tier's rate limit before trying again.
4   */
5  import { useEffect, useState } from 'react';
6  import { waitMessage } from '../services/slips';
7  import { T } from './components';
8
9  // Rounded, so a countdown that starts at 20 never flashes 21 because of a millisecond of drift.
10 const secondsLeft = (until: number) => Math.max(0, Math.round((until - Date.now()) / 1000));
11
12 export function WaitNotice({ until, micro }: { until: number; micro?: boolean }) {
13   const [left, setLeft] = useState(() => secondsLeft(until));
14   useEffect(() => {
15     const tick = () => setLeft(secondsLeft(until));
16     tick();
17     const id = setInterval(tick, 1000);
18     return () => clearInterval(id);
19   }, [until]);

```



```
20   return <T v={micro ? 'micro' : 'small'}>{left > 0 ? waitMessage(left) : 'กำลังลองอ่านอีกครั้ง...'}</T>;  
21 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/WaitNotice.tsx:1`

บรรทัด	ความหมาย
9–10	วินาทีที่เหลือ ปิดใกล้สุด (ไม่ปิดขึ้น) เพราะนับเริ่ม 20 แล้วเศษมิลลิวินาทีทำให้ขึ้น 21 แวบหนึ่งได้
12	รับเวลาเป้าหมาย (ms) และขนาดตัวอักษร
13	state วินาทีที่เหลือ
14–19	effect: คำนวณทันที แล้วทุก 1,000 ms · return ล้างตัวตั้งเวลา (ไม่จับนับต่อแม้การ์ดหายไปแล้ว)
20	เหลือ > 0 → "AI มีคิวเยอะ รออีก N วินาทีแล้วอ่านต่อให้เอง" · ถึง 0 → "กำลังลองอ่านอีกครั้ง..."

อ่านโค้ด: parse-slip/index.ts — ฟังก์ชันอ่านสลิปบนเซิร์ฟเวอร์

Edge Function 154 บรรทัด รับบน Supabase ด้วย Deno (ภาค ก อธิบายว่า Edge Function คืออะไร)

ส่วนที่ 1: คอมเมนต์ออกแบบ import และขีดจำกัด (บรรทัด 1–23)

supabase/functions/parse-slip/index.tsบรรทัด 1–23

```
1 // FR-4: read one Thai bank or e-wallet slip image and return structured fields.
2 //
3 // Accuracy: every slip is read twice, at the same time, by two different AI
4 // models that do not see each other's answer (_shared/helpers.ts
5 // mergeReadings). Where they agree the value is kept; where they disagree on the
6 // amount, date, direction or who was paid, the slip waits for the user with
7 // both values to choose from. A slip that could be read only once is never
8 // counted without a look. The app then checks the reading against the slip's
9 // QR code (bank and reference) on the phone.
10 //
11 // Privacy: the image is used only for this request. It is not written to the
12 // database or to storage, and only images that already looked like slips on
13 // the phone are sent here. The AI service (Claude or Gemini, see
14 // _shared/common.ts) receives the image to read it.
15 import { aiProvider, BusyError, callAIWithModel, corsHeaders, envLimit, fail, json,
16   NotConfiguredError, requireUser, takeQuota } from '../_shared/common.ts';
17 import { mergeReadings, normalizeReading, parseJsonText, type SlipReadingOut } from
18   '../_shared/helpers.ts';
19
20 const DAILY_LIMIT = envLimit('SLIP_DAILY_LIMIT', 300);
21 const MAX_BASE64_CHARS = 6_000_000; // about 4.5 MB of image
22 /** Checked before the body is read, so a huge upload is refused without filling memory. */
23 const MAX_BODY_BYTES = MAX_BASE64_CHARS + 100_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → supabase/functions/parse-slip/index.ts:1

บรรทัด	ความหมาย
1	FR-4: อ่านสลิปธนาคารหรือ e-wallet 1 รูป คืนข้อมูลเป็นช่อง ๆ
3–9	ความแม่นยำ: ทุกสลิปอ่าน 2 ครั้ง พร้อมกัน ด้วย AI 2 รุ่นที่ไม่เห็นคำตอบกัน ตรงกัน = เกือบ ไม่ตรง (ยอด วันที่ ทิศทาง ผู้รับ) = รอผู้ใช้เลือก อ่านได้ครั้งเดียว = ไม่นับโดยไมดู แล้วแอปตรวจกับ QR อีกชั้น
11–14	ความเป็นส่วนตัว: รูปใช้แค่ในคำขอนี้ ไม่เขียนลงฐานข้อมูลหรือที่เก็บไฟล์ และมาเฉพาะรูปที่ดูเหมือนสลิปบนมือถือแล้ว
15	ของใช้ร่วมจาก common.ts: เลือก AI, ชนิด error, เรียก AI, CORS, ขีดจำกัดจากตั้งค่า, ตอบ error/JSON, ตรวจผู้ใช้, โควตา
16	จาก helpers.ts: รวม 2 ผล, ทำความสะอาด, แปลง JSON
18	โควตาต่อวัน: ค่าจากตั้งค่า SLIP_DAILY_LIMIT หรือ 300

บรรทัด	ความหมาย
19	base64 ไม่เกิน 6,000,000 ตัว (~4.5 MB) · 6_000_000 ชัดล่างช่วยอ่านตัวเลขยาว (เหมือนจุลภาค)
21	ขนาด body สูงสุด เช็กจาก header ก่อน อ่าน body
23	ชนิด "ข้อความหรือ null" ใน JSON Schema

ส่วนที่ 2: SCHEMA – รูปแบบคำตอบที่บังคับ AI (บรรทัด 25–57)

supabase/functions/parse-slip/index.ts

บรรทัด 25–57

```

25 const SCHEMA = {
26   type: 'object',
27   additionalProperties: false,
28   required: [
29     'isSlip', 'direction', 'amount', 'amountPrinted', 'fee', 'dateText', 'dateIso', 'time',
30     'fromName', 'toName', 'counterparty', 'bank', 'reference', 'confidence',
31   ],
32   properties: {
33     isSlip: { type: 'boolean' },
34     direction: { type: 'string', enum: ['expense', 'income', 'unknown'] },
35     amount: nullableString,
36     amountPrinted: nullableString,
37     fee: nullableString,
38     dateText: nullableString,
39     dateIso: nullableString,
40     time: nullableString,
41     fromName: nullableString,
42     toName: nullableString,
43     counterparty: nullableString,
44     bank: nullableString,
45     reference: nullableString,
46     confidence: {
47       type: 'object',
48       additionalProperties: false,
49       required: ['amount', 'date', 'counterparty'],
50       properties: {
51         amount: { type: 'number' },
52         date: { type: 'number' },
53         counterparty: { type: 'number' },
54       },
55     },
56   },
57 };

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/parse-slip/index.ts:25

บรรทัด	ความหมาย
25–27	JSON Schema = "แบบฟอร์ม" ที่บอก AI ว่าต้องตอบเป็นอ็อบเจกต์ และ ห้ามมีช่องอื่นนอกจากที่กำหนด (additionalProperties: false)
28–31	ต้องมีครบ 14 ช่อง (ช่องที่อ่านไม่ได้ให้เป็น null ไม่ใช่ละไว้)
33–34	isSlip เป็น true/false · direction เลือกได้แค่ 3 ค่า (enum)

บรรทัด	ความหมาย
35–37	amount, amountPrinted (ยอดแบบที่พิมพ์ ใช้ตรวจสอบตัวเอง), fee (ค่าธรรมเนียม ใช้กันหยิบผัดบรรทัด)
38–45	วันที่ 2 แบบ, เวลา, ผู้โอน, ผู้รับ, อีเมล, ธนาคาร, เลขอ้างอิง
46–55	ความมั่นใจ 3 ช่อง เป็นตัวเลขทั้งหมด

ส่วนที่ 3: SYSTEM — คำสั่งให้ AI (บรรทัด 59–79)

supabase/functions/parse-slip/index.ts

บรรทัด 59–79

```

59 const SYSTEM = `You read images from a Thai user's phone gallery and extract transfer / payment
    slip data for a personal finance app. Accuracy matters more than anything: a wrong amount or date
    is worse than no answer.
60
61 Return JSON only, matching the schema.
62
63 isSlip is true only for a COMPLETED transfer, payment, top-up or withdrawal confirmation made by
    any Thai bank or e-wallet app, for example: K PLUS or MAKE by KBank (Kasikorn), SCB EASY,
    Krungthai NEXT or เป๋าตัง, Bualuang mBanking (Bangkok Bank), KMA (Krungsri), ttb touch, MyMo (GSB
    ออมสิน), BAAC A-Mobile (ธ.ก.ส.), UOB TMRW, CIMB THAI, KKP Mobile, LH Bank, TISCO, GHB (ธอส.), ICBC,
    Thai Credit, LINE BK, TrueMoney Wallet, ShopeePay, Rabbit LINE Pay, dime!, including PromptPay
    (พร้อมเพย์) transfers and bill payments. Typical words: "โอนเงินสำเร็จ", "ชำระเงินสำเร็จ", "ทำรายการสำเร็จ",
    "รายการสำเร็จ", "เติมเงินสำเร็จ", "จ่ายบิลสำเร็จ", "Transfer successful", "Payment successful", "Completed".
64 isSlip is false for: chats, bills or invoices not yet paid, a QR code to pay (PromptPay QR),
    balance screens, statements with many rows, failed or pending transfers ("ไม่สำเร็จ", "รอดำเนินการ",
    "Failed"), and any other photo. When false, set every other string to null, direction "unknown"
    and all confidences 0.
65
66 Fields:
67 - amount: the amount transferred or paid, digits with a dot for decimals (e.g. "1250.00"). It is
    labelled "จำนวนเงิน", "จำนวน", "ยอดเงิน", "ยอดชำระ", "Amount", or printed large in the middle. Never
    the fee ("ค่าธรรมเนียม", "Fee"), never a balance ("ยอดคงเหลือ", "ยอดคงเหลือ", "Balance"), never a
    reference, phone or account number.
68 - amountPrinted: the same amount exactly as printed, with its commas and decimals (e.g.
    "1,250.00"). It must be the same number as amount.
69 - fee: the fee as digits (e.g. "0.00") when printed, else null.
70 - dateText: the date exactly as printed (e.g. "27 ก.ย. 69", "27 Sep 2026", "27/09/2569").
71 - dateIso: that date as YYYY-MM-DD in the Gregorian calendar. Thai slips print Buddhist Era
    years: 2569 = 2026, and a 2-digit "69" means 2569 = 2026. English slips print Gregorian years.
    Thai months: ม.ค. 01, ก.พ. 02, มี.ค. 03, เม.ย. 04, พ.ค. 05, มิ.ย. 06, ก.ค. 07, ส.ค. 08, ก.ย. 09,
    ต.ค. 10, พ.ย. 11, ธ.ค. 12.
72 - time: HH:MM (24h) as printed ("14:05 น." gives "14:05"), or null.
73 - fromName: the payer as printed ("จาก", "From", "ผู้โอน"), usually at the top. toName: the
    receiver, shop or biller as printed ("ไปยัง", "ไปที่", "To", "ผู้รับ"). Keep masked parts as printed
    (e.g. "นาย สมชาย {***}"). Never include account numbers.
74 - direction: "expense" for a slip that shows money sent or paid (the normal slip the payer's app
    makes); "income" only when the image itself says money was received ("ได้รับเงิน", "รับเงินสำเร็จ", "เงิน
    เข้า", "You received"); otherwise "unknown".
75 - counterparty: for an expense the receiver (toName), for income the sender (fromName).
76 - bank: the bank or wallet app that made the slip, from its logo or name at the top, e.g.
    "KBank", "SCB", "Krungthai", "Bangkok Bank", "Krungsri", "ttb", "GSB", "BAAC", "UOB", "CIMB",
    "KKP", "LH Bank", "TrueMoney", "ShopeePay".
77 - reference: the transaction reference ("เลขที่รายการ", "รหัสอ้างอิง", "หมายเลขอ้างอิง", "Ref No.",
    "Transaction ID"), letters and digits only.

```

78 - confidence: for amount, date and counterparty, your certainty from 0 to 1 that the value is exactly right, character for character. Use below 0.8 if any character is blurred, cropped, covered, small, or could be read two ways (1 or 7, 3 or 8, 5 or 6, 0 or 8, comma or dot). Use 0 when the value is null.

79 - Never guess, and never compute a value that is not printed. If a field cannot be read, return null.`;

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/parse-slip/index.ts:59`

Prompt นี้เขียนเป็นภาษาอังกฤษ (AI เข้าใจคำสั่งภาษาอังกฤษได้แม่นยำที่สุด) แต่มีคำภาษาไทยที่ต้องเจอบนสลิปจริง ทุกประโยคมีเหตุผล:

บรรทัด	ความหมาย และทำไมต้องเขียน
59	บทบาท + "ยอดหรือวันที่ผิด แยกว่าไม่มีคำตอบ" ตั้งนิสัยให้ AI ระวังมากกว่าเดา
61	ตอบ JSON ตามแบบฟอร์มเท่านั้น
63	isSlip จริงเฉพาะการทำรายการ ที่สำเร็จแล้ว พร้อมรายชื่อแอปธนาคารและ e-wallet ไทย (K PLUS, SCB EASY, เป๋าตัง, TrueMoney ...) และคำที่มักเจอ "โอนเงินสำเร็จ" "ชำระเงินสำเร็จ"
64	isSlip เป็นเท็จสำหรับ: แชต บิลที่ยังไม่จ่าย QR สำหรับจ่ายเงิน หน้ายอดคงเหลือ statement หลายแถว และ รายการไม่สำเร็จ/รอดำเนินการ ถ้าเท็จให้ทุกช่องเป็น null ความมั่นใจ 0
67	amount: ป้ายที่มักเจอ ("จำนวนเงิน" "ยอดชำระ" "Amount") และ ห้าม เป็นค่าธรรมเนียม ยอดคงเหลือ เลขอ้างอิง เบอร์โทร หรือเลขบัญชี (ความผิดพลาดที่พบจริง)
68–69	amountPrinted ต้องเป็นเลขเดียวกับ amount · fee เป็นตัวเลข
70–71	วันที่ตามที่พิมพ์ และแบบ ค.ศ. พร้อม สอนการแปลง พ.ศ. ("69" = 2569 = 2026) และตารางเดือนไทยทั้ง 12
72	เวลา 24 ชม. ("14:05 น." → "14:05")
73	ผู้โอน/ผู้รับตามที่พิมพ์ เก็บส่วนที่ถูกปิดไว้ตามเดิม และ ห้ามใส่เลขบัญชี (ความเป็นส่วนตัว)
74	direction: expense สำหรับสลิปที่ส่งเงินออก · income เฉพาะ เมื่อภาพบอกว่าได้รับเงิน · นอกนั้น unknown
75–77	counterparty, bank (ซื้อสันตามรายการ), reference (ตัวอักษรและตัวเลขเท่านั้น)
78	ความมั่นใจ: ความแน่ใจว่าถูก "ทุกตัวอักษร" ต่ำกว่า 0.8 ถ้าเบลอ ตัด บัง เล็ก หรืออ่านได้ 2 แบบ (1/7, 3/8, 5/6, 0/8, จุลภาค/จุด) และ 0 เมื่อเป็น null · ตรงกับเกณฑ์ 80% ของแอป
79	ห้ามเดา และห้ามคำนวณค่าที่ไม่ได้พิมพ์ไว้

ส่วนที่ 4: readOnce — อ่าน 1 รอบ (บรรทัด 81–94)

`supabase/functions/parse-slip/index.ts`

บรรทัด 81–94

```
81 async function readOnce(variant: 'primary' | 'verify', image: string, mediaType: string):
    Promise<{ reading: SlipReadingOut; model: string }> {
82   const { text, model } = await callAIWithModel({
83     task: 'slip',
84     variant,
85     system: SYSTEM,
86     maxTokens: 900,
87     schema: SCHEMA,
```

```

88     content: [
89       { type: 'image', mediaType, data: image },
90       { type: 'text', text: 'Extract the slip fields from this image.' },
91     ],
92   });
93   return { reading: normalizeReading(parseJsonText(text)), model };
94 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/parse-slip/index.ts:81`

บรรทัด	ความหมาย
81	รับชนิดการอ่าน (primary/verify) รูป และชนิดรูป คำนวณที่ทำความสะอาดแล้ว + ชื่อรุ่นที่ตอบ
82–92	เรียก AI: งาน slip · system prompt · คำตอบไม่เกิน 900 token · แบบฟอร์ม · เนื้อหา = รูป + ข้อความ "Extract the slip fields from this image."
93	แปลงข้อความเป็น JSON (ทราฟฟิคหรือประโยชน์รอบ) แล้วทำความสะอาด

ส่วนที่ 5: ตัวจัดการคำขอ (บรรทัด 96–154)

`supabase/functions/parse-slip/index.ts`

บรรทัด 96–154

```

96 Deno.serve(async (req) => {
97   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
98   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
99
100  const userId = await requireUser(req);
101  if (!userId) return fail('unauthorized', 'Sign in again', 401);
102
103  if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
    'Image is too large', 413);
104  let body: { imageBase64?: string; mediaType?: string };
105  try {
106    body = await req.json();
107  } catch {
108    return fail('bad_request', 'Body must be JSON', 400);
109  }
110  if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
    400);
111  const image = body.imageBase64;
112  const mediaType = body.mediaType ?? 'image/jpeg';
113  if (!image || typeof image !== 'string') return fail('bad_request', 'imageBase64 is required',
    400);
114  if (image.length > MAX_BASE64_CHARS) return fail('too_large', 'Image is too large', 413);
115  if (!['image/jpeg', 'image/png', 'image/webp'].includes(mediaType)) {
116    return fail('bad_request', 'Unsupported image type', 400);
117  }
118
119  if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
120
121  try {
122    if (!(await takeQuota(userId, 'slip', DAILY_LIMIT))) {
123      return fail('quota', 'Daily slip limit reached, try again tomorrow', 429);
124    }
125    // Two independent reads at the same time (one slip = one use of the daily limit).
126    const [first, second] = await Promise.allSettled([readOnce('primary', image, mediaType),
      readOnce('verify', image, mediaType)]);

```

```

127     const a = first.status === 'fulfilled' ? first.value : null;
128     const b = second.status === 'fulfilled' ? second.value : null;
129     if (!a && !b) {
130         const reasons = [first, second].map((r) => (r as PromiseRejectedResult).reason);
131         throw reasons.find((r) => r instanceof NotConfiguredError) ?? reasons.find((r) => r
instanceof BusyError) ?? reasons[0];
132     }
133     if (!a || !b) {
134         const why = (!a ? first : second) as PromiseRejectedResult;
135         console.warn(JSON.stringify({ slip: 'single_read', failed: !a ? 'primary' : 'verify',
error: String(why.reason?.message ?? why.reason).slice(0, 200) }));
136     }
137     // Both reads by the same model (no separate verifier configured) are not independent.
138     const sameModel = !!a && !!b && a.model === b.model;
139     const { reading, check } = mergeReadings(a?.reading ?? null, b?.reading ?? null, { sameModel
});
140     console.log(JSON.stringify({ slip: 'checked', reads: check.reads, verified: check.verified,
sameModel, disagree: Object.keys(check.disagree) }));
141     return json({ reading, check });
142 } catch (e) {
143     if (e instanceof NotConfiguredError) {
144         console.error('parse-slip: AI key missing or refused', e.message);
145         return fail('not_configured', 'AI key is not set on the server or was refused', 503);
146     }
147     if (e instanceof BusyError) {
148         console.warn('parse-slip: AI busy', e.message);
149         return fail('busy', 'The AI service is busy, try again shortly', 503);
150     }
151     console.error('parse-slip failed', e);
152     return fail('reader_error', 'Could not read this image', 502);
153 }
154 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/parse-slip/index.ts:96

บรรทัด	ความหมาย
96	<code>Deno.serve</code> = เริ่มเว็บเซิร์ฟเวอร์ ทุกคำขอเรียกฟังก์ชันนี้
97	<code>OPTIONS</code> = เบราวเซอร์ "ถามก่อน" ว่าข้ามโดเมนได้ไหม (CORS) ตอบ ok พร้อม header
98	รับเฉพาะ POST
100–101	ด้าน 1 ผู้ใช้: ไม่มี token หรือ token ใช้ไม่ได้ → 401
103	ด้าน 2 ขนาดจาก header → 413
104–110	อ่าน JSON ไม่ได้ หรือไม่ใช่ไอบีเจกต์ → 400
111–117	ต้องมี imageBase64 เป็นข้อความ ไม่เกินขนาด และชนิดรูปเป็น JPEG/PNG/WebP
119	ไม่มี key ของ AI เลย → 503 not_configured (แอปแสดงข้อความถึงผู้ดูแล)
122–124	ด้าน 3 โควตา: ใช้ครบแล้ว → 429 quota
126	อ่าน 2 รอบพร้อมกัน ด้วย <code>Promise.allSettled</code> (ต่างจาก <code>Promise.all</code> ตรงที่รอจนครบทั้งคู่แม้ตัวหนึ่งพัง และบอกผลแต่ละตัวว่าสำเร็จหรือพัง)
127–128	เอาผลของตัวที่สำเร็จ ตัวที่พังเป็น null

บรรทัด	ความหมาย
129–132	พังทั้งคู่ → โยน error ที่ "สำคัญที่สุด": ไม่มี key ก่อน แล้ว AI ยุ่ง แล้วอย่างอื่น
133–136	พังตัวเดียว → เขียน log ว่าอ่านได้รอบเดียว (เฉพาะข้อความ error 200 ตัว ไม่มีข้อมูลสลิป)
138	2 รอบใช้รุ่นเดียวกันไหม (ไม่ได้ตั้งตัวตรวจแยก)
139	รวม 2 ผล
140	log เฉพาะข้อมูลเมตา: อ่านกี่รอบ ตรงไหม รุ่นเดียวกันไหม ช่องที่ไม่ตรง (แค่ชื่อช่อง ไม่มีค่า)
141	ตอบ { reading, check }
142–153	แปลง error เป็นรหัสที่แอปเข้าใจ: key ผิด/ไม่มี → 503 not_configured · AI ยุ่ง → 503 busy · อื่น ๆ → 502 reader_error

อ่านโค้ด: helpers.ts — เลือกรุ่น AI ทำความสะอาดและรวม 2 ผล

supabase/functions/_shared/helpers.ts เป็นฟังก์ชันล้นทั่วทั้งไฟล์ จึงทดสอบด้วย Vitest ได้ (helpers.test.ts) ส่วนบรรทัด 1–45 อ่านแล้วในภาค ก

ส่วนที่ 1: รุ่น AI และการตัดสินใจเมื่อเรียกไม่สำเร็จ (บรรทัด 47–99)

supabase/functions/_shared/helpers.ts

บรรทัด 47–99

```

47 /** Gemini models to try in order. A model the key cannot use (404, or no free quota) falls
    through to the next. */
48 export function geminiModels(preferred?: string | null): string[] {
49   const list = [preferred?.trim(), 'gemini-3.5-flash-lite', 'gemini-3.1-flash-lite', 'gemini-
    flash-lite-latest', 'gemini-flash-latest'];
50   return [...new Set(list.filter((m): m is string => !!m))];
51 }
52
53 /**
54  * Models for the second, independent read of a slip. A different (larger) model
55  * than the first read makes the two readings independent, so a misread digit
56  * shows up as a disagreement. Each model has its own free quota, so running both
57  * at once does not halve how many slips can be read.
58  */
59 export function verifierModels(preferred?: string | null): string[] {
60   const list = [preferred?.trim(), 'gemini-flash-latest', 'gemini-3.5-flash', 'gemini-3.1-flash-
    lite', 'gemini-flash-lite-latest'];
61   return [...new Set(list.filter((m): m is string => !!m))];
62 }
63
64 /** The key itself was refused (wrong, deleted, or not allowed for this API). */
65 export function isInvalidKey(status: number, body: string): boolean {
66   if (status === 401 || status === 403) return true;
67   return status === 400 && /API_KEY_INVALID|API key not valid|invalid x-api-key/i.test(body);
68 }
69
70 /** A 429 that means "this model has no quota on this key at all" rather than "slow down". */
71 export function isZeroQuota(body: string): boolean {
72   return /limit:\s*0\b/.test(body);
73 }
74
75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';

```

```

88 if (status === 404) return 'next_model';
89 if (status === 429 && isZeroQuota(body)) return 'next_model';
90 if (status === 400 && usingJsonSchema) return 'plain_schema';
91 if (status === 429 || status === 500 || status === 503) return 'busy';
92 return 'fail';
93 }
94
95 /** Gemini 429 bodies carry RetryInfo such as "retryDelay": "7s". Returns milliseconds, or null.
96 */
97 export function retryDelayMs(body: string): number | null {
98   const m = body.match(/"retryDelay"\s*:\s*"(\d+(?:\.\d+)?)s"/);
99   return m ? Math.ceil(Number(m[1]) * 1000) : null;
100 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:47

บรรทัด	ความหมาย
48–51	geminiModels : รุ่นที่ผู้ดูแลตั้ง (ถ้ามี) ขึ้นก่อน ตามด้วย 4 รุ่น flash-lite → flash · กรองคำว่าง และตัดซ้ำด้วย Set
53–62	verifierModels : รุ่นสำหรับการอ่านครั้งที่ 2 เริ่มจากรุ่น ใหญ่กว่า และต่างจากตัวหลัก ทำให้ 2 ผลเป็นอิสระต่อกัน ตัวเลขที่อ่านผิดจะโผล่เป็น "ไม่ตรงกัน" · แต่ละรุ่นมีโควตาฟรีของตัวเอง
65–68	isInvalidKey : 401/403 หรือ 400 ที่มีข้อความ key ไม่ถูกต้อง
71–73	isZeroQuota : 429 ที่ข้อความมี "limit: 0" = key นี้ใช้รุ่นนี้ไม่ได้เลย (ไม่ใช่แค่ "ช้าลงหน่อย")
75–84	5 ทางเลือกหลังเรียก Gemini ไม่สำเร็จ: key ผิด (หยุด) · ลองแบบฟอร์มในข้อความแทน · ไปรุ่นถัดไป · ยุ่ง (ลองรุ่นอื่นก่อน รอเมื่อทุกรุ่นยุ่ง) · ล้มเหลว
86–93	geminiStep ตามลำดับ: key ผิด → invalid_key · 404 (ไม่มีรุ่นนี้) → next_model · 429 + limit 0 → next_model · 400 ขณะใช้ JSON Schema → plain_schema · 429/500/503 → busy · อื่น → fail (ผลจริง: 418 → fail)
96–99	retryDelayMs : หา "retryDelay": "7s" ในข้อความของ Google → 7,000 ms (ไม่เจอ → null)

ส่วนที่ 2: อ่านข้อความตอบของ AI (บรรทัด 101–122)

supabase/functions/_shared/helpers.ts

บรรทัด 101–122

```

101 /** Text of the first Gemini candidate, without "thought" parts. */
102 export function geminiText(data: unknown): string {
103   const parts = (data as { candidates?: { content?: { parts?: { text?: string; thought?: boolean }[] } } })
104     ?.candidates?.[0]?.content?.parts;
105   return (parts ?? [])
106     .filter((p) => typeof p.text === 'string' && !p.thought)
107     .map((p) => p.text)
108     .join('');
109 }
110
111 /** Parse a JSON reply; tolerates ``json fences or a sentence around the object. */
112 export function parseJsonText(text: string): unknown {
113   const trimmed = text.trim().replace(/^``(?:json)?\s*/i, '').replace(/\s*``$/, '');
114   try {
115     return JSON.parse(trimmed);
116   } catch {
117     const start = trimmed.indexOf('{');

```

```

118     const end = trimmed.lastIndexOf('}');
119     if (start >= 0 && end > start) return JSON.parse(trimmed.slice(start, end + 1));
120     throw new Error('AI reply was not JSON');
121 }
122 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:101

บรรทัด	ความหมาย
102–109	geminiText: ดึงข้อความของคำตอบแรก ตัดส่วน "ความคิด" (thought) ของรุ่นที่คิดก่อนตอบ แล้วต่อกัน
112–113	parseJsonText: ตัดช่องว่าง และรัวโค้ด ```json ... ``` ที่บางรุ่นชอบใส่
114–115	ลองแปลง JSON ตรง ๆ
116–119	ไม่ได้ → หา { แรก ถึง } สุดท้ายแล้วลองใหม่ (เพื่อมีประโยชน์นำ เช่น "Here: {...} done")
120	ยังไม่ได้ → error

ส่วนที่ 3: SlipReadingOut และตัวช่วยทำความสะอาด (บรรทัด 136–194)

supabase/functions/_shared/helpers.ts

บรรทัด 136–194

```

136 export interface SlipReadingOut {
137   isSlip: boolean;
138   direction: 'expense' | 'income' | 'unknown';
139   amount: string | null;
140   dateText: string | null;
141   dateIso: string | null;
142   time: string | null;
143   counterparty: string | null;
144   bank: string | null;
145   reference: string | null;
146   /** Payer and receiver as printed (the app works out which one is the user). */
147   fromName: string | null;
148   toName: string | null;
149   confidence: { amount: number; date: number; counterparty: number };
150 }
151
152 /** Longest text kept from a reading: names become the transaction title (the database allows
153    120). */
154 export const MAX_TEXT = 100;
155 /** Largest amount a slip can carry, in baht (the app takes up to $1,000,000,000). */
156 export const MAX_AMOUNT_BAHT = 1_000_000_000;
157
158 function text(v: unknown): string | null {
159   if (typeof v === 'number' && Number.isFinite(v)) return String(v);
160   if (typeof v !== 'string') return null;
161   const s = v.trim();
162   return s && s.toLowerCase() !== 'null' ? s.slice(0, MAX_TEXT).trim() : null;
163 }
164
165 function amountText(v: unknown): string | null {
166   if (typeof v === 'number') return Number.isFinite(v) && v > 0 && v <= MAX_AMOUNT_BAHT ?
167     v.toFixed(2) : null;
168   const s = text(v);
169   if (!s) return null;

```

```

168   const cleaned = s.replace(/฿|บาท|THB|,|\s/gi, '').replace(/^[+\\-]/, '');
169   return /^\\d+(\\.\\d{1,2})?$/i.test(cleaned) && Number(cleaned) > 0 && Number(cleaned) <=
      MAX_AMOUNT_BAHT ? cleaned : null;
170 }
171
172 function isoDate(v: unknown): string | null {
173   const s = text(v);
174   const m = s?.match(/^(\\d{4})-(\\d{2})-(\\d{2})$/);
175   if (!m) return null;
176   let year = Number(m[1]);
177   if (year > 2400) year -= 543; // Buddhist Era year left unconverted
178   return `${year}-${m[2]}-${m[3]}`;
179 }
180
181 function clockTime(v: unknown): string | null {
182   const m = text(v)?.match(/^(\\d{1,2})[:.](\\d{2})/);
183   if (!m) return null;
184   const h = Number(m[1]);
185   const min = Number(m[2]);
186   if (h > 23 || min > 59) return null;
187   return `${String(h).padStart(2, '0')}:${m[2]}`;
188 }
189
190 function score(v: unknown): number {
191   const n = typeof v === 'string' ? Number(v) : v;
192   if (typeof n !== 'number' || !Number.isFinite(n)) return 0;
193   return Math.min(1, Math.max(0, n));
194 }

```

▶ เปิดใน VS Code (เต็ม) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:136

บรรทัด	ความหมาย
136–150	SlipReadingOut หน้าตาผลที่ส่งกลับแอป (ตรงกับ SlipReading ของแอป)
153	ข้อความยาวไม่เกิน 100 ตัว (ชื่อกลายเป็นชื่อรายการ ฐานข้อมูลรับ 120)
155	ยอดสูงสุด 1,000,000,000 บาท
157–162	text : ตัวเลขแปลงเป็นข้อความ · ไม่ใช่ข้อความ → null · ตัดช่องว่าง · ว่างหรือ "null" → null · ตัดให้ไม่เกิน 100
164–170	amountText : ตัวเลข → ทศนิยม 2 ตำแหน่งถ้า > 0 และไม่เกินสูงสุด · ข้อความ → ลบ ฿ บาท THB คอมมา ช่องว่าง (ธง = ทุกตัว ไม่สนตัวพิมพ์) และเครื่องหมายนำหน้า + - แล้วต้องเป็นตัวเลขทศนิยมไม่เกิน 2 ตำแหน่ง
172–179	isoDate : ต้องเป็น YYYY-MM-DD · ปี > 2400 = ลืมแปลง พ.ศ. → ลบ 543
181–188	clockTime : ชั่วโมง:นาที หรือ ชั่วโมง.นาที ตรวจสอบเพิ่ม 0
190–194	score : ความมั่นใจเป็นตัวเลข 0–1 (ข้อความ "0.9" ก็รับ) อย่างอื่น → 0

ส่วนที่ 4: normalizeReading ฝั่งเซิร์ฟเวอร์ (บรรทัด 196–253)

supabase/functions/_shared/helpers.ts

บรรทัด 196–253

```

196 /**
197  * Make a slip reading safe for the app, whichever model produced it:
198  * unknown values become null, and a null value never keeps a high confidence
199  * (so an unreadable field always sends the slip to "รอยืนยัน").

```

```

200 *
201 * The amount is also checked against itself: the model writes it twice (as
202 * digits, and exactly as printed with commas), and a slip whose two amounts do
203 * not match, or whose amount equals the fee, gets no confidence in its amount.
204 */
205 export function normalizeReading(raw: unknown): SlipReadingOut {
206   const r = (raw ?? {}) as Record<string, unknown>;
207   const conf = (r.confidence ?? {}) as Record<string, unknown>;
208   const empty: SlipReadingOut = {
209     isSlip: false,
210     direction: 'unknown',
211     amount: null,
212     dateText: null,
213     dateIso: null,
214     time: null,
215     counterparty: null,
216     bank: null,
217     reference: null,
218     fromName: null,
219     toName: null,
220     confidence: { amount: 0, date: 0, counterparty: 0 },
221   };
222   if (r.isSlip !== true) return empty;
223
224   const amount = amountText(r.amount);
225   const printed = r.amountPrinted === undefined ? undefined : amountText(r.amountPrinted);
226   const fee = amountText(r.fee);
227   // Written twice and the two differ, or it is the fee: the amount is not trusted.
228   const amountDoubtful =
229     !!amount && ((printed !== undefined && (printed === null || Number(printed) !==
Number(amount))) || (!!fee && Number(fee) === Number(amount)));
230   const dateText = text(r.dateText);
231   const dateIso = isoDate(r.dateIso);
232   const counterparty = text(r.counterparty);
233   const reference = text(r.reference)?.replace(/[^\0-9A-Za-z]/g, '') || null;
234   const direction = r.direction === 'expense' || r.direction === 'income' ? r.direction :
'unknown';
235   return {
236     isSlip: true,
237     direction,
238     amount,
239     dateText,
240     dateIso,
241     time: clockTime(r.time),
242     counterparty,
243     bank: text(r.bank),
244     reference,
245     fromName: text(r.fromName),
246     toName: text(r.toName),
247     confidence: {
248       amount: amount && !amountDoubtful ? score(conf.amount) : 0,
249       date: dateIso || dateText ? score(conf.date) : 0,
250       counterparty: counterparty ? score(conf.counterparty) : 0,
251     },
252   };
253 }

```

บรรทัด	ความหมาย
196–204	ถ้าผลของรุ่นไหนก็ได้ให้ปลอดภัย: ค่าที่ไม่รู้ → null และ ค่า null ต้องไม่มีความมั่นใจสูง · ยอดเงินตรวจกับตัวเอง (เขียน 2 แบบ) และกับค่าธรรมเนียม
206–207	รับอะไรก็ได้ (unknown) แปลงเป็นอ็อบเจกต์ (null → {})
208–221	ผลว่างมาตรฐาน: ไม่ใช่สลิป ทุกช่อง null ความมั่นใจ 0
222	isSlip ต้องเป็น true จริง ๆ (ไม่ใช่ "true" หรือ 1) ไม่จับคืนผลว่าง
224–226	ยอด, ยอดแบบพิมพ์ (ไม่ส่งมาเลย = undefined แยกจาก null), ค่าธรรมเนียม
228–229	amountDoubtful: มียอด และ (มียอดแบบพิมพ์แต่อ่านไม่ได้หรือไม่เท่ากัน หรือ มีค่าธรรมเนียมเท่ากับยอด)
230–234	วันที่ 2 แบบ ชื่อ เลขอ้างอิง (เฉพาะตัวอักษรและตัวเลข) ทิศทาง (ไม่ใช่ 2 ค่าที่รู้จัก → unknown)
235–246	ประกอบผลที่สะอาด
248	ความมั่นใจยอด = ของ AI เฉพาะเมื่อมียอดและไม่น่าสงสัย ไม่เกิน 0
249	ความมั่นใจวันที่ = ของ AI เฉพาะเมื่อมีวันที่อย่างน้อย 1 แบบ
250	ความมั่นใจชื่อ = ของ AI เฉพาะเมื่อมีชื่อ

ส่วนที่ 5: คำคงที่และตัวช่วยเทียบ (บรรทัด 255–341)

supabase/functions/_shared/helpers.ts

บรรทัด 255–341

```

255 // -----
256 // Two independent reads of the same slip
257 // -----
258
259 /** Below the app's 0.8 rule, so the field is shown to the user to check. */
260 export const UNSURE = 0.5;
261 /** A single read that could not be double-checked is never counted without a look. */
262 export const UNCHECKED_MAX = 0.79;
263
264 export type CheckedField = 'amount' | 'date' | 'direction' | 'counterparty';
265
266 export interface SlipCheck {
267   /** Both reads agreed on every key field (or were merged without a doubt). */
268   verified: boolean;
269   /** How many independent reads were made (1 when the second one could not be made). */
270   reads: number;
271   /** Fields the two reads disagree on, with both values, for the user to choose from. */
272   disagree: Partial<Record<CheckedField, [string | null, string | null]>>;
273 }
274
275 const THAI_MONTHS: Record<string, string> = {
276   'ม.ค.': '01', 'ก.พ.': '02', 'มี.ค.': '03', 'พ.ย.': '04', 'พ.ค.': '05', 'มิ.ย.': '06',
277   'ก.ค.': '07', 'ส.ค.': '08', 'ก.ย.': '09', 'ต.ค.': '10', 'พ.ย.': '11', 'ธ.ค.': '12',
278 };
279
280 /** "27 ก.ย. 69" and "27ก.ย.2569" and "27 ก.ย. 2569" are the same date. */
281 function dateTextKey(s: string | null): string | null {
282   if (!s) return null;
283   let t = s.replace(/\\s+/g, '');

```

► ပေါ်ရဲ VS Code (လိပ်) GitHub

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/helpers.ts:255`

บรรทัด	ความหมาย
260	UNSURE = 0.5 ต่ำกว่า 0.8 ของแอป ช่งนั้นจะถูกแสดงให้ตรวจ
262	UNCHECKED_MAX = 0.79 ผลที่ตรวจซ้ำไม่ได้ ไม่ถูกนับโดยไม่ดู
266–273	SlipCheck ฝั่งเซิร์ฟเวอร์ (ตรงกับของแอป)
275–278	ตารางเดือนไทยแบบมีจุด → เลขเดือน
281–291	dateTextKey : ลบช่องว่าง แทนชื่อเดือนด้วย /เลข/ หา วัน เดือน ปี · ปี 2 หลัก: $\geq 40 \rightarrow 25xx$ (พ.ศ.) ไม่เกิน $20xx$ · ปี > 2400 ลบ 543 · ได้คีย์ YYYY-MM-DD ("27 ก.ย. 69" และ "27ก.ย.2569" ได้คีย์เดียวกัน)
293–299	sameDate : ISO ตรงกัน → ใช่ · ไม่เกินเทียบคีย์ของตัวหนังสือ · ไม่มีวันที่ทั้งคู่ → ถือว่าตรง
301	คำนำหน้าชื่อ (เหมือนในแอป)
304–311	nameLetters : ตัดคำนำหน้า ตัวพิมพ์เล็ก ลบช่องว่าง จุด ชัด และเครื่องหมายปิดบัง ("นาย สมชาย ใ***" → "สมชายใ")
313–325	editDistance (Levenshtein): จำนวนครั้งน้อยที่สุดที่ต้อง เพิ่ม ลบ หรือเปลี่ยนตัวอักษร ให้ 2 คำเหมือนกัน คำนวณด้วยตาราง แต่เก็บแค่แถวเดียว (prev) เพื่อประหยัดหน่วยความจำ
331–339	similarName : ว้างทั้งคู่ = ตรง ว้างข้างเดียว = ไม่ตรง · ตัวอักษรเหมือนกัน · ตัวสั้น (≥ 3) เป็นจุดเริ่มของตัวยาว · ต่างไม่เกิน floor(ความยาว ÷ 5)
341	higher ตัวช่วยเลือกค่าที่มากกว่า

ส่วนที่ 6: mergeReadings และ mergeTwo (บรรทัด 343–441)

supabase/functions/_shared/helpers.ts

บรรทัด 343–441

```

343 /**
344  * Put two independent readings of one slip together. Where they agree the
345  * value is kept (with the higher confidence); where they disagree on the
346  * amount, date, direction or who was paid, the field is marked unsure so the
347  * slip waits for the user, and both values are returned to choose from.
348  * One reading only (the second could not be made), or both made by the same model
349  * (their mistakes would agree): confidences are capped so nothing is counted
350  * without a look.
351  */
352 export function mergeReadings(
353   a: SlipReadingOut | null,
354   b: SlipReadingOut | null,
355   opts: { sameModel?: boolean } = {},
356 ): { reading: SlipReadingOut; check: SlipCheck } {
357   const merged = mergeTwo(a, b);
358   if (!opts.sameModel || merged.check.reads < 2) return merged;
359   const c = merged.reading.confidence;
360   return {
361     reading: { ...merged.reading, confidence: { amount: Math.min(c.amount, UNCHECKED_MAX), date:
Math.min(c.date, UNCHECKED_MAX), counterparty: Math.min(c.counterparty, UNCHECKED_MAX) } },
362     check: { ...merged.check, verified: false },
363   };
364 }
365
366 function mergeTwo(a: SlipReadingOut | null, b: SlipReadingOut | null): { reading: SlipReadingOut;
check: SlipCheck } {

```



```

367   const only = a ?? b;
368   if (!only) throw new Error('No reading to merge');
369   if (!a || !b) {
370     const c = only.confidence;
371     return {
372       reading: { ...only, confidence: { amount: Math.min(c.amount, UNCHECKED_MAX), date:
Math.min(c.date, UNCHECKED_MAX), counterparty: Math.min(c.counterparty, UNCHECKED_MAX) } },
373       check: { verified: false, reads: 1, disagree: {} },
374     };
375   }
376   if (!a.isSlip && !b.isSlip) return { reading: a, check: { verified: true, reads: 2, disagree:
{} } };
377   if (a.isSlip !== b.isSlip) {
378     // One model sees a slip, the other does not: keep the slip, but nothing in it is sure.
379     const slip = a.isSlip ? a : b;
380     return {
381       reading: { ...slip, confidence: { amount: Math.min(slip.confidence.amount, UNSURE), date:
Math.min(slip.confidence.date, UNSURE), counterparty: Math.min(slip.confidence.counterparty,
UNSURE) } },
382       check: { verified: false, reads: 2, disagree: {} },
383     };
384   }
385
386   const disagree: SlipCheck['disagree'] = {};
387   const conf = { ...a.confidence };
388
389   const amountAgrees = !!a.amount && !!b.amount && Number(a.amount) === Number(b.amount);
390   // Agreeing reads keep the higher confidence, unless one of them doubted its own amount.
391   const lowest = Math.min(a.confidence.amount, b.confidence.amount);
392   if (amountAgrees) conf.amount = lowest < UNSURE ? lowest : higher(a.confidence.amount,
b.confidence.amount);
393   else {
394     conf.amount = Math.min(a.confidence.amount, UNSURE);
395     if (a.amount || b.amount) disagree.amount = [a.amount, b.amount];
396   }
397
398   if (sameDate(a, b)) conf.date = higher(a.confidence.date, b.confidence.date);
399   else {
400     conf.date = Math.min(a.confidence.date, UNSURE);
401     disagree.date = [a.dateText ?? a.dateIso, b.dateText ?? b.dateIso];
402   }
403
404   let direction = a.direction;
405   if (a.direction !== b.direction) {
406     if (a.direction === 'unknown') direction = b.direction;
407     else if (b.direction !== 'unknown') {
408       direction = 'unknown';
409       disagree.direction = [a.direction, b.direction];
410     }
411   }
412
413   let counterparty = a.counterparty ?? b.counterparty;
414   if (similarName(a.counterparty, b.counterparty)) {
415     conf.counterparty = higher(a.confidence.counterparty, b.confidence.counterparty);
416     // Prefer the fuller spelling when one of them is masked or cut short.
417     const [la, lb] = [nameLetters(a.counterparty), nameLetters(b.counterparty)];
418     if (lb.length > la.length && lb.startsWith(la)) counterparty = b.counterparty;
419   } else {
420     conf.counterparty = Math.min(a.confidence.counterparty, UNSURE);
421     disagree.counterparty = [a.counterparty, b.counterparty];
422   }

```

```

423
424 // A value only one read found is kept (still unsure, so the user checks it) instead of lost.
425 const dateFrom = a.dateIso || a.dateText ? a : b;
426 const reading: SlipReadingOut = {
427   ...a,
428   amount: a.amount ?? b.amount,
429   dateText: dateFrom.dateText,
430   dateIso: dateFrom.dateIso,
431   direction,
432   counterparty,
433   time: a.time ?? b.time,
434   bank: a.bank ?? b.bank,
435   reference: a.reference ?? b.reference,
436   fromName: similarName(a.fromName, b.fromName) ? a.fromName ?? b.fromName : null,
437   toName: similarName(a.toName, b.toName) ? a.toName ?? b.toName : null,
438   confidence: conf,
439 };
440 return { reading, check: { verified: Object.keys(disagree).length === 0, reads: 2, disagree }
441 };

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/helpers.ts:343`

บรรทัด	ความหมาย
352–356	mergeReadings รับ 2 ผล (อาจเป็น null) และบอกว่าเป็นรุ่นเดียวกันไหม
357–358	รวมด้วย mergeTwo · ไม่ใช่รุ่นเดียวกัน หรืออ่านได้รอบเดียว → ค้นเลย
359–363	รุ่นเดียวกัน: ทดทุกความมั่นใจไม่เกิน 0.79 และ verified = false (รุ่นเดียวกันผิดเหมือนกันได้ ความตรงกันจึงไม่ใช่หลักฐาน)
367–368	ไม่มีผลเลย → error (ไม่ควรเกิด เพราะผู้เรียกเช็คแล้ว)
369–375	ผลเดียว: ทดทุกช่อง $\leq 0.79 \cdot \text{reads} = 1$
376	ทั้งคู่ว่าไม่ใช่สลิป → ไม่ใช่สลิป (verified)
377–384	เห็นต่างว่าเป็นสลิปไหม → เก็บตัวที่ว่าเป็นสลิป แต่ทุกช่อง ≤ 0.5
386–387	เริ่มรายการช่องที่ไม่ตรง และความมั่นใจตั้งต้นจากผลแรก
389–396	ยอดเงิน: ตรงกันเมื่อมีทั้งคู่และเป็นเลขเดียวกัน · ตรงกัน → ความมั่นใจ "ที่สูงกว่า" ยกเว้นตัวใดตัวหนึ่งสงสัยตัวเอง (< 0.5) ให้ใช้ตัวต่ำ · ไม่ตรง → ≤ 0.5 และจดทั้ง 2 ค่า
398–402	วันที่: ตรง → สูงกว่า · ไม่ตรง → ≤ 0.5 และจด (ตัวหนังสือหรือ ISO)
404–411	ทิศทาง: ตัวแรกไม่รู้ ใช้ตัวที่สอง · รู้ทั้งคู่แต่ต่างกัน → unknown และจด
413–422	ชื่อ: คล้ายกัน → สูงกว่า และเลือกสะกดเต็ม (ถ้าตัวที่ 2 ยาวกว่าและขึ้นต้นด้วยตัวแรก) · ไม่คล้าย → ≤ 0.5 และจด
424–439	ประกอบผลรวม: ค่าที่มีแค่ฝั่งเดียว เก็บไว้ (ยังติดธงให้ตรวจ) ดีกว่าทั้ง · ผู้โอน/ผู้รับเก็บเฉพาะเมื่อ 2 ฝั่งคล้ายกัน
440	verified = ไม่มีช่องที่ไม่ตรงเลย · reads = 2

อ่านโค้ด: callGemini ใน common.ts — เรียก Gemini อย่างทนทาน

supabase/functions/_shared/common.ts บรรทัด 202–268 (ส่วนอื่นของไฟล์อ่านแล้วในภาค ก)

supabase/functions/_shared/common.ts

บรรทัด 202–268

```

202 async function callGemini(req: AiRequest, deadline: number): Promise<{ text: string; model:
    string }> {
203   const key = geminiKey(!);
204   const parts = req.content.map((c) =>
205     c.type === 'text' ? { text: c.text } : { inlineData: { mimeType: c.mediaType, data: c.data }
    },
206   );
207   let useJsonSchema = !!req.schema;
208   let lastError = 'Gemini: no model available';
209   const models = req.variant === 'verify' ? verifierModels(Deno.env.get('GEMINI_VERIFY_MODEL')) :
    geminiModels(Deno.env.get('GEMINI_MODEL'));
210
211   // Try every model; if some were only busy, wait once and try them all again.
212   for (let round = 0; round < 2; round++) {
213     let busyDelayMs: number | null = null;
214     for (const model of models) {
215       for (let attempt = 0; attempt < 2; attempt++) {
216         // maxOutputTokens also covers the model's internal thinking, so leave room.
217         const generationConfig: Record<string, unknown> = { maxOutputTokens: Math.max(2048,
    req.maxTokens * 4) };
218         let system = req.system;
219         if (req.schema) {
220           generationConfig.responseMimeType = 'application/json';
221           if (useJsonSchema) generationConfig.responseJsonSchema = req.schema;
222           else system += '\n\nReply with one JSON object that matches this JSON
    Schema:\n${JSON.stringify(req.schema)}';
223         }
224         const res = await timedFetch(
225           `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent`,
226           {
227             method: 'POST',
228             headers: { 'x-goog-api-key': key, 'content-type': 'application/json' },
229             body: JSON.stringify({
230               systemInstruction: { parts: [{ text: system }] },
231               contents: [{ role: 'user', parts }],
232               generationConfig,
233             }),
234           },
235           deadline,
236         );
237         if (res.ok) {
238           const data = await res.json();
239           const text = geminiText(data);
240           if (!text) {
241             const why = data?.candidates?.[0]?.finishReason ?? data?.promptFeedback?.blockReason
    ?? 'unknown';
242             throw new Error(`Gemini ${model} returned no text (${why})`);
243           }

```

```

244         return { text, model };
245     }
246
247     const detail = await res.text();
248     lastError = `Gemini ${model} ${res.status}: ${detail.slice(0, 300)}`;
249     const step = geminiStep(res.status, detail, useJsonSchema);
250     if (step === 'invalid_key') throw new NotConfiguredError(lastError);
251     if (step === 'fail') throw new Error(lastError);
252     if (step === 'plain_schema') {
253         useJsonSchema = false; // describe the schema in the prompt instead
254         continue;
255     }
256     if (step === 'busy') busyDelayMs = Math.max(busyDelayMs ?? 0, retryDelayMs(detail) ??
3000);
257     break; // next_model or busy: the next model has its own quota
258 }
259 }
260 if (busyDelayMs === null) break; // no model is usable with this key
261 if (round === 1) throw new BusyError(lastError);
262 const waitMs = Math.min(busyDelayMs, 12000);
263 if (Date.now() + waitMs > deadline - FETCH_TIMEOUT_MS / 2) throw new BusyError(lastError);
264 console.warn(JSON.stringify({ ai: 'gemini', task: req.task, waitMs }));
265 await sleep(waitMs);
266 }
267 throw new Error(lastError);
268 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:202`

บรรทัด	ความหมาย
203	key จาก Supabase Secrets (มีแน่ เพราะเลือก provider แล้ว:  บอก TypeScript ว่าไม่ใช่ null)
204–206	แปลงเนื้อหาเป็นรูปแบบของ Gemini: ข้อความ → { text } · รูป → { imageData: { mimeType, data } }
207–208	ใช้ JSON Schema ใหม่ และ error ล่าสุด
209	รายการรุ่น: ตัวตรวจใช้ verifierModels ตัวหลักใช้ geminiModels (ผู้ดูแลกำหนดรุ่นแรกได้ด้วย GEMINI_VERIFY_MODEL / GEMINI_MODEL)
211–212	ลองได้ 2 "ยก": ยกแรกลองทุกรุ่น ถ้ามีรุ่นที่แค่ยุ่ง รอแล้วลองทุกรุ่นอีกยก
213	เวลารอที่ต้องใช้ (null = ไม่มีรุ่นไหนยุ่ง)
214–215	ทุกรุ่น ลองได้ 2 ครั้ง (ครั้งที่ 2 สำหรับแบบฟอร์มในข้อความ)
217	จำนวน token สูงสุด เพื่อการ "คิด" ของรุ่นใหม่ (อย่างน้อย 2,048 หรือ 4 เท่าของที่ขอ)
218–223	ขอคำตอบเป็น JSON · ใช้ JSON Schema ได้ → ส่งแบบฟอร์มทางเทคนิค · ไม่ได้ → แปะแบบฟอร์มต่อท้าย system prompt เป็นข้อความ
224–236	เรียก API ของ Google ด้วย key ใน header (ไม่ใส่ใน URL จึงไม่หลุดใน log) และจำกัดเวลา
237–245	สำเร็จ: ถึงข้อความ ว่าง → error พร้อมเหตุผล (เช่นถูกบล็อก) · มี → คืนข้อความและชื่อรุ่น
247–249	ไม่สำเร็จ: เก็บรายละเอียด 300 ตัว และถามว่าทำอะไรต่อ
250–251	key ผิด → หยุดทันที (not_configured) · ลืมเลวอื่น → หยุด

บรรทัด	ความหมาย
252-255	schema ไม่รองรับ → ลองรุ่นเดิมใหม่แบบฟอร์มในข้อความ
256	ยุง → จำเวลารอที่ยาวที่สุด (จาก retryDelay หรือ 3 วินาที)
257	รุ่นถัดไป (แต่ละรุ่นมีโควตาของตัวเอง)
260	ไม่มีรุ่นไหนแค่งุง (ใช้ไม่ได้ทั้งหมด) → เลิก
261	ยกที่ 2 ยังยุง → BusyError (แอปจะรอ 20/40 วินาทีเอง)
262-265	รอไม่เกิน 12 วินาที และต้องเหลือเวลาพอ (ไม่ขึ้น BusyError) แล้วรอ
267	หลุดลูปมา = ไม่มีรุ่นที่ใช้ได้ → error

≈ นึกภาพโทรสั่งอาหาร

โทรร้านแรก ถ้าเบอร์ไม่มีอยู่จริง (404) หรือร้านไม่รับลูกค้าแบบเรา (limit 0) → โทรร้านถัดไป · ถ้าสายไม่ว่าง (429) → จดไว้แล้วลองร้านถัดไปก่อน · ทุกร้านสายไม่ว่าง → รอสักครู่แล้วโทรวนอีกรอบ · ยังไม่ว่างอีก → บอกลูกค้าว่า "ตอนนี้คนเยอะ" (แอปรอ 20/40 วินาทีแล้วลองใหม่เอง) · ถ้าบัตรเครดิตเราใช้ไม่ได้ (key ผิด) → หยุดเลย โทรร้านไหนก็ไม่ได้

การทดสอบ FR-4

ฉันพิสูจน์ยังงี้ว่าแอปอ่านสลิปถูก ไม่นับซ้ำ และไม่นับสิ่งที่ไม่แน่ใจ

157/157

unit test ทั้งโปรเจกต์ผ่าน (18 ไฟล์ รับซ้ำ 9 ต.ค. 2569)

38 รหัส

unit test ของ FR-4 (TC-04-07, 19-30, 35-42, 52, 54-57, 62-65, 83-85, 87-88)

5 สถานการณ์

E2E ในเบราว์เซอร์จริงกับเซิร์ฟเวอร์จำลอง + MT 6 ขอบบนมือถือจริง

Unit test ทั้งหมดของ FR-4

กลุ่ม	เทสต์	ทดสอบอะไร
วันขึ้นสลิป	TC-04, 05, 06, 07, 85	"ก.ย." ทุกแบบ (บ๊ิกเดิม) · รูปแบบอื่น ๆ · วันที่เป็นไปไม่ได้ (31 ก.พ.) · เวลา · วันที่ที่อยู่ติดเวลาแบบมีจุด ("27 ก.ย. 69 เวลา 14.30.25")
ผลอ่าน → ผู้สมัคร	TC-19, 20, 21, 22, 25	สลิปชัด = ready · < 0.8 = needs_review · วันที่หายหรืออนาคต = งง ไม่เดา · นอกช่วง = ข้าม · หมวดจากชื่อผู้รับ
ซ้ำ	TC-23, 83	ซ้ำด้วยเลขอ้างอิง / แฮชรูป / ยอด+นาท+ผู้รับ · กาแฟแก้วที่ 2 คนละนาทไม่ซ้ำ · สลิปไม่มีเลขอ้างอิงหลายกรณี
QR	TC-24, 57, 64	แกะ QR สลิปและปฏิเสธ QR พร้อมเพย์ · เลือก QR สลิปจากหลาย QR · ดูละเอียดเฉพาะรูปขนาดหน้าจอ และส่วนที่ตัดอยู่ในรูปเสมอ · ธนาคารจาก QR ชนะโลโก้ · บันทึก 2 คำ
state machine	TC-26-30	ใหม่ไปเก่า · เปลี่ยนช่วงระหว่างสแกนไม่ได้ (บ๊ิกเดิม) · ทั้งผลของรอบเก่า · หยุดแล้วทำต่อ · หลีกเวลาเต็มตามลำดับ
สแกนอัตโนมัติ	TC-35, 36, 37, 56	หน้าต่างเวลา 30 วัน/กับ 1 ชม. · ไม่สแกนซ้ำใน 5 นาที · สถานะเริ่มต้นตาม 80% + สวิตช์ · รูปที่อ่านไม่ได้ยังอยู่ในหน้าต่างรอบหน้า
ชื่อของเงิน	TC-65, 84	รายรับ/รายจ่ายจากชื่อ (ไทย อังกฤษ ปิดบาง) · ยังเรียนชื่อใหม่ได้เมื่อตารางเต็ม 30 ชื่อ
ช่วงเวลา	TC-52	วันนี้ = 00:00-23:59 เวลาไทย, 7 วัน = 7 วันเต็ม (ใช้ในผลการสแกน)
เซิร์ฟเวอร์	TC-38-42, 54, 55, 62, 63, 87, 88	เลือก Claude/Gemini · แยก key ผิดกับโควตา 0 · รูนุ่งส่งต่อรุ่นถัดไป · อ่านข้อความตอบ · ทนชื่อ key พิมพ์ผิด · ทำความสะอาดค่า · ค่าว่างไม่มีความมั่นใจ · ยอดตรวจกับตัวเอง · กติการวม 2 ผล · ตัดความยาวให้ฐานข้อมูลรับได้

ตัวอย่างเทสต์ 1: TC-23 กันซ้ำ

src/domain/__tests__/domain.test.tsบรรทัด 264-282

```
264 it('TC-23 duplicates by reference, image hash, or amount+time+payee', () => {
265   const existing = [
266     tx({ day: '2026-09-27', amount: 85, source: 'slip', slipRef: '015270081500ABC123', status: 'draft' } ),
```

```

267     tx({ day: '2026-09-26', amount: 40, source: 'slip', slipImageHash: 'hash-1' })),
268   ];
269   const idx = buildDuplicateIndex(existing);
270   const range = rangeDays('7d', NOW);
271   expect(classifyCandidate(normalizeReading(good, { now: NOW }), { range, index: idx
    })).toBe('duplicate');
272   const sameImage = normalizeReading({ ...good, reference: null }, { imageHash: 'hash-1', now:
    NOW });
273   expect(classifyCandidate(sameImage, { range, index: idx })).toBe('duplicate');
274
275   const noRef = { ...good, reference: null };
276   const saved = tx({ day: '2026-09-27', amount: 85, source: 'slip', title: 'ร้านค้าแพดอยช้าง' });
277   saved.occurredAt = bkkToIso('2026-09-27', '08:15');
278   const idx2 = buildDuplicateIndex([saved]);
279   expect(classifyCandidate(normalizeReading(noRef, { now: NOW }), { range, index: idx2
    })).toBe('duplicate');
280   // Same shop, same amount, different minute: a second coffee, not a duplicate.
281   expect(classifyCandidate(normalizeReading({ ...noRef, time: '15:40' }, { now: NOW }), {
    range, index: idx2 })).toBe('ready');
282   });

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/___tests___/domain.test.ts:264

บรรทัด	ความหมาย
265–268	รายการเดิม 2 รายการ: สลิป ฿85 ที่มีเลขอ้างอิง และสลิป ฿40 ที่มีแฮชรูป "hash-1" แล้วสร้างสมุดกันซ้ำ
270	สลิปดี (เลขอ้างอิงเดียวกัน) → duplicate
271–272	ไม่มีเลขอ้างอิง แต่แฮชรูปเหมือน → duplicate
274–278	ไม่มีเลขอ้างอิงทั้งคู่ ยอด วัน นาที (08:15) และร้านเดียวกัน → duplicate
279–280	ร้านเดิม ยอดเดิม แต่ 15:40 = "กาแฟแก้วที่สอง" ไม่ใช่สลิปซ้ำ → ready (เทสตอนนี้กันไม่ให้เกิดการกันซ้ำ "เขมเกิน" จนรายการจริงหาย)

ตัวอย่างเทส 2: TC-63 รวม 2 ผลอ่าน

supabase/functions/_shared/___tests___/helpers.test.ts

บรรทัด 200–239

```

200   it('TC-63 two reads: agreement is kept, any disagreement on a key field waits for the user', ()
    => {
201     const agreed = mergeReadings(slip(), slip({ confidence: { amount: 0.99, date: 0.9,
    counterparty: 0.9 }, dateText: '27ก.ย.2569' })));
202     expect(agreed.check).toEqual({ verified: true, reads: 2, disagree: {} });
203     expect(agreed.reading.confidence).toEqual({ amount: 0.99, date: 0.96, counterparty: 0.95 });
204
205     const amount = mergeReadings(slip(), slip({ amount: '1280.00', amountPrinted: '1,280.00' })));
206     expect(amount.check.verified).toBe(false);
207     expect(amount.check.disagree.amount).toEqual(['1250.00', '1280.00']);
208     expect(amount.reading.confidence.amount).toBeLessThan(0.8);
209     expect(amount.reading.confidence.date).toBe(0.96);
210
211     const date = mergeReadings(slip(), slip({ dateText: '21 ก.ย. 69', dateIso: '2026-09-21' })));
212     expect(date.check.disagree.date).toEqual(['27 ก.ย. 69', '21 ก.ย. 69']);
213     expect(date.reading.confidence.date).toBeLessThan(0.8);
214
215     const direction = mergeReadings(slip(), slip({ direction: 'income' })));

```

```

216     expect(direction.reading.direction).toBe('unknown');
217     expect(mergeReadings(slip({ direction: 'unknown' }),
    slip()).reading.direction).toBe('expense');
218
219     // A masked name and the full name are the same person; a different shop is not.
220     const masked = mergeReadings(slip({ counterparty: 'ร้านข้าวมันไก่ป่า***' }), slip());
221     expect(masked.check.verified).toBe(true);
222     expect(masked.reading.counterparty).toBe('ร้านข้าวมันไก่ป่าแดง');
223     expect(similarName('นาย สมชาย ใจดี', 'สมชาย ใจดี')).toBe(true);
224     expect(similarName('MR. SOMCHAI JAIDEE', 'Somchai Jaidee')).toBe(true);
225     expect(similarName('ร้านกาแฟดอยช้าง', 'ร้านข้าวมันไก่ป่าแดง')).toBe(false);
226     const other = mergeReadings(slip(), slip({ counterparty: 'ร้านกาแฟดอยช้าง' }));
227     expect(other.check.disagree.counterparty).toBeTruthy();
228
229     // Only one read could be made: nothing counts without a look.
230     const single = mergeReadings(slip(), null);
231     expect(single.check).toEqual({ verified: false, reads: 1, disagree: {} });
232
233     expect(Math.max(...Object.values(single.reading.confidence))).toBeLessThanOrEqual(UNCHECKED_MAX);
234     // One model sees a slip, the other does not.
235     const notSure = mergeReadings(slip(), normalizeReading({ isSlip: false }));
236     expect(notSure.reading.isSlip).toBe(true);
237     expect(notSure.check.verified).toBe(false);
238     expect(notSure.reading.confidence.amount).toBeLessThan(0.8);
239     expect(mergeReadings(normalizeReading({ isSlip: false }), normalizeReading({ isSlip: false
    })).reading.isSlip).toBe(false);
240 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/__tests__/helpers.test.ts:200

บรรทัด	ความหมาย
201–203	ตรงกัน (แม้ตัวหนังสือวันที่เขียนต่างแบบ "27ก.ย.2569") → verified และความมั่นใจเอาตัวสูงกว่า 0.99 / 0.96 / 0.95
205–209	ยอดต่าง → ไม่ verified จดทั้ง 2 ค่า ความมั่นใจยอด < 0.8 แต่ ช่องอื่นไม่โดนลดตาม (วันที่ยัง 0.96)
211–213	วันที่ต่าง → จด 2 ค่าเป็นตัวหนังสือ
215–217	ทิศทางขัดกัน → unknown · ตัวหนึ่งไม่รู้ → ใช้ของอีกตัว
219–227	ชื่อปิดบางกับชื่อเต็ม = คนเดียวกัน (ได้สะกดเต็ม) · ร้านต่างกัน = ไม่ตรง
229–232	อ่านได้รอบเดียว → ทุกช่อง ≤ 0.79
233–238	ตัวหนึ่งว่าเป็นสลิป อีกตัวว่าไม่ใช่ → เกิดเป็นสลิปแต่ไม่แน่ใจ · ทั้งคู่ว่าไม่ใช่ → ไม่ใช่สลิป

E2E: เบราวเซอร์จริง เซิร์ฟเวอร์จำลอง (e2e/web-auth.mjs)

ข้อ	สถานการณ์	สิ่งที่ตรวจ
9	สแกนบนเว็บ: เลือกรูปครั้งเดียว	ไม่มีให้เลือกช่วงก่อน · AI ยุ่ง → ข้อความรอฟพร้อมตัวเลขที่ ลดลงจริง (≤ 20 และลดอย่างน้อย 4 ในช่วงที่วัด) · อ่านจบเอง · ที่ชักรวมยอดทันที · วันนี้ 00:00–23:59 ออก \$120 · 7 วันมีรายรับเมื่อวาน · แยกตามวัน · "เสร็จ" กลับหน้าหลัก

ข้อ	สถานการณ์	สิ่งที่ตรวจ
15	สลิปไม่ชัด + เน็ตหลุด	เน็ตหลุด 2 ครั้ง (ครั้งแรก + ลองซ้ำหลัง 3 วิ) → หยุดพร้อมเหตุผลและปุ่ม "อ่านต่อ" (ไม่นับว่าอ่านไม่ได้) · อ่านต่อ → ได้ 2 รายการ นับ 1 · ที่ไม่ชัดอยู่ "ต้องตรวจสอบก่อน (1)" · ยืนยันแล้วจึงเป็น confirmed · "เคลียร์หมดแล้ว"
16	QR เล็กมุมขวาล่าง	หา QR เจอ · slip_ref = เลขจาก QR (016271094231BTF05678) ไม่ใช่ที่ AI อ่าน · บันทึก = "สลิปจาก KBank" (จากระหัส 004 ไม่ใช่โลโก้)
17	Android บนแท็บเล็ตจอ	เปิดแอปแล้วอ่านเองโดยไม่เลือกรูป · 6 รูป ส่ง AI 3 ครั้งพอดี · เลขอ้างอิงจาก QR · ยอด ฿5,730 กันที · หน้าสแกนคืนเองแล้วบอก "ทุกรูปเคยตรวจแล้ว (6 รูป)"
18	AI 2 ตัวอ่านยอดไม่ตรง	เป็นแบบร่างที่มีธง amount · หน้าตรวจแสดง "AI อ่านได้ 2 แบบ" + "ดูที่สลิปแล้วเลือก" · แต่ละ ฿1,280.00 แล้วยืนยัน → บันทึก 128,000 สตางค์ และ confirmed

MT: ทดสอบด้วยมือบนมือถือจริง (docs/TRACEABILITY.md)

MT	ขั้นตอน	ผลที่ต้องได้
MT-04	สแกนแท็บเล็ตที่มีสลิปจริง 3 ใบ	ได้รายการ 3 รายการ ยอดตรงสลิป
MT-05	ปฏิเสธสิทธิ์รูปภาพ	แอปอธิบายและเสนอ "เลือกรูปเอง"
MT-06	ยืนยันแบบร่างทั้งหมด	ยอดคงเหลือรวมรายการเหล่านั้น
MT-07	สแกนรูปเดิมซ้ำ (ล้างประวัติสแกนก่อน)	ขึ้น "ซ้ำ" ไม่เกิดรายการใหม่
MT-09	หยุดชั่วคราวระหว่างสแกน แล้วอ่านต่อ	ทำต่อจากรูปที่ค้าง
MT-11	เซฟสลิปโอนเงินจริงแล้วเปิดแอป	ไม่กี่วินาทีขึ้นการ์ด ยอดและ Runway เปลี่ยนกันที

! พุดตรง ๆ เรื่องสถานะ MT

ในเอกสาร TRACEABILITY ช่องผลของ MT ยังเป็น ☐ (ยังไม่ได้บันทึกผล) เพราะต้องใช้สลิปจริงบนมือถือ Android จริง ก่อนนำเสนอควรทำ MT-04, 07 และ 11 อย่างน้อย แล้วจดผลลงตาราง เพราะ unit test และ E2E ใช้ผลอ่านของ AI ที่กำหนดไว้ ส่วน MT คือที่เดียวที่ทดสอบ "AI อ่านสลิปจริง"

FR-4 เชื่อมกับ FR อื่นยังไ

FR-4 เป็น "ทางเข้า" ของข้อมูลที่ใหญ่ที่สุด และใช้ของคนอื่นหลายชั้น

FR	FR-4 ใช้อะไรของเขา / เขาใช้อะไรของ FR-4
FR-1 จัดการรายการ	สลิปทุกใบกลายเป็นรายการในตาราง transactions ผ่าน <code>repo.insert</code> ตัวเดียวกัน · หน้าตรวจสลิป = ฟอรั่ม <code>transaction.tsx</code> ของ FR-1 · หมวดจาก <code>suggestCategory</code> · แถวรายการ <code>TxRow</code> แสดงธง · แปลงเงินด้วย <code>parseBahtToSatang</code>
FR-2 ภาพรวม	ยอดคงเหลือ/ยอดตามช่วงนับเฉพาะ <code>confirmed</code> · หน้าสแกนใช้ <code>PeriodSummary</code> และ <code>groupByDay</code> · การ์ด "สลิปรอยืนยัน" อยู่บนหน้าหลัก · การ์ดสแกนอัตโนมัติวางบนหน้าหลัก
FR-3 พุดจด	เป็นทางเข้าอีกทางของรายการ (<code>source = voice</code>) ไม่ได้ใช้โค้ดร่วมกันโดยตรง แต่ใช้ตารางและหมวดเดียวกัน
FR-5 โค้ช AI	ใช้ <code>Edge Function</code> และ <code>AI</code> ตัวเดียวกัน (<code>common.ts</code>) แต่โค้ชเห็นแค่ยอดรวมตามหมวด ไม่เห็นรูปสลิป เลขบัญชี หรือชื่อคนที่โอนให้ · โควตา <code>AI</code> แยกกัน (<code>slip / coach</code>)
FR-6 Money Runway	กันที่ที่สลิปถูกนับในยอด Runway คำนวณใหม่ (ในภาพหน้าจอ: ยังคำนวณไม่ได้ → 22 วัน ถึง 31 ต.ค.) · แบบร่างไม่กระทบ Runway

ทำไมฉันออกแบบแบบนี้

การตัดสินใจหลัก ทางเลือกที่ไม่เลือก และเหตุผล

การตัดสินใจ	เหตุผล
หา QR บนมือถือก่อน ไม่ส่งทุกรูปให้ AI	แกลเลอรีมีรูปเป็นพัน ส่งทุกรูป = แพงมาก ช้า และรูปส่วนตัวรัว · QR บอกได้ฟรีว่ารูปไหนน่าจะเป็นสลิป และให้เลขอ้างอิง/ธนาคารที่แม่นยำกว่า AI
อ่าน 2 รอบด้วย AI 2 รุ่น	ความมั่นใจที่ AI ให้เองเชื่อไม่ได้ 100% (มันใจผิดได้) · 2 รุ่นที่เป็นอิสระผิดเหมือนกัน ยาก · ไม่ตรงกัน = ให้คนเลือกจาก 2 คำ ซึ่งเร็วกว่าพิมพ์ใหม่
แบบร่างไม่นับในยอดใด ๆ	ยอดผิดแยกว่าไม่มีคำตอบ ผู้ใช้วางแผนเงินจากตัวเลขนี้ (FR-6)
เปิด "รวมยอดทันทีเมื่ออ่านชัด" เป็นค่าเริ่มต้น	ตัดสินใจร่วมกันในทีม 27 ก.ย. 2569: ถ้าผู้ใช้ต้องกดยืนยันทุกใบ ระบบอัตโนมัติก็ไม่ใช่ช่วยอะไร · ป้องกันด้วยเกณฑ์ 80% + อ่าน 2 รอบ + ปุ่ม "ยังไม่รวมยอด" + ปิดได้ในตั้งค่า
อ่านวันที่ด้วยตัวอ่านของเราเอง ก่อนเชื่อ AI	บ๊ิก "ก.ย." ใน prototype · ตัวอ่านของเราทดสอบได้ทุกแบบ (TC-04-06) และถ้าตัวหนังสือกับที่ AI แปลงไม่ตรงกัน = ตัดธง
เรียนชื่อจากสลิป แทนการถามชื่อผู้ใช้	ผู้ใช้ไม่ต้องตั้งค่า · ชื่อบนสลิปมักถูกปิดบัง/พิมพ์อังกฤษ ซึ่งชื่อที่ผู้ใช้พิมพ์เองจะไม่ตรง · เก็บแค่ลายนิ้วมือ
state machine เป็น reducer ล้วน	บ๊ิกเปลี่ยนช่วงระหว่างสแกนใน prototype · reducer ทำให้สถานะที่ผิดเกิดไม่ได้ และทดสอบได้โดยไม่ต้องมีมือถือ
processSlip ใช้ร่วมกัน 2 ที่	หน้าสแกนกับการสแกนอัตโนมัติใช้กฎชุดเดียวกัน แยกที่เดียวได้ทั้งคู่ (DRY)
ไม่เก็บรูปบนเซิร์ฟเวอร์	ไม่จำเป็นต่อการทำงาน (ใช้แค่แฮช) ลดความเสี่ยงข้อมูลรั่ว และไม่เสียค่าที่เก็บ
เน็ตหลุด/AI ล่ม → หยุดทั้งคิว	ปัญหาที่เกิดขึ้นทุกรูป ถ้าข้ามไปเรื่อย ๆ ทุกรูปจะถูกนับว่า "อ่านไม่ได้" แล้วหายไปจากรอบหน้า
unique index ในฐานข้อมูลอีกชั้น	การค้นซ้ำในแอปไม่เห็นรายการที่อีกเครื่องเพิ่งบันทึก ฐานข้อมูลเป็นด่านเดียวที่เห็นทุกเครื่องพร้อมกัน
ขอสลิปรูปเองแค่ครั้งเดียว	ครั้งแรกถามเพื่อให้ "ไม่ต้องทำอะไร" ได้จริง หลังจากนั้นไม่รบกวนอีก (Android บล็อกการถามซ้ำหลังถูกปฏิเสธหลายครั้งอยู่แล้ว)

ข้อจำกัด และสิ่งที่จะทำต่อ

สิ่งที่ฉันรู้ว่ายังไม่สมบูรณ์ พร้อมวิธีแก้ที่คิดไว้

ข้อจำกัด	ผลกระทบ และวิธีแก้ที่เสนอ
ค่า <code>crcValid</code> ถูกคำนวณแต่ยังไม่ถูกใช้	QR ที่ถูกแก้ไขข้อมูลยังให้เลขอ้างอิงได้ (เสี่ยงต่ำ เพราะผู้ใช้สแกนสลิปของตัวเอง) · แก้: ถ้า CRC ไม่ถูก ไม่ใช้เลขจาก QR (หรือติดธง) และเพิ่มเกสต์
คำว่า "ร้าน" อยู่ในคำค้นหมวดอาหาร	"ร้านรองเท้า" ได้หมวดอาหาร (เห็นในภาพหน้าจอ) · แก้: ให้คำทั่วไปอย่าง "ร้าน" มีลำดับต่ำกว่าคำเฉพาะ หรือเพิ่มคำค้นหมวดช้อปปิ้ง (รองเท้า เสื้อผ้า)
เรียนชื่อต้องมีสลิปของตัวเองอย่างน้อย 2 ใบ	ช่วงแรกสลิปที่เพื่อนส่งมาอาจถูกจัดเป็นรายจ่าย · ผู้ใช้แก้ไขเป็นรายรับได้ในหน้าตรวจ/แก้ไข
เว็บค้นเกลเลเองไม่ได้	ข้อจำกัดของเบราว์เซอร์ (ความปลอดภัย) ผู้ใช้ iPhone ผ่านเว็บต้องกดเลือกรูป · แก้ได้ด้วยแอป iOS (Expo รองรับ แต่ทีมทดสอบเฉพาะ Android)
สลิปที่ไม่มี QR (รูปถ่ายสลิปกระดาษ ภาพที่ครอบ QR ออก)	สแกนอัตโนมัติจะไม่เจอ · ต้องเลือกรูปเอง (หน้าสแกนแนะนำเรื่องนี้เมื่อไม่เจอสลิป)
AI แบบฟรีของ Gemini	มีลิมิตต่อนาที (สแกนครั้งแรกที่มีสลิปเยอะต้องรอ) และ Google อาจใช้ข้อมูลตามเงื่อนไข (แจ้งในตั้งค่าแล้ว) · แก้: ใช้แบบเสียเงินเมื่อเปิดให้ใช้จริง
AI 2 ตัวอาจผิดเหมือนกัน	โอกาสต่ำแต่ไม่ใช่ศูนย์ · มีด้านอื่นช่วย: ยอดตรวจกับตัวเอง, QR, วันที่ของเราเอง และผู้ใช้แก้ไขได้ทุกเมื่อ
สแกนอัตโนมัติย้อนไม่เกิน 30 วัน	สลิปเก่ากว่านั้นต้องกด "ค้นหาย้อนหลัง 1 ปี" ในหน้าสแกน
ปี 2 หลักที่เกินปีหน้า	เช่น "28" ตอนนี้ถูกตีความเป็น พ.ศ. 2528 (ค.ศ. 1985) → นอกช่วง ไม่บันทึก · ไม่กระทบจนถึงปี 2570 ขึ้นไป (สลิปจริงไม่พิมพ์ปีอนาคต)
MT บนมือถือจริงยังไม่บันทึกผล	ต้องทดสอบกับสลิปจริงหลายธนาคารก่อนส่งงาน (หัวข้อการทดสอบ)

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ตอบด้วยตัวเลขและชื่อไฟล์ จะดูเข้าใจระบบจริง

คำถาม	คำตอบ
ถ้า AI อ่านยอดผิด ยอดเงินของผู้ใช้จะผิดไหม	มีด่านกัน 4 ชั้นก่อนนับ: AI ต้องมั่นใจ $\geq 80\%$ ทุกช่อง · AI 2 รุ่นต้องอ่านตรงกัน · ยอดต้องตรงกับยอดแบบพิมพ์และไม่เท่าค่าธรรมเนียม · ไม่ผ่านข้อใด = แบบร่าง ไม่นับ · และถ้านับไปแล้วผู้ใช้กด "ยังไม่รวมยอด" ได้
ตัวเลข 80% วัดยังง	AI ให้ความมั่นใจ 0–1 ต่อช่องตามคำสั่งใน prompt (ต่ำกว่า 0.8 ถ้ามีตัวเลข/อ่านได้ 2 แบบ) จากนั้นเซิร์ฟเวอร์ปรับลง: ค่าว่าง = 0, ยอดไม่ตรงกับตัวเอง = 0, 2 รุ่นไม่ตรงกัน = ไม่เกิน 0.5, อ่านได้รอบเดียว = ไม่เกิน 0.79 แอปเทียบ <code>CONFIDENCE_THRESHOLD = 0.8</code> ในไฟล์ slip.ts
Requirement บอก "ตรวจเมื่อผู้ใช้เริ่ม Scan Gallery" ทำโมสแกนเองตอนเปิดแอปด้วย	หน้าสแกนตรง Requirement (ผู้ใช้กดเข้าหน้าสแกน แอปค้นและอ่านทันที) ส่วนสแกนอัตโนมัติเป็นส่วนเสริมที่ทีมตัดสินใจเพิ่มจากความเห็นผู้ใช้ ปิดได้ในตั้งค่า และใช้กฎ 80% กับกันซ้ำชุดเดียวกัน
ทำไมไม่ใช้ OCR สรรดา	OCR ให้แค่ตัวหนังสือ แต่ไม่รู้ตัวเลขไหนคือยอด เลขไหนคือค่าธรรมเนียมหรือยอดคงเหลือ และสลิปแต่ละธนาคารวางต่างกัน AI แบบดูรูปเข้าใจความหมายของป้าย ส่วนที่ต้องแม่นยำแบบตายตัว (QR, วันที่, กันซ้ำ) เราเขียนโค้ดเองและทดสอบได้
รูปในเครื่องปลอดภัยไหม	รูปที่ไม่มี QR ไม่ออกจากเครื่อง · รูปสลิปถูกย่อแล้วส่งผ่าน HTTPS ใช้แค่ระหว่างคำขอ ไม่เก็บบนเซิร์ฟเวอร์ · ข้อมูลรายการเห็นได้เฉพาะเจ้าของ (RLS) · ชื่อผู้ใช้เก็บแค่ลายนิ้วมือในเครื่อง
สแกนสลิปใบเดิม 2 ครั้งเกิดอะไรขึ้น	รูปเดิมถูกข้ามตั้งแต่แรก (จำไว้แล้ว) · ถ้าล้างประวัติ: แหะรูปเดิม → ชำก่อนส่ง AI · รูปใหม่ของสลิปเดิม: เลขอ้างอิง → ชำ · ไม่มีเลข: ยอด+นาทียอมรับ → ชำ · หลุดทุกด้าน: unique index ในฐานะข้อมูลปฏิเสธ
ถ้าเน็ตหลุดกลางการสแกน	ลองรูปเดิมซ้ำหลัง 3 วินาที ไม่ได้ → หยุดทั้งคิวพร้อมบอกเหตุผล กด "อ่านต่อ" ทำต่อจากรูปที่ค้าง · สแกนอัตโนมัติไม่เลื่อนหน้าต่างเวลา รอบหน้าเห็นรูปเดิม (E2E ข้อ 15, TC-56)
สลิปที่เพื่อนส่งมาให้ เป็นรายรับได้ยังง	แอปเรียนชื่อของผู้ใช้จากบรรทัด "จาก" ของสลิปที่ผู้ใช้โอนเอง (≥ 2 ใบ และ $\geq 20\%$) ถ้าผู้รับในสลิปคือชื่อผู้ใช้ = รายรับ (slipNames.ts, TC-65)
ทดสอบยังงถึงที่ AI ตอบไม่เหมือนเดิมทุกครั้ง	แยกส่วนที่แน่นอนออกมาเป็นฟังก์ชันล้วนแล้ว unit test (38 สหสเทศ) · E2E ใช้เซิร์ฟเวอร์จำลองที่ตอบผลอ่านตายตัว แต่ทุกอย่างหลังจากนั้นเป็นโค้ดจริง · ส่วน "AI อ่านรูปจริง" ทดสอบด้วยมือ (MT-04)
สลิป TrueMoney ไม่มี QR ธนาคาร อ่านได้ยังง	กฎเสริม: รูปขนาดหน้าจอที่มี QR อะไรก็ได้ ส่งให้ AI ตัดสิน (E2E ข้อ 17: wallet-qr ถูกอ่าน แต่รูปกล้องที่มี QR เดียวกันไม่ถูกส่ง)
ผู้ใช้ไม่ยอมให้สิทธิ์รูปภาพ	แอปไม่ถามซ้ำ มีการแจ้งเตือนอนุญาตเองทีหลัง และหน้าสแกนยังมี "เลือกรูปเอง" (MT-05)
สลับบัญชีระหว่างสแกนอัตโนมัติ	เลขรุ่น (runGen) เปลี่ยน รอบค่าหยุดเจียบ และก่อนบันทึกทุกใบเช็ก <code>stillValid</code> จึงไม่มีสลิปของคนหนึ่งไปอยู่บัญชีอีกคน

คำถาม	คำตอบ
ค่าใช้จ่าย AI ต่อเดือน	ช่วงพัฒนาใช้ Gemini แบบฟรี · ลดต้นทุนด้วย: ส่งเฉพาะรูปที่มี QR, ย่อรูป ~150 KB, จับช้ำก่อนส่ง, รุ่น flash-lite · กั้นบิลบานด้วยโควตา 300 ใบ/วัน/บัญชี (นับแบบลือกในฐานข้อมูล)

แบบฝึกหัดเช็กความเข้าใจ

ทุกข้อฉันรับโค้ดจริงเพื่อหาคำตอบแล้ว (สมมติวันนี้ 9 ต.ค. 2569)

- QR มีข้อความ 0041000600000101030140220016272184455CKQ112205102TH91040ECC ธนาคารอะไร เลขอ้างอิงอะไร CRC ถูกไหม
- AI ให้ความมั่นใจ ยอด 0.85 วันที่ 0.80 ชื่อ 0.78 จะติดธงอะไร ความมั่นใจต่ำสุดที่แสดงคือเท่าไร และถ้าเปิด "รวมยอดทันที" สถานะเป็นอะไร
- สลิปวันที่ "1 ม.ค. 70" parseSlipDate ได้วันอะไร และ normalizeReading ทำอะไรกับวันนี้
- รูปขนาด 720×1600 ส่วน "मुखवास" ที่ถูกตัดไปสแกนใหม่ เริ่มที่พิกเซลไหน กว้างและสูงเท่าไร
- AI ตัวแรกตอบยอด "1250.00" ตัวที่สองตอบเป็นตัวเลข 1250 ถือว่าตรงกันไหม
- มีรายการจากสลิป ร้านกาแฟดอยช้าง ฿85 วันที่ 27 ก.ย. 08:15 (ไม่มีเลขอ้างอิง) (ก) สลิปใหม่ยอด วัน นาทีย่อยกันแต่อ่านชื่อร้านไม่ได้ ชำโหม (ข) สลิปใหม่ที่เหมือนกันทุกอย่างแต่มีเลขอ้างอิง ZZ998877 ชำโหม
- สแกนอัตโนมัติครั้งสุดท้ายเมื่อ 3 ชั่วโมงก่อน ครั้งนี้ค้นรูปย้อนหลังกี่ชั่วโมง
- ปี "68" บนสลิปคือ ค.ศ. อะไร
- "น.ส. มะลิ แสนดี" กับ "มะลิ แ****" ได้ลายนิ้วมือเดียวกันไหม แล้ว "มะลิ" เฉย ๆ ละ
- AI ตัวแรกตอบ expense ตัวที่สองตอบ income ผลรวมเป็นอะไร และแอปติดธงอะไร

เฉลย

- รหัส 014 = **SCB** · เลขอ้างอิง 016272184455CKQ11220 · CRC 0ECC ถูกต้อง (crcValid: true)
- ธง **counterparty** อย่างเดียว (0.80 ไม่ติด เพราะใช้ < 0.8) · ความมั่นใจต่ำสุด **0.78** (แสดง 78%) · มีธง = needs_review → **draft** แม้เปิดรวมยอดทันที
- 2027-01-01 (ปี 70 = พ.ศ. 2570) · เป็นอนาคต (เลขพຽງนี้) → ติดธง date และ **ทั้งวันที่** (dayKey = null) ใช้เวลาของรูปแทน
- เริ่ม (324, 880) กว้าง 396 สูง 720 ($0.45 \times 720 = 324$, $0.55 \times 1600 = 880$, $0.55 \times 720 = 396$, $0.45 \times 1600 = 720$)
- ตรง** (เทียบด้วย `Number("1250.00") === 1250`) ผลรวม verified: true ยอด "1250.00"
- (ก) **ชำ** เพราะสลิปใหม่ไม่รู้ผู้รับ ถือว่าตรงไว้ก่อน · (ข) **ไม่ชำ** เพราะมีเลขอ้างอิงที่ไม่เคยเห็น จึงไม่ใช่กฎยอด+นาทีย่อย+ผู้รับ
- 4 ชั่วโมง** (ครั้งล่าสุด - 1 ชม. ที่ย้อนกับ)
- 2025** (พ.ศ. 2568 · แบบ ค.ศ. 2068 เป็นอนาคตไกลจึงถูกตัด)
- 2 ชื่อแรกได้ 574998de เหมือนกัน (มะลิ|แ) · "มะลิ" เฉย ๆ ได้ 8372882b คนละรหัส (ไม่มีอักษรนามสกุล)
- direction = **unknown** และ disagree.direction = [expense, income] → แอปติดธง **direction** ("ตรวจเงินเข้าหรือออก") และบันทึก "AI อ่านได้ 2 แบบ: เงินเข้าหรือออก"

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงได้เลย ใช้เวลาประมาณ 1 นาที

“ สคริปต์ 1 นาที

"FR-4 คือการจดยรายการจากสลิปโดยอัตโนมัติค่ะ/ครับ ทุกครั้งที่โอนเงิน แอปธนาคารจะเซฟสลิปไว้ในแกลเลอรี MindPay จะหาสลิปเหล่านั้นเอง โดยดูจาก QR ตรวจสอบสลิปที่ธนาคารพิมพ์ไว้ ซึ่งมือถือถอดได้ฟรีไม่ต้องใช้เน็ต รูปที่ไม่ใช่สลิปจะไม่ถูกส่งไปไหนเลย

รูปที่เป็นสลิปถูกย่อและส่งให้ AI อ่าน 2 ครั้งด้วย 2 รุ่นที่ไม่เห็นคำตอบกัน ถ้าตรงกันและมั่นใจตั้งแต่ 80% ขึ้นไปทุกช่องตามเกณฑ์ของ Requirement รายการจะเข้ายอดเงินทันที ถ้าไม่ชัดหรืออ่านได้ 2 แบบ จะเป็นแบบร่างที่ไม่นับในยอดจนกว่าผู้ใช้จะแตะเลือกค่าที่ถูกและยืนยัน

เราเก็บรายการซ้ำ 4 ชั้น ทั้งเลขอ้างอิงจาก QR ลายนิ้วมือของรูป ยอดกับเวลาถึงนาที และ unique index ในฐานข้อมูลทั้งหมดนี้ทดสอบด้วย unit test 38 รหัสทดสอบ และ E2E 5 สถานการณ์ เช่น เปิดแอปแล้ว 6 รูปถูกส่งให้ AI แค 3 รูป และยอดเงินอัปเดตทันที"

คำศัพท์ของ FR-4

คำ	ความหมาย
QR ตรวจสอบสลิป	QR เลิกบนสลิปที่เก็บรหัสธนาคารและเลขอ้างอิงของการโอน (ไม่ใช่ QR สำหรับจ่ายเงิน)
TLV	รูปแบบข้อมูล Tag-Length-Value ใช้ใน QR การเงิน
CRC	ตัวเลขตรวจสอบที่คำนวณจากข้อมูล ใช้จับว่าข้อมูลถูกแก้ไขหรือเสียหาย
Confidence (ความมั่นใจ)	ตัวเลข 0-1 ที่บอกว่า AI แน่ใจแค่ไหนว่าอ่านชื่อนั้นถูก
ธง (flag)	เครื่องหมายว่าช่องไหนต้องให้คนตรวจ: amount, date, counterparty, direction
แบบร่าง (draft) / ยืนยันแล้ว (confirmed)	สถานะของรายการ เฉพาะ confirmed ที่นับในยอด
ready / needs_review	ผลการตัดสินสลิป: ไม่มีธง / มีธง
Auto-confirm	สวิตช์ "รวมยอดทันทีเมื่ออ่านชัด"
แฮช (hash) / ลายนิ้วมือ	ค่าสั้น ๆ ที่คำนวณจากข้อมูล ข้อมูลเดิมได้ค่าเดิมเสมอ ใช้เทียบโดยไม่ต้องเก็บข้อมูลจริง (SHA-256 สำหรับรูป, FNV-1a สำหรับชื่อ)
Duplicate index	"สมุดจำ" ของเลขอ้างอิง แฮช และยอด+นาที ที่มีอยู่แล้ว
Unique index	กฎในฐานข้อมูลที่ห้ามมี 2 แถวค่าเดียวกัน
State machine / reducer	การออกแบบสถานะที่ชัดเจน และฟังก์ชันที่รับ (สถานะ, คำสั่ง) คืนสถานะใหม่

คำ	ความหมาย
runId	เลขรอบของคิว ใช้ทั้งผลที่มาจากรอบเก่า
Edge Function	ฟังก์ชันที่รันบนเซิร์ฟเวอร์ Supabase (parse-slip)
JSON Schema	แบบฟอร์มบังคับหน้าตาคำตอบของ AI
base64	วิธีแปลงไฟล์รูปเป็นตัวอักษร เพื่อส่งใน JSON
Rate limit / busy	AI จำกัดจำนวนครั้งต่อนาที เกินแล้วต้องรอ (แอดโปร 20/40 วินาที)
Quota	จำนวนครั้งต่อวันต่อบัญชี (สลิป 300 ใบ)
Provider / Context	กลไกของ React ที่แชร์สถานะ (เช่นการสแกนอัตโนมัติ) ให้ทุกหน้าจอ
e-wallet	กระเป๋าเงินอิเล็กทรอนิกส์ เช่น TrueMoney, ShopeePay